

Python 3 — Biblioteca Estándar

Tomo 5 de 6

Contenido de este tomo:

1. `tkinter.dnd`— Drag and drop support
2. `tkinter.ttk`— Tk themed widgets
3. IDLE — Python editor and shell
4. `turtle`— Turtle graphics
5. Development Tools
6. `typing`— Support for type hints
7. `pydoc`— Documentation generator and online help system
8. Python Development Mode
9. `doctest`— Test interactive Python examples
10. `unittest`— Unit testing framework
11. `unittest.mock`— mock object library
12. `unittest.mock`— getting started
13. `test`— Regression tests package for Python
14. Debugging and Profiling
15. Audit events table
16. `bdb`— Debugger framework
17. `faulthandler`— Dump the Python traceback
18. `pdb`— The Python Debugger
19. The Python Profilers
20. `timeit`— Measure execution time of small code snippets
21. `trace`— Trace or track Python statement execution
22. `tracemalloc`— Trace memory allocations
23. Software Packaging and Distribution
24. `ensurepip`— Bootstrapping the pip installer
25. `venv`— Creation of virtual environments

- 26. `zipapp`— Manage executable Python zip archives
- 27. Python Runtime Services
- 28. `sys`— System-specific parameters and functions
- 29. `sys.monitoring`— Execution event monitoring
- 30. `sysconfig`— Provide access to Python's configuration information
- 31. `builtins`— Built-in objects
- 32. `__main__`— Top-level code environment
- 33. `warnings`— Warning control
- 34. `dataclasses`— Data Classes
- 35. `contextlib`— Utilities for with-statement contexts
- 36. `abc`— Abstract Base Classes
- 37. `atexit`— Exit handlers
- 38. `traceback`— Print or retrieve a stack traceback
- 39. `__future__`— Future statement definitions
- 40. `gc`— Garbage Collector interface
- 41. `inspect`— Inspect live objects
- 42. `annotationlib`— Functionality for introspecting annotations
- 43. `site`— Site-specific configuration hook
- 44. Custom Python Interpreters
- 45. `code`— Interpreter base classes

tkinter.dnd— Drag and drop support

Source code: `Lib/tkinter/dnd.py`

■ Nota: Note This is experimental and due to be deprecated when it is replaced with the Tk DND.

The `tkinter.dnd` module provides drag-and-drop support for objects within a single application, within the same window or between windows. To enable an object to be dragged, you must create an event binding for it that starts the drag-and-drop process. Typically, you bind a `ButtonPress` event to a callback function that you write (see [Bindings and Events](#)). The function should call `dnd_start()`, where 'source' is the object to be dragged, and 'event' is the event that invoked the call (the argument to your callback function).

Selection of a target object occurs as follows:

- Top-down search of area under mouse for target widget
- Target widget should have a callable `dnd_accept` attribute
- If `dnd_accept` is not present or returns `None`, search moves to parent widget
- If no target widget is found, then the target object is `None`
- Call to `<old_target>.dnd_leave(source, event)`
- Call to `<new_target>.dnd_enter(source, event)`
- Call to `<target>.dnd_commit(source, event)` to notify of drop
- Call to `<source>.dnd_end(target, event)` to signal end of drag-and-drop

`class tkinter.dnd.DndHandler(source, event)`

`cancel(event=None)`

`finish(event, commit=0)`

`on_motion(event)`

`on_release(event)`

The `DndHandler` class handles drag-and-drop events tracking `Motion` and `ButtonRelease` events on the root of the event widget. `cancel(event=None)` Cancel the drag-and-drop process. `finish(event, commit=0)` Execute end of drag-and-drop functions. `on_motion(event)` Inspect area below mouse for target objects while drag is performed. `on_release(event)` Signal end of drag when the release pattern is triggered.

Cancel the drag-and-drop process.

Execute end of drag-and-drop functions.

Inspect area below mouse for target objects while drag is performed.

Signal end of drag when the release pattern is triggered.

`tkinter.dnd.dnd_start(source, event)`

Factory function for drag-and-drop process.

See also

[Bindings and Events](#)

tkinter.ttk— Tk themed widgets

Source code: `Lib/tkinter/ttk.py`

The `tkinter.ttk` module provides access to the Tk themed widget set, introduced in Tk 8.5. It provides additional benefits including anti-aliased font rendering under X11 and window transparency (requiring a composition window manager on X11).

The basic idea for `tkinter.ttk` is to separate, to the extent possible, the code implementing a widget's behavior from the code implementing its appearance.

See also

Tk Widget Styling Support

A document introducing theming support for Tk

Using Ttk

To start using Ttk, import its module:

```
from tkinter import ttk
```

To override the basic Tk widgets, the import should follow the Tk import:

```
from tkinter import *
from tkinter.ttk import *
```

That code causes several `tkinter.ttk` widgets (`Button`, `Checkbutton`, `Entry`, `Frame`, `Label`, `LabelFrame`, `Menubutton`, `PanedWindow`, `Radiobutton`, `Scale` and `Scrollbar`) to automatically replace the Tk widgets.

This has the direct benefit of using the new widgets which gives a better look and feel across platforms; however, the replacement widgets are not completely compatible. The main difference is that widget options such as “fg”, “bg” and others related to widget styling are no longer present in Ttk widgets. Instead, use the `ttk.Style` class for improved styling effects.

See also

Converting existing applications to use Tk widgets

A monograph (using Tcl terminology) about differences typically encountered when moving applications to use the new widgets.

Ttk Widgets

Ttk comes with 18 widgets, twelve of which already existed in `tkinter`: `Button`, `Checkbutton`, `Entry`, `Frame`, `Label`, `LabelFrame`, `Menubutton`, `PanedWindow`, `Radiobutton`, `Scale`, `Scrollbar`, and `Spinbox`. The other six are new: `Combobox`, `Notebook`, `Progressbar`, `Separator`, `Sizegrip` and `Treeview`. And all them are subclasses of `Widget`.

Using the Ttk widgets gives the application an improved look and feel. As discussed above, there are differences in how the styling is coded.

Tk code:

```
l1 = tkinter.Label(text="Test", fg="black", bg="white")
l2 = tkinter.Label(text="Test", fg="black", bg="white")
```

Ttk code:

```
style = ttk.Style()
```

```
style.configure("BW.TLabel", foreground="black", background="white")

l1 = ttk.Label(text="Test", style="BW.TLabel")
l2 = ttk.Label(text="Test", style="BW.TLabel")
```

For more information about `TtkStyling`, see the `Styleclass` documentation.

Widget

`ttk.Widget` defines standard options and methods supported by Tk themed widgets and is not supposed to be directly instantiated.

Standard Options

All the `ttk.Widgets` accept the following options:

Option	Description
<code>class</code>	Specifies the window class. The class is used when querying the option database for the window's other options, to determine the default bindtags for the window, and to select the widget's default layout and style. This option is read-only, and may only be specified when the window is created.
<code>cursor</code>	Specifies the mouse cursor to be used for the widget. If set to the empty string (the default), the cursor is inherited for the parent widget.
<code>takefocus</code>	Determines whether the window accepts the focus during keyboard traversal. 0, 1 or an empty string is returned. If 0 is returned, it means that the window should be skipped entirely during keyboard traversal. If 1, it means that the window should receive the input focus as long as it is viewable. And an empty string means that the window should receive the focus only when it is the active window.

Scrollable Widget Options

The following options are supported by widgets that are controlled by a scrollbar.

Option	Description
<code>xscrollcommand</code>	Used to communicate with horizontal scrollbars. When the view in the widget's window change, the widget will generate a Tcl command based on the scrollcommand. Usually this option consists of the method <code>Scrollbar.set()</code> of some scrollbar. This will cause the scrollbar to be updated whenever the view in the window changes.
<code>yscrollcommand</code>	Used to communicate with vertical scrollbars. For some more information, see above.

Label Options

The following options are supported by labels, buttons and other button-like widgets.

Option	Description
<code>text</code>	Specifies a text string to be displayed inside the widget.
<code>textvariable</code>	Specifies a name whose value will be used in place of the text option resource.
<code>underline</code>	If set, specifies the index (0-based) of a character to underline in the text string. The underline character is used for mnemonic activation.
<code>image</code>	Specifies an image to display. This is a list of 1 or more elements. The first element is the default image name. The rest of the list is a sequence of statespec/value pairs as defined by <code>Style.map()</code> , specifying different images to use when the widget is in a particular state or a combination of states. All images in the list should have the same size.
<code>compound</code>	Specifies how to display the image relative to the text, in the case both

Compatibility Options

Option	Description
<code>state</code>	May be set to "normal" or "disabled" to control the "disabled" state bit. This is a write-only option: setting it changes the widget state, but the <code>Widget.state()</code> method does not affect this option.

Widget States

The widget state is a bitmap of independent state flags.

Flag	Description
<code>active</code>	The mouse cursor is over the widget and pressing a mouse button will cause some action to occur
<code>disabled</code>	Widget is disabled under program control
<code>focus</code>	Widget has keyboard focus
<code>pressed</code>	Widget is being pressed
<code>selected</code>	"On", "true", or "current" for things like Checkbuttons and radiobuttons
<code>background</code>	Windows and Mac have a background color, while Unix has a background pixmap.

notion of an “active” or foreground window. The | background | state is set for widgets in a background window, and cleared for those in the foreground window | readonly | Widget should not allow user modification | alternate | A widget-specific alternate display format | invalid | The widget’s value is invalid

A state specification is a sequence of state names, optionally prefixed with an exclamation point indicating that the bit is off.

ttk.Widget

Besides the methods described below, the `ttk.Widget` supports the methods `tkinter.Widget.cget()` and `tkinter.Widget.configure()`.

class `tkinter.ttk.Widget`

`identify(x,y)`

`instate(statespec,callback=None,*args,**kw)`

`state(statespec=None)`

`identify(x,y)` Returns the name of the element at position `xy`, or the empty string if the point does not lie within any element. `x` and `y` are pixel coordinates relative to the widget. `instate(statespec,callback=None,*args,**kw)` Test the widget’s state. If a callback is not specified, returns `True` if the widget state matches `statespec` and `False` otherwise. If callback is specified then it is called with `args` if widget state matches `statespec`. `state(statespec=None)` Modify or inquire widget state. If `statespec` is specified,

Returns the name of the element at position `xy`, or the empty string if the point does not lie within any element. `x` and `y` are pixel coordinates relative to the widget.

Test the widget’s state. If a callback is not specified, returns `True` if the widget state matches `statespec` and `False` otherwise. If callback is specified then it is called with `args` if widget state matches `statespec`.

Modify or inquire widget state. If `statespec` is specified, sets the widget state according to it and return a new `statespec` indicating which flags were changed. If `statespec` is not specified, returns the currently enabled state flags.

Combobox

The `ttk.Combobox` widget combines a text field with a pop-down list of values. This widget is a subclass of `Entry`.

Besides the methods inherited

from `Widget`: `Widget.cget()`, `Widget.configure()`, `Widget.identify()`, `Widget.instate()` and `Widget.state()`, and the following inherited

from `Entry`: `Entry.bbox()`, `Entry.delete()`, `Entry.icursor()`, `Entry.index()`, `Entry.insert()`, `Entry.selection()`, `Entry.xview()`, it has some other methods, described at `ttk.Combobox`.

Options

This widget accepts the following specific options:

Option	Description
exportselection	Boolean value. If set, the widget selection is linked to the Window Manager selection (which can be returned by invoking <code>Misc.selection_get</code> , for example).
justify	Specifies how the text is aligned within the widget. One of “left”, “center”, or “right”.
height	Specifies the height of the pop-down listbox, in rows.
postcommand	A script (possibly registered with <code>Misc.register</code>) that is called immediately before displaying the values. It may specify which values to display.
state	One of “normal”, “readonly”, or “disabled”. In the “readonly” state, the value may not be edited directly, and the user can only selection of the values from the dropdown list. In the “normal” state, the text field is directly editable. In the “disabled” state, n

Virtual events

The combobox widget generates a `<<ComboboxSelected>>` virtual event when the user selects an element from the list of values.

ttk.Combobox

`class tkinter.ttk.Combobox`

`current(newindex=None)`

`get()`

`set(value)`

`current(newindex=None)` If `newindex` is specified, sets the combobox value to the element position `newindex`. Otherwise, returns the index of the current value or -1 if the current value is not in the values list. `get()` Returns the current value of the combobox. `set(value)` Sets the value of the combobox to `value`.

If `newindex` is specified, sets the combobox value to the element position `newindex`. Otherwise, returns the index of the current value or -1 if the current value is not in the values list.

Returns the current value of the combobox.

Sets the value of the combobox to `value`.

Spinbox

The `ttk.Spinbox` widget is `ttk.Entry` enhanced with increment and decrement arrows. It can be used for numbers or lists of string values. This widget is a subclass of `Entry`.

Besides the methods inherited

from `Widget`: `Widget.cget()`, `Widget.configure()`, `Widget.identify()`, `Widget.instate()` and `Widget.state()`, and the following inherited from `Entry`: `Entry.bbox()`, `Entry.delete()`, `Entry.icursor()`, `Entry.index()`, `Entry.insert()`, `Entry.xview()`, it has some other methods, described at `ttk.Spinbox`.

Options

This widget accepts the following specific options:

Option	Description
<code>from</code>	Float value. If set, this is the minimum value to which the decrement button will decrement. Must be spelled as <code>from_</code> when used as an argument, since <code>from</code> is a Python keyword.
<code>to</code>	Float value. If set, this is the maximum value to which the increment button will increment.
<code>increment</code>	Float value. Specifies the amount which the increment/decrement buttons change the value. Defaults to 1.0.
<code>values</code>	Sequence of string or float values. If specified, the increment/decrement buttons will cycle through the items in this sequence rather than incrementing or decrementing numbers.
<code>wrap</code>	Boolean value. If <code>True</code> , increment and decrement buttons will cycle from the <code>to</code> value to the <code>from</code> value or the <code>from</code> value to the <code>to</code> value, respectively.

Virtual events

The spinbox widget generates an `<<Increment>>` virtual event when the user presses `<Up>`, and a `<<Decrement>>` virtual event when the user presses `<Down>`.

ttk.Spinbox

`class tkinter.ttk.Spinbox`

`get()`

`set(value)`

`get()`Returns the current value of the spinbox.`set(value)`Sets the value of the spinbox to `value`.

Returns the current value of the spinbox.

Sets the value of the spinbox to `value`.

Notebook

Ttk Notebook widget manages a collection of windows and displays a single one at a time. Each child window is associated with a tab, which the user may select to change the currently displayed window.

Options

This widget accepts the following specific options:

Option	Description
<code>height</code>	If present and greater than zero, specifies the desired height of the pane area (not including internal padding or tabs). Otherwise, the maximum height of all panes is used.
<code>padding</code>	Specifies the amount of extra space to add around the outside of the notebook. The padding is a list up to four length specifications left top right bottom. If fewer than four elements are specified, bottom defaults to top, right defaults to left, and top defaults to left.
<code>width</code>	If present and greater than zero, specified the desired width of the pane area (not including internal padding). Otherwise, the maximum width of all panes is used.

Tab Options

There are also specific options for tabs:

Option	Description
<code>state</code>	Either "normal", "disabled" or "hidden". If "disabled", then the tab is not selectable. If "hidden", then the tab is not shown.
<code>sticky</code>	Specifies how the child window is positioned within the pane area. Value is a string containing zero or more of the characters "n", "s", "e" or "w". Each letter refers to a side (north, south, east or west) that the child window will stick to, as per the <code>grid()</code> geometry manager.
<code>padding</code>	Specifies the amount of extra space to add between the notebook and this pane. Syntax is the same as for the option padding used by this widget.
<code>text</code>	Specifies a text to be displayed in the tab.
<code>image</code>	Specifies an image to display in the tab. See the option image described in <code>Widget</code> .
<code>compound</code>	Specifies how to display

Tab Identifiers

The `tab_id` present in several methods of `ttk.Notebook` may take any of the following forms:

- An integer between zero and the number of tabs
- The name of a child window
- A positional specification of the form "@x,y", which identifies the tab
- The literal string "current", which identifies the currently selected tab
- The literal string "end", which returns the number of tabs (only valid for `Notebook.index()`)

Virtual Events

This widget generates a `<<NotebookTabChanged>>` virtual event after a new tab is selected.

ttk.Notebook

`class tkinter.ttk.Notebook`

`add(child,**kw)`

`forget(tab_id)`

`hide(tab_id)`

identify(x,y)

index(tab_id)

insert(pos,child,kw)**

select(tab_id=None)

tab(tab_id,option=None,kw)**

tabs()

enable_traversal()

add(child,kw)** Adds a new tab to the notebook. If window is currently managed by the notebook but hidden, it is restored to its previous position. See [Tab Options](#) for the list of available options. **forget(tab_id)** Removes the tab specified by `tab_id`, unmaps and unmanages the associated window. **hide(tab_id)** Hides the tab specified by `tab_id`. The tab will not be displayed, but the associated window remains managed by the notebook and its configuration remembered. Hidden tabs may be restored with `thead()`.

Adds a new tab to the notebook. If window is currently managed by the notebook but hidden, it is restored to its previous position. See [Tab Options](#) for the list of available options.

Removes the tab specified by `tab_id`, unmaps and unmanages the associated window.

Hides the tab specified by `tab_id`. The tab will not be displayed, but the associated window remains managed by the notebook and its configuration remembered. Hidden tabs may be restored with the `thead()` command.

Returns the name of the tab element at position `x,y`, or the empty string if none.

Returns the numeric index of the tab specified by `tab_id`, or the total number of tabs if `tab_id` is the string "end".

Inserts a pane at the specified position. `pos` is either the string "end", an integer index, or the name of a managed child. If `child` is already managed by the notebook, moves it to the specified position. See [Tab Options](#) for the list of available options.

Selects the specified `tab_id`. The associated child window will be displayed, and the previously selected window (if different) is unmapped. If `tab_id` is omitted, returns the widget name of the currently selected pane.

Query or modify the options of the specified `tab_id`. If `kw` is not given, returns a dictionary of the tab option values. If `option` is specified, returns the value of that option. Otherwise, sets the options to the corresponding values.

Returns a list of windows managed by the notebook.

Enable keyboard traversal for a toplevel window containing this notebook. This will extend the bindings for the toplevel window containing the notebook as follows: **Control-Tab**: selects the tab following the currently selected one. **Shift-Control-Tab**: selects the tab preceding the currently selected one. **Alt-K**: where `K` is the mnemonic (underlined) character of any tab, will select that tab. Multiple notebooks in a single toplevel may be enabled for traversal, including nested notebooks. However, notebook

Progressbar

The `tk.Progressbar` widget shows the status of a long-running operation. It can operate in two modes: 1) the determinate mode which shows the amount completed relative to the total amount of work to be done and 2) the indeterminate mode which provides an animated display to let the user know that work is progressing.

Options

This widget accepts the following specific options:

Option	Description
<code>orient</code>	One of "horizontal" or "vertical". Specifies the orientation of the progress bar.
<code>length</code>	Specifies the length of the long axis of the progress bar (width if

horizontal, height if vertical). | mode | One of "determinate" or "indeterminate". | maximum | A number specifying the maximum value. Defaults to 100. | value | The current value of the progress bar. In "determinate" mode, this represents the amount of work completed. In "indeterminate" mode, it is interpreted as modulo | maximum | ; that is, the progress bar completes one "cycle" when its value increases by | maximum | . | variable | A name which is linked to the option value. If specified, the value of the progress bar is automatically set to the value of this name whenever the latter is modified.

ttk.Progressbar

class `tkinter.ttk.Progressbar`

start(interval=None)

step(amount=None)

stop()

start(interval=None) Begin autoincrement mode: schedules a recurring timer event that calls `Progressbar.step()` every `interval` milliseconds. If omitted, `interval` defaults to 50 milliseconds. **step**(amount=None) Increments the progress bar's value by `amount`. `amount` defaults to 1.0 if omitted. **stop**() Stop autoincrement mode: cancels any recurring timer event initiated by `Progressbar.start()` for this progress bar.

Begin autoincrement mode: schedules a recurring timer event that calls `Progressbar.step()` every `interval` milliseconds. If omitted, `interval` defaults to 50 milliseconds.

Increments the progress bar's value by `amount`. `amount` defaults to 1.0 if omitted.

Stop autoincrement mode: cancels any recurring timer event initiated by `Progressbar.start()` for this progress bar.

Separator

The `ttk.Separator` widget displays a horizontal or vertical separator bar.

It has no other methods besides the ones inherited from `ttk.Widget`.

Options

This widget accepts the following specific option:

Option	Description
<code>orient</code>	One of "horizontal" or "vertical". Specifies the orientation of the separator.

Sizegrip

The `ttk.Sizegrip` widget (also known as a grow box) allows the user to resize the containing toplevel window by pressing and dragging the grip.

This widget has neither specific options nor specific methods, besides the ones inherited from `ttk.Widget`.

Platform-specific notes

- On macOS, toplevel windows automatically include a built-in size grip by default. Adding a `Sizegrip` is harmless, since the built-in grip will just mask the widget.

Bugs

- If the containing toplevel's position was specified relative to the right or bottom of the screen (e.g.), theSizegripwidget will not resize the window.
- This widget supports only "southeast" resizing.

Treeview

The `Tk.Ttreeviewwidget` displays a hierarchical collection of items. Each item has a textual label, an optional image, and an optional list of data values. The data values are displayed in successive columns after the tree label.

The order in which data values are displayed may be controlled by setting the widget option `displaycolumns`. The tree widget can also display column headings. Columns may be accessed by number or symbolic names listed in the widget option `columns`. See [Column Identifiers](#).

Each item is identified by a unique name. The widget will generate item IDs if they are not supplied by the caller. There is a distinguished root item, named `{}`. The root item itself is not displayed; its children appear at the top level of the hierarchy.

Each item also has a list of tags, which can be used to associate event bindings with individual items and control the appearance of the item.

The Treeview widget supports horizontal and vertical scrolling, according to the options described in [Scrollable Widget Options](#) and the methods `Treeview.xview()` and `Treeview.yview()`.

Options

This widget accepts the following specific options:

Option	Description
<code>columns</code>	A list of column identifiers, specifying the number of columns and their names.
<code>displaycolumns</code>	A list of column identifiers (either symbolic or integer indices) specifying which data columns are displayed and the order in which they appear, or the string <code>"#all"</code> .
<code>height</code>	Specifies the number of rows which should be visible. Note: the requested width is determined from the sum of the column widths.
<code>padding</code>	Specifies the internal padding for the widget. The padding is a list of up to four length specifications.
<code>selectmode</code>	Controls how the built-in class bindings manage the selection. One of <code>"extended"</code> , <code>"browse"</code> or <code>"none"</code> . If set to <code>"extended"</code> (the default), multiple items may be selected. If <code>"browse"</code> , only a single item will be selected at a time.

Item Options

The following item options may be specified for items in the `insert` and `item` widget commands.

Option	Description
<code>text</code>	The textual label to display for the item.
<code>image</code>	A Tk Image, displayed to the left of the label.
<code>values</code>	The list of values associated with the item. Each item should have the same number of values as the widget option <code>columns</code> . If there are fewer values than columns, the remaining values are assumed empty. If there are more values than columns, the extra values are ignored.
<code>open</code>	True / False value indicating whether the item's children should be displayed or hidden.
<code>tags</code>	A list of tags associated with this item.

Tag Options

The following options may be specified on tags:

Option	Description
<code>foreground</code>	Specifies the text foreground color.
<code>background</code>	Specifies the cell or item background color.
<code>font</code>	Specifies the font to use when drawing text.
<code>image</code>	Specifies the item image, in case the item's image option is empty.

Column Identifiers

Column identifiers take any of the following forms:

- A symbolic name from the list of columns option.

- An integer n, specifying the nth data column.
- A string of the form #n, where n is an integer, specifying the nth display column.

Notes:

- Item's option values may be displayed in a different order than the order in which they are stored.
- Column #0 always refers to the tree column, even if show="tree" is not specified.

A data column number is an index into an item's option values list; a display column number is the column number in the tree where the values are displayed. Tree labels are displayed in column #0. If option displaycolumns is not set, then data column n is displayed in column #n+1. Again, column #0 always refers to the tree column.

Virtual Events

The Treeview widget generates the following virtual events.

Event	Description
<<TreeviewSelect>>	Generated whenever the selection changes.
<<TreeviewOpen>>	Generated just before setting the focus item to open=True.
<<TreeviewClose>>	Generated just after setting the focus item to open=False.

The `Treeview.focus()` and `Treeview.selection()` methods can be used to determine the affected item or items.

ttk.Treeview

class `tkinter.ttk.Treeview`

bbox(item,column=None)

get_children(item=None)

set_children(item,*newchildren)

column(column,option=None,**kw)

id

anchor: One of the standard Tk anchor values.

minwidth: width

stretch: True/False

width: width

delete(*items)

detach(*items)

exists(item)

focus(item=None)

heading(column,option=None,**kw)

text: text

image: imageName

anchor: anchor

command: callback

identify(component,x,y)

identify_row(y)

identify_column(x)
identify_region(x,y)
identify_element(x,y)
index(item)
insert(parent,index,iid=None,kw)**
item(item,option=None,kw)**
move(item,parent,index)
next(item)
parent(item)
prev(item)
reattach(item,parent,index)
see(item)
selection()
selection_set(*items)
selection_add(*items)
selection_remove(*items)
selection_toggle(*items)
set(item,column=None,value=None)
tag_bind(tagname,sequence=None,callback=None)
tag_configure(tagname,option=None,kw)**
tag_has(tagname,item=None)
xview(*args)
yview(*args)

bbox(item,column=None) Returns the bounding box (relative to the treeview widget's window) of the specified *item* in the form (x, y, width, height). If *column* is specified, returns the bounding box of that cell. If *item* is not visible (i.e., if it is a descendant of a closed item or is scrolled offscreen), returns an empty string. **get_children(item=None)** Returns the list of children belonging to *item*. If *item* is not specified, returns root children. **set_children(item,*newchildren)** Replaces *item*'s child with *newchildren*.

Returns the bounding box (relative to the treeview widget's window) of the specified *item* in the form (x, y, width, height). If *column* is specified, returns the bounding box of that cell. If *item* is not visible (i.e., if it is a descendant of a closed item or is scrolled offscreen), returns an empty string.

Returns the list of children belonging to *item*. If *item* is not specified, returns root children.

Replaces *item*'s child with *newchildren*. Children present in *item* that are not present in *newchildren* are detached from the tree. No items in *newchildren* may be an ancestor of *item*. Note that not specifying *newchildren* results in detaching *item*'s children.

Query or modify the options for the specified *column*. If *kw* is not given, returns a dict of the column option values. If *option* is specified then the value for that *option* is returned. Otherwise, sets the options to the corresponding values. The valid options/values are: *id* Returns the column name. This is a read-only option. *anchor*: One of the standard Tk anchor values. Specifies how the text in this column should be aligned

with respect to the cell.
minwidth: widthThe minimum width of the column in pixels.

Returns the column name. This is a read-only option.

Specifies how the text in this column should be aligned with respect to the cell.

The minimum width of the column in pixels. The treeview widget will not make the column any smaller than specified by this option when the widget is resized or the user drags a column.

Specifies whether the column's width should be adjusted when the widget is resized.

The width of the column in pixels.

Delete all specified items and all their descendants. The root item may not be deleted.

Unlinks all of the specified items from the tree. The items and all of their descendants are still present, and may be reinserted at another point in the tree, but will not be displayed. The root item may not be detached.

Returns True if the specified item is present in the tree.

If item is specified, sets the focus item to item. Otherwise, returns the current focus item, or "" if there is none.

Query or modify the heading options for the specified column. If kw is not given, returns a dict of the heading option values. If option is specified then the value for that option is returned. Otherwise, sets the options to the corresponding values. The valid options/values are: text: textThe text to display in the column heading. image: imageNameSpecifies an image to display to the right of the column heading. anchor: anchorSpecifies how the heading text should be aligned. One of the standard Tk anchor values.

The text to display in the column heading.

Specifies an image to display to the right of the column heading.

Specifies how the heading text should be aligned. One of the standard Tk anchor values.

A callback to be invoked when the heading label is pressed.

Returns a description of the specified component under the point given by x and y, or the empty string if no such component is present at that position.

Returns the item ID of the item at position y.

Returns the data column identifier of the cell at position x. The tree column has ID #0.

Returns one of: region meaning heading Tree heading area. separator Space between two columns. headings. tree The tree area. cell A data cell. Availability: Tk 8.6.

Returns the element at position x, y. Availability: Tk 8.6.

Returns the integer index of item within its parent's list of children.

Creates a new item and returns the item identifier of the newly created item. parent is the item ID of the parent item, or the empty string to create a new top-level item. index is an integer, or the value "end", specifying where in the list of parent's children to insert the new item. If index is less than or equal to zero, the new node is inserted at the beginning; if index is greater than or equal to the current number of children, it is inserted at the end. If id is specified, it is used as the item

Query or modify the options for the specified item. If no options are given, a dict with options/values for the item is returned. If option is specified then the value for that option is returned. Otherwise, sets the options to the corresponding values as given by kw.

Moves item to position index in parent's list of children. It is illegal to move an item under one of its descendants. If index is less than or equal to zero, item is moved to the beginning; if greater than or equal to the number of children, it is moved to the end. If item was detached it is reattached.

Returns the identifier of item's next sibling, or "" if item is the last child of its parent.

Returns the ID of the parent of item, or "" if item is at the top level of the hierarchy.

Returns the identifier of item's previous sibling, or "" if item is the first child of its parent.

An alias for `Treeview.move()`.

Ensure that item is visible. Sets all of item's ancestors open option to `True`, and scrolls the widget if necessary so that item is within the visible portion of the tree.

Returns a tuple of selected items. Changed in version 3.8: `selection()` no longer takes arguments. For changing the selection state use the following selection methods.

items becomes the new selection. Changed in version 3.6: items can be passed as separate arguments, not just as a single tuple.

Add item to the selection. Changed in version 3.6: items can be passed as separate arguments, not just as a single tuple.

Remove items from the selection. Changed in version 3.6: items can be passed as separate arguments, not just as a single tuple.

Toggle the selection state of each item in items. Changed in version 3.6: items can be passed as separate arguments, not just as a single tuple.

With one argument, returns a dictionary of column/value pairs for the specified item. With two arguments, returns the current value of the specified column. With three arguments, sets the value of given column in given item to the specified value.

Bind a callback for the given event sequence to the tag name. When an event is delivered to an item, the callbacks for each of the item's tags option are called.

Query or modify the options for the specified tag name. If `kw` is not given, returns a dict of the option settings for tag name. If `option` is specified, returns the value for that option for the specified tag name. Otherwise, sets the options to the corresponding values for the given tag name.

If item is specified, returns 1 or 0 depending on whether the specified item has the given tag name. Otherwise, returns a list of all items that have the specified tag. Availability: Tk 8.6

Query or modify horizontal position of the treeview.

Query or modify vertical position of the treeview.

Ttk Styling

Each widget in Tk is assigned a style, which specifies the set of elements making up the widget and how they are arranged, along with dynamic and default settings for element options. By default the style name is the same as the widget's class name, but it may be overridden by the widget's style option. If you don't know the class name of a widget, use the method `Misc.winfo_class()` (some widget. `winfo_class()`).

See also

Tcl'2004 conference presentation

This document explains how the theme engine works

`class tkinter.ttk.Style`

`configure(style, query_opt=None, **kw)`

`map(style, query_opt=None, **kw)`

lookup(style,option,state=None,default=None)
layout(style,layoutspec=None)
element_create(elementname,etype,*args,kw)**
border=padding
height=height
padding=padding
sticky=spec
width=width
padding=padding
margins=padding
width=width
height=height
element_names()
element_options(elementname)
theme_create(themename,parent=None,settings=None)
theme_settings(themename,settings)
theme_names()
theme_use(themename=None)

This class is used to manipulate the style database.`configure(style,query_opt=None,**kw)` Query or set the default value of the specified option(s) in style. Each key in kw is an option and each value is a string identifying the value for that option. For example, to change every default button to be a flat button with some padding and a different background color: `from tkinter import ttk; root = tkinter.Tk(); ttk.Style().configure("TButton",padding=6,relief="flat",background="#ccc"); btn = ttk.Button(t`

Query or set the default value of the specified option(s) in style. Each key in kw is an option and each value is a string identifying the value for that option. For example, to change every default button to be a flat button with some padding and a different background color: `from tkinter import ttk; root = tkinter.Tk(); ttk.Style().configure("TButton",padding=6,relief="flat",background="#ccc"); btn = ttk.Button(text="Sample"); btn.pack(); root.mainloop()`

Query or sets dynamic values of the specified option(s) in style. Each key in kw is an option and each value should be a list or a tuple (usually) containing statespecs grouped in tuples, lists, or some other preference. A statespec is a compound of one or more states and then a value. An example may make it more understandable: `import tkinter; from tkinter import ttk; root = tkinter.Tk(); style = ttk.Style(); style.map("C.TButton",foreground=[('pressed','red'),('active','blue')],background=[('pressed','!disabled','black'),('active','!disabled','black')])`

Returns the value specified for option in style. If state is specified, it is expected to be a sequence of one or more states. If the default argument is set, it is used as a fallback value in case no specification for option is found. To check what font a Button uses by default: `from tkinter import ttk; print(ttk.Style().lookup("TButton","font"))`

Define the widget layout for given style. If layouts spec is omitted, return the layout specification for given style. layouts spec, if specified, is expected to be a list or some other sequence type (excluding strings), where each item should be a tuple and the first item is the layout name and the second item should have the format described in Layouts. To understand the format, see the following example (it is not intended to do anything useful): `from tkinter import ttk; root = tkinter.Tk(); style =`

Create a new element in the current theme, of the given `type` which is expected to be either "image", "from" or "vsapi". The latter is only available in Tk 8.6 on Windows. If "image" is used, `args` should contain the default image name followed by `state spec/value` pairs (this is the `image spec`), and `kw` may have the following options: `border=padding` `padding` is a list of up to four integers, specifying the left, top, right, and bottom borders, respectively. `height=height` Specifies a minimum height for the element

`padding` is a list of up to four integers, specifying the left, top, right, and bottom borders, respectively.

Specifies a minimum height for the element. If less than zero, the base image's height is used as a default.

Specifies the element's interior padding. Defaults to border's value if not specified.

Specifies how the image is placed within the final parcel. `spec` contains zero or more characters "n", "s", "w", or "e".

Specifies a minimum width for the element. If less than zero, the base image's width is used as a default.

Specify the element's interior padding. `padding` is a list of up to four integers specifying the left, top, right and bottom padding quantities respectively. If fewer than four elements are specified, bottom defaults to top, right defaults to left, and top defaults to left. In other words, a list of three numbers specify the left, vertical, and right padding; a list of two numbers specify the horizontal and the vertical padding; a single number specifies the same padding all the way around the width

Specifies the element's exterior padding. `padding` is a list of up to four integers specifying the left, top, right and bottom padding quantities respectively. This option may not be mixed with any other options.

Specifies the width for the element. If this option is set then the Visual Styles API will not be queried for the recommended size of the part. If this option is set then `height` should also be set. The `width` and `height` options cannot be mixed with the `padding` or `margin` options.

Specifies the height of the element. See the comments for `width`.

Returns the list of elements defined in the current theme.

Returns the list of `elementname`'s options.

Create a new theme. It is an error if `themename` already exists. If `parent` is specified, the new theme will inherit styles, elements and layouts from the parent theme. If `settings` are present they are expected to have the same syntax used for `theme_settings()`.

Temporarily sets the current theme to `themename`, apply specified `settings` and then restore the previous theme. Each key in `settings` is a style and each value may contain the keys 'configure', 'map', 'layout' and 'element create' and they are expected to have the same format as specified by the methods `Style.configure()`, `Style.map()`, `Style.layout()` and `Style.element_create()` respectively. As an example, let's change the Combobox for the default theme a bit: `from tkinter import ttk; import tkinter; root = tkinter.Tk(); sty`

Returns a list of all known themes.

If `themename` is not given, returns the theme in use. Otherwise, sets the current theme to `themename`, refreshes all widgets and emits a `<<ThemeChanged>>` event.

Layouts

A layout can be just `None`, if it takes no options, or a dict of options specifying how to arrange the element. The layout mechanism uses a simplified version of the pack geometry manager: given an initial cavity, each element is allocated a parcel.

The valid options/values are:

side: whichside

sticky: nswe

unit: 0 or 1

children: [sublayout...]

Specifies which side of the cavity to place the element; one of top, right, bottom or left. If omitted, the element occupies the entire cavity.

Specifies where the element is placed inside its allocated parcel.

If set to 1, causes the element and all of its descendants to be treated as a single element for the purposes of `Widget.identify()` et al. It's used for things like scrollbar thumbs with grips.

Specifies a list of elements to place inside the element. Each element is a tuple (or other sequence type) where the first item is the layout name, and the other is a `Layout`.

IDLE — Python editor and shell

Source code: `Lib/idlelib/`

IDLE is Python's Integrated Development and Learning Environment.

IDLE has the following features:

- cross-platform: works mostly the same on Windows, Unix, and macOS
- Python shell window (interactive interpreter) with colorizing of code input, output, and error messages
- multi-window text editor with multiple undo, Python colorizing, smart indent, call tips, auto completion, and other features
- search within any window, replace within editor windows, and search through multiple files (grep)
- debugger with persistent breakpoints, stepping, and viewing of global and local namespaces
- configuration, browsers, and other dialogs

The IDLE application is implemented in the `idlelib` package.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements for optional modules](#).

Menus

IDLE has two main window types, the Shell window and the Editor window. It is possible to have multiple editor windows simultaneously. On Windows and Linux, each has its own top menu. Each menu documented below indicates which window type it is associated with.

Output windows, such as used for `Edit => Find in Files`, are a subtype of editor window. They currently have the same top menu but a different default title and context menu.

On macOS, there is one application menu. It dynamically changes according to the window currently selected. It has an IDLE menu, and some entries described below are moved around to conform to Apple guidelines.

File menu (Shell and Editor)

New File

Open...

Open Module...

Recent Files

Create a new file editing window.

Open an existing file with an Open dialog.

Open an existing module (searches `sys.path`).

Open a list of recent files. Click one to open it.

Module Browser

Path Browser

Save

Save As...

Save Copy As...

Print Window

Close Window

Exit IDLE

Show functions, classes, and methods in the current Editor file in a tree structure. In the shell, open a module first.

Show sys.path directories, modules, functions, classes and methods in a tree structure.

Save the current window to the associated file, if there is one. Windows that have been changed since being opened or last saved have a * before and after the window title. If there is no associated file, do Save As instead.

Save the current window with a Save As dialog. The file saved becomes the new associated file for the window. (If your file manager is set to hide extensions, the current extension will be omitted in the file name box. If the new filename has no '.', '.py' and '.txt' will be added for Python and text files, except that on macOS Aqua, '.py' is added for all files.)

Save the current window to different file without changing the associated file. (See Save As note above about filename extensions.)

Print the current window to the default printer.

Close the current window (if an unsaved editor, ask to save; if an unsaved Shell, ask to quit execution). Calling exit() or close() in the Shell window also closes Shell. If this is the only window, also exit IDLE.

Close all windows and quit IDLE (ask to save unsaved edit windows).

Edit menu (Shell and Editor)

Undo

Redo

Select All

Cut

Copy

Paste

Undo the last change to the current window. A maximum of 1000 changes may be undone.

Redo the last undone change to the current window.

Select the entire contents of the current window.

Copy selection into the system-wide clipboard; then delete the selection.

Copy selection into the system-wide clipboard.

Insert contents of the system-wide clipboard into the current window.

The clipboard functions are also available in context menus.

Find...

Find Again

Find Selection

Find in Files...

Replace...

Go to Line

Show Completions

Expand Word

Show Call Tip

Show Surrounding Paresns

Open a search dialog with many options

Repeat the last search, if there is one.

Search for the currently selected string, if there is one.

Open a file search dialog. Put results in a new output window.

Open a search-and-replace dialog.

Move the cursor to the beginning of the line requested and make that line visible. A request past the end of the file goes to the end. Clear any selection and update the line and column status.

Open a scrollable list allowing selection of existing names. See *Completions* in the Editing and Navigation section below.

Expand a prefix you have typed to match a full word in the same window; repeat to get a different expansion.

After an unclosed parenthesis for a function, open a small window with function parameter hints. See *Calltips* in the Editing and Navigation section below.

Highlight the surrounding parenthesis.

Format menu (Editor window only)

Format Paragraph

Indent Region

Dedent Region

Comment Out Region

Uncomment Region

Tabify Region

Untabify Region

Toggle Tabs

New Indent Width

Strip Trailing Whitespace

Rewrap the text block containing the text insert cursor. Avoid code lines. See *Format block* in the Editing and Navigation section below.

Shift selected lines right by the indent width (default 4 spaces).

Shift selected lines left by the indent width (default 4 spaces).

Insert ## in front of selected lines.

Remove leading # or ## from selected lines.

Turn leading stretches of spaces into tabs. (Note: We recommend using 4 space blocks to indent Python code.)

Turn all tabs into the correct number of spaces.

Open a dialog to switch between indenting with spaces and tabs.

Open a dialog to change indent width. The accepted default by the Python community is 4 spaces.

Remove trailing space and other whitespace characters after the last non-whitespace character of a line by applying `str.rstrip()` to each line, including lines within multiline strings. Except for Shell windows, remove extra newlines at the end of the file.

Run menu (Editor window only)

Run Module

Do Check Module. If no error, restart the shell to clean the environment, then execute the module. Output is displayed in the Shell window. Note that output requires use of `print` or `write`. When execution is complete, the Shell retains focus and displays a prompt. At this point, one may interactively explore the result of execution. This is similar to executing a file with `python -i file` at a command line.

Run... Customized

Same as Run Module, but run the module with customized settings. Command Line Argument extends `sys.argv` as if passed on a command line. The module can be run in the Shell without restarting.

Check Module

Check the syntax of the module currently open in the Editor window. If the module has not been saved IDLE will either prompt the user to save or autosave, as selected in the General tab of the Idle Settings dialog. If there is a syntax error, the approximate location is indicated in the Editor window.

Python Shell

Open or wake up the Python Shell window.

Shell menu (Shell window only)

View Last Restart

Restart Shell

Previous History

Next History

Interrupt Execution

Scroll the shell window to the last Shell restart.

Restart the shell to clean the environment and reset display and exception handling.

Cycle through earlier commands in history which match the current entry.

Cycle through later commands in history which match the current entry.

Stop a running program.

Debug menu (Shell window only)

Go to File/Line

Look on the current line, with the cursor, and the line above for a filename and line number. If found, open the file if not already open, and show the line. Use this to view source lines referenced in an exception traceback

and lines found by Find in Files. Also available in the context menu of the Shell window and Output windows.

Debugger (toggle)

Stack Viewer

Auto-open Stack Viewer

When activated, code entered in the Shell or run from an Editor will run under the debugger. In the Editor, breakpoints can be set with the context menu. This feature is still incomplete and somewhat experimental.

Show the stack traceback of the last exception in a tree widget, with access to locals and globals.

Toggle automatically opening the stack viewer on an unhandled exception.

Options menu (Shell and Editor)

Configure IDLE

Open a configuration dialog and change preferences for the following: fonts, indentation, keybindings, text color themes, startup windows and size, additional help sources, and extensions. On macOS, open the configuration dialog by selecting Preferences in the application menu. For more details, see [Setting preferences](#) under Help and preferences.

Most configuration options apply to all windows or all future windows. The option items below only apply to the active window.

Show/Hide Code Context (Editor Window only)

Show/Hide Line Numbers (Editor Window only)

Zoom/Restore Height

Open a pane at the top of the edit window which shows the block context of the code which has scrolled above the top of the window. See [Code Context](#) in the Editing and Navigation section below.

Open a column to the left of the edit window which shows the number of each line of text. The default is off, which may be changed in the preferences (see [Setting preferences](#)).

Toggles the window between normal size and maximum height. The initial size defaults to 40 lines by 80 chars unless changed on the General tab of the Configure IDLE dialog. The maximum height for a screen is determined by momentarily maximizing a window the first time one is zoomed on the screen. Changing screen settings may invalidate the saved height. This toggle has no effect when a window is maximized.

Window menu (Shell and Editor)

Lists the names of all open windows; select one to bring it to the foreground (deiconifying it if necessary).

Help menu (Shell and Editor)

About IDLE

IDLE Help

Python Docs

Turtle Demo

Display version, copyright, license, credits, and more.

Display this IDLE document, detailing the menu options, basic editing and navigation, and other tips.

Access local Python documentation, if installed, or start a web browser and open docs.python.org showing the latest Python documentation.

Run the `turtledemo` module with example Python code and turtle drawings.

Additional help sources may be added here with the Configure IDLE dialog under the General tab. See the `Help` subsection below for more on Help menu choices.

Context menus

Open a context menu by right-clicking in a window (Control-click on macOS). Context menus have the standard clipboard functions also on the Edit menu.

Cut

Copy

Paste

Copy selection into the system-wide clipboard; then delete the selection.

Copy selection into the system-wide clipboard.

Insert contents of the system-wide clipboard into the current window.

Editor windows also have breakpoint functions. Lines with a breakpoint set are specially marked. Breakpoints only have an effect when running under the debugger. Breakpoints for a file are saved in the `user's.idlerc` directory.

Set Breakpoint

Clear Breakpoint

Set a breakpoint on the current line.

Clear the breakpoint on that line.

Shell and Output windows also have the following.

Go to file/line

Same as in Debug menu.

The Shell window also has an output squeezing facility explained in the `Python Shell windows` subsection below.

Squeeze

If the cursor is over an output line, squeeze all the output between the code above and the prompt below down to a 'Squeezed text' label.

Editing and Navigation

Editor windows

IDLE may open editor windows when it starts, depending on settings and how you start IDLE. Thereafter, use the File menu. There can be only one open editor window for a given file.

The title bar contains the name of the file, the full path, and the version of Python and IDLE running the window. The status bar contains the line number ('Ln') and column number ('Col'). Line numbers start with 1; column numbers with 0.

IDLE assumes that files with a known `.py*` extension contain Python code and that other files do not. Run Python code with the Run menu.

Key bindings

The IDLE insertion cursor is a thin vertical bar between character positions. When characters are entered, the insertion cursor and everything to its right moves right one character and the new character is entered in the new space.

Several non-character keys move the cursor and possibly delete characters. Deletion does not put text on the clipboard, but IDLE has an undo list. Wherever this doc discusses keys, 'C' refers to the Control key on Windows and Unix and the Command key on macOS. (And all such discussions assume that the keys have not been re-bound to something else.)

- Arrow keys move the cursor one character or line.
- C-Left Arrow and C-Right Arrow moves left or right one word.
- Home and End go to the beginning or end of the line.
- Page Up and Page Down go up or down one screen.
- C-Home and C-End go to beginning or end of the file.
- Backspace and Del (or C-d) delete the previous or next character.
- C-Backspace and C-Del delete one word left or right.
- C-k deletes ('kills') everything to the right.

Standard keybindings (like C-cto copy and C-vto paste) may work. Keybindings are selected in the Configure IDLE dialog.

Automatic indentation

After a block-opening statement, the next line is indented by 4 spaces (in the Python Shell window by one tab). After certain keywords (break, return etc.) the next line is dedented. In leading indentation, Backspace deletes up to 4 spaces if they are there. Tab inserts spaces (in the Python Shell window one tab), number depends on Indent width. Currently, tabs are restricted to four spaces due to Tcl/Tk limitations.

See also the indent/dedent region commands on the Format menu.

Search and Replace

Any selection becomes a search target. However, only selections within a line work because searches are only performed within lines with the terminal newline removed. If ☒ Regular expression is checked, the target is interpreted according to the Python re module.

Completions

Completions are supplied, when requested and available, for module names, attributes of classes or functions, or filenames. Each request method displays a completion box with existing names. (See tab completions below for an exception.) For any box, change the name being completed and the item highlighted in the box by typing and deleting characters; by hitting Up, Down, Page Up, Page Down, Home, and End keys; and by a single click within the box. Close the box with Escape, Enter, and double Tab keys or clicks outside the box. A double click within the box selects and closes.

One way to open a box is to type a key character and wait for a predefined interval. This defaults to 2 seconds; customize it in the settings dialog. (To prevent auto popups, set the delay to a large number of milliseconds, such as 100000000.) For imported module names or class or function attributes, type '.'. For filenames in the root directory, type os.sep or os.altsep immediately after an opening quote. (On Windows, one can specify a drive first.) Move into subdirectories by typing a directory name and a separator.

Instead of waiting, or after a box is closed, open a completion box immediately with Show Completions on the Edit menu. The default hot key is C-space. If one types a prefix for the desired name before opening the box, the first match or near miss is made visible. The result is the same as if one enters a prefix after the box is

displayed. Show Completions after a quote completes filenames in the current directory instead of a root directory.

Hitting Tab after a prefix usually has the same effect as Show Completions. (With no prefix, it indents.) However, if there is only one match to the prefix, that match is immediately added to the editor text without opening a box.

Invoking 'Show Completions', or hitting Tab after a prefix, outside of a string and without a preceding '.' opens a box with keywords, builtin names, and available module-level names.

When editing code in an editor (as oppose to Shell), increase the available module-level names by running your code and not restarting the Shell thereafter. This is especially useful after adding imports at the top of a file. This also increases possible attribute completions.

Completion boxes initially exclude names beginning with '_' or, for modules, not included in '__all__'. The hidden names can be accessed by typing '_' after '.', either before or after the box is opened.

Calltips

A calltip is shown automatically when one types (after the name of an accessible function. A function name expression may include dots and subscripts. A calltip remains until it is clicked, the cursor is moved out of the argument area, or) is typed. Whenever the cursor is in the argument part of a definition, select Edit and "Show Call Tip" on the menu or enter its shortcut to display a calltip.

The calltip consists of the function's signature and docstring up to the latter's first blank line or the fifth non-blank line. (Some builtin functions lack an accessible signature.) A '/' or '*' in the signature indicates that the preceding or following arguments are passed by position or name (keyword) only. Details are subject to change.

In Shell, the accessible functions depends on what modules have been imported into the user process, including those imported by Idle itself, and which definitions have been run, all since the last restart.

For example, restart the Shell and enter `itertools.count()`. A calltip appears because Idle imports `itertools` into the user process for its own use. (This could change.) Enter `turtle.write()` and nothing appears. Idle does not itself import `turtle`. The menu entry and shortcut also do nothing. Enter `import turtle`. Thereafter, `turtle.write()` will display a calltip.

In an editor, import statements have no effect until one runs the file. One might want to run a file after writing import statements, after adding function definitions, or after opening an existing file.

Format block

Reformat Paragraph rewraps a block ('paragraph') of contiguous equally indented non-blank comments, a similar block of text within a multiline string, or a selected subset of either. If needed, add a blank line to separate string from code. Partial lines in a selection expand to complete lines. The resulting lines have the same indent as before but have maximum total length of N columns (characters). Change the default N of 72 on the Window tab of IDLE Settings.

Code Context

Within an editor window containing Python code, code context can be toggled in order to show or hide a pane at the top of the window. When shown, this pane freezes the opening lines for block code, such as those beginning with `class`, `def`, or `if` keywords, that would have otherwise scrolled out of view. The size of the pane will be expanded and contracted as needed to show the all current levels of context, up to the maximum number of lines defined in the Configure IDLE dialog (which defaults to 15). If there are no current context lines and the feature is toggled on, a single blank line will display. Clicking on a line in the context pane will move that line to the top of the editor.

The text and background colors for the context pane can be configured under the Highlights tab in the Configure IDLE dialog.

Shell window

In IDLE's Shell, enter, edit, and recall complete statements. (Most consoles and terminals only work with a single physical line at a time).

Submit a single-line statement for execution by hitting `Return` with the cursor anywhere on the line. If a line is extended with Backslash (`\`), the cursor must be on the last physical line. Submit a multi-line compound statement by entering a blank line after the statement.

When one pastes code into Shell, it is not compiled and possibly executed until one hits `Return`, as specified above. One may edit pasted code first. If one pastes more than one statement into Shell, the result will be a `SyntaxError` when multiple statements are compiled as if they were one.

Lines containing `RESTART` mean that the user execution process has been re-started. This occurs when the user execution process has crashed, when one requests a restart on the Shell menu, or when one runs code in an editor window.

The editing features described in previous subsections work when entering code interactively. IDLE's Shell window also responds to the following:

- `C`- attempts to interrupt statement execution (but may fail).
- `C-d` closes Shell if typed at `a>>>` prompt.
- `Alt-p` and `Alt-n` (`C-p` and `C-n` on macOS) retrieve to the current prompt the previous or next previously entered statement that matches anything already typed.
- `Return` while the cursor is on any previous statement appends the latter to anything already typed at the prompt.

Text colors

Idle defaults to black on white text, but colors text with special meanings. For the shell, these are shell output, shell error, user output, and user error. For Python code, at the shell prompt or in an editor, these are keywords, builtin class and function names, names following `class` and `def`, strings, and comments. For any text window, these are the cursor (when present), found text (when possible), and selected text.

IDLE also highlights the soft keywords `match`, `case`, and `_in` pattern-matching statements. However, this highlighting is not perfect and will be incorrect in some rare cases, including some `_s` in `case` patterns.

Text coloring is done in the background, so uncolorized text is occasionally visible. To change the color scheme, use the Configure IDLE dialog Highlighting tab. The marking of debugger breakpoint lines in the editor and text in popups and dialogs is not user-configurable.

Startup and Code Execution

Upon startup with the `-soption`, IDLE will execute the file referenced by the environment variables `IDLESTARTUP` or `PYTHONSTARTUP`. IDLE first checks for `IDLESTARTUP`; if `IDLESTARTUP` is present the file referenced is run. If `IDLESTARTUP` is not present, IDLE checks for `PYTHONSTARTUP`. Files referenced by these environment variables are convenient places to store functions that are used frequently from the IDLE shell, or for executing import statements to import common modules.

In addition, `Tk` also loads a startup file if it is present. Note that the `Tk` file is loaded unconditionally. This additional file is `Idle.py` and is looked for in the user's home directory. Statements in this file will be executed in the `Tk` namespace, so this file is not useful for importing functions to be used from IDLE's Python shell.

Command-line usage

IDLE can be invoked from the command line with various options. The general syntax is:

```
python -m idlelib [options] [file ...]
```

The following options are available:

-c<command>

Run the specified Python command in the shell window. For example, pass `-c"print('Hello,World!')"`. On Windows, the outer quotes must be double quotes as shown.

-d

Enable the debugger and open the shell window.

-e

Open an editor window.

-h

Print a help message with legal combinations of options and exit.

-i

Open a shell window.

-r<file>

Run the specified file in the shell window.

-s

Run the startup file (as defined by the environment variables `IDLESTARTUP` or `PYTHONSTARTUP`) before opening the shell window.

-t<title>

Set the title of the shell window.

-

Read and execute standard input in the shell window. This option must be the last one before any arguments.

If arguments are provided:

- If `-c`, `-r` or `-i` is used, all arguments are placed in `sys.argv[1:]`, and `sys.argv[0]` is set to `"-c"`, `-r` or `-i` respectively. No editor window is opened, even if that is the default set in the `OptionsDialog`.
- Otherwise, arguments are treated as files to be opened for editing, and `sys.argv` reflects the arguments passed to IDLE itself.

Startup failure

IDLE uses a socket to communicate between the IDLE GUI process and the user code execution process. A connection must be established whenever the Shell starts or restarts. (The latter is indicated by a divider line that says 'RESTART'). If the user process fails to connect to the GUI process, it usually displays a `TkError` box with a 'cannot connect' message that directs the user here. It then exits.

One specific connection failure on Unix systems results from misconfigured masquerading rules somewhere in a system's network setup. When IDLE is started from a terminal, one will see a message starting with `**Invalid host: .`. The valid value is `127.0.0.1(idlelib.rpc.LOCALHOST)`. One can diagnose with `htcpconnect -i rv127.0.0.16543` in one terminal window and `tcplisten <same args>` in another.

A common cause of failure is a user-written file with the same name as a standard library module, such as `random.py` and `tkinter.py`. When such a file is located in the same directory as a file that is about to be run, IDLE cannot import the `stdlib` file. The current fix is to rename the user file.

Though less common than in the past, an antivirus or firewall program may stop the connection. If the program cannot be taught to allow the connection, then it must be turned off for IDLE to work. It is safe to allow this internal connection because no data is visible on external ports. A similar problem is a network mis-configuration that blocks connections.

Python installation issues occasionally stop IDLE: multiple versions can clash, or a single installation might need admin access. If one undo the clash, or cannot or does not want to run as admin, it might be easiest to completely remove Python and start over.

A zombie pythonw.exe process could be a problem. On Windows, use Task Manager to check for one and stop it if there is. Sometimes a restart initiated by a program crash or Keyboard Interrupt (control-C) may fail to connect. Dismissing the error box or using Restart Shell on the Shell menu may fix a temporary problem.

When IDLE first starts, it attempts to read user configuration files in `~/.idlerc/` (`~` is one's home directory). If there is a problem, an error message should be displayed. Leaving aside random disk glitches, this can be prevented by never editing the files by hand. Instead, use the configuration dialog, under Options. Once there is an error in a user configuration file, the best solution may be to delete it and start over with the settings dialog.

If IDLE quits with no message, and it was not started from a console, try starting it from a console or terminal (`python-midlelib`) and see if this results in an error message.

On Unix-based systems with tcl/tk older than 8.6.11 (see About IDLE) certain characters of certain fonts can cause a tk failure with a message to the terminal. This can happen either if one starts IDLE to edit a file with such a character or later when entering such a character. If one cannot upgrade tcl/tk, then re-configure IDLE to use a font that works better.

Running user code

With rare exceptions, the result of executing Python code with IDLE is intended to be the same as executing the same code by the default method, directly with Python in a text-mode system console or terminal window. However, the different interface and operation occasionally affect visible results. For instance, `sys.modules` starts with more entries, and `threading.active_count()` returns 2 instead of 1.

By default, IDLE runs user code in a separate OS process rather than in the user interface process that runs the shell and editor. In the execution process, it replaces `sys.stdin`, `sys.stdout`, and `sys.stderr` with objects that get input from and send output to the Shell window. The original values stored in `sys.__stdin__`, `sys.__stdout__`, and `sys.__stderr__` are not touched, but may be `None`.

Sending print output from one process to a text widget in another is slower than printing to a system terminal in the same process. This has the most effect when printing multiple arguments, as the string for each argument, each separator, the newline are sent separately. For development, this is usually not a problem, but if one wants to print faster in IDLE, format and join together everything one wants displayed together and then print a single string. Both format strings and `str.join()` can help combine fields and lines.

IDLE's standard stream replacements are not inherited by subprocesses created in the execution process, whether directly by user code or by modules such as multiprocessing. If such subprocess use input from `sys.stdin` or print or write to `sys.stdout` or `sys.stderr`, IDLE should be started in a command line window. (On Windows, use `pythonorpyr` rather than `pythonworpyw`.) The secondary subprocess will then be attached to that window for input and output.

If `sys` is reset by user code, such as with `importlib.reload(sys)`, IDLE's changes are lost and input from the keyboard and output to the screen will not work correctly.

When Shell has the focus, it controls the keyboard and screen. This is normally transparent, but functions that directly access the keyboard and screen will not work. These include system-specific functions that determine whether a key has been pressed and if so, which.

The IDLE code running in the execution process adds frames to the call stack that would not be there otherwise. IDLE wraps `sys.getrecursionlimit` and `sys.setrecursionlimit` to reduce the effect of the additional stack frames.

When user code raises `SystemExit` either directly or by calling `sys.exit`, IDLE returns to a Shell prompt instead of exiting.

User output in Shell

When a program outputs text, the result is determined by the corresponding output device. When IDLE executes user code, `sys.stdout` and `sys.stderr` are connected to the display area of IDLE's Shell. Some of its features are inherited from the underlying Tk Text widget. Others are programmed additions. Where it matters, Shell is designed for development rather than production runs.

For instance, Shell never throws away output. A program that sends unlimited output to Shell will eventually fill memory, resulting in a memory error. In contrast, some system text windows only keep the last *n* lines of output. A Windows console, for instance, keeps a user-settable 1 to 9999 lines, with 300 the default.

A Tk Text widget, and hence IDLE's Shell, displays characters (codepoints) in the BMP (Basic Multilingual Plane) subset of Unicode. Which characters are displayed with a proper glyph and which with a replacement box depends on the operating system and installed fonts. Tab characters cause the following text to begin after the next tab stop. (They occur every 8 'characters'). Newline characters cause following text to appear on a new line. Other control characters are ignored or displayed as a space, box, or something else, depending on the operating system and font. (Moving the text cursor through such output with arrow keys may exhibit some surprising spacing behavior.)

```
>>> s = 'a\tb\ac<\x02><\r>\bc\nd' # Enter 22 chars.
>>> len(s)
14
>>> s # Display repr(s)
'a\tb\x07<\x02><\r>\x08c\nd'
>>> print(s, end='') # Display s as is.
# Result varies by OS and font. Try it.
```

The `repr` function is used for interactive echo of expression values. It returns an altered version of the input string in which control codes, some BMP codepoints, and all non-BMP codepoints are replaced with escape codes. As demonstrated above, it allows one to identify the characters in a string, regardless of how they are displayed.

Normal and error output are generally kept separate (on separate lines) from code input and each other. They each get different highlight colors.

For `SyntaxError` tracebacks, the normal '^' marking where the error was detected is replaced by coloring the text with an error highlight. When code run from a file causes other exceptions, one may right click on a traceback line to jump to the corresponding line in an IDLE editor. The file will be opened if necessary.

Shell has a special facility for squeezing output lines down to a 'Squeezed text' label. This is done automatically for output over *N* lines (*N* = 50 by default). *N* can be changed in the PyShell section of the General page of the Settings dialog. Output with fewer lines can be squeezed by right clicking on the output. This can be useful lines long enough to slow down scrolling.

Squeezed output is expanded in place by double-clicking the label. It can also be sent to the clipboard or a separate view window by right-clicking the label.

Developing tkinter applications

IDLE is intentionally different from standard Python in order to facilitate development of tkinter programs. Enter `import tkinter; root = tk.Tk()` in standard Python and nothing appears. Enter the same in IDLE and a tk window appears. In standard Python, one must also enter `root.update()` to see the window. IDLE does the equivalent in the background, about 20 times a second, which is about every 50 milliseconds. Next enter `b = tk.Button(root, text='button'); b.pack()`. Again, nothing visibly changes in standard Python until one

enters `root.update()`.

Most tkinter programs run `root.mainloop()`, which usually does not return until the tk app is destroyed. If the program is run with `python -i` or from an IDLE editor, a `>>>` shell prompt does not appear until `mainloop()` returns, at which time there is nothing left to interact with.

When running a tkinter program from an IDLE editor, one can comment out the `mainloop` call. One then gets a shell prompt immediately and can interact with the live application. One just has to remember to re-enable the `mainloop` call when running in standard Python.

Running without a subprocess

By default, IDLE executes user code in a separate subprocess via a socket, which uses the internal loopback interface. This connection is not externally visible and no data is sent to or received from the internet. If firewall software complains anyway, you can ignore it.

If the attempt to make the socket connection fails, IDLE will notify you. Such failures are sometimes transient, but if persistent, the problem may be either a firewall blocking the connection or misconfiguration of a particular system. Until the problem is fixed, one can run IDLE with the `-n` command line switch.

If IDLE is started with the `-n` command line switch it will run in a single process and will not create the subprocess which runs the RPC Python execution server. This can be useful if Python cannot create the subprocess or the RPC socket interface on your platform. However, in this mode user code is not isolated from IDLE itself. Also, the environment is not restarted when Run/Run Module (F5) is selected. If your code has been modified, you must `reload()` the affected modules and re-import any specific items (e.g. `from foo import baz`) if the changes are to take effect. For these reasons, it is preferable to run IDLE with the default subprocess if at all possible.

Deprecated since version 3.4.

Help and Preferences

Help sources

Help menu entry "IDLE Help" displays a formatted html version of the IDLE chapter of the Library Reference. The result, in a read-only tkinter text window, is close to what one sees in a web browser. Navigate through the text with a mousewheel, the scrollbar, or up and down arrow keys held down. Or click the TOC (Table of Contents) button and select a section header in the opened box.

Help menu entry "Python Docs" opens the extensive sources of help, including tutorials, available at docs.python.org/x.y, where 'x.y' is the currently running Python version. If your system has an off-line copy of the docs (this may be an installation option), that will be opened instead.

Selected URLs can be added or removed from the help menu at any time using the General tab of the Configure IDLE dialog.

Setting preferences

The font preferences, highlighting, keys, and general preferences can be changed via Configure IDLE on the Option menu. Non-default user settings are saved in `.idlercdirectory` in the user's home directory. Problems caused by bad user configuration files are solved by editing or deleting one or more of the files in `.idlerc`.

On the Font tab, see the text sample for the effect of font face and size on multiple characters in multiple languages. Edit the sample to add other characters of personal interest. Use the sample to select monospaced fonts. If particular characters have problems in Shell or an editor, add them to the top of the sample and try changing first size and then font.

On the Highlights and Keys tab, select a built-in or custom color theme and key set. To use a newer built-in color theme or key set with older IDLEs, save it as a new custom theme or key set and it will be accessible to older IDLEs.

IDLE on macOS

Under System Preferences: Dock, one can set “Prefer tabs when opening documents” to “Always”. This setting is not compatible with the tk/tkinter GUI framework used by IDLE, and it breaks a few IDLE features.

Extensions

IDLE contains an extension facility. Preferences for extensions can be changed with the Extensions tab of the preferences dialog. See the beginning of config-extensions.def in the idlelib directory for further information. The only current default extension is zzdummy, an example also used for testing.

idlelib — implementation of IDLE application

Source code: `Lib/idlelib`

The `Lib/idlelib` package implements the IDLE application. See the rest of this page for how to use IDLE.

The files in `idlelib` are described in `idlelib/README.txt`. Access it either in `idlelib` or click Help => About IDLE on the IDLE menu. This file also maps IDLE menu items to the code that implements the item. Except for files listed under ‘Startup’, the `idlelib` code is ‘private’ in sense that feature changes can be backported (see PEP 434).

turtle— Turtle graphics

Source code: `Lib/turtle.py`

Introduction

Turtle graphics is an implementation of the popular geometric drawing tools introduced in Logo, developed by Wally Feurzeig, Seymour Papert and Cynthia Solomon in 1967.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements for optional modules](#).

Get started

Imagine a robotic turtle starting at (0, 0) in the x-y plane. After `import turtle`, give it the command `turtle.forward(15)`, and it moves (on-screen!) 15 pixels in the direction it is facing, drawing a line as it moves. Give it the command `turtle.right(25)`, and it rotates in-place 25 degrees clockwise.

Turtle star

Turtle can draw intricate shapes using programs that repeat simple moves.

In Python, turtle graphics provides a representation of a physical “turtle” (a little robot with a pen) that draws on a sheet of paper on the floor.

It’s an effective and well-proven way for learners to encounter programming concepts and interaction with software, as it provides instant, visible feedback. It also provides convenient access to graphical output in general.

Turtle drawing was originally created as an educational tool, to be used by teachers in the classroom. For the programmer who needs to produce some graphical output it can be a way to do that without the overhead of introducing more complex or external libraries into their work.

Tutorial

New users should start here. In this tutorial we’ll explore some of the basics of turtle drawing.

Starting a turtle environment

In a Python shell, import all the objects of the `turtle` module:

```
from turtle import *
```

If you run into a `ModuleNotFoundError: No module named '_tkinter'`, you’ll have to install the `Tkinter` package on your system.

Basic drawing

Send the turtle forward 100 steps:

```
forward(100)
```

You should see (most likely, in a new window on your display) a line drawn by the turtle, heading East. Change the direction of the turtle, so that it turns 120 degrees left (anti-clockwise):

```
left(120)
```

Let's continue by drawing a triangle:

```
forward(100)
left(120)
forward(100)
```

Notice how the turtle, represented by an arrow, points in different directions as you steer it.

Experiment with those commands, and also with `backward()` and `right()`.

Pen control

Try changing the color - for example, `color('blue')`- and width of the line - for example, `width(3)`- and then drawing again.

You can also move the turtle around without drawing, by lifting up the pen: `up()` before moving. To start drawing again, use `down()`.

The turtle's position

Send your turtle back to its starting-point (useful if it has disappeared off-screen):

```
home()
```

The home position is at the center of the turtle's screen. If you ever need to know them, get the turtle's x-y coordinates with:

```
pos()
```

Home is at (0,0).

And after a while, it will probably help to clear the window so we can start anew:

```
clearscreen()
```

Making algorithmic patterns

Using loops, it's possible to build up geometric patterns:

```
for steps in range(100):
    for c in ('blue', 'red', 'green'):
        color(c)
        forward(steps)
        right(30)
```

- which of course, are limited only by the imagination!

Let's draw the star shape at the top of this page. We want red lines, filled in with yellow:

```
color('red')
fillcolor('yellow')
```

Just as `up()` and `down()` determine whether lines will be drawn, filling can be turned on and off:

```
begin_fill()
```

Next we'll create a loop:

```
while True:
    forward(200)
    left(170)
    if abs(pos()) < 1:
        break
```

`abs(pos()) < 1` is a good way to know when the turtle is back at its home position.

Finally, complete the filling:

```
end_fill()
```

(Note that filling only actually takes place when you give the `end_fill()` command.)

How to...

This section covers some typical turtle use-cases and approaches.

Get started as quickly as possible

One of the joys of turtle graphics is the immediate, visual feedback that's available from simple commands - it's an excellent way to introduce children to programming ideas, with a minimum of overhead (not just children, of course).

The turtle module makes this possible by exposing all its basic functionality as functions, available with `from turtle import *`. The turtle graphics tutorial covers this approach.

It's worth noting that many of the turtle commands also have even more terse equivalents, such as `fd()` for `forward()`. These are especially useful when working with learners for whom typing is not a skill.

You'll need to have the Tkinter package installed on your system for turtle graphics to work. Be warned that this is not always straightforward, so check this in advance if you're planning to use turtle graphics with a learner.

Automatically begin and end filling

Starting with Python 3.14, you can use the `fill()` context manager instead of `begin_fill()` and `end_fill()` to automatically begin and end fill. Here is an example:

```
with fill():
    for i in range(4):
        forward(100)
        right(90)
```

```
forward(200)
```

The code above is equivalent to:

```
begin_fill()
for i in range(4):
    forward(100)
    right(90)
end_fill()
```

```
forward(200)
```

Use the turtle module namespace

Using `from turtle import *` is convenient - but be warned that it imports a rather large collection of objects, and if you're doing anything but turtle graphics you run the risk of a name conflict (this becomes even more an issue if you're using turtle graphics in a script where other modules might be imported).

The solution is to use `import turtle` - `fd()` becomes `turtle.fd()`, `width()` becomes `turtle.width()` and so on. (If typing "turtle" over and over again becomes tedious, use for example `import turtle as t` instead.)

Use turtle graphics in a script

It's recommended to use the turtle module namespace as described immediately above, for example:

```
import turtle as t
from random import random

for i in range(100):
    steps = int(random() * 100)
```

```
angle = int(random() * 360)
t.right(angle)
t.fd(steps)
```

Another step is also required though - as soon as the script ends, Python will also close the turtle's window. Add:

```
t.mainloop()
```

to the end of the script. The script will now wait to be dismissed and will not exit until it is terminated, for example by closing the turtle graphics window.

Use object-oriented turtle graphics

See also

Explanation of the object-oriented interface

Other than for very basic introductory purposes, or for trying things out as quickly as possible, it's more usual and much more powerful to use the object-oriented approach to turtle graphics. For example, this allows multiple turtles on screen at once.

In this approach, the various turtle commands are methods of objects (mostly of `Turtle` objects). You can use the object-oriented approach in the shell, but it would be more typical in a Python script.

The example above then becomes:

```
from turtle import Turtle
from random import random

t = Turtle()
for i in range(100):
    steps = int(random() * 100)
    angle = int(random() * 360)
    t.right(angle)
    t.fd(steps)

t.screen.mainloop()
```

Note the last line. `t.screen` is an instance of the `Screen` that a `Turtle` instance exists on; it's created automatically along with the turtle.

The turtle's screen can be customised, for example:

```
t.screen.title('Object-oriented turtle demo')
t.screen.bgcolor("orange")
```

Turtle graphics reference

■ **Note:** In the following documentation the argument list for functions is given. Methods, of course, have the additional first argument `self` which is omitted here.

Turtle methods

Turtle motion

Move and draw

Tell Turtle's state

Setting and measurement

Pen control

Drawing state

Color control

Filling

More drawing control

Turtle state

Visibility

Appearance

Using events

Special Turtle methods

Move and draw `forward()`|`fd()`|`backward()`|`bk()`|`back()`|`right()`|`rt()`|`left()`|`lt()`|`goto()`|`setpos()`|`setposition()`|`teleport()`|`setx()`|`sety()`|`setheading()`|`seth()`|`home()`|`circle()`|`dot()`|`stamp()`|`clearstamp()`|`clearstamps()`|`undo()`|`speed()` Tell Turtle's state `position()`|`pos()`|`towards()`|`xcor()`|`ycor()`|`heading()`|`distance()` Setting and measurement `degrees()`|`radians()`

`forward()`|`fd()`|`backward()`|`bk()`|`back()`|`right()`|`rt()`|`left()`|`lt()`|`goto()`|`setpos()`|`setposition()`|`teleport()`|`setx()`|`sety()`|`setheading()`|`seth()`|`home()`|`circle()`|`dot()`|`stamp()`|`clearstamp()`|`clearstamps()`|`undo()`|`speed()`

`position()`|`pos()`|`towards()`|`xcor()`|`ycor()`|`heading()`|`distance()`

`degrees()`|`radians()`

Drawing state `pendown()`|`pd()`|`down()`|`penup()`|`pu()`|`up()`|`pensize()`|`width()`|`pen()`|`isdown()` Color control `color()`|`pencolor()`|`fillcolor()` Filling `filling()`|`fill()`|`begin_fill()`|`end_fill()` More drawing control `reset()`|`clear()`|`write()`

`pendown()`|`pd()`|`down()`|`penup()`|`pu()`|`up()`|`pensize()`|`width()`|`pen()`|`isdown()`

`color()`|`pencolor()`|`fillcolor()`

`filling()`|`fill()`|`begin_fill()`|`end_fill()`

`reset()`|`clear()`|`write()`

Visibility `showturtle()`|`st()`|`hideturtle()`|`ht()`|`isvisible()` Appearance `shape()`|`resizemode()`|`shapesize()`|`turtlesize()`|`shearfactor()`|`tiltangle()`|`tilt()`|`shapetransform()`|`get_shapepoly()`

`showturtle()`|`st()`|`hideturtle()`|`ht()`|`isvisible()`

`shape()`|`resizemode()`|`shapesize()`|`turtlesize()`|`shearfactor()`|`tiltangle()`|`tilt()`|`shapetransform()`|`get_shapepoly()`

`onclick()`|`onrelease()`|`ondrag()`

`poly()`|`begin_poly()`|`end_poly()`|`get_poly()`|`clone()`|`getturtle()`|`getpen()`|`getscreen()`|`setundobuffer()`|`undobufferentries()`

Methods of TurtleScreen/Screen

Window control

Animation control

Using screen events

Settings and special methods

Input methods

Methods specific to Screen

`bgcolor()`|`bgpic()`|`clearscreen()`|`resetscreen()`|`screensize()`|`setworldcoordinates()`

```
no_animation()delay()tracer()update()
listen()onkey()onkeyrelease()onkeypress()onclick()onscreenclick()ontimer()mainloop()done()
mode()colormode()getcanvas()getshapes()register_shape()addshape()turtles()window_height()window_width()
textinput()numinput()
bye()exitonclick()save()setup()title()
```

Methods of RawTurtle/Turtle and corresponding functions

Most of the examples in this section refer to a Turtle instance called `turtle`.

Turtle motion

`turtle.forward(distance)`

`turtle.fd(distance)`

Parameters:

Parameters: `distance`— a number (integer or float) Move the turtle forward by the specified `distance`, in the direction the turtle is headed. `>>>turtle.position()(0.00,0.00)>>>turtle.forward(25)>>>turtle.position()(25.00,0.00)>>>turtle.forward(-75)>>>turtle.position()(-50.00,0.00)`

`distance`— a number (integer or float)

`turtle.back(distance)`

`turtle.bk(distance)`

`turtle.backward(distance)`

Parameters:

Parameters: `distance`— a number Move the turtle backward by `distance`, opposite to the direction the turtle is headed. Do not change the turtle's heading. `>>>turtle.position()(0.00,0.00)>>>turtle.backward(30)>>>turtle.position()(-30.00,0.00)`

`distance`— a number

`turtle.right(angle)`

`turtle.rt(angle)`

Parameters:

Parameters: `angle`— a number (integer or float) Turn turtle right by `angle` units. (Units are by default degrees, but can be set via the `degrees()` and `radians()` functions.) Angle orientation depends on the turtle mode, see `mode()`. `>>>turtle.heading()22.0>>>turtle.right(45)>>>turtle.heading()337.0`

`angle`— a number (integer or float)

`turtle.left(angle)`

`turtle.lt(angle)`

Parameters:

Parameters: `angle`— a number (integer or float) Turn turtle left by `angle` units. (Units are by default degrees, but can be set via the `degrees()` and `radians()` functions.) Angle orientation depends on the turtle mode, see `mode()`. `>>>turtle.heading()22.0>>>turtle.left(45)>>>turtle.heading()67.0`

angle– a number (integer or float)

turtle.goto(x,y=None)

turtle.setpos(x,y=None)

turtle.setposition(x,y=None)

Parameters:

Parameters:x– a number or a pair/vector of numbersy– a number orNoneIfyisNone,xmust be a pair of coordinates or aVec2D(e.g. as returned bypos()).Move turtle to an absolute position. If the pen is down, draw line. Do not change the turtle's orientation.>>>tp=turtle.pos()>>>tp(0.00,0.00)>>>turtle.setpos(60,30)>>>turtle.p
os()(60.00,30.00)>>>turtle.setpos((20,80))>>>turtle.pos()(20.00,80.00)>>>turtle.setpos(tp)>>>turtle.pos()(0.00,
0.00)

x– a number or a pair/vector of numbersy– a number orNone

turtle.teleport(x,y=None,*,fill_gap=False)

Parameters:

Parameters:x– a number orNoney– a number orNonefill_gap– a booleanMove turtle to an absolute position. Unlike goto(x, y), a line will not be drawn. The turtle's orientation does not change. If currently filling, the polygon(s) teleported from will be filled after leaving, and filling will begin again after teleporting. This can be disabled with fill_gap=True, which makes the imaginary line traveled during teleporting act as a fill barrier like in goto(x, y).>>>tp=turtle.pos()>>>tp(0.00,0.00)>>>t

x– a number orNoney– a number orNonefill_gap– a boolean

turtle.setx(x)

Parameters:

Parameters:x– a number (integer or float)Set the turtle's first coordinate tox, leave second coordinate unchanged.>>>turtle.position()(0.00,240.00)>>>turtle.setx(10)>>>turtle.position()(10.00,240.00)

x– a number (integer or float)

turtle.sety(y)

Parameters:

Parameters:y– a number (integer or float)Set the turtle's second coordinate toy, leave first coordinate unchanged.>>>turtle.position()(0.00,40.00)>>>turtle.sety(-10)>>>turtle.position()(0.00,-10.00)

y– a number (integer or float)

turtle.setheading(to_angle)

turtle.seth(to_angle)

Parameters:

Parameters:to_angle– a number (integer or float)Set the orientation of the turtle toto_angle. Here are some common directions in degrees:standard modelogo mode0 - east0 - north90 - north90 - east180 - west180 - south270 - south270 - west>>>turtle.setheading(90)>>>turtle.heading()90.0

to_angle– a number (integer or float)

turtle.home()

Move turtle to the origin – coordinates (0,0) – and set its heading to its start-orientation (which depends on the mode, seemode()).>>>turtle.heading()90.0>>>turtle.position()(0.00,-10.00)>>>turtle.home()>>>turtle.position()(0.00,0.00)>>>turtle.heading()0.0

turtle.circle(radius,extent=None,steps=None)

Parameters:

Parameters:radius– a numberextent– a number (orNone)steps– an integer (orNone)Draw a circle with givenradius. The center isradiusunits left of the turtle;extent– an angle – determines which part of the circle is drawn. Ifextentis not given, draw the entire circle. Ifextentis not a full circle, one endpoint of the arc is the current pen position. Draw the arc in counterclockwise direction ifradiusis positive, otherwise in clockwise direction. Finally the direction of the turtle is changed by

radius– a numberextent– a number (orNone)steps– an integer (orNone)

turtle.dot()

turtle.dot(size)

turtle.dot(color,/)

turtle.dot(size,color,/)

turtle.dot(size,r,g,b,/)

Parameters:

Parameters:size– an integer ≥ 1 (if given)color– a colorstring or a numeric color tupleDraw a circular dot with diametersize, usingcolor. Ifsizeis not given, the maximum ofpensize+4and2*pensizeis used.>>>turtle.home()>>>turtle.dot()>>>turtle.fd(50);turtle.dot(20,"blue");turtle.fd(50)>>>turtle.position()(100.00,-0.00)>>>turtle.heading()0.0

size– an integer ≥ 1 (if given)color– a colorstring or a numeric color tuple

turtle.stamp()

Stamp a copy of the turtle shape onto the canvas at the current turtle position. Return a stamp_id for that stamp, which can be used to delete it by callingclearstamp(stamp_id).>>>turtle.color("blue")>>>stamp_id=turtle.stamp()>>>turtle.fd(50)

turtle.clearstamp(stampid)

Parameters:

Parameters:stampid– an integer, must be return value of previousstamp()callDelete stamp with givenstampid.>>>turtle.position()(150.00,-0.00)>>>turtle.color("blue")>>>astamp=turtle.stamp()>>>turtle.fd(50)>>>turtle.position()(200.00,-0.00)>>>turtle.clearstamp(astamp)>>>turtle.position()(200.00,-0.00)

stampid– an integer, must be return value of previousstamp()call

turtle.clearstamps(n=None)

Parameters:

Parameters:n– an integer (orNone)Delete all or first/lastnof turtle's stamps. IfnisNone, delete all stamps, ifn> 0 delete firstnstamps, else ifn< 0 delete lastnstamps.>>>foriinrange(8):...unused_stamp_id=turtle.stamp()...turtle.f d(30)>>>turtle.clearstamps(2)>>>turtle.clearstamps(-2)>>>turtle.clearstamps()

n– an integer (orNone)

turtle.undo()

Undo (repeatedly) the last turtle action(s). Number of available undo actions is determined by the size of the undobuffer.>>>foriinrange(4):...turtle.fd(50);turtle.lt(80)...>>>foriinrange(8):...turtle.undo()

turtle.speed(speed=None)

Parameters:

Parameters: speed— an integer in the range 0..10 or a speedstring (see below) Set the turtle's speed to an integer value in the range 0..10. If no argument is given, return current speed. If input is a number greater than 10 or smaller than 0.5, speed is set to 0. Speedstrings are mapped to speedvalues as follows: "fastest": 0 "fast": 10 "normal": 6 "slow": 3 "slowest": 1 Speeds from 1 to 10 enforce increasingly faster animation of line drawing and turtle turning. Attention: speed= 0 means that no ani

speed— an integer in the range 0..10 or a speedstring (see below)

Tell Turtle's state

turtle.position()

turtle.pos()

Return the turtle's current location (x,y) (as a Vec2D vector).>>>turtle.pos()(440.00,-0.00)

turtle.towards(x,y=None)

Parameters:

Parameters: x— a number or a pair/vector of numbers or a turtle instance y— a number if x is a number, else None Return the angle between the line from turtle position to position specified by (x,y), the vector or the other turtle. This depends on the turtle's start orientation which depends on the mode - "standard"/"world" or "logo".>>>turtle.goto(10,10)>>>turtle.towards(0,0)225.0

x— a number or a pair/vector of numbers or a turtle instance y— a number if x is a number, else None

turtle.xcor()

Return the turtle's x coordinate.>>>turtle.home()>>>turtle.left(50)>>>turtle.forward(100)>>>turtle.pos()(64.28,76.60)>>>print(round(turtle.xcor(),5))64.27876

turtle.ycor()

Return the turtle's y coordinate.>>>turtle.home()>>>turtle.left(60)>>>turtle.forward(100)>>>print(turtle.pos())(50.00,86.60)>>>print(round(turtle.ycor(),5))86.60254

turtle.heading()

Return the turtle's current heading (value depends on the turtle mode, see mode()).>>>turtle.home()>>>turtle.left(67)>>>turtle.heading()67.0

turtle.distance(x,y=None)

Parameters:

Parameters: x— a number or a pair/vector of numbers or a turtle instance y— a number if x is a number, else None Return the distance from the turtle to (x,y), the given vector, or the given other turtle, in turtle step units.>>>turtle.home()>>>turtle.distance(30,40)50.0>>>turtle.distance((30,40))50.0>>>joe=Turtle()>>>joe.forward(77)>>>turtle.distance(joe)77.0

x— a number or a pair/vector of numbers or a turtle instance y— a number if x is a number, else None

Settings for measurement

turtle.degrees(fullcircle=360.0)

Parameters:

Parameters: fullcircle— a number Set angle measurement units, i.e. set number of "degrees" for a full circle. Default value is 360 degrees.>>>turtle.home()>>>turtle.left(90)>>>turtle.heading()90.0>>># Change angle measurement unit to grad (also known as gon,>>># grade, or gradian and equals 1/100-th of the right angle.)>>>turtle.degrees(400.0)>>>turtle.heading()100.0>>>turtle.degrees(360)>>>turtle.heading()90.0

fullcircle– a number

turtle.radians()

Set the angle measurement units to radians. Equivalent to `degrees(2*math.pi)`.
`>>> turtle.home()>>> turtle.left(90)>>> turtle.heading()90.0>>> turtle.radians()>>> turtle.heading()1.5707963267948966`

Pen control

Drawing state

turtle.pendown()

turtle.pd()

turtle.down()

Pull the pen down – drawing when moving.

turtle.penup()

turtle.pu()

turtle.up()

Pull the pen up – no drawing when moving.

turtle.pensize(width=None)

turtle.width(width=None)

Parameters:

Parameters: `width`– a positive number Set the line thickness to `width` or return it. If `resizemode` is set to “auto” and `turtle.shape` is a polygon, that polygon is drawn with the same line thickness. If no argument is given, the current pensize is returned.
`>>> turtle.pensize(1)>>> turtle.pensize(10)` # from here on lines of width 10 are drawn

`width`– a positive number

turtle.pen(pen=None, **pendict)

Parameters:

Parameters: `pen`– a dictionary with some or all of the below listed keys `pendict`– one or more keyword-arguments with the below listed keys as keywords Return or set the pen’s attributes in a “pen-dictionary” with the following key/value pairs: “shown”: True/False “pendown”: True/False “pencolor”: color-string or color-tuple “fillcolor”: color-string or color-tuple “pensize”: positive number “speed”: number in range 0..10 “resizemode”: “auto” or “user” or “noresize” “stretchfactor”: (positive number, positive number)

`pen`– a dictionary with some or all of the below listed keys `pendict`– one or more keyword-arguments with the below listed keys as keywords

turtle.isdown()

Return True if pen is down, False if it's up.
`>>> turtle.penup()>>> turtle.isdown()False>>> turtle.pendown()>>> turtle.isdown()True`

Color control

turtle.pencolor()

turtle.pencolor(color, l)

turtle.pencolor(r, g, b, l)

pencolor()

pencolor(colorstring)

pencolor((r,g,b))

pencolor(r,g,b)

Return or set the pencolor. Four input formats are allowed: **pencolor()** Return the current pencolor as color specification string or as a tuple (see example). May be used as input to another color/pencolor/fillcolor/bgcolor call. **pencolor(colorstring)** Set pencolor to colorstring, which is a Tk color specification string, such as "red", "yellow", or "#33cc8c". **pencolor((r,g,b))** Set pencolor to the RGB color represented by the tuple of r, g, and b. Each of r, g, and b must be in the range 0..colormode, where color

Return the current pencolor as color specification string or as a tuple (see example). May be used as input to another color/pencolor/fillcolor/bgcolor call.

Set pencolor to colorstring, which is a Tk color specification string, such as "red", "yellow", or "#33cc8c".

Set pencolor to the RGB color represented by the tuple of r, g, and b. Each of r, g, and b must be in the range 0..colormode, where colormode is either 1.0 or 255 (see colormode()).

Set pencolor to the RGB color represented by r, g, and b. Each of r, g, and b must be in the range 0..colormode.

turtle.fillcolor()

turtle.fillcolor(color,/)

turtle.fillcolor(r,g,b,/)

fillcolor()

fillcolor(colorstring)

fillcolor((r,g,b))

fillcolor(r,g,b)

Return or set the fillcolor. Four input formats are allowed: **fillcolor()** Return the current fillcolor as color specification string, possibly in tuple format (see example). May be used as input to another color/pencolor/fillcolor/bgcolor call. **fillcolor(colorstring)** Set fillcolor to colorstring, which is a Tk color specification string, such as "red", "yellow", or "#33cc8c". **fillcolor((r,g,b))** Set fillcolor to the RGB color represented by the tuple of r, g, and b. Each of r, g, and b must be in the range 0..col

Return the current fillcolor as color specification string, possibly in tuple format (see example). May be used as input to another color/pencolor/fillcolor/bgcolor call.

Set fillcolor to colorstring, which is a Tk color specification string, such as "red", "yellow", or "#33cc8c".

Set fillcolor to the RGB color represented by the tuple of r, g, and b. Each of r, g, and b must be in the range 0..colormode, where colormode is either 1.0 or 255 (see colormode()).

Set fillcolor to the RGB color represented by r, g, and b. Each of r, g, and b must be in the range 0..colormode.

turtle.color()

turtle.color(color,/)

turtle.color(r,g,b,/)

turtle.color(pencolor,fillcolor,/)

color()

color(colorstring),color((r,g,b)),color(r,g,b)

color(colorstring1,colorstring2),color((r1,g1,b1),(r2,g2,b2))

Return or set pencolor and fillcolor. Several input formats are allowed. They use 0 to 3 arguments as follows: `color()` Return the current pencolor and the current fillcolor as a pair of color specification strings or tuples as returned by `pencolor()` and `fillcolor()`. `color(colorstring)`, `color((r,g,b))`, `color(r,g,b)` Inputs as in `pencolor()`, set both, fillcolor and pencolor, to the given value. `color(colorstring1,colorstring2)`, `color((r1,g1,b1),(r2,g2,b2))` Equivalent to `pencolor(colorstring1)` and `fillcolor(colorst`

Return the current pencolor and the current fillcolor as a pair of color specification strings or tuples as returned by `pencolor()` and `fillcolor()`.

Inputs as in `pencolor()`, set both, fillcolor and pencolor, to the given value.

Equivalent to `pencolor(colorstring1)` and `fillcolor(colorstring2)` and analogously if the other input format is used.

See also: Screen method `colormode()`.

Filling

`turtle.filling()`

Return fillstate (True if filling, False else).
`>>> turtle.begin_fill() >>> if turtle.filling(): ... turtle.pensize(5) ... else: ... turtle.pensize(3)`

`turtle.fill()`

Fill the shape drawn in the with `turtle.fill(): block`.
`>>> turtle.color("black", "red") >>> with turtle.fill(): ... turtle.circle(80)` Using `fill()` is equivalent to adding the `begin_fill()` before the fill-block and `end_fill()` after the fill-block.
`>>> turtle.color("black", "red") >>> turtle.begin_fill() >>> turtle.circle(80) >>> turtle.end_fill()` Added in version 3.14.

`turtle.begin_fill()`

To be called just before drawing a shape to be filled.

`turtle.end_fill()`

Fill the shape drawn after the last call to `begin_fill()`. Whether or not overlap regions for self-intersecting polygons or multiple shapes are filled depends on the operating system graphics, type of overlap, and number of overlaps. For example, the Turtle star above may be either all yellow or have some white regions.
`>>> turtle.color("black", "red") >>> turtle.begin_fill() >>> turtle.circle(80) >>> turtle.end_fill()`

More drawing control

`turtle.reset()`

Delete the turtle's drawings from the screen, re-center the turtle and set variables to the default values.
`>>> turtle.goto(0,-22) >>> turtle.left(100) >>> turtle.position()(0.00,-22.00) >>> turtle.heading()100.0 >>> turtle.reset() >>> turtle.position()(0.00,0.00) >>> turtle.heading()0.0`

`turtle.clear()`

Delete the turtle's drawings from the screen. Do not move turtle. State and position of the turtle as well as drawings of other turtles are not affected.

`turtle.write(arg, move=False, align='left', font=('Arial', 8, 'normal'))`

Parameters:

Parameters: `arg`— object to be written to the TurtleScreen `move`— True/False `align`— one of the strings "left", "center" or "right" `font`— a triple (fontname, fontsize, fonttype) Write text - the string representation of `arg` - at the current turtle position according to `align` ("left", "center" or "right") and with the given font. If `move` is true, the pen is moved to the bottom-right corner of the text. By default, `move` is False.
`>>> turtle.write("Home =`

```
",True,align="center")>>>turtle.write((0,0),True)
```

arg— object to be written to the TurtleScreen
move— True/False
align— one of the strings “left”, “center” or “right”
font— a triple (fontname, fontsize, fonttype)

Turtle state

Visibility

turtle.hideturtle()

turtle.ht()

Make the turtle invisible. It's a good idea to do this while you're in the middle of doing some complex drawing, because hiding the turtle speeds up the drawing observably.>>>turtle.hideturtle()

turtle.showturtle()

turtle.st()

Make the turtle visible.>>>turtle.showturtle()

turtle.isvisible()

Return True if the Turtle is shown, False if it's hidden.>>>turtle.hideturtle()>>>turtle.isvisible()False>>>turtle.showturtle()>>>turtle.isvisible()True

Appearance

turtle.shape(name=None)

Parameters:

Parameters: name— a string which is a valid shapename
Set turtle shape to shape with given name or, if name is not given, return name of current shape. Shape with name must exist in the TurtleScreen's shape dictionary. Initially there are the following polygon shapes: “arrow”, “turtle”, “circle”, “square”, “triangle”, “classic”. To learn about how to deal with shapes see Screen
method register_shape().>>>turtle.shape('classic')>>>turtle.shape("turtle")>>>turtle.shape() 'turtle'

name— a string which is a valid shapename

turtle.resizemode(rmode=None)

Parameters:

Parameters: rmode— one of the strings “auto”, “user”, “noresize”
Set resizemode to one of the values: “auto”, “user”, “noresize”. If rmode is not given, return current resizemode. Different resizemodes have the following effects: “auto”: adapts the appearance of the turtle corresponding to the value of pensize. “user”: adapts the appearance of the turtle according to the values of stretchfactor and outlinewidth (outline), which are set by shape_size(). “noresize”: no adaption of the turtle's appearance

rmode— one of the strings “auto”, “user”, “noresize”

turtle.shapesize(stretch_wid=None, stretch_len=None, outline=None)

turtle.turtlesize(stretch_wid=None, stretch_len=None, outline=None)

Parameters:

Parameters: stretch_wid— positive number
stretch_len— positive number
outline— positive number
Return or set the pen's attributes x/y-stretchfactors and/or outline. Set resizemode to “user”. If and only if resizemode is set to “user”, the turtle will be displayed stretched according to its stretchfactors: stretch_wid is stretchfactor perpendicular to its orientation, stretch_len is stretchfactor in direction of its orientation, outline determines the

width of the shape's outline.>>>turtle.shapesize()(1.

stretch_wid– positive numberstretch_len– positive numberoutline– positive number

turtle.shearfactor(shear=None)

Parameters:

Parameters:shear– number (optional)Set or return the current shearfactor. Shear the turtleshape according to the given shearfactor shear, which is the tangent of the shear angle. Donotchange the turtle's heading (direction of movement). If shear is not given: return the current shearfactor, i. e. the tangent of the shear angle, by which lines parallel to the heading of the turtle are

sheared.>>>turtle.shape("circle")>>>turtle.shapesize(5,2)>>>turtle.shearfactor(0.5)>>>turtle.shearfactor()0.5

shear– number (optional)

turtle.tilt(angle)

Parameters:

Parameters:angle– a numberRotate the turtleshape byanglefrom its current tilt-angle, but donotchange the turtle's heading (direction of movement).>>>turtle.reset()>>>turtle.shape("circle")>>>turtle.shapesize(5,2)>>>turtle.tilt(30)>>>turtle.fd(50)>>>turtle.tilt(30)>>>turtle.fd(50)

angle– a number

turtle.tiltangle(angle=None)

Parameters:

Parameters:angle– a number (optional)Set or return the current tilt-angle. If angle is given, rotate the turtleshape to point in the direction specified by angle, regardless of its current tilt-angle. Donotchange the turtle's heading (direction of movement). If angle is not given: return the current tilt-angle, i. e. the angle between the orientation of the turtleshape and the heading of the turtle (its direction of movement).>>>turtle.reset()>>>turtle.shape("circle")>>>turtle.shapesize(5,2)>>>t

angle– a number (optional)

turtle.shapetransform(t11=None,t12=None,t21=None,t22=None)

Parameters:

Parameters:t11– a number (optional)t12– a number (optional)t21– a number (optional)t22– a number (optional)Set or return the current transformation matrix of the turtle shape.If none of the matrix elements are given, return the transformation matrix as a tuple of 4 elements. Otherwise set the given elements and transform the turtleshape according to the matrix consisting of first row t11, t12 and second row t21, t22. The determinant $t11 * t22 - t12 * t21$ must not be zero, otherwise an error is r

t11– a number (optional)t12– a number (optional)t21– a number (optional)t22– a number (optional)

turtle.get_shapepoly()

Return the current shape polygon as tuple of coordinate pairs. This can be used to define a new shape or components of a compound

shape.>>>turtle.shape("square")>>>turtle.shapetransform(4,-1,0,2)>>>turtle.get_shapepoly()((50, -20), (30, 20), (-50, 20), (-30, -20))

Using events

turtle.onclick(fun,btn=1,add=None)

Parameters:

Parameters:fun– a function with two arguments which will be called with the coordinates of the clicked point on the canvasbtn– number of the mouse-button, defaults to 1 (left mouse button)add–TrueorFalse– ifTrue, a new binding will be added, otherwise it will replace a former bindingBindfunto mouse-click events on this turtle. IffunisNone, existing bindings are removed. Example for the anonymous turtle, i.e. the procedural way:>>>defturn(x,y):...left(180)...>>>onclick(turn)# Now clicking into

fun– a function with two arguments which will be called with the coordinates of the clicked point on the canvasbtn– number of the mouse-button, defaults to 1 (left mouse button)add–TrueorFalse– ifTrue, a new binding will be added, otherwise it will replace a former binding

turtle.onrelease(fun,btn=1,add=None)

Parameters:

Parameters:fun– a function with two arguments which will be called with the coordinates of the clicked point on the canvasbtn– number of the mouse-button, defaults to 1 (left mouse button)add–TrueorFalse– ifTrue, a new binding will be added, otherwise it will replace a former bindingBindfunto mouse-button-release events on this turtle. IffunisNone, existing bindings are removed.>>>classMyTurtle(Turtle):...defglow(self,x,y):...self.fillcolor("red")...defunglow(self,x,y):...self.fillcolor("")...>

fun– a function with two arguments which will be called with the coordinates of the clicked point on the canvasbtn– number of the mouse-button, defaults to 1 (left mouse button)add–TrueorFalse– ifTrue, a new binding will be added, otherwise it will replace a former binding

turtle.ondrag(fun,btn=1,add=None)

Parameters:

Parameters:fun– a function with two arguments which will be called with the coordinates of the clicked point on the canvasbtn– number of the mouse-button, defaults to 1 (left mouse button)add–TrueorFalse– ifTrue, a new binding will be added, otherwise it will replace a former bindingBindfunto mouse-move events on this turtle. IffunisNone, existing bindings are removed.Remark: Every sequence of mouse-move-events on a turtle is preceded by a mouse-click event on that turtle.>>>turtle.ondrag(turtl

fun– a function with two arguments which will be called with the coordinates of the clicked point on the canvasbtn– number of the mouse-button, defaults to 1 (left mouse button)add–TrueorFalse– ifTrue, a new binding will be added, otherwise it will replace a former binding

Special Turtle methods

turtle.poly()

Record the vertices of a polygon drawn in thewithturtle.poly():block. The first and last vertices will be connected.>>>withturtle.poly():...turtle.forward(100)...turtle.right(60)...turtle.forward(100)Added in version 3.14.

turtle.begin_poly()

Start recording the vertices of a polygon. Current turtle position is first vertex of polygon.

turtle.end_poly()

Stop recording the vertices of a polygon. Current turtle position is last vertex of polygon. This will be connected with the first vertex.

turtle.get_poly()

Return the last recorded polygon.>>>turtle.home()>>>turtle.begin_poly()>>>turtle.fd(100)>>>turtle.left(20)>>>turtle.fd(30)>>>turtle.left(60)>>>turtle.fd(50)>>>turtle.end_poly()>>>p=turtle.get_poly()>>>register_shape("myFavouriteShape",p)

turtle.clone()

Create and return a clone of the turtle with same position, heading and turtle properties.>>>mick=Turtle()>>>joe=mick.clone()

turtle.getturtle()

turtle.getpen()

Return the Turtle object itself. Only reasonable use: as a function to return the “anonymous turtle”:>>>pet=getturtle()>>>pet.fd(50)>>>pet<turtle.Turtle object at 0x...>

turtle.getscreen()

Return the TurtleScreen object the turtle is drawing on. TurtleScreen methods can then be called for that object.>>>ts=turtle.getscreen()>>>ts<turtle._Screen object at 0x...>>>ts.bgcolor("pink")

turtle.setundobuffer(size)

Parameters:

Parameters: size— an integer or None Set or disable undobuffer. If size is an integer, an empty undobuffer of given size is installed. size gives the maximum number of turtle actions that can be undone by the undo() method/function. If size is None, the undobuffer is disabled.>>>turtle.setundobuffer(42)

size— an integer or None

turtle.undobufferentries()

Return number of entries in the undobuffer.>>>while undobufferentries():...undo()

Compound shapes

To use compound turtle shapes, which consist of several polygons of different color, you must use the helper class Shape explicitly as described below:

- Create an empty Shape object of type “compound”.
- Add as many components to this object as desired, using the addcomponent() method. For example:>>>s=Shape("compound")>>>poly1=((0,0),(10,-5),(0,10),(-10,-5))>>>s.addcomponent(poly1,"red","blue")>>>poly2=((0,0),(10,-5),(-10,-5))>>>s.addcomponent(poly2,"blue","red")
- Now add the Shape to the Screen's shapelist and use it:>>>register_shape("myshape",s)>>>shape("myshape")

■ Nota: NoteTheShape class is used internally by the register_shape() method in different ways. The application programmer has to deal with the Shape class only when using compound shapes like shown above!

Methods of TurtleScreen/Screen and corresponding functions

Most of the examples in this section refer to a TurtleScreen instance called screen.

Window control

turtle.bgcolor()

turtle.bgcolor(color, /)

turtle.bgcolor(r,g,b, /)

bgcolor()

bgcolor(colorstring)

bgcolor((r,g,b))

bgcolor(r,g,b)

Return or set the background color of the TurtleScreen. Four input formats are allowed: `bgcolor()` Return the current background color as color specification string or as a tuple (see example). May be used as input to another `color/pencolor/fillcolor/bgcolor` call. `bgcolor(colorstring)` Set the background color to `colorstring`, which is a Tk color specification string, such as "red", "yellow", or "#33cc8c". `bgcolor((r,g,b))` Set the background color to the RGB color represented by the tuple of `r`, `g`, and `b`. Each of

Return the current background color as color specification string or as a tuple (see example). May be used as input to another `color/pencolor/fillcolor/bgcolor` call.

Set the background color to `colorstring`, which is a Tk color specification string, such as "red", "yellow", or "#33cc8c".

Set the background color to the RGB color represented by the tuple of `r`, `g`, and `b`. Each of `r`, `g`, and `b` must be in the range 0..`colormode`, where `colormode` is either 1.0 or 255 (see `colormode()`).

Set the background color to the RGB color represented by `r`, `g`, and `b`. Each of `r`, `g`, and `b` must be in the range 0..`colormode`.

turtle.bgpic(picname=None)

Parameters:

Parameters: `picname`— a string, name of an image file (PNG, GIF, PGM, and PPM) or "nopic", or `None` Set background image or return name of current background image. If `picname` is a filename, set the corresponding image as background. If `picname` is "nopic", delete background image, if present. If `picname` is `None`, return the filename of the current

background image. >>> `screen.bgpic('nopic')` >>> `screen.bgpic("landscape.gif")` >>> `screen.bgpic()` "landscape.gif"

`picname`— a string, name of an image file (PNG, GIF, PGM, and PPM) or "nopic", or `None`

turtle.clear()

Note This TurtleScreen method is available as a global function only under the name `clearscreen`. The global function `clear` is a different one derived from the Turtle method `clear`.

turtle.clearscreen()

Delete all drawings and all turtles from the TurtleScreen. Reset the now empty TurtleScreen to its initial state: white background, no background image, no event bindings and tracing on.

turtle.reset()

Note This TurtleScreen method is available as a global function only under the name `resetscreen`. The global function `reset` is another one derived from the Turtle method `reset`.

turtle.resetscreen()

Reset all Turtles on the Screen to their initial state.

turtle.screensize(canvwidth=None, canvheight=None, bg=None)

Parameters:

Parameters: `canvwidth`— positive integer, new width of canvas in pixels `canvheight`— positive integer, new height of canvas in pixels `bg`— colorstring or color-tuple, new background color If no arguments are given, return current (`canvaswidth`, `canvasheight`). Else resize the canvas the turtles are drawing on. Do not alter the drawing window. To observe hidden parts of the canvas, use the scrollbars. With this method, one can make visible those parts of a drawing which were outside the canvas before.>

`canvwidth`— positive integer, new width of canvas in pixels `canvheight`— positive integer, new height of canvas in pixels `bg`— colorstring or color-tuple, new background color

turtle.setworldcoordinates(llx,lly,urx,ury)

Parameters:

Parameters:llx– a number, x-coordinate of lower left corner of canvaslly– a number, y-coordinate of lower left corner of canvasurx– a number, x-coordinate of upper right corner of canvasury– a number, y-coordinate of upper right corner of canvasSet up user-defined coordinate system and switch to mode “world” if necessary. This performs ascreen.reset(). If mode “world” is already active, all drawings are redrawn according to the new coordinates.ATTENTION: in user-defined coordinate systems angl

llx– a number, x-coordinate of lower left corner of canvaslly– a number, y-coordinate of lower left corner of canvasurx– a number, x-coordinate of upper right corner of canvasury– a number, y-coordinate of upper right corner of canvas

Animation control

turtle.no_animation()

Temporarily disable turtle animation. The code written inside the no_animation block will not be animated; once the code block is exited, the drawing will appear.>>>withscreen.no_animation():...fordistinrange(2,400,2):...fd(dist)...rt(90)Added in version 3.14.

turtle.delay(delay=None)

Parameters:

Parameters:delay– positive integerSet or return the drawing delay in milliseconds. (This is approximately the time interval between two consecutive canvas updates.) The longer the drawing delay, the slower the animation.Optional argument:>>>screen.delay()10>>>screen.delay(5)>>>screen.delay()5

delay– positive integer

turtle.tracer(n=None,delay=None)

Parameters:

Parameters:n– nonnegative integerdelay– nonnegative integerTurn turtle animation on/off and set delay for update drawings. If n is given, only each n-th regular screen update is really performed. (Can be used to accelerate the drawing of complex graphics.) When called without arguments, returns the currently stored value of n. Second argument sets delay value (seedelay()).>>>screen.tracer(8,25)>>>dist=2>>>foriinrange(200):...fd(dist)...rt(90)...dist+=2

n– nonnegative integerdelay– nonnegative integer

turtle.update()

Perform a TurtleScreen update. To be used when tracer is turned off.

See also the RawTurtle/Turtle methods speed().

Using screen events

turtle.listen(xdummy=None,ydummy=None)

Set focus on TurtleScreen (in order to collect key-events). Dummy arguments are provided in order to be able to pass listen() to the onclick method.

turtle.onkey(fun,key)

turtle.onkeyrelease(fun,key)

Parameters:

Parameters:fun– a function with no arguments orNonekey– a string: key (e.g. “a”) or key-symbol (e.g. “space”)Bindfunto key-release event of key. IffunisNone, event bindings are removed. Remark: in order to be able to register key-events, TurtleScreen must have the focus. (See methodlisten().)>>>deff():...fd(50)...lt(60)...>>>screen.onkey(f,"Up")>>>screen.listen()

fun– a function with no arguments orNonekey– a string: key (e.g. “a”) or key-symbol (e.g. “space”)

turtle.onkeypress(fun,key=None)

Parameters:

Parameters:fun– a function with no arguments orNonekey– a string: key (e.g. “a”) or key-symbol (e.g. “space”)Bindfunto key-press event of key if key is given, or to any key-press-event if no key is given. Remark: in order to be able to register key-events, TurtleScreen must have focus. (See methodlisten().)>>>deff():...fd(50)...>>>screen.onkey(f,"Up")>>>screen.listen()

fun– a function with no arguments orNonekey– a string: key (e.g. “a”) or key-symbol (e.g. “space”)

turtle.onclick(fun,btn=1,add=None)

turtle.onscreenclick(fun,btn=1,add=None)

Parameters:

Parameters:fun– a function with two arguments which will be called with the coordinates of the clicked point on the canvasbtn– number of the mouse-button, defaults to 1 (left mouse button)add–TrueorFalse– ifTrue, a new binding will be added, otherwise it will replace a former bindingBindfunto mouse-click events on this screen. IffunisNone, existing bindings are removed.Example for a TurtleScreen instance namedscreenand a Turtle instance namedturtle:>>>screen.onclick(turtle.goto)# Subsequently c

fun– a function with two arguments which will be called with the coordinates of the clicked point on the canvasbtn– number of the mouse-button, defaults to 1 (left mouse button)add–TrueorFalse– ifTrue, a new binding will be added, otherwise it will replace a former binding

turtle.ontimer(fun,t=0)

Parameters:

Parameters:fun– a function with no argumentst– a number >= 0Install a timer that callsfunaftertmilliseconds.>>>running=True>>>deff():...ifrunning:...fd(50)...lt(60)...screen.ontimer(f,250)>>>f()#### makes the turtle march around>>>running=False

fun– a function with no argumentst– a number >= 0

turtle.mainloop()

turtle.done()

Starts event loop - calling Tkinter's mainloop function. Must be the last statement in a turtle graphics program. Mustnotbe used if a script is run from within IDLE in -n mode (No subprocess) - for interactive use of turtle graphics.>>>screen.mainloop()

Input methods

turtle.textinput(title,prompt)

Parameters:

Parameters:title– stringprompt– stringPop up a dialog window for input of a string. Parameter title is the title of the dialog window, prompt is a text mostly describing what information to input. Return the string input. If the dialog is canceled, returnNone.>>>screen.textinput("NIM","Name of first player:")

title– stringprompt– string

turtle.numinput(title,prompt,default=None,minval=None,maxval=None)

Parameters:

Parameters:title– stringprompt– stringdefault– number (optional)minval– number (optional)maxval– number (optional)Pop up a dialog window for input of a number. title is the title of the dialog window, prompt is a text mostly describing what numerical information to input. default: default value, minval: minimum value for input, maxval: maximum value for input. The number input must be in the range minval .. maxval if these are given. If not, a hint is issued and the dialog remains open for correction

title– stringprompt– stringdefault– number (optional)minval– number (optional)maxval– number (optional)

Settings and special methods

turtle.mode(mode=None)

Parameters:

Parameters:mode– one of the strings “standard”, “logo” or “world”Set turtle mode (“standard”, “logo” or “world”) and perform reset. If mode is not given, current mode is returned.Mode “standard” is compatible with oldturtle. Mode “logo” is compatible with most Logo turtle graphics. Mode “world” uses user-defined “world coordinates”.Attention: in this mode angles appear distorted ifx/yunit-ratio doesn't equal 1.ModeInitial turtle headingpositive angles“standard”to the right (east)counterclockw

mode– one of the strings “standard”, “logo” or “world”

turtle.colormode(cmode=None)

Parameters:

Parameters:cmode– one of the values 1.0 or 255Return the colormode or set it to 1.0 or 255.

Subsequentlyr,g,bvalues of color triples have to be in the range

```
0..*cmode*.>>>screen.colormode(1)>>>turtle.pencolor(240,160,80)Traceback (most recent call last):...TurtleGraphicsError:bad color sequence: (240, 160, 80)>>>screen.colormode(1.0)>>>screen.colormode(255)>>>screen.colormode(255)>>>turtle.pencolor(240,160,80)
```

cmode– one of the values 1.0 or 255

turtle.getcanvas()

Return the Canvas of this TurtleScreen. Useful for insiders who know what to do with a Tkinter Canvas.>>>cv=screen.getcanvas()>>>cv<tkinter.ScrolledCanvas object ...>

turtle.getshapes()

Return a list of names of all currently available turtle shapes.>>>screen.getshapes()['arrow', 'blank', 'circle', ..., 'turtle']

turtle.register_shape(name,shape=None)

turtle.addshape(name,shape=None)

There are four different ways to call this function:nameis the name of an image file (PNG, GIF, PGM, and PPM) andshapeisNone: Install the corresponding image shape.>>>screen.register_shape("turtle.gif")Note!Image shapesdo notrotate when turning the turtle, so they do not display the heading of the turtle!nameis an arbitrary string andshapeis the name of an image file (PNG, GIF, PGM, and PPM): Install the corresponding image shape.>>>screen.register_shape("turtle","turtle.gif")Note!Image shapesdo not

turtle.turtles()

Return the list of turtles on the screen.>>>forturtleinscreen.turtles():...turtle.color("red")

turtle.window_height()

Return the height of the turtle window.>>>screen.window_height()480

turtle.window_width()

Return the width of the turtle window.>>>screen.window_width()640

Methods specific to Screen, not inherited from TurtleScreen

turtle.bye()

Shut the turtlegraphics window.

turtle.exitonclick()

Bindbye()method to mouse clicks on the Screen.If the value “using_IDLE” in the configuration dictionary isFalse(default value), also enter mainloop. Remark: If IDLE with the-ns switch (no subprocess) is used, this value should be set toTrueinturtle.cfg. In this case IDLE’s own mainloop is active also for the client script.

turtle.save(filename,overwrite=False)

Parameters:

Save the current turtle drawing (and turtles) as a PostScript file.Parameters:filename– the path of the saved PostScript fileoverwrite– ifFalseand there already exists a file with the given filename, then the function will raise aFileExistsError. If it isTrue, the file will be overwritten.>>>screen.save("my_drawing.ps")>>>screen.save("my_drawing.ps",overwrite=True)Added in version 3.14.

filename– the path of the saved PostScript fileoverwrite– ifFalseand there already exists a file with the given filename, then the function will raise aFileExistsError. If it isTrue, the file will be overwritten.

turtle.setup(width=_CFG['width'],height=_CFG['height'],startx=_CFG['leftright'],starty=_CFG['topbottom'])

Parameters:

Set the size and position of the main window. Default values of arguments are stored in the configuration dictionary and can be changed via aturtle.cfgfile.Parameters:width– if an integer, a size in pixels, if a float, a fraction of the screen; default is 50% of screenheight– if an integer, the height in pixels, if a float, a fraction of the screen; default is 75% of screenstartx– if positive, starting position in pixels from the left edge of the screen, if negative from the right edge, ifNone,

width– if an integer, a size in pixels, if a float, a fraction of the screen; default is 50% of screenheight– if an integer, the height in pixels, if a float, a fraction of the screen; default is 75% of screenstartx– if positive, starting position in pixels from the left edge of the screen, if negative from the right edge, ifNone, center window horizontallystarty– if positive, starting position in pixels from the top edge of the screen, if negative from the bottom edge, ifNone, center window ver

turtle.title(titlestring)

Parameters:

Parameters:titlestring– a string that is shown in the titlebar of the turtle graphics windowSet title of turtle window totitlestring.>>>screen.title("Welcome to the turtle zoo!")

titlestring– a string that is shown in the titlebar of the turtle graphics window

Public classes

classturtle.RawTurtle(canvas)

class turtle.RawPen(canvas)

Parameters:

Parameters: canvas – `tkinter.Canvas`, `ScrolledCanvas` or `TurtleScreen` Create a turtle. The turtle has all methods described above as “methods of Turtle/RawTurtle”.

canvas – `tkinter.Canvas`, `ScrolledCanvas` or `TurtleScreen`

class turtle.Turtle

Subclass of `RawTurtle`, has the same interface but draws on a default `Screen` object created automatically when needed for the first time.

class turtle.TurtleScreen(cv)

Parameters:

Parameters: cv – `tkinter.Canvas` Provides screen oriented methods like `bgcolor()` etc. that are described above.

cv – `tkinter.Canvas`

class turtle.Screen

Subclass of `TurtleScreen`, with four methods added.

class turtle.ScrolledCanvas(master)

Parameters:

Parameters: master – some `Tkinter` widget to contain the `ScrolledCanvas`, i.e. a `Tkinter`-canvas with scrollbars added Used by class `Screen`, which thus automatically provides a `ScrolledCanvas` as playground for the turtles.

master – some `Tkinter` widget to contain the `ScrolledCanvas`, i.e. a `Tkinter`-canvas with scrollbars added

class turtle.Shape(type_, data)

Parameters:

addcomponent(poly, fill, outline=None)

Parameters:

Parameters: type_ – one of the strings “polygon”, “image”, “compound” Data structure modeling shapes. The pair (type_, data) must follow this specification: type_data “polygon” a polygon-tuple, i.e. a tuple of pairs of coordinates “image” an image (in this form only used internally!) “compound” `None` (a compound shape has to be constructed using the `addcomponent()` method) `addcomponent(poly, fill, outline=None)` Parameters: poly – a polygon, i.e. a tuple of pairs of numbers fill – a color the poly will be filled with outline –

type_ – one of the strings “polygon”, “image”, “compound”

Parameters: poly – a polygon, i.e. a tuple of pairs of numbers fill – a color the poly will be filled with outline – a color for the poly’s outline (if given) Example: `>>> poly = ((0,0),(10,-5),(0,10),(-10,-5))` `>>> s = Shape("compound")` `>>> s.addcomponent(poly, "red", "blue")` `>>> # ... add more components and then use register_shape()` See Compound shapes.

poly – a polygon, i.e. a tuple of pairs of numbers fill – a color the poly will be filled with outline – a color for the poly’s outline (if given)

class turtle.Vec2D(x,y)

A two-dimensional vector class, used as a helper class for implementing turtle graphics. May be useful for turtle graphics programs too. Derived from tuple, so a vector is a tuple! Provides (for a, b vectors, k number): `a + b` vector addition `a - b` vector subtraction `a * b` inner product `k * a` and `a * k` multiplication with scalar `abs(a)` absolute value of a `a.rotate(angle)` rotation

Explanation

A turtle object draws on a screen object, and there a number of key classes in the turtle object-oriented interface that can be used to create them and relate them to each other.

A Turtle instance will automatically create a Screen instance if one is not already present.

Turtle is a subclass of RawTurtle, which doesn't automatically create a drawing surface - a canvas will need to be provided or created for it. The canvas can be an `tkinter.Canvas`, `ScrolledCanvas` or `TurtleScreen`.

`TurtleScreen` is the basic drawing surface for a turtle. `Screen` is a subclass of `TurtleScreen`, and includes some additional methods for managing its appearance (including size and title) and behaviour. `TurtleScreen`'s constructor needs an `tkinter.Canvas` or a `ScrolledCanvas` as an argument.

The functional interface for turtle graphics uses the various methods of `Turtle` and `TurtleScreen/Screen`. Behind the scenes, a screen object is automatically created whenever a function derived from a `Screen` method is called. Similarly, a turtle object is automatically created whenever any of the functions derived from a `Turtle` method is called.

To use multiple turtles on a screen, the object-oriented interface must be used.

Help and configuration

How to use help

The public methods of the `Screen` and `Turtle` classes are documented extensively via docstrings. So these can be used as online-help via the Python help facilities:

- When using IDLE, tooltips show the signatures and first lines of the docstrings of typed in function-/method calls.
- Calling `help()` on methods or functions displays the docstrings:

```
>>>help(Screen.bgcolor)Help on method bgcolor in module turtle:bgcolor(self, *args) unbound turtle.Screen methodSet or return backgroundcolor of the TurtleScreen.Arguments (if given): a color string or three numbers in the range 0..colormode or a 3-tuple of such numbers.>>> screen.bgcolor("orange")>>> screen.bgcolor()>>> screen.bg
```
- The docstrings of the functions which are derived from methods have a modified form:

```
>>>help(bgcolor)Help on function bgcolor in module turtle:bgcolor(*args)Set or return backgroundcolor of the TurtleScreen.Arguments (if given): a color string or three numbers in the range 0..colormode or a 3-tuple of such numbers.Example::>>> bgcolor("orange")>>> bgcolor()>>> bgcolor(0.5,0,0.5)>>> bgcolor()
```

These modified docstrings are created automatically together with the function definitions that are derived from the methods at import time.

Translation of docstrings into different languages

There is a utility to create a dictionary the keys of which are the method names and the values of which are the docstrings of the public methods of the classes `Screen` and `Turtle`.

`turtle.write_docstringdict(filename='turtle_docstringdict')`

Parameters:

Parameters: `filename`— a string, used as filename. Create and write docstring-dictionary to a Python script with the given filename. This function has to be called explicitly (it is not used by the turtle graphics classes). The docstring dictionary will be written to the Python script `filename.py`. It is intended to serve as a template for translation of the docstrings into different languages.

filename— a string, used as filename

If you (or your students) want to use turtle with online help in your native language, you have to translate the docstrings and save the resulting file as e.g. `turtle_docstringdict_german.py`.

If you have an appropriate entry in your `turtle.cfg` file this dictionary will be read in at import time and will replace the original English docstrings.

At the time of this writing there are docstring dictionaries in German and in Italian. (Requests please to toglingl@aon.at.)

How to configure Screen and Turtles

The built-in default configuration mimics the appearance and behaviour of the old turtle module in order to retain best possible compatibility with it.

If you want to use a different configuration which better reflects the features of this module or which better fits to your needs, e.g. for use in a classroom, you can prepare a configuration file `turtle.cfg` which will be read at import time and modify the configuration according to its settings.

The built in configuration would correspond to the following `turtle.cfg`:

```
width = 0.5
height = 0.75
leftright = None
topbottom = None
canvwidth = 400
canvheight = 300
mode = standard
colormode = 1.0
delay = 10
undobuffersize = 1000
shape = classic
pencolor = black
fillcolor = black
resizemode = noresize
visible = True
language = english
exampleturtle = turtle
examplescreen = screen
title = Python Turtle Graphics
using_IDLE = False
```

Short explanation of selected entries:

- The first four lines correspond to the arguments of the `Screen.setup` method.
- Line 5 and 6 correspond to the arguments of the method `Screen.screensize`.
- `shape` can be any of the built-in shapes, e.g.: `arrow`, `turtle`, etc. For more info try `help(shape)`.
- If you want to use no fill color (i.e. make the turtle transparent), you have to write `fillcolor=""` (but all nonempty strings must not have quotes in the `cfg` file).
- If you want to reflect the turtle its state, you have to use `resizemode=auto`.
- If you set e.g. `language=italian` the `docstringdict_turtle_docstringdict_italian.py` will be loaded at import time (if present on the import path, e.g. in the same directory as `turtle`).
- The entries `exampleturtle` and `examplescreen` define the names of these objects as they occur in the docstrings. The transformation of method-docstrings to function-docstrings will delete these names from the docstrings.
- `using_IDLE`: Set this to `True` if you regularly work with IDLE and its `-nswitch` ("no subprocess"). This will prevent `exitonclick()` to enter the mainloop.

There can be a `turtle.cfg` file in the directory where `turtle` is stored and an additional one in the current working directory. The latter will override the settings of the first one.

The `Lib/turtledemo` directory contains a `turtle.cfg` file. You can study it as an example and see its effects when running the demos (preferably not from within the demo-viewer).

turtledemo— Demo scripts

The `turtledemo` package includes a set of demo scripts. These scripts can be run and viewed using the supplied demo viewer as follows:

```
python -m turtledemo
```

Alternatively, you can run the demo scripts individually. For example,

```
python -m turtledemo.bytedesign
```

The `turtledemo` package directory contains:

- A demo viewer `__main__.py` which can be used to view the sourcecode of the scripts and run them at the same time.
- Multiple scripts demonstrating different features of the `turtle` module. Examples can be accessed via the Examples menu. They can also be run standalone.
- A `turtle.cfg` file which serves as an example of how to write and use such files.

The demo scripts are:

```
Name | Description | Features | bytedesign | complex classical turtle graphics pattern |
tracer() | , | delay() | , | update() | chaos | graphs Verhulst dynamics, shows that computer's
computations can generate results sometimes against the common sense expectations | world
coordinates | clock | analog clock showing time of your computer | turtles as clock's hands, |
ontimer() | colormixer | experiment with r, g, b | ondrag() | forest | 3 breadth-first trees |
randomization | fractalcurves | Hilbert & Koch curves | recursion | lindenmayer |
ethnomathematics (indian kolams) | L-System | minimal_hanoi | Towers of Hanoi | Rectangular
Turtles as Hanoi discs ( | shape() | , | shapesize() | ) | nim | play the classical nim game
with three heaps of sticks against the computer. | turtles as nimsti
```

Have fun!

Changes since Python 2.6

- The methods `Turtle.tracer`, `Turtle.window_width` and `Turtle.window_height` have been eliminated. Methods with these names and functionality are now available only as methods of `Screen`. The functions derived from these remain available. (In fact already in Python 2.6 these methods were merely duplications of the corresponding `TurtleScreen/Screen` methods.)
- The method `Turtle.fill()` has been eliminated. The behaviour of `begin_fill()` and `end_fill()` have changed slightly: now every filling process must be completed with an `end_fill()` call.
- A method `Turtle.filling` has been added. It returns a boolean value: `True` if a filling process is under way, `False` otherwise. This behaviour corresponds to a `fill()` call without arguments in Python 2.6.

Changes since Python 3.0

- The `Turtle` methods `shearfactor()`, `shapetransform()` and `get_shapepoly()` have been added. Thus the full range of regular linear transforms is now available for transforming turtle shapes. `tiltangle()` has been enhanced in

functionality: it now can be used to get or set the tilt angle.

- The `Screen` method `onkeypress()` has been added as a complement to `onkey()`. As the latter binds actions to the key release event, an alias: `onkeyrelease()` was also added for it.
- The method `Screen.mainloop()` has been added, so there is no longer a need to use the `standalone_mainloop()` function when working with `Screen` and `Turtle` objects.
- Two input methods have been added: `Screen.textinput()` and `Screen.numinput()`. These pop up input dialogs and return strings and numbers respectively.

Development Tools

The modules described in this chapter help you write software. For example, the `pydoc` module takes a module and generates documentation based on the module's contents. The `doctest` and `unittest` modules contain frameworks for writing unit tests that automatically exercise code and verify that the expected output is produced.

The list of modules described in this chapter is:

- `typing`— Support for type hintsSpecification for the Python Type SystemType aliasesNewTypeAnnotating callable objectsGenericsAnnotating tuplesThe type of class objectsAnnotating generators and coroutinesUser-defined generic typesTheAnytypeNominal vs structural subtypingModule contentsSpecial typing primitivesSpecial typesSpecial formsBuilding generic types and type aliasesOther special directivesPr
- `pydoc`— Documentation generator and online help system
- Python Development ModeEffects of the Python Development ModeResourceWarning ExampleBad file descriptor error example
- `doctest`— Test interactive Python examplesSimple Usage: Checking Examples in DocstringsSimple Usage: Checking Examples in a Text FileCommand-line UsageHow It WorksWhich Docstrings Are Examined?How are Docstring Examples Recognized?What's the Execution Context?What About Exceptions?Option FlagsDirectivesWarningsBasic APIUnittest APIAdvanced APIDocTest ObjectsExample ObjectsDocTestFinder objectsDocTe
- `unittest`— Unit testing frameworkBasic exampleCommand-Line InterfaceCommand-line optionsTest DiscoveryOrganizing test codeRe-using old test codeSkipping tests and expected failuresDistinguishing test iterations using subtestsClasses and functionsTest casesGrouping testsLoading and running testsload_tests ProtocolClass and Module FixturessetUpClass and tearDownClasssetUpModule and tearDownModuleSign
- `unittest.mock`— mock object libraryQuick GuideThe Mock ClassCallingDeleting AttributesMock names and the name attributeAttaching Mocks as AttributesThe patcherspachpatch.objectpatch.dictpatch.multiplepatch methods: start and stoppatch builtinsTEST_PREFIXNesting Patch DecoratorsWhere to patchPatching Descriptors and Proxy ObjectsMagicMock and magic method supportMocking Magic MethodsMagic MockHelpe
- `unittest.mock`— getting startedUsing MockMock patching methodsMock for method calls on an objectMocking classesNaming your mocksTracking all callsSetting return values and attributesRaising exceptions with mocksSide effect functions and iterablesMocking asynchronous iteratorsMocking asynchronous context managerCreating a mock from an existing objectUsing side_effect to return per file contentPatch
- `test`— Regression tests package for PythonWriting Unit Tests for thetestpackageRunning tests using the command-line interface
- `test.support`— Utilities for the Python test suite
- `test.support.socket_helper`— Utilities for socket tests
- `test.support.script_helper`— Utilities for the Python execution tests
- `test.support.bytecode_helper`— Support tools for testing correct bytecode generation
- `test.support.threading_helper`— Utilities for threading tests
- `test.support.os_helper`— Utilities for os tests
- `test.support.import_helper`— Utilities for import tests
- `test.support.warnings_helper`— Utilities for warnings tests

typing— Support for type hints

Added in version 3.5.

Source code: [Lib/typing.py](#)

■ **Nota:** NoteThe Python runtime does not enforce function and variable type annotations. They can be used by third party tools such as type checkers, IDEs, linters, etc.

This module provides runtime support for type hints.

Consider the function below:

```
def surface_area_of_cube(edge_length: float) -> str:
    return f"The surface area of the cube is {6 * edge_length ** 2}."
```

The function `surface_area_of_cube` takes an argument expected to be an instance of `float`, as indicated by the type hint `edge_length: float`. The function is expected to return an instance of `str`, as indicated by the `-> str` hint.

While type hints can be simple classes like `float` or `str`, they can also be more complex. The `typing` module provides a vocabulary of more advanced type hints.

New features are frequently added to the `typing` module. The `typing_extensions` package provides backports of these new features to older versions of Python.

See also

Typing cheat sheet

Type System Reference section of the mypy docs

Static Typing with Python

A quick overview of type hints (hosted at the mypy docs)

The Python typing system is standardised via PEPs, so this reference should broadly apply to most Python type checkers. (Some parts may still be specific to mypy.)

Type-checker-agnostic documentation written by the community detailing type system features, useful typing related tools and typing best practices.

Specification for the Python Type System

The canonical, up-to-date specification of the Python type system can be found at [Specification for the Python type system](#).

Type aliases

A type alias is defined using the `typing` statement, which creates an instance of `typing.TypeAliasType`. In this example, `Vector` and `list[float]` will be treated equivalently by static type checkers:

```
type Vector = list[float]
```

```
def scale(scalar: float, vector: Vector) -> Vector:
    return [scalar * num for num in vector]
```

```
# passes type checking; a list of floats qualifies as a Vector.
new_vector = scale(2.0, [1.0, -4.2, 5.4])
```

Type aliases are useful for simplifying complex type signatures. For example:

```

from collections.abc import Sequence

type ConnectionOptions = dict[str, str]
type Address = tuple[str, int]
type Server = tuple[Address, ConnectionOptions]

def broadcast_message(message: str, servers: Sequence[Server]) -> None:
    ...

# The static type checker will treat the previous type signature as
# being exactly equivalent to this one.
def broadcast_message(
    message: str,
    servers: Sequence[tuple[tuple[str, int], dict[str, str]]]
) -> None:
    ...

```

The `type` statement is new in Python 3.12. For backwards compatibility, type aliases can also be created through simple assignment:

```
Vector = list[float]
```

Or marked with `TypeAlias` to make it explicit that this is a type alias, not a normal variable assignment:

```

from typing import TypeAlias

Vector: TypeAlias = list[float]

```

NewType

Use the `NewType` helper to create distinct types:

```

from typing import NewType

UserId = NewType('UserId', int)
some_id = UserId(524313)

```

The static type checker will treat the new type as if it were a subclass of the original type. This is useful in helping catch logical errors:

```

def get_user_name(user_id: UserId) -> str:
    ...

# passes type checking
user_a = get_user_name(UserId(42351))

# fails type checking; an int is not a UserId
user_b = get_user_name(-1)

```

You may still perform all integer operations on a variable of type `UserId`, but the result will always be of type `int`. This lets you pass in a `UserId` wherever an `int` might be expected, but will prevent you from accidentally creating a `UserId` in an invalid way:

```

# 'output' is of type 'int', not 'UserId'
output = UserId(23413) + UserId(54341)

```

Note that these checks are enforced only by the static type checker. At runtime, the statement `Derived = NewType('Derived', Base)` will make `Derived` a callable that immediately returns whatever parameter you pass it. That means the expression `Derived(some_value)` does not create a new class or introduce much overhead beyond that of a regular function call.

More precisely, the expression `some_value is Derived(some_value)` is always true at runtime.

It is invalid to create a subtype of `Derived`:

```
from typing import NewType

UserId = NewType('UserId', int)

# Fails at runtime and does not pass type checking
class AdminUserId(UserId): pass
```

However, it is possible to create a `NewType` based on a 'derived' `NewType`:

```
from typing import NewType

UserId = NewType('UserId', int)

ProUserId = NewType('ProUserId', UserId)

and typechecking for ProUserId will work as expected.
```

See [PEP 484](#) for more details.

■ **Note:** Recall that the use of a type alias declares two types to be equivalent to one another. Doing `typeAlias = Original` will make the static type checker treat `Alias` as being exactly equivalent to `Original` in all cases. This is useful when you want to simplify complex type signatures. In contrast, `NewType` declares one type to be a subtype of another. Doing `Derived = NewType('Derived', Original)` will make the static type checker treat `Derived` as a subclass of `Original`, which means a value of type `Original` cannot be used in places where a value of type `Derived` is expected. This is useful when you want to prevent logic errors w

Added in version 3.5.2.

Changed in version 3.10: `NewType` is now a class rather than a function. As a result, there is some additional runtime cost when calling `NewType` over a regular function.

Changed in version 3.11: The performance of calling `NewType` has been restored to its level in Python 3.9.

Annotating callable objects

Functions – or other callable objects – can be annotated using `collections.abc.Callable` or deprecated `typing.Callable`. `Callable[[int], str]` signifies a function that takes a single parameter of type `int` and returns a `str`.

For example:

```
from collections.abc import Callable, Awaitable

def feeder(get_next_item: Callable[[], str]) -> None:
    ... # Body

def async_query(on_success: Callable[[int], None],
               on_error: Callable[[int, Exception], None]) -> None:
    ... # Body

async def on_update(value: str) -> None:
    ... # Body

callback: Callable[[str], Awaitable[None]] = on_update
```

The subscription syntax must always be used with exactly two values: the argument list and the return type. The argument list must be a list of types, a `ParamSpec`, `Concatenate`, or an ellipsis (...). The return type must be a single type.

If a literal ellipsis... is given as the argument list, it indicates that a callable with any arbitrary parameter list would be acceptable:

```
def concat(x: str, y: str) -> str:
    return x + y
```

```
x: Callable[..., str]
x = str          # OK
x = concat       # Also OK
```

Callable cannot express complex signatures such as functions that take a variadic number of arguments, overloaded functions, or functions that have keyword-only parameters. However, these signatures can be expressed by defining a Protocol class with a `__call__()` method:

```
from collections.abc import Iterable
from typing import Protocol

class Combiner(Protocol):
    def __call__(self, *vals: bytes, maxlen: int | None = None) -> list[bytes]: ...

def batch_proc(data: Iterable[bytes], cb_results: Combiner) -> bytes:
    for item in data:
        ...

def good_cb(*vals: bytes, maxlen: int | None = None) -> list[bytes]:
    ...
def bad_cb(*vals: bytes, maxitems: int | None) -> list[bytes]:
    ...

batch_proc([], good_cb) # OK
batch_proc([], bad_cb) # Error! Argument 2 has incompatible type because of
                        # different name and kind in the callback
```

Callables which take other callables as arguments may indicate that their parameter types are dependent on each other using `ParamSpec`. Additionally, if that callable adds or removes arguments from other callables, the `Concatenate` operator may be used. They take the form `Callable[ParamSpecVariable, ReturnType]` and `Callable[Concatenate[Arg1Type, Arg2Type, ..., ParamSpecVariable], ReturnType]` respectively.

Changed in version 3.10: Callable now supports `ParamSpec` and `Concatenate`. See PEP 612 for more details.

See also

The documentation for `ParamSpec` and `Concatenate` provides examples of usage in Callable.

Generics

Since type information about objects kept in containers cannot be statically inferred in a generic way, many container classes in the standard library support subscription to denote the expected types of container elements.

```
from collections.abc import Mapping, Sequence

class Employee: ...

# Sequence[Employee] indicates that all elements in the sequence
# must be instances of "Employee".
# Mapping[str, str] indicates that all keys and all values in the mapping
# must be strings.
def notify_by_email(employees: Sequence[Employee],
                   overrides: Mapping[str, str]) -> None: ...
```

Generic functions and classes can be parameterized by using type parameter syntax:

```
from collections.abc import Sequence

def first[T](l: Sequence[T]) -> T: # Function is generic over the TypeVar "T"
```

```
    return l[0]
```

Or by using the `TypeVar` factory directly:

```
from collections.abc import Sequence
from typing import TypeVar

U = TypeVar('U')                                # Declare type variable "U"

def second(l: Sequence[U]) -> U: # Function is generic over the TypeVar "U"
    return l[1]
```

Changed in version 3.12: Syntactic support for generics is new in Python 3.12.

Annotating tuples

For most containers in Python, the typing system assumes that all elements in the container will be of the same type. For example:

```
from collections.abc import Mapping

# Type checker will infer that all elements in ``x`` are meant to be ints
x: list[int] = []

# Type checker error: ``list`` only accepts a single type argument:
y: list[int, str] = [1, 'foo']

# Type checker will infer that all keys in ``z`` are meant to be strings,
# and that all values in ``z`` are meant to be either strings or ints
z: Mapping[str, str | int] = {}
```

`list` only accepts one type argument, so a type checker would emit an error on the assignment above. Similarly, `Mapping` only accepts two type arguments: the first indicates the type of the keys, and the second indicates the type of the values.

Unlike most other Python containers, however, it is common in idiomatic Python code for tuples to have elements which are not all of the same type. For this reason, tuples are special-cased in Python's typing system. `tuple` accepts any number of type arguments:

```
# OK: ``x`` is assigned to a tuple of length 1 where the sole element is an int
x: tuple[int] = (5,)

# OK: ``y`` is assigned to a tuple of length 2;
# element 1 is an int, element 2 is a str
y: tuple[int, str] = (5, "foo")

# Error: the type annotation indicates a tuple of length 1,
# but ``z`` has been assigned to a tuple of length 3
z: tuple[int] = (1, 2, 3)
```

To denote a tuple which could be of any length, and in which all elements are of the same type `T`, use the literal ellipsis...: `tuple[T,...]`. To denote an empty tuple, use `tuple[()]`. Using `tuple[Any,...]` as an annotation is equivalent to using `tuple[Any,...]`:

```
x: tuple[int, ...] = (1, 2)
# These reassignments are OK: ``tuple[int, ...]`` indicates x can be of any length
x = (1, 2, 3)
x = ()
# This reassignment is an error: all elements in ``x`` must be ints
x = ("foo", "bar")

# ``y`` can only ever be assigned to an empty tuple
y: tuple[()] = ()
```



```

z: tuple = ("foo", "bar")
# These reassignments are OK: plain ``tuple`` is equivalent to ``tuple[Any, ...]``
z = (1, 2, 3)
z = ()

```

The type of class objects

A variable annotated with `C` may accept a value of type `C`. In contrast, a variable annotated with `type[C]` (or deprecated `typing.Type[C]`) may accept values that are classes themselves – specifically, it will accept the class object of `C`. For example:

```

a = 3          # Has type ``int``
b = int        # Has type ``type[int]``
c = type(a)    # Also has type ``type[int]``

```

Note that `type[C]` is covariant:

```

class User: ...
class ProUser(User): ...
class TeamUser(User): ...

```

```

def make_new_user(user_class: type[User]) -> User:
    # ...
    return user_class()

```

```

make_new_user(User)      # OK
make_new_user(ProUser)   # Also OK: ``type[ProUser]`` is a subtype of ``type[User]``
make_new_user(TeamUser)  # Still fine
make_new_user(User())    # Error: expected ``type[User]`` but got ``User``
make_new_user(int)       # Error: ``type[int]`` is not a subtype of ``type[User]``

```

The only legal parameters for `type` are classes, `Any`, type variables, and unions of any of these types. For example:

```

def new_non_team_user(user_class: type[BasicUser | ProUser]): ...

new_non_team_user(BasicUser) # OK
new_non_team_user(ProUser)   # OK
new_non_team_user(TeamUser)  # Error: ``type[TeamUser]`` is not a subtype
                             # of ``type[BasicUser | ProUser]``
new_non_team_user(User)      # Also an error

```

`type[Any]` is equivalent to `type`, which is the root of Python's metaclass hierarchy.

Annotating generators and coroutines

A generator can be annotated using the generic type `Generator[YieldType, SendType, ReturnType]`. For example:

```

def echo_round() -> Generator[int, float, str]:
    sent = yield 0
    while sent >= 0:
        sent = yield round(sent)
    return 'Done'

```

Note that unlike many other generic classes in the standard library, the `SendType` of `Generator` behaves contravariantly, not covariantly or invariantly.

The `SendType` and `ReturnType` parameters default to `None`:

```

def infinite_stream(start: int) -> Generator[int]:
    while True:
        yield start

```

```
start += 1
```

It is also possible to set these types explicitly:

```
def infinite_stream(start: int) -> Generator[int, None, None]:
    while True:
        yield start
        start += 1
```

Simple generators that only ever yield values can also be annotated as having a return type of either `Iterable[YieldType]` or `Iterator[YieldType]`:

```
def infinite_stream(start: int) -> Iterator[int]:
    while True:
        yield start
        start += 1
```

Async generators are handled in a similar fashion, but don't expect a `ReturnType` argument (`AsyncGenerator[YieldType, SendType]`). The `SendType` argument defaults to `None`, so the following definitions are equivalent:

```
async def infinite_stream(start: int) -> AsyncGenerator[int]:
    while True:
        yield start
        start = await increment(start)

async def infinite_stream(start: int) -> AsyncGenerator[int, None]:
    while True:
        yield start
        start = await increment(start)
```

As in the synchronous case, `AsyncIterable[YieldType]` and `AsyncIterator[YieldType]` are available as well:

```
async def infinite_stream(start: int) -> AsyncIterator[int]:
    while True:
        yield start
        start = await increment(start)
```

Coroutines can be annotated using `Coroutine[YieldType, SendType, ReturnType]`. Generic arguments correspond to those of `Generator`, for example:

```
from collections.abc import Coroutine
c: Coroutine[list[str], str, int] # Some coroutine defined elsewhere
x = c.send('hi')                 # Inferred type of 'x' is list[str]
async def bar() -> None:
    y = await c                   # Inferred type of 'y' is int
```

User-defined generic types

A user-defined class can be defined as a generic class.

```
from logging import Logger

class LoggedVar[T]:
    def __init__(self, value: T, name: str, logger: Logger) -> None:
        self.name = name
        self.logger = logger
        self.value = value

    def set(self, new: T) -> None:
        self.log('Set ' + repr(self.value))
        self.value = new

    def get(self) -> T:
```

```

        self.log('Get ' + repr(self.value))
        return self.value

    def log(self, message: str) -> None:
        self.logger.info('%s: %s', self.name, message)

```

This syntax indicates that the class `LoggedVar` is parameterised around a single type variable `T`. This also makes `T` valid as a type within the class body.

Generic classes implicitly inherit from `Generic`. For compatibility with Python 3.11 and lower, it is also possible to inherit explicitly from `Generic` to indicate a generic class:

```
from typing import TypeVar, Generic
```

```
T = TypeVar('T')
```

```
class LoggedVar(Generic[T]):
    ...
```

Generic classes have `__class_getitem__()` methods, meaning they can be parameterised at runtime (e.g. `LoggedVar[int]` below):

```
from collections.abc import Iterable
```

```
def zero_all_vars(vars: Iterable[LoggedVar[int]]) -> None:
    for var in vars:
        var.set(0)
```

A generic type can have any number of type variables. All varieties of `TypeVar` are permissible as parameters for a generic type:

```
from typing import TypeVar, Generic, Sequence
```

```
class WeirdTrio[T, B: Sequence[bytes], S: (int, str)]:
    ...
```

```
OldT = TypeVar('OldT', contravariant=True)
OldB = TypeVar('OldB', bound=Sequence[bytes], covariant=True)
OldS = TypeVar('OldS', int, str)
```

```
class OldWeirdTrio(Generic[OldT, OldB, OldS]):
    ...
```

Each type variable argument to `Generic` must be distinct. This is thus invalid:

```
from typing import TypeVar, Generic
...
```

```
class Pair[M, M]: # SyntaxError
    ...
```

```
T = TypeVar('T')
```

```
class Pair(Generic[T, T]): # INVALID
    ...
```

Generic classes can also inherit from other classes:

```
from collections.abc import Sized
```

```
class LinkedList[T](Sized):
    ...
```

When inheriting from generic classes, some type parameters could be fixed:

```
from collections.abc import Mapping
```

```
class MyDict[T](Mapping[str, T]):
    ...
```

In this case `MyDict` has a single parameter, `T`.

Using a generic class without specifying type parameters assumes `Any` for each position. In the following example, `MyIterable` is not generic but implicitly inherits from `Iterable[Any]`:

```
from collections.abc import Iterable

class MyIterable(Iterable): # Same as Iterable[Any]
    ...
```

User-defined generic type aliases are also supported. Examples:

```
from collections.abc import Iterable

type Response[S] = Iterable[S] | int

# Return type here is same as Iterable[str] | int
def response(query: str) -> Response[str]:
    ...

type Vec[T] = Iterable[tuple[T, T]]

def inproduct[T: (int, float, complex)](v: Vec[T]) -> T: # Same as Iterable[tuple[T, T]]
    return sum(x*y for x, y in v)
```

For backward compatibility, generic type aliases can also be created through a simple assignment:

```
from collections.abc import Iterable
from typing import TypeVar

S = TypeVar("S")
Response = Iterable[S] | int
```

Changed in version 3.7: `Generic` no longer has a custom metaclass.

Changed in version 3.12: Syntactic support for generics and type aliases is new in version 3.12. Previously, generic classes had to explicitly inherit from `Generic` or contain a type variable in one of their bases.

User-defined generics for parameter expressions are also supported via parameter specification variables in the form `[**P]`. The behavior is consistent with type variables' described above as parameter specification variables are treated by the `typing` module as a specialized type variable. The one exception to this is that a list of types can be used to substitute a `ParamSpec`:

```
>>> class Z[T, **P]: ... # T is a TypeVar; P is a ParamSpec
...
>>> Z[int, [dict, float]]
__main__.Z[int, [dict, float]]
```

Classes generic over a `ParamSpec` can also be created using explicit inheritance from `Generic`. In this case, `**` is not used:

```
from typing import ParamSpec, Generic

P = ParamSpec('P')

class Z(Generic[P]):
    ...
```

Another difference between `TypeVar` and `ParamSpec` is that a generic with only one parameter specification variable will accept parameter lists in the forms `X[[Type1, Type2, ...]]` and also `X[Type1, Type2, ...]` for aesthetic reasons. Internally, the latter is converted to the former, so the following are equivalent:

```
>>> class X[**P]: ...
```

```
...
>>> X[int, str]
__main__.X[[int, str]]
>>> X[[int, str]]
__main__.X[[int, str]]
```

Note that generics with `ParamSpec` may not have correct `__parameters__` after substitution in some cases because they are intended primarily for static type checking.

Changed in version 3.10: `Generic` can now be parameterized over parameter expressions. See `ParamSpec` and [PEP 612](#) for more details.

A user-defined generic class can have ABCs as base classes without a metaclass conflict. Generic metaclasses are not supported. The outcome of parameterizing generics is cached, and most types in the `typing` module are hashable and comparable for equality.

The `Any` type

A special kind of type is `Any`. A static type checker will treat every type as being compatible with `Any` and `Any` as being compatible with every type.

This means that it is possible to perform any operation or method call on a value of type `Any` and assign it to any variable:

```
from typing import Any

a: Any = None
a = []          # OK
a = 2           # OK

s: str = ''
s = a           # OK

def foo(item: Any) -> int:
    # Passes type checking; 'item' could be any type,
    # and that type might have a 'bar' method
    item.bar()
    ...
```

Notice that no type checking is performed when assigning a value of type `Any` to a more precise type. For example, the static type checker did not report an error when assigning `atoseven` though `s` was declared to be of type `str` and receives an `int` value at runtime!

Furthermore, all functions without a return type or parameter types will implicitly default to using `Any`:

```
def legacy_parser(text):
    ...
    return data

# A static type checker will treat the above
# as having the same signature as:
def legacy_parser(text: Any) -> Any:
    ...
    return data
```

This behavior allows `Any` to be used as an escape hatch when you need to mix dynamically and statically typed code.

Contrast the behavior of `Any` with the behavior of `object`. Similar to `Any`, every type is a subtype of `object`. However, unlike `Any`, the reverse is not true: `object` is not a subtype of every other type.

That means when the type of a value is `object`, a type checker will reject almost all operations on it, and assigning it to a variable (or using it as a return value) of a more specialized type is a type error. For example:

```
def hash_a(item: object) -> int:
    # Fails type checking; an object does not have a 'magic' method.
    item.magic()
    ...

def hash_b(item: Any) -> int:
    # Passes type checking
    item.magic()
    ...

# Passes type checking, since ints and strs are subclasses of object
hash_a(42)
hash_a("foo")

# Passes type checking, since Any is compatible with all types
hash_b(42)
hash_b("foo")
```

Use `object` to indicate that a value could be any type in a typesafe manner. Use `Any` to indicate that a value is dynamically typed.

Nominal vs structural subtyping

Initially PEP 484 defined the Python static type system as using nominal subtyping. This means that a class `A` is allowed where a class `B` is expected if and only if `A` is a subclass of `B`.

This requirement previously also applied to abstract base classes, such as `Iterable`. The problem with this approach is that a class had to be explicitly marked to support them, which is unpythonic and unlike what one would normally do in idiomatic dynamically typed Python code. For example, this conforms to PEP 484:

```
from collections.abc import Sized, Iterable, Iterator

class Bucket(Sized, Iterable[int]):
    ...
    def __len__(self) -> int: ...
    def __iter__(self) -> Iterator[int]: ...
```

PEP 544 solves this problem by allowing users to write the above code without explicit base classes in the class definition, allowing `Bucket` to be implicitly considered a subtype of both `Sized` and `Iterable[int]` by static type checkers. This is known as structural subtyping (or static duck-typing):

```
from collections.abc import Iterator, Iterable

class Bucket: # Note: no base classes
    ...
    def __len__(self) -> int: ...
    def __iter__(self) -> Iterator[int]: ...

def collect(items: Iterable[int]) -> int: ...
result = collect(Bucket()) # Passes type check
```

Moreover, by subclassing a special class `Protocol`, a user can define new custom protocols to fully enjoy structural subtyping (see examples below).

Module contents

The `typing` module defines the following classes, functions and decorators.

Special typing primitives

Special types

These can be used as types in annotations. They do not support subscription using `[]`.

`typing.Any`

Special type indicating an unconstrained type. Every type is compatible with `Any`. `Any` is compatible with every type. Changed in version 3.11: `Any` can now be used as a base class. This can be useful for avoiding type checker errors with classes that can duck type anywhere or are highly dynamic.

`typing.AnyStr`

A constrained type variable. Definition: `AnyStr = TypeVar('AnyStr', str, bytes)` `AnyStr` is meant to be used for functions that may accept `str` or `bytes` arguments but cannot allow the two to mix. For example: `def concat(a: AnyStr, b: AnyStr) -> AnyStr: return a + b` `concat("foo", "bar")` # OK, output has type `'str'` `concat(b"foo", b"bar")` # OK, output has type `'bytes'` `concat("foo", b"bar")` # Error, cannot mix `str` and `bytes` Note that, despite its name, `AnyStr` has nothing to do with the `Any` type, nor does it mean “any string”. In particular,

`typing.LiteralString`

Special type that includes only literal strings. Any string literal is compatible with `LiteralString`, as is another `LiteralString`. However, an object typed as just `str` is not. A string created by composing `LiteralString`-typed objects is also acceptable as a `LiteralString`. Example: `def run_query(sql: LiteralString) -> None: ...` `def caller(arbitrary_string: str, literal_string: LiteralString) -> None: run_query("SELECT * FROM students")` # OK `run_query(literal_string)` # OK `run_query("SELECT * FROM " + literal_string)` # OK `run_q`

`typing.Never`

`typing.NoReturn`

`Never` and `NoReturn` represent the bottom type, a type that has no members. They can be used to indicate that a function never returns, such as `sys.exit()`: `from typing import Never` # or `NoReturn` `def stop() -> Never: raise RuntimeError('no way')` Or to define a function that should never be called, as there are no valid arguments, such as `assert_never()`: `from typing import Never` # or `NoReturn` `def never_call_me(arg: Never) -> None: pass` `def int_or_str(arg: int | str) -> None: never_call_me(arg)` # type checker error `match arg: case int(): print`

`typing.Self`

Special type to represent the current enclosed class. For example: `from typing import Self` `reveal_type` `class Foo: def return_self(self) -> Self: ...` `return self` `class SubclassOfFoo(Foo): pass` `reveal_type(Foo().return_self())` # Revealed type is `"Foo"` `reveal_type(SubclassOfFoo().return_self())` # Revealed type is `"SubclassOfFoo"` This annotation is semantically equivalent to the following, albeit in a more succinct fashion: `from typing import TypeVar` `Self = TypeVar("Self", bound="Foo")` `class Foo: def return_self(self: Self) -> Self: ...`

`typing.TypeAlias`

Special annotation for explicitly declaring a type alias. For example: `from typing import TypeAlias` `Factors: TypeAlias = list[int]` `TypeAlias` is particularly useful on older Python versions for annotating aliases that make use of forward references, as it can be hard for type checkers to distinguish these from normal variable assignments: `from typing import Generic, TypeAlias` `TypeVar T = TypeVar("T")` # "Box" does not exist yet, # so we have to use quotes for the forward reference on Python <3.12. # Using ```TypeAlias```

Special forms

These can be used as types in annotations. They all support subscription using `[]`, but each has a unique syntax.

classtyping.Union

Union type; `Union[X,Y]` is equivalent to `X|Y` and means either X or Y. To define a union, use e.g. `Union[int,str]` or the shorthand `int|str`. Using that shorthand is recommended. Details: The arguments must be types and there must be at least one. Unions of unions are flattened, e.g.: `Union[Union[int,str],float]==Union[int,str,float]` However, this does not apply to unions referenced through a type alias, to avoid forcing evaluation of the underlying type. Alias type: `typeA=Union[int,str]` `Union[A,float]!=Union[int,str]`

typing.Optional

`Optional[X]` is equivalent to `X|None` (or `Union[X,None]`). Note that this is not the same concept as an optional argument, which is one that has a default. An optional argument with a default does not require the `Optional` qualifier on its type annotation just because it is optional. For example: `def foo(arg:int=0)->None:...` On the other hand, if an explicit value of `None` is allowed, the use of `Optional` is appropriate, whether the argument is optional or not. For example: `def foo(arg:Optional[int]=None)->None:...`

typing.Concatenate

Special form for annotating higher-order functions. `Concatenate` can be used in conjunction with `Callable` and `ParamSpec` to annotate a higher-order callable which adds, removes, or transforms parameters of another callable. Usage is in the form `Concatenate[Arg1Type,Arg2Type,...,ParamSpecVariable]`. `Concatenate` is currently only valid when used as the first argument to a `Callable`. The last parameter to `Concatenate` must be a `ParamSpec` or ellipsis (...). For example, to annotate a decorator with `_lock` which provides a

typing.Literal

Special typing form to define "literal types". `Literal` can be used to indicate to type checkers that the annotated object has a value equivalent to one of the provided literals. For example: `def validate_simple(data:Any)->Literal[True]:# always returns True...type Mode=Literal['r','rb','w','wb']def open_helper(file:str,mode:Mode)->str:...open_helper('/some/path','r')# Passes type checkopen_helper('/other/path','typo')# Error in type checker` `Literal[...]` cannot be subclassed. At runtime, an arbitrary value

typing.ClassVar

Special type construct to mark class variables. As introduced in PEP 526, a variable annotation wrapped in `ClassVar` indicates that a given attribute is intended to be used as a class variable and should not be set on instances of that class. Usage: `class Starship: stats:ClassVar[dict[str,int]]={}# class variable damage:int=10# instance variable` `ClassVar` accepts only types and cannot be further subscribed. `ClassVar` is not a class itself, and should not be used with `isinstance()` or `issubclass()`. `ClassVar` does not

typing.Final

Special typing construct to indicate final names to type checkers. Final names cannot be reassigned in any scope. Final names declared in class scopes cannot be overridden in subclasses. For example: `MAX_SIZE:Final=9000MAX_SIZE+=1# Error reported by type checker` `class Connection: TIMEOUT:Final[int]=10` `class FastConnector(Connection): TIMEOUT=1# Error reported by type checker` There is no runtime checking of these properties. See PEP 591 for more details. Added in version 3.8. Changed in version 3.13: `Final` can

typing.Required

Special typing construct to mark a `TypedDict` key as required. This is mainly useful for `total=False` `TypedDict`s. See `TypedDict` and PEP 655 for more details. Added in version 3.11.

typing.NotRequired

Special typing construct to mark a `TypedDict` key as potentially missing. See `TypedDict` and PEP 655 for more details. Added in version 3.11.

typing.ReadOnly

A special typing construct to mark an item of a `TypedDict` as read-only. For example: `class Movie(TypedDict): title: ReadOnly[str] year: int`
`def mutate_movie(m: Movie) -> None: m["year"] = 1999 # allowed`
`m["title"] = "The Matrix" # typechecker error` There is no runtime checking for this property. See `TypedDict` and PEP 705 for more details. Added in version 3.13.

typing.Annotated

`Annotated[<type>, <metadata>]`

PEP 593- Flexible function and variable annotations

Special typing form to add context-specific metadata to an annotation. Add metadata to a given type `T` by using the annotation `Annotated[T, x]`. Metadata added using `Annotated` can be used by static analysis tools or at runtime. At runtime, the metadata is stored in a `__metadata__` attribute. If a library or tool encounters an annotation `Annotated[T, x]` and has no special logic for the metadata, it should ignore the metadata and simply treat the annotation as `T`. As such, `Annotated` can be useful for code that wants

The PEP introducing `Annotated` to the standard library.

typing.TypeIs

Special typing construct for marking user-defined type predicate functions. `TypeIs` can be used to annotate the return type of a user-defined type predicate function. `TypeIs` only accepts a single type argument. At runtime, functions marked this way should return a boolean and take at least one positional argument. `TypeIs` aims to benefit type narrowing— a technique used by static type checkers to determine a more precise type of an expression within a program's code flow. Usually type narrowing is done

typing.TypeGuard

Special typing construct for marking user-defined type predicate functions. Type predicate functions are user-defined functions that return whether their argument is an instance of a particular type. `TypeGuard` works similarly to `TypeIs`, but has subtly different effects on type checking behavior (see below). Using `-> TypeGuard` tells the static type checker that for a given function: The return value is a boolean. If the return value is `True`, the type of its argument is the type inside `TypeGuard`. `TypeGuard` also

typing.Unpack

Typing operator to conceptually mark an object as having been unpacked. For example, using the unpack operator `*` on a type variable `tuple` is equivalent to using `Unpack` to mark the type variable `tuple` as having been unpacked: `Ts = TypeVarTuple('Ts'); tup: tuple[*Ts] # Effectively does: tup: tuple[Unpack[Ts]]` In fact, `Unpack` can be used interchangeably with `*` in the context of typing. `TypeVarTuple` and `builtins.tuple` types. You might see `Unpack` being used explicitly in older versions of Python, where `*` couldn't be used in certa

Building generic types and type aliases

The following classes should not be used directly as annotations. Their intended purpose is to be building blocks for creating generic types and type aliases.

These objects can be created through special syntax (type parameter lists and the `type` statement). For compatibility with Python 3.11 and earlier, they can also be created without the dedicated syntax, as documented below.

classtyping.Generic

Abstract base class for generic types. A generic type is typically declared by adding a list of type parameters after the class name: `class Mapping[K, V]: def __getitem__(self, key: K) -> V: ... # Etc.` Such a class implicitly

inherits from `Generic`. The runtime semantics of this syntax are discussed in the [Language Reference](#). This class can then be used as follows:

```
def lookup_name[X, Y](mapping: Mapping[X, Y], key: X, default: Y) -> Y: try: return mapping[key] except KeyError: return default
```

Here the brackets after the function

`typing.TypeVar(name, *constraints, bound=None, covariant=False, contravariant=False, infer_variance=False, default=typing.NoDefault)`

`__name__`
`__covariant__`
`__contravariant__`
`__infer_variance__`
`__bound__`
`evaluate_bound()`
`__constraints__`
`evaluate_constraints()`
`__default__`
`evaluate_default()`
`has_default()`

Type variable. The preferred way to construct a type variable is via the dedicated syntax for generic functions, generic classes, and generic type aliases:

```
class Sequence[T]: # T is a TypeVar... This syntax can also be used to create bounded and constrained type variables: class StrSequence[S: str]: # S is a TypeVar with a `str` upper bound; ... # we can say that S is "bounded by `str`" class StrOrBytesSequence[A: (str, bytes)]: # A is a TypeVar constrained to str or bytes... However, if desired, reusable type vari
```

The name of the type variable.

Whether the type var has been explicitly marked as covariant.

Whether the type var has been explicitly marked as contravariant.

Whether the type variable's variance should be inferred by type checkers. Added in version 3.12.

The upper bound of the type variable, if any. Changed in version 3.12: For type variables created through type parameter syntax, the bound is evaluated only when the attribute is accessed, not when the type variable is created (see [Lazy evaluation](#)).

An evaluate function corresponding to the `__bound__` attribute. When called directly, this method supports only the `VALUE` format, which is equivalent to accessing the `__bound__` attribute directly, but the method object can be passed to `annotationlib.call_evaluate_function()` to evaluate the value in a different format. Added in version 3.14.

A tuple containing the constraints of the type variable, if any. Changed in version 3.12: For type variables created through type parameter syntax, the constraints are evaluated only when the attribute is accessed, not when the type variable is created (see [Lazy evaluation](#)).

An evaluate function corresponding to the `__constraints__` attribute. When called directly, this method supports only the `VALUE` format, which is equivalent to accessing the `__constraints__` attribute directly, but the method object can be passed to `annotationlib.call_evaluate_function()` to evaluate the value in a different format. Added in version 3.14.

The default value of the type variable, or `typing.NoDefault` if it has no default. Added in version 3.13.

An evaluate function corresponding to the `__default__` attribute. When called directly, this method supports only the `VALUE` format, which is equivalent to accessing the `__default__` attribute directly, but the method object can

be passed to `annotationlib.call_evaluate_function()` to evaluate the value in a different format. Added in version 3.14.

Return whether or not the type variable has a default value. This is equivalent to checking whether `__default__` is not `typing.NoDefaultSingleton`, except that it does not force evaluation of the lazily evaluated default value. Added in version 3.13.

`typing.TypeVarTuple(name, *, default=typing.NoDefault)`

`__name__`

`__default__`

`evaluate_default()`

`has_default()`

Type variable tuple. A specialized form of type variable that enables variadic generics. Type variable tuples can be declared in type parameter lists using a single asterisk (*) before the name: `def move_first_element_to_last[T, *Ts](tup: tuple[T, *Ts]) -> tuple[*Ts, T]: return (*tup[1:], tup[0])` Or by explicitly invoking the `TypeVarTuple` constructor: `T = TypeVar("T") Ts = TypeVarTuple("Ts") def move_first_element_to_last(tup: tuple[T, *Ts]) -> tuple[*Ts, T]: return (*tup[1:], tup[0])` A normal type variable enables parameterization

The name of the type variable tuple.

The default value of the type variable tuple, or `typing.NoDefault` if it has no default. Added in version 3.13.

A `evaluate` function corresponding to the `__default__` attribute. When called directly, this method supports only the `VALUE` format, which is equivalent to accessing the `__default__` attribute directly, but the method object can be passed to `annotationlib.call_evaluate_function()` to evaluate the value in a different format. Added in version 3.14.

Return whether or not the type variable tuple has a default value. This is equivalent to checking whether `__default__` is not `typing.NoDefaultSingleton`, except that it does not force evaluation of the lazily evaluated default value. Added in version 3.13.

`typing.ParamSpec(name, *, bound=None, covariant=False, contravariant=False, default=typing.NoDefault)`

`args`

`kwargs`

`__name__`

`__default__`

`evaluate_default()`

`has_default()`

Parameter specification variable. A specialized version of type variables. In type parameter lists, parameter specifications can be declared with two asterisks (**): `type IntFunc[**P] = Callable[P, int]` For compatibility with Python 3.11 and earlier, `ParamSpec` objects can also be created as follows: `P = ParamSpec('P')` Parameter specification variables exist primarily for the benefit of static type checkers. They are used to forward the parameter types of one callable to another callable – a pattern commonly

Since `ParamSpec` captures both positional and keyword parameters, `P.args` and `P.kwargs` can be used to split a `ParamSpec` into its components. `P.args` represents the tuple of positional parameters in a given call and should only be used to annotate `*args`. `P.kwargs` represents the mapping of keyword parameters to their values in a given call, and should only be used to annotate `**kwargs`. Both attributes require the annotated parameter to be in scope. At runtime, `P.args` and `P.kwargs` are instances respectively of `Param`

The name of the parameter specification.

The default value of the parameter specification, or `typing.NoDefault` if it has no default. Added in version 3.13.

An evaluate function corresponding to the `__default__` attribute. When called directly, this method supports only the `VALUE` format, which is equivalent to accessing the `__default__` attribute directly, but the method object can be passed to `annotationlib.call_evaluate_function()` to evaluate the value in a different format. Added in version 3.14.

Return whether or not the parameter specification has a default value. This is equivalent to checking whether `__default__` is not `typing.NoDefault` singleton, except that it does not force evaluation of the lazily evaluated default value. Added in version 3.13.

typing.ParamSpecArgs

typing.ParamSpecKwargs

Arguments and keyword arguments attributes of a `ParamSpec`. The `P.args` attribute of a `ParamSpec` is an instance of `ParamSpecArgs`, and `P.kwargs` is an instance of `ParamSpecKwargs`. They are intended for runtime introspection and have no special meaning to static type checkers. Calling `get_origin()` on either of these objects will return the original `ParamSpec`:

```
>>> from typing import ParamSpec, get_origin
>>> P = ParamSpec("P")
>>> get_origin(P.args) is P
True
>>> get_origin(P.kwargs) is P
True
```

 Added in version 3.10.

class typing.TypeAliasType(name, value, *, type_params=())

`__name__`
`__module__`
`__type_params__`
`__value__`

evaluate_value()

The type of type aliases created through the `type` statement. Example:

```
>>> typeAlias = int
>>> typeAlias
<class 'typing.TypeAliasType'>
```

 Added in version 3.12.
`__name__` The name of the type alias:

```
>>> typeAlias = int
>>> Alias.__name__
'Alias'
```

`__module__` The name of the module in which the type alias was defined:

```
>>> typeAlias = int
>>> Alias.__module__
'__main__'
```

`__type_params__` The type parameters of the type alias, or an empty tuple if the alias is not generic:

```
>>> typeListOrSet[T] = list[T] | set[T]
>>> ListOrSet.__type_params__
(T,)
```

The name of the type alias:

```
>>> typeAlias = int
>>> Alias.__name__
'Alias'
```

The name of the module in which the type alias was defined:

```
>>> typeAlias = int
>>> Alias.__module__
'__main__'
```

The type parameters of the type alias, or an empty tuple if the alias is not generic:

```
>>> typeListOrSet[T] = list[T] | set[T]
>>> ListOrSet.__type_params__(T)
(T,)
```

```
>>> typeNotGeneric = int
>>> NotGeneric.__type_params__()
()
```

The type alias's value. This is lazily evaluated, so names used in the definition of the alias are not resolved until the `__value__` attribute is accessed:

```
>>> typeMutually = Recursive
>>> typeRecursive = Mutually
>>> MutuallyMutually
>>> RecursiveRecursive
>>> Mutually.__value__
Recursive
>>> Recursive.__value__
Mutually
```

An evaluate function corresponding to the `__value__` attribute. When called directly, this method supports only the `VALUE` format, which is equivalent to accessing the `__value__` attribute directly, but the method object can be passed to `annotationlib.call_evaluate_function()` to evaluate the value in a different format:

```
>>> typeAlias = undefined
>>> Alias.__value__
Traceback (most recent call last):
...NameError: name 'undefined' is not defined
>>> from annotationlib import Format, call_evaluate_function
>>> Alias.evaluate_value(Format())
```

Other special directives

These functions and classes should not be used directly as annotations. Their intended purpose is to be building blocks for creating and declaring types.

classtyping.NamedTuple

Typed version of `collections.namedtuple()`. Usage: `class Employee(NamedTuple): name: str; id: int` This is equivalent to: `Employee = collections.namedtuple('Employee', ['name', 'id'])` To give a field a default value, you can assign to it in the class

body: `class Employee(NamedTuple): name: str; id: int = 3`
`employee = Employee('Guido')`
`assert employee.id == 3`
Fields with a default value must come after any fields without a default. The resulting class has an extra attribute `__annotations__` giving a dict that maps the field names to t

classtyping.NewType(name, tp)

`__module__`

`__name__`

`__supertype__`

Helper class to create low-overhead distinct types. A `NewType` is considered a distinct type by a type checker. At runtime, however, calling a `NewType` returns its argument unchanged. Usage: `UserId = NewType('UserId', int)`
Declare the `NewType` "UserId"
`first_user = UserId(1)`
"UserId" returns the argument unchanged at runtime
`__module__` The name of the module in which the new type is defined.
`__name__` The name of the new type.
`__supertype__` The type that the new type is based on. Added in version 3.5.2. Changed in versi

The name of the module in which the new type is defined.

The name of the new type.

The type that the new type is based on.

classtyping.Protocol(Generic)

Base class for protocol classes. Protocol classes are defined like this: `class Proto(Protocol): def meth(self) -> int: ...` Such classes are primarily used with static type checkers that recognize structural subtyping (static duck-typing), for example: `class C: def meth(self) -> int: return 0`
`def func(x: Proto) -> int: return x.meth()`
`func(C())` # Passes static type check
See PEP 544 for more details. Protocol classes decorated with `runtime_checkable()` (described later) act as simple-minded runtime protocols that check only the

@typing.runtime_checkable

Mark a protocol class as a runtime protocol. Such a protocol can be used with `isinstance()` and `issubclass()`. This allows a simple-minded structural check, very similar to "one-trick ponies" in `collections.abc` such as `Iterable`. For example: `@runtime_checkable class Closable(Protocol): def close(self): ...`
`assert isinstance(open('/some/file'), Closable)`
`@runtime_checkable class Named(Protocol): name: str`
`import threading`
`assert isinstance(threading.Thread(name='Bob'), Named)`
This decorator raises `TypeError` when applied to a

classtyping.TypedDict(dict)

`__total__`

`__required_keys__`

`__optional_keys__`

`__readonly_keys__`

`__mutable_keys__`

Special construct to add type hints to a dictionary. At runtime "TypedDict instances" are simply dicts. `TypedDict` declares a dictionary type that expects all of its instances to have a certain set of keys,

where each key is associated with a value of a consistent type. This expectation is not checked at runtime but is only enforced by type checkers.

```
Usage: class Point2D(TypedDict): x: int; y: int; label: str; Point2D={'x':1, 'y':2, 'label': 'good'}#
```

```
OK; Point2D={'z':3, 'label': 'bad'}# Fails type check; assert Point2D
```

`Point2D.__total__` gives the value of the `total` argument. Example: `>>> from typing import TypedDict >>> class Point2D(TypedDict): pass >>> Point2D.__total__ True >>> class Point2D(TypedDict, total=False): pass >>> Point2D.__total__ False >>> class Point3D(Point2D): pass >>> Point3D.__total__ True` This attribute reflects only the value of the `total` argument to the current `TypedDict` class, not whether the class is semantically total. For example, a `TypedDict` with `__total__` set to `True` may have keys marked with `NotRequired`, or it may inherit

Added in version 3.9.

`Point2D.__required_keys__` and `Point2D.__optional_keys__` return `frozenset` objects containing required and non-required keys, respectively. Keys marked with `Required` will always appear in `__required_keys__` and keys marked with `NotRequired` will always appear in `__optional_keys__`. For backwards compatibility with Python 3.10 and below, it is also possible to use inheritance to declare both required and non-required keys in the same `TypedDict`. This is done by declaring a `TypedDict` with one value for the `total` argument

A `frozenset` containing the names of all read-only keys. Keys are read-only if they carry the `ReadOnly` qualifier. Added in version 3.13.

A `frozenset` containing the names of all mutable keys. Keys are mutable if they do not carry the `ReadOnly` qualifier. Added in version 3.13.

Protocols

The following protocols are provided by the `typing` module. All are decorated with `@runtime_checkable`.

`classtyping.SupportsAbs`

An ABC with one abstract method `__abs__` that is covariant in its return type.

`classtyping.SupportsBytes`

An ABC with one abstract method `__bytes__`.

`classtyping.SupportsComplex`

An ABC with one abstract method `__complex__`.

`classtyping.SupportsFloat`

An ABC with one abstract method `__float__`.

`classtyping.SupportsIndex`

An ABC with one abstract method `__index__`. Added in version 3.8.

`classtyping.SupportsInt`

An ABC with one abstract method `__int__`.

`classtyping.SupportsRound`

An ABC with one abstract method `__round__` that is covariant in its return type.

ABCs and Protocols for working with I/O

`classtyping.IO[AnyStr]`

`classtyping.TextIO`

`classtyping.BinaryIO`

Generic class `IO[AnyStr]` and its subclasses `TextIO(IO[str])` and `BinaryIO(IO[bytes])` represent the types of I/O streams such as returned by `open()`. Please note that these classes are not protocols, and their interface is fairly broad.

The protocols `io.Reader` and `io.Writer` offer a simpler alternative for argument types, when only `read()` or `write()` methods are accessed, respectively:

```
def read_and_write(reader: Reader[str], writer: Writer[bytes]):
    data = reader.read()
    writer.write(data.encode())
```

Also consider using `collections.abc.Iterable` for iterating over the lines of an input stream:

```
def read_config(stream: Iterable[str]):
    for line in stream:
        ...
```

Functions and decorators

`typing.cast(typ, val)`

Cast a value to a type. This returns the value unchanged. To the type checker this signals that the return value has the designated type, but at runtime we intentionally don't check anything (we want this to be as fast as possible).

`typing.assert_type(val, typ, /)`

Ask a static type checker to confirm that `val` has an inferred type of `typ`. At runtime this does nothing: it returns the first argument unchanged with no checks or side effects, no matter the actual type of the argument. When a static type checker encounters a call to `assert_type()`, it emits an error if the value is not of the specified type: `def greet(name: str) -> None: assert_type(name, str) # OK, inferred type of `name` is `str`` `assert_type(name, int) # type checker error` This function is useful for ensuring t

`typing.assert_never(arg, /)`

Ask a static type checker to confirm that a line of code is unreachable. Example: `def int_or_str(arg: int | str) -> None: match arg: case int(): print("It's an int") case str(): print("It's a str") case _ as unreachable: assert_never(unreachable)` Here, the annotations allow the type checker to infer that the last case can never execute, because `arg` is either an `int` or a `str`, and both options are covered by earlier cases. If a type checker finds that a call to `assert_never()` is reachable, it will emit an error. For example, i

`typing.reveal_type(obj, /)`

Ask a static type checker to reveal the inferred type of an expression. When a static type checker encounters a call to this function, it emits a diagnostic with the inferred type of the argument. For example: `x: int = 1` `reveal_type(x)` # Revealed type is "builtins.int" This can be useful when you want to debug how your type checker handles a particular piece of code. At runtime, this function prints the runtime type of its argument to `sys.stderr` and returns the argument unchanged (allowing the call to be u

`@typing.dataclass_transform(*, eq_default=True, order_default=False, kw_only_default=False, frozen_default=False, field_specifiers=(), **kwargs)`

Parameters:

Decorator to mark an object as providing `dataclass`-like behavior. `dataclass_transform` may be used to decorate a class, metaclass, or a function that is itself a decorator. The presence of `@dataclass_transform()` tells a static type checker that the decorated object performs runtime "magic" that transforms a class in a similar way to `@dataclasses.dataclass`. Example usage with a decorator function: `@dataclass_transform() def create_model[T](cls: type[T]) -> type[T]: ... return cls @ create_model class CustomerModel: id`

`eq_default(bool)` – Indicates whether the `eq` parameter is assumed to be `True` or `False` if it is omitted by the caller. Defaults to `True`. `order_default(bool)` – Indicates whether the `order` parameter is assumed to be `True` or `False` if it is omitted by the caller. Defaults to `False`. `kw_only_default(bool)` – Indicates whether the `kw_only` parameter is assumed to be `True` or `False` if it is omitted by the caller. Defaults to `False`. `frozen_default(bool)` – Indicates whether the `frozen` parameter is assumed to be `True` or `False` if it is omitted by the caller. Defaults to `False`.

@typing.overload

Decorator for creating overloaded functions and methods. The `@overload` decorator allows describing functions and methods that support multiple different combinations of argument types. A series of `@overload`-decorated definitions must be followed by exactly one non-`@overload`-decorated definition (for the same function/method). `@overload`-decorated definitions are for the benefit of the type checker only, since they will be overwritten by the non-`@overload`-decorated definition. The non-`@overload`-decorated definition is the one that is actually used at runtime.

typing.get_overloads(func)

Return a sequence of `@overload`-decorated definitions for `func`. `func` is the function object for the implementation of the overloaded function. For example, given the definition of `process` in the documentation for `@overload`, `get_overloads(process)` will return a sequence of three function objects for the three defined overloads. If called on a function with no overloads, `get_overloads()` returns an empty sequence. `get_overloads()` can be used for introspecting an overloaded function at runtime. Added in version 3.7.

typing.clear_overloads()

Clear all registered overloads in the internal registry. This can be used to reclaim the memory used by the registry. Added in version 3.11.

@typing.final

Decorator to indicate final methods and final classes. Decorating a method with `@final` indicates to a type checker that the method cannot be overridden in a subclass. Decorating a class with `@final` indicates that it cannot be subclassed. For example: `class Base: @final def done(self) -> None: ...` `class Sub(Base): def done(self) -> None: # Error reported by type checker...` `@final class Leaf: ...` `class Other(Leaf): # Error reported by type checker...` There is no runtime checking of these properties. See PEP 591 for more details.

@typing.no_type_check

Decorator to indicate that annotations are not type hints. This works as a class or function decorator. With a class, it applies recursively to all methods and classes defined in that class (but not to methods defined in its superclasses or subclasses). Type checkers will ignore all annotations in a function or class with this decorator. `@no_type_check` mutates the decorated object in place.

@typing.no_type_check_decorator

Decorator to give another decorator the `@no_type_check()` effect. This wraps the decorator with something that wraps the decorated function in `@no_type_check()`. Deprecated since version 3.13, will be removed in version 3.15: No type checker ever added support for `@no_type_check_decorator`. It is therefore deprecated, and will be removed in Python 3.15.

@typing.override

Decorator to indicate that a method in a subclass is intended to override a method or attribute in a superclass. Type checkers should emit an error if a method decorated with `@override` does not, in fact, override anything. This helps prevent bugs that may occur when a base class is changed without an equivalent change to a child class. For example: `class Base: def log_status(self) -> None: ...` `class Sub(Base): @override def log_status(self) -> None: # Okay: overrides Base.log_status...` `@overridedef done(self) -> None: # Error: does not override anything`

@typing.type_check_only

Decorator to mark a class or function as unavailable at runtime. This decorator is itself not available at runtime. It is mainly intended to mark classes that are defined in type stub files if an implementation returns an instance of a private class: `@type_check_only class Response: # private or not available at runtime`
`code: int`
`def get_header(self, name: str) -> str: ...`
`def fetch_response() -> Response: ...`
Note that returning instances of private classes is not recommended. It is usually preferable to make such

Introspection helpers

typing.get_type_hints(obj, globalns=None, localns=None, include_extras=False)

Return a dictionary containing type hints for a function, method, module, class object, or other callable object. This is often the same as `obj.__annotations__`, but this function makes the following changes to the annotations dictionary: Forward references encoded as string literals or `ForwardRef` objects are handled by evaluating them in `globalns`, `localns`, and (where applicable) `obj`'s type parameter namespace. If `globalns` or `localns` is not given, appropriate namespace dictionaries are inferred from `obj`. `None` is

typing.get_origin(tp)

Get the unsubscripted version of a type: for a typing object of the form `X[Y, Z, ...]` return `X`. If `X` is a typing-module alias for a builtin or `collections` class, it will be normalized to the original class. If `X` is an instance of `ParamSpecArgs` or `ParamSpecKwargs`, return the underlying `ParamSpec`. Return `None` for unsupported objects. Examples: `assert get_origin(str) is None`, `assert get_origin(Dict[str, int]) is dict`, `assert get_origin(Union[int, str]) is Union`, `assert get_origin(Annotated[str, 'metadata']) is Annotated`, `P = ParamSpec('P')`, `assert get_origin(P) is P`

typing.get_args(tp)

Get type arguments with all substitutions performed: for a typing object of the form `X[Y, Z, ...]` return `(Y, Z, ...)`. If `X` is a union or `Literal` contained in another generic type, the order of `(Y, Z, ...)` may be different from the order of the original arguments `[Y, Z, ...]` due to type caching. Return `()` for unsupported objects. Examples: `assert get_args(int) == ()`, `assert get_args(Dict[int, str]) == (int, str)`, `assert get_args(Union[int, str]) == (int, str)` Added in version 3.8.

typing.get_protocol_members(tp)

Return the set of members defined in a `Protocol`.
`>>> from typing import Protocol, get_protocol_members`
`>>> class SP(Protocol): ...`
`def a(self) -> str: ...`
`def b(self) -> int: ...`
`>>> get_protocol_members(P) == frozenset({'a', 'b'})`
`True`
Raise `TypeError` for arguments that are not `Protocols`. Added in version 3.13.

typing.is_protocol(tp)

Determine if a type is a `Protocol`. For example: `class P(Protocol):`
`def a(self) -> str: ...`
`def b(self) -> int: ...`
`>>> is_protocol(P) # => True`
`is_protocol(int) # => False` Added in version 3.13.

typing.is_typeddict(tp)

Check if a type is a `TypedDict`. For example: `class Film(TypedDict):`
`title: str`
`year: int`
`>>> is_typeddict(Film) # True`
`is_typeddict(list[str]) # False`
`is_typeddict(TypedDict) # False` Added in version 3.10.

typing.ForwardRef

Class used for internal typing representation of string forward references. For example, `List["SomeClass"]` is implicitly transformed into `List[ForwardRef("SomeClass")]`. `ForwardRef` should not be instantiated by a user, but may be used by introspection tools. Note PEP 585 generic types such as `list["SomeClass"]` will not be implicitly transformed into `list[ForwardRef("SomeClass")]` and thus will not automatically resolve to `list[SomeClass]`. Added in version 3.7.4. Changed in version 3.14: This is now an alias for `ann`

typing.evaluate_forward_ref(forward_ref, *, owner=None, globals=None, locals=None, type_params=None, format=annotationlib.Format.VALUE)

Evaluate an annotation `lib.ForwardRef` as a type hint. This is similar to calling `annotationlib.ForwardRef.evaluate()`, but unlike that method, `evaluate_forward_ref()` also recursively evaluates forward references nested within the type hint. See the documentation for `annotationlib.ForwardRef.evaluate()` for the meaning of the `owner`, `globals`, `locals`, `type_params`, and `format_parameters`. Caution This function may execute arbitrary code contained in annotations. See Security implications of introspecting annotations for more.

typing.NoDefault

A sentinel object used to indicate that a type parameter has no default value. For example:

```
>>> T = TypeVar("T")
>>> T.__default__ is typing.NoDefault
True
>>> S = TypeVar("S", default=None)
>>> S.__default__ is None
True
Added in version 3.13.
```

Constant

typing.TYPE_CHECKING

A special constant that is assumed to be `True` by 3rd party static type checkers. It's `False` at runtime. A module which is expensive to import, and which only contains types used for typing annotations, can be safely imported inside an `if TYPE_CHECKING:` block. This prevents the module from actually being imported at runtime; annotations aren't eagerly evaluated (see PEP 649) so using undefined symbols in annotations is harmless—as long as you don't later examine them. Your static type analysis tool will see

Deprecated aliases

This module defines several deprecated aliases to pre-existing standard library classes. These were originally included in the `typing` module in order to support parameterizing these generic classes using []. However, the aliases became redundant in Python 3.9 when the corresponding pre-existing classes were enhanced to support [] (see PEP 585).

The redundant types are deprecated as of Python 3.9. However, while the aliases may be removed at some point, removal of these aliases is not currently planned. As such, no deprecation warnings are currently issued by the interpreter for these aliases.

If at some point it is decided to remove these deprecated aliases, a deprecation warning will be issued by the interpreter for at least two releases prior to removal. The aliases are guaranteed to remain in the `typing` module without deprecation warnings until at least Python 3.14.

Type checkers are encouraged to flag uses of the deprecated types if the program they are checking targets a minimum Python version of 3.9 or newer.

Aliases to built-in types

typing.Dict(dict, MutableMapping[KT, VT])

Deprecated alias to `dict`. Note that to annotate arguments, it is preferred to use an abstract collection type such as `Mapping` rather than to use `dict` or `typing.Dict`. Deprecated since version 3.9: `builtins.dict` now supports subscripting []. See PEP 585 and Generic Alias Type.

typing.List(list, MutableSequence[T])

Deprecated alias to `list`. Note that to annotate arguments, it is preferred to use an abstract collection type such as `Sequence` or `Iterable` rather than to use `list` or `typing.List`. Deprecated since version 3.9: `builtins.list` now supports subscripting []. See PEP 585 and Generic Alias Type.

typing.Set(set, MutableSet[T])

Deprecated alias to `builtins.set`. Note that to annotate arguments, it is preferred to use an abstract collection type such as `collections.abc.Set` rather than to use `set` or `typing.Set`. Deprecated since version 3.9: `builtins.set` now supports subscripting []. See PEP 585 and Generic Alias Type.

classtyping.FrozenSet(frozenset,AbstractSet[T_co])

Deprecated alias to `builtins.frozenset`. Deprecated since version 3.9: `builtins.frozenset` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

typing.Tuple

Deprecated alias for `tuple`. `tuple` and `Tuple` are special-cased in the type system; see `Annotating tuples` for more details. Deprecated since version 3.9: `builtins.tuple` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

classtyping.Type(Generic[CT_co])

Deprecated alias to `type`. See `The type of class objects` for details on using `type` or `typing.Type` in type annotations. Added in version 3.5.2. Deprecated since version 3.9: `builtins.type` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

Aliases to types in collections

classtyping.DefaultDict(collections.defaultdict,MutableMapping[KT,VT])

Deprecated alias to `collections.defaultdict`. Added in version 3.5.2. Deprecated since version 3.9: `collections.defaultdict` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

classtyping.OrderedDict(collections.OrderedDict,MutableMapping[KT,VT])

Deprecated alias to `collections.OrderedDict`. Added in version 3.7.2. Deprecated since version 3.9: `collections.OrderedDict` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

classtyping.ChainMap(collections.ChainMap,MutableMapping[KT,VT])

Deprecated alias to `collections.ChainMap`. Added in version 3.6.1. Deprecated since version 3.9: `collections.ChainMap` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

classtyping.Counter(collections.Counter,Dict[T,int])

Deprecated alias to `collections.Counter`. Added in version 3.6.1. Deprecated since version 3.9: `collections.Counter` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

classtyping.Deque(deque,MutableSequence[T])

Deprecated alias to `collections.deque`. Added in version 3.6.1. Deprecated since version 3.9: `collections.deque` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

Aliases to other concrete types

classtyping.Pattern

classtyping.Match

Deprecated aliases corresponding to the return types from `re.compile()` and `re.match()`. These types (and the corresponding functions) are generic over `AnyStr`. `Pattern` can be specialised as `Pattern[str]` or `Pattern[bytes]`; `Match` can be specialised as `Match[str]` or `Match[bytes]`. Deprecated since version 3.9: `ClassesPattern` and `Match` from `re` now support `[]`. See PEP 585 and Generic Alias Type.

classtyping.Text

Deprecated alias for `str`. `Text` is provided to supply a forward compatible path for Python 2 code: in Python 2, `Text` is an alias for `unicode`. Use `Text` to indicate that a value must contain a unicode string in a manner that is compatible with both Python 2 and Python 3: `def add_unicode_checkmark(text: Text) -> Text: return text + u'\u2713'` Added in version 3.5.2. Deprecated since version 3.11: Python 2 is no longer supported, and most type checkers also no longer support type checking Python 2 code. Removal of the ali

Aliases to container ABCs in collections.abc

classtyping.AbstractSet(Collection[T_co])

Deprecated alias to collections.abc.Set. Deprecated since version 3.9: collections.abc.Set now supports subscripting ([]). See PEP 585 and Generic Alias Type.

classtyping.ByteString(Sequence[int])

Deprecated alias to collections.abc.ByteString. Use isinstance(obj, collections.abc.Buffer) to test if obj implements the buffer protocol at runtime. For use in type annotations, either use Buffer or a union that explicitly specifies the types your code supports (e.g., bytes|bytearray|memoryview). ByteString was originally intended to be an abstract class that would serve as a supertype of both bytes and bytearray. However, since the ABC never had any methods, knowing that an object was an instance of ByteString

classtyping.Collection(Sized, Iterable[T_co], Container[T_co])

Deprecated alias to collections.abc.Collection. Added in version 3.6. Deprecated since version 3.9: collections.abc.Collection now supports subscripting ([]). See PEP 585 and Generic Alias Type.

classtyping.Container(Generic[T_co])

Deprecated alias to collections.abc.Container. Deprecated since version 3.9: collections.abc.Container now supports subscripting ([]). See PEP 585 and Generic Alias Type.

classtyping.ItemsView(MappingView, AbstractSet[tuple[KT_co, VT_co]])

Deprecated alias to collections.abc.ItemsView. Deprecated since version 3.9: collections.abc.ItemsView now supports subscripting ([]). See PEP 585 and Generic Alias Type.

classtyping.KeysView(MappingView, AbstractSet[KT_co])

Deprecated alias to collections.abc.KeysView. Deprecated since version 3.9: collections.abc.KeysView now supports subscripting ([]). See PEP 585 and Generic Alias Type.

classtyping.Mapping(Collection[KT], Generic[KT, VT_co])

Deprecated alias to collections.abc.Mapping. Deprecated since version 3.9: collections.abc.Mapping now supports subscripting ([]). See PEP 585 and Generic Alias Type.

classtyping.MappingView(Sized)

Deprecated alias to collections.abc.MappingView. Deprecated since version 3.9: collections.abc.MappingView now supports subscripting ([]). See PEP 585 and Generic Alias Type.

classtyping.MutableMapping(Mapping[KT, VT])

Deprecated alias to collections.abc.MutableMapping. Deprecated since version 3.9: collections.abc.MutableMapping now supports subscripting ([]). See PEP 585 and Generic Alias Type.

classtyping.MutableSequence(Sequence[T])

Deprecated alias to collections.abc.MutableSequence. Deprecated since version 3.9: collections.abc.MutableSequence now supports subscripting ([]). See PEP 585 and Generic Alias Type.

classtyping.MutableSet(AbstractSet[T])

Deprecated alias to collections.abc.MutableSet. Deprecated since version 3.9: collections.abc.MutableSet now supports subscripting ([]). See PEP 585 and Generic Alias Type.

classtyping.Sequence(Reversible[T_co], Collection[T_co])

Deprecated alias to collections.abc.Sequence. Deprecated since version 3.9: collections.abc.Sequence now supports subscripting ([]). See PEP 585 and Generic Alias Type.

classtyping.ValuesView(MappingView,Collection[_VT_co])

Deprecated alias to `collections.abc.ValuesView`. Deprecated since version 3.9: `collections.abc.ValuesView` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

Aliases to asynchronous ABCs in collections.abc

classtyping.Coroutine(Awaitable[ReturnType],Generic[YieldType,SendType,ReturnType])

Deprecated alias to `collections.abc.Coroutine`. See `Annotating generators and coroutines` for details on using `collections.abc.Coroutine` and `typing.Coroutine` in type annotations. Added in version 3.5.3. Deprecated since version 3.9: `collections.abc.Coroutine` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

classtyping.AsyncGenerator(AsyncIterator[YieldType],Generic[YieldType,SendType])

Deprecated alias to `collections.abc.AsyncGenerator`. See `Annotating generators and coroutines` for details on using `collections.abc.AsyncGenerator` and `typing.AsyncGenerator` in type annotations. Added in version 3.6.1. Deprecated since version 3.9: `collections.abc.AsyncGenerator` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type. Changed in version 3.13: The `SendType` parameter now has a default.

classtyping.AsyncIterable(Generic[T_co])

Deprecated alias to `collections.abc.AsyncIterable`. Added in version 3.5.2. Deprecated since version 3.9: `collections.abc.AsyncIterable` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

classtyping.AsyncIterator(AsyncIterable[T_co])

Deprecated alias to `collections.abc.AsyncIterator`. Added in version 3.5.2. Deprecated since version 3.9: `collections.abc.AsyncIterator` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

classtyping.Awaitable(Generic[T_co])

Deprecated alias to `collections.abc.Awaitable`. Added in version 3.5.2. Deprecated since version 3.9: `collections.abc.Awaitable` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

Aliases to other ABCs in collections.abc

classtyping.Iterable(Generic[T_co])

Deprecated alias to `collections.abc.Iterable`. Deprecated since version 3.9: `collections.abc.Iterable` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

classtyping.Iterator(Iterable[T_co])

Deprecated alias to `collections.abc.Iterator`. Deprecated since version 3.9: `collections.abc.Iterator` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

typing.Callable

Deprecated alias to `collections.abc.Callable`. See `Annotating callable objects` for details on how to use `collections.abc.Callable` and `typing.Callable` in type annotations. Deprecated since version 3.9: `collections.abc.Callable` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type. Changed in version 3.10: `Callable` now supports `ParamSpec` and `Concatenate`. See PEP 612 for more details.

classtyping.Generator(Iterator[YieldType],Generic[YieldType,SendType,ReturnType])

Deprecated alias to `collections.abc.Generator`. See `Annotating generators and coroutines` for details on using `collections.abc.Generator` and `typing.Generator` in type annotations. Deprecated since version 3.9: `collections.abc.Generator` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type. Changed in version 3.13: Default values for the `send` and `return` types were added.

classtyping.Hashable

Deprecated alias `tocollections.abc.Hashable`. Deprecated since version 3.12: Use `collections.abc.Hashable` directly instead.

`classtyping.Reversible(Iterable[T_co])`

Deprecated alias `tocollections.abc.Reversible`. Deprecated since version 3.9: `collections.abc.Reversible` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type.

`classtyping.Sized`

Deprecated alias `tocollections.abc.Sized`. Deprecated since version 3.12: Use `collections.abc.Sized` directly instead.

Aliases to contextlib ABCs

`classtyping.ContextManager(Generic[T_co, ExitT_co])`

Deprecated alias `tocontextlib.AbstractContextManager`. The first type parameter, `T_co`, represents the type returned by the `__enter__()` method. The optional second type parameter, `ExitT_co`, which defaults to `bool|None`, represents the type returned by the `__exit__()` method. Added in version 3.5.4. Deprecated since version 3.9: `contextlib.AbstractContextManager` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type. Changed in version 3.13: Added the optional second type parameter, `ExitT_co`.

`classtyping.AsyncContextManager(Generic[T_co, AExitT_co])`

Deprecated alias `tocontextlib.AbstractAsyncContextManager`. The first type parameter, `T_co`, represents the type returned by the `__aenter__()` method. The optional second type parameter, `AExitT_co`, which defaults to `bool|None`, represents the type returned by the `__aexit__()` method. Added in version 3.6.2. Deprecated since version 3.9: `contextlib.AbstractAsyncContextManager` now supports subscripting (`[]`). See PEP 585 and Generic Alias Type. Changed in version 3.13: Added the optional second type parameter, `AExitT_co`.

Deprecation Timeline of Major Features

Certain features in typing are deprecated and may be removed in a future version of Python. The following table summarizes major deprecations for your convenience. This is subject to change, and not all deprecations are listed.

Feature	Deprecated in	Projected removal	PEP/issue	typing	versions of standard collections
<code>typing.ByteString</code>	3.9	Undecided (see gh-91896)	PEP 585		
<code>typing.Hashable</code> and <code>typing.Sized</code>	3.9 3.17	Undecided gh-92332			
<code>typing.TypeAlias</code>	3.12	Undecided gh-94309			
<code>@typing.no_type_check_decorator</code>	3.13 3.15	Undecided gh-106309			
<code>typing.AnyStr</code>	3.13 3.18	Undecided gh-105578			

pydoc— Documentation generator and online help system

Source code: `Lib/pydoc.py`

The `pydoc` module automatically generates documentation from Python modules. The documentation can be presented as pages of text on the console, served to a web browser, or saved to HTML files.

For modules, classes, functions and methods, the displayed documentation is derived from the docstring (i.e. the `__doc__` attribute) of the object, and recursively of its documentable members. If there is no docstring, `pydoc` tries to obtain a description from the block of comment lines just above the definition of the class, function or method in the source file, or at the top of the module (see `inspect.getcomments()`).

The built-in function `help()` invokes the online help system in the interactive interpreter, which uses `pydoc` to generate its documentation as text on the console. The same text documentation can also be viewed from outside the Python interpreter by running `pydoc` as a script at the operating system's command prompt. For example, running

```
python -m pydoc sys
```

at a shell prompt will display documentation on the `sys` module, in a style similar to the manual pages shown by the Unix `man` command. The argument to `pydoc` can be the name of a function, module, or package, or a dotted reference to a class, method, or function within a module or module in a package. If the argument to `pydoc` looks like a path (that is, it contains the path separator for your operating system, such as a slash in Unix), and refers to an existing Python source file, then documentation is produced for that file.

■ **Nota:** In order to find objects and their documentation, `pydoc` imports the module(s) to be documented. Therefore, any code on module level will be executed on that occasion. Use `if __name__ == '__main__':` guard to only execute code when a file is invoked as a script and not just imported.

When printing output to the console, `pydoc` attempts to paginate the output for easier reading. If either the `MANPAGER` or the `PAGER` environment variable is set, `pydoc` will use its value as a pagination program. When both are set, `MANPAGER` is used.

Specifying a `-w` flag before the argument will cause HTML documentation to be written out to a file in the current directory, instead of displaying text on the console.

Specifying a `-k` flag before the argument will search the synopsis lines of all available modules for the keyword given as the argument, again in a manner similar to the Unix `man` command. The synopsis line of a module is the first line of its documentation string.

You can also use `pydoc` to start an HTTP server on the local machine that will serve documentation to visiting web browsers. `python -m pydoc -p 1234` will start a HTTP server on port 1234, allowing you to browse the documentation at `http://localhost:1234/` in your preferred web browser. Specifying `0` as the port number will select an arbitrary unused port.

`python -m pydoc -n <hostname>` will start the server listening at the given hostname. By default the hostname is 'localhost' but if you want the server to be reached from other machines, you may want to change the host name that the server responds to. During development this is especially useful if you want to run `pydoc` from within a container.

`python -m pydoc -b` will start the server and additionally open a web browser to a module index page. Each served page has a navigation bar at the top where you can get help on an individual item, search all modules with a keyword in their synopsis line, and go to the Module index, Topics and Keywords pages.

When `pydoc` generates documentation, it uses the current environment and path to locate modules. Thus, invoking `pydoc spam` documents precisely the version of the module you would get if you started the Python

interpreter and `typedimportspam`.

Module docs for core modules are assumed to reside in `https://docs.python.org/X.Y/library/whereXandY` are the major and minor version numbers of the Python interpreter. This can be overridden by setting the `PYTHONDOCSenvironment` variable to a different URL or to a local directory containing the Library Reference Manual pages.

Changed in version 3.2: Added the `-b` option.

Changed in version 3.3: The `-g` command line option was removed.

Changed in version 3.4: `pydocnow` uses `inspect.signature()` rather than `inspect.getfullargspec()` to extract signature information from callables.

Changed in version 3.7: Added the `-n` option.

Python Development Mode

Added in version 3.7.

The Python Development Mode introduces additional runtime checks that are too expensive to be enabled by default. It should not be more verbose than the default if the code is correct; new warnings are only emitted when an issue is detected.

It can be enabled using the `-Xdev` command line option or by setting the `PYTHONDEVMODE` environment variable to `1`.

See also [Python debug build](#).

Effects of the Python Development Mode

Enabling the Python Development Mode is similar to the following command, but with additional effects described below:

```
PYTHONMALLOC=debug PYTHONASYNCIODEBUG=1 python -W default -X faulthandler
```

Effects of the Python Development Mode:

- Add default warning filter. The following warnings are shown: `DeprecationWarning`, `ImportWarning`, `PendingDeprecationWarning`, `ResourceWarning`. Normally, the above warnings are filtered by the default warning filters. It behaves as if the `-W default` command line option is used. Use the `-W error` command line option or set the `PYTHONWARNINGSENVIRONMENT` variable to `error` to treat warnings as errors.
- Install debug hooks on memory allocators to check for: `Buffer underflow`, `Buffer overflow`, `Memory allocator API violation`, `Unsafe usage of the GIL`. See the `PyMem_SetupDebugHooks()` C function. It behaves as if the `PYTHONMALLOC` environment variable is set to `debug`. To enable the Python Development Mode without installing debug hooks on memory allocators, set the `PYTHONMALLOC` environment variable to `default`.
- Call `faulthandler.enable()` at Python startup to install handlers for the `SIGSEGV`, `SIGFPE`, `SIGABRT`, `SIGBUS` and `SIGILL` signals to dump the Python traceback on a crash. It behaves as if the `-X faulthandler` command line option is used or if the `PYTHONFAULTHANDLER` environment variable is set to `1`.
- Enable `asyncio` debug mode. For example, `asyncio` checks for coroutines that were not awaited and logs them. It behaves as if the `PYTHONASYNCIODEBUG` environment variable is set to `1`.
- Check the `encoding` and `errors` arguments for string encoding and decoding operations. Examples: `open()`, `str.encode()` and `bytes.decode()`. By default, for best performance, the `errors` argument is only checked at the first encoding/decoding error and the `encoding` argument is sometimes ignored for empty strings.
- The `io.IOBase` destructor logs `close()` exceptions.
- Set the `dev_mode` attribute of `sys.flags` to `True`.

The Python Development Mode does not enable the `tracemalloc` module by default, because the overhead cost (to performance and memory) would be too large. Enabling the `tracemalloc` module provides additional information on the origin of some errors. For example, `ResourceWarning` logs the traceback where the resource was allocated, and a buffer overflow error logs the traceback where the memory block was allocated.

The Python Development Mode does not prevent the `-O` command line option from removing `assert` statements nor from setting `__debug__` to `False`.

The Python Development Mode can only be enabled at the Python startup. Its value can be read from `sys.flags.dev_mode`.

Changed in version 3.8: `Theio.IOBasedestructor` now logs `close()` exceptions.

Changed in version 3.9: `Theencodinganderrors` arguments are now checked for string encoding and decoding operations.

ResourceWarning Example

Example of a script counting the number of lines of the text file specified in the command line:

```
import sys

def main():
    fp = open(sys.argv[1])
    nlines = len(fp.readlines())
    print(nlines)
    # The file is closed implicitly

if __name__ == "__main__":
    main()
```

The script does not close the file explicitly. By default, Python does not emit any warning. Example using `README.txt`, which has 269 lines:

```
$ python script.py README.txt
269
```

Enabling the Python Development Mode displays a `ResourceWarning`:

```
$ python -X dev script.py README.txt
269
script.py:10: ResourceWarning: unclosed file <_io.TextIOWrapper name='README.rst' mode='r' encoding='UTF-8'>
    main()
ResourceWarning: Enable tracemalloc to get the object allocation traceback
```

In addition, enabling `tracemalloc` shows the line where the file was opened:

```
$ python -X dev -X tracemalloc=5 script.py README.rst
269
script.py:10: ResourceWarning: unclosed file <_io.TextIOWrapper name='README.rst' mode='r' encoding='UTF-8'>
    main()
Object allocated at (most recent call last):
  File "script.py", line 10
    main()
  File "script.py", line 4
    fp = open(sys.argv[1])
```

The fix is to close explicitly the file. Example using a context manager:

```
def main():
    # Close the file explicitly when exiting the with block
    with open(sys.argv[1]) as fp:
        nlines = len(fp.readlines())
    print(nlines)
```

Not closing a resource explicitly can leave a resource open for way longer than expected; it can cause severe issues upon exiting Python. It is bad in CPython, but it is even worse in PyPy. Closing resources explicitly makes an application more deterministic and more reliable.

Bad file descriptor error example

Script displaying the first line of itself:

```
import os

def main():
    fp = open(__file__)
    firstline = fp.readline()
    print(firstline.rstrip())
    os.close(fp.fileno())
    # The file is closed implicitly

main()
```

By default, Python does not emit any warning:

```
$ python script.py
import os
```

The Python Development Mode shows a `ResourceWarning` and logs a “Bad file descriptor” error when finalizing the file object:

```
$ python -X dev script.py
import os
script.py:10: ResourceWarning: unclosed file <_io.TextIOWrapper name='script.py' mode='r' encoding='UTF-8'>
    main()
ResourceWarning: Enable tracemalloc to get the object allocation traceback
Exception ignored in: <_io.TextIOWrapper name='script.py' mode='r' encoding='UTF-8'>
Traceback (most recent call last):
  File "script.py", line 10, in <module>
    main()
OSError: [Errno 9] Bad file descriptor
```

`os.close(fp.fileno())` closes the file descriptor. When the file object finalizer tries to close the file descriptor again, it fails with the `BadFileDescriptorError`. A file descriptor must be closed only once. In the worst case scenario, closing it twice can lead to a crash (see [bpo-18748](#) for an example).

The fix is to remove the `os.close(fp.fileno())` line, or open the file with `closefd=False`.

doctest— Test interactive Python examples

Source code: `Lib/doctest.py`

The `doctest` module searches for pieces of text that look like interactive Python sessions, and then executes those sessions to verify that they work exactly as shown. There are several common ways to use `doctest`:

- To check that a module's docstrings are up-to-date by verifying that all interactive examples still work as documented.
- To perform regression testing by verifying that interactive examples from a test file or a test object work as expected.
- To write tutorial documentation for a package, liberally illustrated with input-output examples. Depending on whether the examples or the expository text are emphasized, this has the flavor of "literate testing" or "executable documentation".

Here's a complete but small example module:

```
"""
This is the "example" module.

The example module supplies one function, factorial().  For example,

>>> factorial(5)
120
"""

def factorial(n):
    """Return the factorial of n, an exact integer >= 0.

    >>> [factorial(n) for n in range(6)]
    [1, 1, 2, 6, 24, 120]
    >>> factorial(30)
    2652528598121910586363084800000000
    >>> factorial(-1)
    Traceback (most recent call last):
    ...
    ValueError: n must be >= 0

    Factorials of floats are OK, but the float must be an exact integer:
    >>> factorial(30.1)
    Traceback (most recent call last):
    ...
    ValueError: n must be exact integer
    >>> factorial(30.0)
    2652528598121910586363084800000000

    It must also not be ridiculously large:
    >>> factorial(1e100)
    Traceback (most recent call last):
    ...
    OverflowError: n too large
    """

    import math
    if not n >= 0:
        raise ValueError("n must be >= 0")
    if math.floor(n) != n:
        raise ValueError("n must be exact integer")
    if n+1 == n: # catch a value like 1e300
        raise OverflowError("n too large")
    result = 1
```

```

factor = 2
while factor <= n:
    result *= factor
    factor += 1
return result

```

```

if __name__ == "__main__":
    import doctest
    doctest.testmod()

```

If you run `example.py` directly from the command line, `doctest` works its magic:

```

$ python example.py
$

```

There's no output! That's normal, and it means all the examples worked. Pass `-v` to the script, and `doctest` prints a detailed log of what it's trying, and prints a summary at the end:

```

$ python example.py -v
Trying:
    factorial(5)
Expecting:
    120
ok
Trying:
    [factorial(n) for n in range(6)]
Expecting:
    [1, 1, 2, 6, 24, 120]
ok

```

And so on, eventually ending with:

```

Trying:
    factorial(1e100)
Expecting:
    Traceback (most recent call last):
      ...
    OverflowError: n too large
ok
2 items passed all tests:
  1 test in __main__
  6 tests in __main__.factorial
7 tests in 2 items.
7 passed.
Test passed.
$

```

That's all you need to know to start making productive use of `doctest`! Jump in. The following sections provide full details. Note that there are many examples of doctests in the standard Python test suite and libraries. Especially useful examples can be found in the standard test file `Lib/test/test_doctest/test_doctest.py`.

Added in version 3.13: Output is colorized by default and can be controlled using environment variables.

Simple Usage: Checking Examples in Docstrings

The simplest way to start using `doctest` (but not necessarily the way you'll continue to do it) is to end each module `M` with:

```

if __name__ == "__main__":
    import doctest
    doctest.testmod()

```

`doctest` then examines docstrings in module `M`.

Running the module as a script causes the examples in the docstrings to get executed and verified:

```
python M.py
```

This won't display anything unless an example fails, in which case the failing example(s) and the cause(s) of the failure(s) are printed to stdout, and the final line of output is `***TestFailed***Nfailures.`, where `N` is the number of examples that failed.

Run it with the `-v` switch instead:

```
python M.py -v
```

and a detailed report of all examples tried is printed to standard output, along with assorted summaries at the end.

You can force verbose mode by passing `verbose=True` to `testmod()`, or prohibit it by passing `verbose=False`. In either of those cases, `sys.argv` is not examined by `testmod()` (so passing `-v` or not has no effect).

There is also a command line shortcut for running `testmod()`, see section [Command-line Usage](#).

For more information on `testmod()`, see section [Basic API](#).

Simple Usage: Checking Examples in a Text File

Another simple application of `doctest` is testing interactive examples in a text file. This can be done with the `testfile()` function:

```
import doctest
doctest.testfile("example.txt")
```

That short script executes and verifies any interactive Python examples contained in the file `example.txt`. The file content is treated as if it were a single giant docstring; the file doesn't need to contain a Python program! For example, perhaps `example.txt` contains this:

```
The ``example`` module
=====
```

```
Using ``factorial``
-----
```

```
This is an example text file in reStructuredText format. First import
``factorial`` from the ``example`` module:
```

```
>>> from example import factorial
```

Now use it:

```
>>> factorial(6)
120
```

Running `doctest.testfile("example.txt")` then finds the error in this documentation:

```
File "./example.txt", line 14, in example.txt
Failed example:
    factorial(6)
Expected:
    120
Got:
    720
```

As with `testmod()`, `testfile()` won't display anything unless an example fails. If an example does fail, then the failing example(s) and the cause(s) of the failure(s) are printed to stdout, using the same format as `testmod()`.

By default, `testfile()` looks for files in the calling module's directory. See section [Basic API](#) for a description of the optional arguments that can be used to tell it to look for files in other locations.

Like `testmod()`, `testfile()`'s verbosity can be set with the `-v` command-line switch or with the optional keyword argument `verbose`.

There is also a command line shortcut for running `testfile()`, see section [Command-line Usage](#).

For more information on `testfile()`, see section [Basic API](#).

Command-line Usage

The `doctest` module can be invoked as a script from the command line:

```
python -m doctest [-v] [-o OPTION] [-f] file [file ...]
```

-v, --verbose

Detailed report of all examples tried is printed to standard output, along with assorted summaries at the end: `python -m doctest -v example.py` This will import `example.py` as a standalone module and run `testmod()` on it. Note that this may not work correctly if the file is part of a package and imports other submodules from that package. If the file name does not end with `.py`, `doctest` infers that it must be run with `testfile()` instead: `python -m doctest -v example.txt`

-o, --option<option>

Option flags control various aspects of `doctest`'s behavior, see section [Option Flags](#). Added in version 3.4.

-f, --fail-fast

This is shorthand for `-o FAIL_FAST`. Added in version 3.4.

How It Works

This section examines in detail how `doctest` works: which docstrings it looks at, how it finds interactive examples, what execution context it uses, how it handles exceptions, and how option flags can be used to control its behavior. This is the information that you need to know to write `doctest` examples; for information about actually running `doctest` on these examples, see the following sections.

Which Docstrings Are Examined?

The module docstring, and all function, class and method docstrings are searched. Objects imported into the module are not searched.

In addition, there are cases when you want tests to be part of a module but not part of the help text, which requires that the tests not be included in the docstring. `Doctest` looks for a module-level variable called `__test__` and uses it to locate other tests. If `M.__test__` exists, it must be a dict, and each entry maps a (string) name to a function object, class object, or string. Function and class object docstrings found from `M.__test__` are searched, and strings are treated as if they were docstrings. In output, a key `in M.__test__` appears with name `M.__test__[key]`.

For example, place this block of code at the top of `example.py`:

```
__test__ = {
    'numbers': """
>>> factorial(6)
720

>>> [factorial(n) for n in range(6)]
```

```
[1, 1, 2, 6, 24, 120]
"""
}
```

The value of `example.__test__["numbers"]` will be treated as a docstring and all the tests inside it will be run. It is important to note that the value can be mapped to a function, class object, or module; if so, `doctest` searches them recursively for docstrings, which are then scanned for tests.

Any classes found are recursively searched similarly, to test docstrings in their contained methods and nested classes.

■ **Nota:** `doctest` can only automatically discover classes and functions that are defined at the module level or inside other classes. Since nested classes and functions only exist when an outer function is called, they cannot be discovered. Define them outside to make them visible.

How are Docstring Examples Recognized?

In most cases a copy-and-paste of an interactive console session works fine, but `doctest` isn't trying to do an exact emulation of any specific Python shell.

```
>>> # comments are ignored
>>> x = 12
>>> x
12
>>> if x == 13:
...     print("yes")
... else:
...     print("no")
...     print("NO")
...     print("NO!!!")
...
no
NO
NO!!!
>>>
```

Any expected output must immediately follow the final `>>>'or'...` line containing the code, and the expected output (if any) extends to the next `>>>'or'` all-whitespace line.

The fine print:

- Expected output cannot contain an all-whitespace line, since such a line is taken to signal the end of expected output. If expected output does contain a blank line, put `<BLANKLINE>` in your doctest example each place a blank line is expected.
- All hard tab characters are expanded to spaces, using 8-column tab stops. Tabs in output generated by the tested code are not modified. Because any hard tabs in the sample output are expanded, this means that if the code output includes hard tabs, the only way the doctest can pass is if the `NORMALIZE_WHITESPACE` option or directive is in effect. Alternatively, the test can be rewritten to capture the output.
- Output to `stdout` is captured, but not output to `stderr` (exception tracebacks are captured via a different means).
- If you continue a line via backslashing in an interactive session, or for any other reason use a backslash, you should use a raw docstring, which will preserve your backslashes exactly as you type them: `>>> def f(x): ... r"""Backslashes in a raw docstring: m\n"...>>> print(f.__doc__)` Backslashes in a raw docstring: `m\n` Otherwise, the backslash will be interpreted as part of the string. For example, the\
- The starting column doesn't matter: `>>> assert "Easy!">>> import math>>> math.floor(1.9)` 1 and as many leading whitespace characters are stripped from the expected output as appeared in the initial `>>>'` line that started the example.

What's the Execution Context?

By default, each `timedoctest` finds a docstring to test, it uses a shallow copy of `M`'s globals, so that running tests doesn't change the module's real globals, and so that one test in `M` can't leave behind crumbs that accidentally allow another test to work. This means examples can freely use any names defined at top-level in `M`, and names defined earlier in the docstring being run. Examples cannot see names defined in other docstrings.

You can force use of your own dict as the execution context by passing `globals=your_dict` to `testmod()` or `testfile()` instead.

What About Exceptions?

No problem, provided that the traceback is the only output produced by the example: just paste in the traceback.^[1] Since tracebacks contain details that are likely to change rapidly (for example, exact file paths and line numbers), this is one case where `doctest` works hard to be flexible in what it accepts.

Simple example:

```
>>> [1, 2, 3].remove(42)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: list.remove(x): x not in list
```

That `doctest` succeeds if `ValueError` is raised, with the `list.remove(x): x not in list` detail as shown.

The expected output for an exception must start with a traceback header, which may be either of the following two lines, indented the same as the first line of the example:

```
Traceback (most recent call last):
Traceback (innermost last):
```

The traceback header is followed by an optional traceback stack, whose contents are ignored by `doctest`. The traceback stack is typically omitted, or copied verbatim from an interactive session.

The traceback stack is followed by the most interesting part: the line(s) containing the exception type and detail. This is usually the last line of a traceback, but can extend across multiple lines if the exception has a multi-line detail:

```
>>> raise ValueError('multi\n    line\ndetail')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: multi
    line
detail
```

The last three lines (starting with `ValueError`) are compared against the exception's type and detail, and the rest are ignored.

Best practice is to omit the traceback stack, unless it adds significant documentation value to the example. So the last example is probably better as:

```
>>> raise ValueError('multi\n    line\ndetail')
Traceback (most recent call last):
...
ValueError: multi
    line
detail
```

Note that tracebacks are treated very specially. In particular, in the rewritten example, the use of `...` is independent of `doctest`'s `ELLIPSIS` option. The ellipsis in that example could be left out, or could just as well be three (or three hundred) commas or digits, or an indented transcript of a Monty Python skit.

Some details you should read once, but won't need to remember:

- For some exceptions, Python displays the position of the error using ^ markers and tildes: >>> 1+NoneFile"<stdin>", line 11+None~^~~~~~TypeError: unsupported operand type(s) for +: 'int' and 'NoneType' Since the lines showing the position of the error come before the exception type and detail, they are not checked by doctest. For example, the following test would pass, even though it puts the ^ marker in

Option Flags

A number of option flags control various aspects of doctest's behavior. Symbolic names for the flags are supplied as module constants, which can be bitwise ORed together and passed to various functions. The names can also be used in doctest directives, and may be passed to the doctest command line interface via the `-o` option.

The first group of options define test semantics, controlling aspects of how doctest decides whether actual output matches an example's expected output:

doctest.DONT_ACCEPT_TRUE_FOR_1

By default, if an expected output block contains just1, an actual output block containing just1or justTrueis considered to be a match, and similarly for0versusFalse. WhenDONT_ACCEPT_TRUE_FOR_1is specified, neither substitution is allowed. The default behavior caters to that Python changed the return type of many functions from integer to boolean; doctests expecting “little integer” output still work in these cases. This option will probably go away, but not for several years.

doctest.DONT_ACCEPT_BLANKLINE

By default, if an expected output block contains a line containing only the string<BLANKLINE>, then that line will match a blank line in the actual output. Because a genuinely blank line delimits the expected output, this is the only way to communicate that a blank line is expected. WhenDONT_ACCEPT_BLANKLINEis specified, this substitution is not allowed.

doctest.NORMALIZE_WHITESPACE

When specified, all sequences of whitespace (blanks and newlines) are treated as equal. Any sequence of whitespace within the expected output will match any sequence of whitespace within the actual output. By default, whitespace must match exactly. `NORMALIZE_WHITESPACE` is especially useful when a line of expected output is very long, and you want to wrap it across multiple lines in your source.

doctest.ELLIPSIS

When specified, an ellipsis marker (...) in the expected output can match any substring in the actual output. This includes substrings that span line boundaries, and empty substrings, so it's best to keep usage of this simple. Complicated uses can lead to the same kinds of "oops, it matched too much!" surprises that `*is` is prone to

in regular expressions.

doctest.IGNORE_EXCEPTION_DETAIL

When specified, doctests expecting exceptions pass so long as an exception of the expected type is raised, even if the details (message and fully qualified exception name) don't match. For example, an example expecting `ValueError:42` will pass if the actual exception raised is `ValueError:3*14`, but will fail if, say, a `TypeError` is raised instead. It will also ignore any fully qualified name included before the exception class, which can vary between implementations and versions of Python and the code/

doctest.SKIP

When specified, do not run the example at all. This can be useful in contexts where doctest examples serve as both documentation and test cases, and an example should be included for documentation purposes, but should not be checked. E.g., the example's output might be random; or the example might depend on resources which would be unavailable to the test driver. The SKIP flag can also be used for temporarily "commenting out" examples.

doctest.COMPARISON_FLAGS

A bitmask or'ing together all the comparison flags above.

The second group of options controls how test failures are reported:

doctest.REPORT_UDIFF

When specified, failures that involve multi-line expected and actual outputs are displayed using a unified diff.

doctest.REPORT_CDIFF

When specified, failures that involve multi-line expected and actual outputs will be displayed using a context diff.

doctest.REPORT_NDIFF

When specified, differences are computed by `difflib.Differ`, using the same algorithm as the popular `ndiff.pyutility`. This is the only method that marks differences within lines as well as across lines. For example, if a line of expected output contains `digit1` where actual output contains `letterl`, a line is inserted with a caret marking the mismatching column positions.

doctest.REPORT_ONLY_FIRST_FAILURE

When specified, display the first failing example in each doctest, but suppress output for all remaining examples. This will prevent doctest from reporting correct examples that break because of earlier failures; but it might also hide incorrect examples that fail independently of the first failure.

When `REPORT_ONLY_FIRST_FAILURE` is specified, the remaining examples are still run, and still count towards the total number of failures reported; only the output is suppressed.

doctest.FAIL_FAST

When specified, exit after the first failing example and don't attempt to run the remaining examples. Thus, the number of failures reported will be at most 1. This flag may be useful during debugging, since examples after the first failure won't even produce debugging output.

doctest.REPORTING_FLAGS

A bitmask or'ing together all the reporting flags above.

There is also a way to register new option flag names, though this isn't useful unless you intend to extend `doctestinternals` via subclassing:

doctest.register_optionflag(name)

Create a new option flag with a given name, and return the new flag's integer value. `register_optionflag()` can be used when subclassing `OutputChecker` or `DocTestRunner` to create new options that are supported by your subclasses. `register_optionflag()` should always be called using the following idiom: `MY_FLAG=register_optionflag('MY_FLAG')`

Directives

Doctest directives may be used to modify the option flags for an individual example. Doctest directives are special Python comments following an example's source code:

```
directive:          "#" "doctest:" directive_options
directive_options:  directive_option ("," directive_option)*
directive_option:   on_or_off directive_option_name
on_or_off:          "+" | "-"
directive_option_name: "DONT_ACCEPT_BLANKLINE" | "NORMALIZE_WHITESPACE" | ...
```

Whitespace is not allowed between the + or - and the directive option name. The directive option name can be any of the option flag names explained above.

An example's doctest directives modify doctest's behavior for that single example. Use + to enable the named behavior, or - to disable it.

For example, this test passes:

```
>>> print(list(range(20))) # doctest: +NORMALIZE_WHITESPACE
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
```

Without the directive it would fail, both because the actual output doesn't have two blanks before the single-digit list elements, and because the actual output is on a single line. This test also passes, and also requires a directive to do so:

```
>>> print(list(range(20))) # doctest: +ELLIPSIS
[0, 1, ..., 18, 19]
```

Multiple directives can be used on a single physical line, separated by commas:

```
>>> print(list(range(20))) # doctest: +ELLIPSIS, +NORMALIZE_WHITESPACE
[0, 1, ..., 18, 19]
```

If multiple directive comments are used for a single example, then they are combined:

```
>>> print(list(range(20))) # doctest: +ELLIPSIS
...                       # doctest: +NORMALIZE_WHITESPACE
[0, 1, ..., 18, 19]
```

As the previous example shows, you can add ... lines to your example containing only directives. This can be useful when an example is too long for a directive to comfortably fit on the same line:

```
>>> print(list(range(5)) + list(range(10, 20)) + list(range(30, 40)))
... # doctest: +ELLIPSIS
[0, ..., 4, 10, ..., 19, 30, ..., 39]
```

Note that since all options are disabled by default, and directives apply only to the example they appear in, enabling options (via + in a directive) is usually the only meaningful choice. However, option flags can also be passed to functions that run doctests, establishing different defaults. In such cases, disabling an option via - in a directive can be useful.

Warnings

doctest is serious about requiring exact matches in expected output. If even a single character doesn't match, the test fails. This will probably surprise you a few times, as you learn exactly what Python does and doesn't guarantee about output. For example, when printing a set, Python doesn't guarantee that the element is printed

in any particular order, so a test like

```
>>> foo()  
{ "spam", "eggs" }
```

is vulnerable! One workaround is to do

```
>>> foo() == { "spam", "eggs" }  
True
```

instead. Another is to do

```
>>> d = sorted(foo())  
>>> d  
[ 'eggs', 'spam' ]
```

There are others, but you get the idea.

Another bad idea is to print things that embed an object address, like

```
>>> id(1.0) # certain to fail some of the time  
7948648  
>>> class C: pass  
>>> C() # the default repr() for instances embeds an address  
<C object at 0x00AC18F0>
```

The `ELLIPSIS` directive gives a nice approach for the last example:

```
>>> C() # doctest: +ELLIPSIS  
<C object at 0x...>
```

Floating-point numbers are also subject to small output variations across platforms, because Python defers to the platform C library for some floating-point calculations, and C libraries vary widely in quality here.

```
>>> 1000**0.1 # risky  
1.9952623149688797  
>>> round(1000**0.1, 9) # safer  
1.995262315  
>>> print(f'{1000**0.1:.4f}') # much safer  
1.9953
```

Numbers of the form $1/2$ are safe across all platforms, and I often contrive doctest examples to produce numbers of that form:

```
>>> 3./4 # utterly safe  
0.75
```

Simple fractions are also easier for people to understand, and that makes for better documentation.

Basic API

The functions `testmod()` and `testfile()` provide a simple interface to doctest that should be sufficient for most basic uses. For a less formal introduction to these two functions, see sections [Simple Usage: Checking Examples in Docstrings](#) and [Simple Usage: Checking Examples in a Text File](#).

`doctest.testfile(filename, module_relative=True, name=None, package=None, globs=None, verbose=None, report=True, optionflags=0, extraglobs=None, raise_on_error=False, parser=DocTestParser(), encoding=None)`

All arguments except `filename` are optional, and should be specified in keyword form. Test examples in the file named `filename`. Return `(failure_count, test_count)`. Optional argument `module_relative` specifies how the filename should be interpreted: If `module_relative` is `True` (the default), then `filename` specifies an OS-independent module-relative path. By default, this path is relative to the calling module's directory; but if the `package` argument is specified, then it is relative to that package. To ensure OS-i

doctest.testmod(m=None,name=None,globs=None,verbose=None,report=True,optionflags=0,extraglobs=None,raise_on_error=False,exclude_empty=False)

All arguments are optional, and all except `m` should be specified in keyword form. Test examples in docstrings in functions and classes reachable from `module` (or `module.__main__` if `m` is not supplied or is `None`), starting with `m.__doc__`. Also test examples reachable from `dict(m.__test__)`, if it exists. `m.__test__` maps names (strings) to functions, classes and strings; function and class docstrings are searched for examples; strings are searched directly, as if they were docstrings. Only docstrings attached to

doctest.run_docstring_examples(f,globs,verbose=False,name='NoName',compileflags=None,optionflags=0)

Test examples associated with object `f`; for example, `f` may be a string, a module, a function, or a class object. A shallow copy of dictionary argument `globs` is used for the execution context. Optional argument `name` is used in failure messages, and defaults to "NoName". If optional argument `verbose` is true, output is generated even if there are no failures. By default, output is generated only in case of an example failure. Optional argument `compileflags` gives the set of flags that should be used by the Python

Unittest API

As your collection of doctest'ed modules grows, you'll want a way to run all their doctests systematically. `doctest` provides two functions that can be used to create `unittest` test suites from modules and text files containing doctests. To integrate with `unittest` test discovery, include a `load_tests` function in your test module:

```
import unittest
import doctest
import my_module_with_doctests

def load_tests(loader, tests, ignore):
    tests.addTests(doctest.DocTestSuite(my_module_with_doctests))
    return tests
```

There are two main functions for creating `unittest.TestSuite` instances from text files and modules with doctests:

doctest.DocFileSuite(*paths,module_relative=True,package=None,setUp=None,tearDown=None,globs=None,optionflags=0,parser=DocTestParser(),encoding=None)

Convert doctest tests from one or more text files to a `unittest.TestSuite`. The returned `unittest.TestSuite` is to be run by the `unittest` framework and runs the interactive examples in each file. If an example in any file fails, then the synthesized unit test fails, and a `failureException` is raised showing the name of the file containing the test and a (sometimes approximate) line number. If all the examples in a file are skipped, then the synthesized unit test is also marked as skipped. Pass

doctest.DocTestSuite(module=None,globs=None,extraglobs=None,test_finder=None,setUp=None,tearDown=None,optionflags=0,checker=None)

Convert doctest tests for a module to a `unittest.TestSuite`. The returned `unittest.TestSuite` is to be run by the `unittest` framework and runs each doctest in the module. Each docstring is run as a separate unit test. If any of the doctests fail, then the synthesized unit test fails, and a `unittest.TestCase.failureException` is raised showing the name of the file containing the test and a (sometimes approximate) line number. If all the examples in a docstring are skipped, then the Optional argument

Under the covers, `DocTestSuite()` creates a `unittest.TestSuite` out of `doctest.DocTestCase` instances, and `DocTestCase` is a subclass of `unittest.TestCase`. `DocTestCase` isn't documented here (it's an internal detail), but studying its code can answer questions about the exact details of `unittest` integration.

Similarly, `DocFileSuite()` creates a `unittest.TestSuite` out of `doctest.DocFileCase` instances, and `DocFileCase` is a subclass of `DocTestCase`.

So both ways of creating `unittest.TestSuite` instances of `DocTestCase`. This is important for a subtle reason: when you run `doctest` functions yourself, you can control the `doctest` options in use directly, by passing option flags to `doctest` functions. However, if you're writing a `unittest` framework, `unittest` ultimately controls when and how tests get run. The framework author typically wants to control `doctest` reporting options (perhaps, e.g., specified by command line options), but there's no way to pass options through `unittest` to `doctest` test runners.

For this reason, `doctest` also supports a notion of `doctest` reporting flags specific to `unittest` support, via this function:

`doctest.set_unittest_reportflags(flags)`

Set the `doctest` reporting flags to use. Argument `flag` takes the bitwise OR of option flags. See section [Option Flags](#). Only "reporting flags" can be used. This is a module-global setting, and affects all future `doctest` runs by module `unittest`: the `runTest()` method of `DocTestCase` looks at the option flags specified for the test case when the `DocTestCase` instance was constructed. If no reporting flags were specified (which is the typical and expected case), `doctest`'s `unittest` reporting flags are bitwise ORed into

Advanced API

The basic API is a simple wrapper that's intended to make `doctest` easy to use. It is fairly flexible, and should meet most users' needs; however, if you require more fine-grained control over testing, or wish to extend `doctest`'s capabilities, then you should use the advanced API.

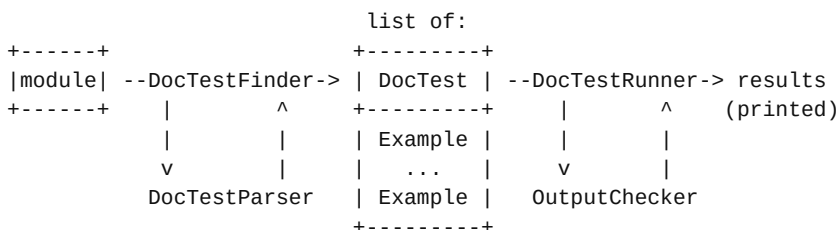
The advanced API revolves around two container classes, which are used to store the interactive examples extracted from `doctest` cases:

- **Example:** A single Python statement, paired with its expected output.
- **DocTest:** A collection of `Examples`, typically extracted from a single docstring or text file.

Additional processing classes are defined to find, parse, and run, and check `doctest` examples:

- **DocTestFinder:** Finds all docstrings in a given module, and uses a `DocTestParser` to create a `DocTest` from every docstring that contains interactive examples.
- **DocTestParser:** Creates a `DocTest` object from a string (such as an object's docstring).
- **DocTestRunner:** Executes the examples in a `DocTest`, and uses an `OutputChecker` to verify their output.
- **OutputChecker:** Compares the actual output from a `doctest` example with the expected output, and decides whether they match.

The relationships among these processing classes are summarized in the following diagram:



DocTest Objects

`class doctest.DocTest(examples, globs, name, filename, lineno, docstring)`

examples

globs

name

filename

lineno

docstring

A collection of doctest examples that should be run in a single namespace. The constructor arguments are used to initialize the attributes of the same names. `DocTest` defines the following attributes. They are initialized by the constructor, and should not be modified directly.

- `examples` A list of `Example` objects encoding the individual interactive Python examples that should be run by this test.
- `globs` The namespace (aka globals) that the examples should be run in. This is a dictionary mapping names to

A list of `Example` objects encoding the individual interactive Python examples that should be run by this test.

The namespace (aka globals) that the examples should be run in. This is a dictionary mapping names to values. Any changes to the namespace made by the examples (such as binding new variables) will be reflected in `globs` after the test is run.

A string name identifying the `DocTest`. Typically, this is the name of the object or file that the test was extracted from.

The name of the file that this `DocTest` was extracted from; or `None` if the filename is unknown, or if the `DocTest` was not extracted from a file.

The line number within `filename` where this `DocTest` begins, or `None` if the line number is unavailable. This line number is zero-based with respect to the beginning of the file.

The string that the test was extracted from, or `None` if the string is unavailable, or if the test was not extracted from a string.

Example Objects

`class doctest.Example(source, want, exc_msg=None, lineno=0, indent=0, options=None)`

source

want

exc_msg

lineno

indent

options

A single interactive example, consisting of a Python statement and its expected output. The constructor arguments are used to initialize the attributes of the same names. `Example` defines the following attributes. They are initialized by the constructor, and should not be modified directly.

- `source` A string containing the example's source code. This source code consists of a single Python statement, and always ends with a newline; the constructor adds a newline when necessary.
- `want` The expected output

A string containing the example's source code. This source code consists of a single Python statement, and always ends with a newline; the constructor adds a newline when necessary.

The expected output from running the example's source code (either from `stdout`, or a traceback in case of exception). `want` ends with a newline unless no output is expected, in which case it's an empty string. The constructor adds a newline when necessary.

The exception message generated by the example, if the example is expected to generate an exception; or `None` if it is not expected to generate an exception. This exception message is compared against the return value of `traceback.format_exception_only().exc_msg` ends with a newline unless it's `None`. The constructor adds

a newline if needed.

The line number within the string containing this example where the example begins. This line number is zero-based with respect to the beginning of the containing string.

The example's indentation in the containing string, i.e., the number of space characters that precede the example's first prompt.

A dictionary mapping from option flags to `True` or `False`, which is used to override default options for this example. Any option flags not contained in this dictionary are left at their default value (as specified by `theDocTestRunner.soptionflags`). By default, no options are set.

DocTestFinder objects

`class doctest.DocTestFinder(verbose=False, parser=DocTestParser(), recurse=True, exclude_empty=True)`

`find(obj[, name][, module][, globs][, extraglobs])`

A processing class used to extract the `DocTests` that are relevant to a given object, from its docstring and the docstrings of its contained objects. `DocTests` can be extracted from modules, classes, functions, methods, static methods, class methods, and properties. The optional argument `verbose` can be used to display the objects searched by the finder. It defaults to `False` (no output). The optional argument `parser` specifies the `DocTestParser` object (or a drop-in replacement) that is used to extract doctests f

Return a list of the `DocTests` that are defined by `obj`'s docstring, or by any of its contained objects' docstrings. The optional argument `name` specifies the object's name; this name will be used to construct names for the returned `DocTests`. If `name` is not specified, then `obj.__name__` is used. The optional parameter `module` is the module that contains the given object. If the module is not specified or is `None`, then the test finder will attempt to automatically determine the correct module. The object's module

DocTestParser objects

`class doctest.DocTestParser`

`get_doctest(string, globs, name, filename, lineno)`

`get_examples(string, name='<string>')`

`parse(string, name='<string>')`

A processing class used to extract interactive examples from a string, and use them to create a `DocTest` object. `DocTestParser` defines the following

methods: `get_doctest(string, globs, name, filename, lineno)` Extract all doctest examples from the given string, and collect them into a `DocTest` object. `globs`, `name`, `filename`, and `lineno` are attributes for the new `DocTest` object. See the documentation for `DocTest` for more information. `get_examples(string, name='<string>')` Extract all doctest examples from the given string,

Extract all doctest examples from the given string, and collect them into a `DocTest` object. `globs`, `name`, `filename`, and `lineno` are attributes for the new `DocTest` object. See the documentation for `DocTest` for more information.

Extract all doctest examples from the given string, and return them as a list of `Example` objects. Line numbers are 0-based. The optional argument `name` is a name identifying this string, and is only used for error messages.

Divide the given string into examples and intervening text, and return them as a list of alternating `Examples` and strings. Line numbers for the `Examples` are 0-based. The optional argument `name` is a name identifying this string, and is only used for error messages.

TestResults objects

`class doctest.TestResults(failed, attempted)`

failed

attempted

skipped

failedNumber of failed tests.attemptedNumber of attempted tests.skippedNumber of skipped tests.Added in version 3.13.

Number of failed tests.

Number of attempted tests.

Number of skipped tests.Added in version 3.13.

DocTestRunner objects

class doctest.DocTestRunner(checker=None, verbose=None, optionflags=0)

report_start(out, test, example)

report_success(out, test, example, got)

report_failure(out, test, example, got)

report_unexpected_exception(out, test, example, exc_info)

run(test, compileflags=None, out=None, clear_globs=True)

summarize(verbose=None)

tries

failures

skips

A processing class used to execute and verify the interactive examples in a DocTest. The comparison between expected outputs and actual outputs is done by an `OutputChecker`. This comparison may be customized with a number of option flags; see section `Option Flags` for more information. If the option flags are insufficient, then the comparison may also be customized by passing a subclass of `OutputChecker` to the constructor. The test runner's display output can be controlled in two ways. First, an output

Report that the test runner is about to process the given example. This method is provided to allow subclasses of `DocTestRunner` to customize their output; it should not be called directly. `example` is the example about to be processed. `test` is the test containing `example`. `out` is the output function that was passed to `DocTestRunner.run()`.

Report that the given example ran successfully. This method is provided to allow subclasses of `DocTestRunner` to customize their output; it should not be called directly. `example` is the example about to be processed. `got` is the actual output from the example. `test` is the test containing `example`. `out` is the output function that was passed to `DocTestRunner.run()`.

Report that the given example failed. This method is provided to allow subclasses of `DocTestRunner` to customize their output; it should not be called directly. `example` is the example about to be processed. `got` is the actual output from the example. `test` is the test containing `example`. `out` is the output function that was passed to `DocTestRunner.run()`.

Report that the given example raised an unexpected exception. This method is provided to allow subclasses of `DocTestRunner` to customize their output; it should not be called directly. `example` is the example about to be processed. `exc_info` is a tuple containing information about the unexpected exception (as returned by `sys.exc_info()`). `test` is the test containing `example`. `out` is the output function that was passed to `DocTestRunner.run()`.

Run the examples `intest(aDocTestobject)`, and display the results using the writer function `out`. Return a `TestResults` instance. The examples are run in the namespace `test.globs`. If `clear_globs` is true (the default), then this namespace will be cleared after the test runs, to help with garbage collection. If you would like to examine the namespace after the test completes, then use `clear_globs=False`. `compileflags` gives the set of flags that should be used by the Python compiler when running the examples.

Print a summary of all the test cases that have been run by this `DocTestRunner`, and return a `TestResults` instance. The optional `verbose` argument controls how detailed the summary is. If the verbosity is not specified, then the `DocTestRunner`'s verbosity is used.

Number of attempted examples.

Number of failed examples.

Number of skipped examples. Added in version 3.13.

OutputChecker objects

`class doctest.OutputChecker`

`check_output(want, got, optionflags)`

`output_difference(example, got, optionflags)`

A class used to check the whether the actual output from a doctest example matches the expected output. `OutputChecker` defines two methods: `check_output()`, which compares a given pair of outputs, and returns `True` if they match; and `output_difference()`, which returns a string describing the differences between two outputs. `OutputChecker` defines the following methods: `check_output(want, got, optionflags)` Return `True` if the actual output from an example (`got`) matches the expected output (`want`). These strings are

Return `True` if the actual output from an example (`got`) matches the expected output (`want`). These strings are always considered to match if they are identical; but depending on what option flags the test runner is using, several non-exact match types are also possible. See section [Option Flags](#) for more information about option flags.

Return a string describing the differences between the expected output for a given example (`example`) and the actual output (`got`). `optionflags` is the set of option flags used to compare `want` and `got`.

Debugging

Doctest provides several mechanisms for debugging doctest examples:

- Several functions convert doctests to executable Python programs, which can be run under the Python debugger, `pdb`.
- The `DebugRunner` class is a subclass of `DocTestRunner` that raises an exception for the first failing example, containing information about that example. This information can be used to perform post-mortem debugging on the example.
- The unittest cases generated by `DocTestSuite()` support the `debug()` method defined by `unittest.TestCase`.
- You can add a call to `pdb.set_trace()` in a doctest example, and you'll drop into the Python debugger when that line is executed. Then you can inspect current values of variables, and so on. For example, suppose `ea.py` contains just this module docstring:

```
""">>> def f(x):... g(x*2)>>> def g(x):... print(x+3)... import pdb;
pdb.set_trace()>>> f(3)9"""
```

 Then an interactive Python session may look

Functions that convert doctests to Python code, and possibly run the synthesized code under the debugger:

`doctest.script_from_examples(s)`

Convert text with examples to a script. `Argument` is a string containing doctest examples. The string is converted to a Python script, where doctest examples are converted to regular code, and everything else is converted to Python comments. The generated script is returned as a string. For example, `import doctest; print(doctest.script_from_examples("""Set x and y to 1 and 2.>>> x, y = 1, 2Print their sum:>>> print(x+y)3"""))` displays: `# Set x and y to 1 and 2.x,y=1,2## Print their sum:print(x+y)#`

doctest.testsource(module,name)

Convert the doctest for an object to a script. `Argument module` is a module object, or dotted name of a module, containing the object whose doctests are of interest. `Argument name` is the name (within the module) of the object with the doctests of interest. The result is a string, containing the object's docstring converted to a Python script, as described for `script_from_examples()` above. For example, if module `a.py` contains a top-level function `f()`, then `import a; doctest.print(doctest.testsource(a,"a.f"))`

doctest.debug(module,name,pm=False)

Debug the doctests for an object. The `module` and `name` arguments are the same as for function `testsource()` above. The synthesized Python script for the named object's docstring is written to a temporary file, and then that file is run under the control of the Python debugger, `pdb`. A shallow copy of `module.__dict__` is used for both local and global execution context. Optional argument `pm` controls whether post-mortem debugging is used. If `pm` has a true value, the script file is run directly, and the debugger gets

doctest.debug_src(src,pm=False,globs=None)

Debug the doctests in a string. This is like function `debug()` above, except that a string containing doctest examples is specified directly, via the `src` argument. Optional argument `pm` has the same meaning as in function `debug()` above. Optional argument `globs` gives a dictionary to use as both local and global execution context. If not specified, or `None`, an empty dictionary is used. If specified, a shallow copy of the dictionary is used.

The `DebugRunner` class, and the special exceptions it may raise, are of most interest to testing framework authors, and will only be sketched here. See the source code, and especially `DebugRunner`'s docstring (which is a doctest!) for more details:

class doctest.DebugRunner(checker=None,verbose=None,optionflags=0)

A subclass of `DocTestRunner` that raises an exception as soon as a failure is encountered. If an unexpected exception occurs, an `UnexpectedException` exception is raised, containing the test, the example, and the original exception. If the output doesn't match, then a `DocTestFailure` exception is raised, containing the test, the example, and the actual output. For information about the constructor parameters and methods, see the documentation for `DocTestRunner` in section `Advanced API`.

There are two exceptions that may be raised by `DebugRunner` instances:

exception doctest.DocTestFailure(test,example,got)

An exception raised by `DocTestRunner` to signal that a doctest example's actual output did not match its expected output. The constructor arguments are used to initialize the attributes of the same names.

`DocTestFailure` defines the following attributes:

DocTestFailure.test

The `DocTest` object that was being run when the example failed.

DocTestFailure.example

The `Example` that failed.

DocTestFailure.got

The example's actual output.

exceptiondoctest.UnexpectedException(test,example,exc_info)

An exception raised by `DocTestRunner` to signal that a doctest example raised an unexpected exception. The constructor arguments are used to initialize the attributes of the same names.

`UnexpectedException` defines the following attributes:

UnexpectedException.test

The `DocTest` object that was being run when the example failed.

UnexpectedException.example

The example that failed.

UnexpectedException.exc_info

A tuple containing information about the unexpected exception, as returned by `sys.exc_info()`.

Soapbox

As mentioned in the introduction, doctest has grown to have three primary uses:

- Checking examples in docstrings.
- Regression testing.
- Executable documentation / literate testing.

These uses have different requirements, and it is important to distinguish them. In particular, filling your docstrings with obscure test cases makes for bad documentation.

When writing a docstring, choose docstring examples with care. There's an art to this that needs to be learned—it may not be natural at first. Examples should add genuine value to the documentation. A good example can often be worth many words. If done with care, the examples will be invaluable for your users, and will pay back the time it takes to collect them many times over as the years go by and things change. I'm still amazed at how often one of my doctest examples stops working after a "harmless" change.

Doctest also makes an excellent tool for regression testing, especially if you don't skimp on explanatory text. By interleaving prose and examples, it becomes much easier to keep track of what's actually being tested, and why. When a test fails, good prose can make it much easier to figure out what the problem is, and how it should be fixed. It's true that you could write extensive comments in code-based testing, but few programmers do. Many have found that using doctest approaches instead leads to much clearer tests. Perhaps this is simply because doctest makes writing prose a little easier than writing code, while writing comments in code is a little harder. I think it goes deeper than just that: the natural attitude when writing a doctest-based test is that you want to explain the fine points of your software, and illustrate them with examples. This in turn naturally leads to test files that start with the simplest features, and logically progress to complications and edge cases. A coherent narrative is the result, instead of a collection of isolated functions that test isolated bits of functionality seemingly at random. It's a different attitude, and produces different results, blurring the distinction between testing and explaining.

Regression testing is best confined to dedicated objects or files. There are several options for organizing tests:

- Write text files containing test cases as interactive examples, and test the files using `testfile()` or `DocFileSuite()`. This is recommended, although is easiest to do for new projects, designed from the start to use doctest.
- Define functions named `_regtest_topic` that consist of single docstrings, containing test cases for the named topics. These functions can be included in the same file as the module, or separated out into a separate test file.
- Define a `__test__` dictionary mapping from regression test topics to docstrings containing test cases.

When you have placed your tests in a module, the module can itself be the test runner. When a test fails, you can arrange for your test runner to re-run only the failing doctest while you debug the problem. Here is a minimal example of such a test runner:

```
if __name__ == '__main__':
    import doctest
    flags = doctest.REPORT_NDIFF|doctest.FAIL_FAST
    if len(sys.argv) > 1:
        name = sys.argv[1]
        if name in globals():
            obj = globals()[name]
        else:
            obj = __test__[name]
        doctest.run_docstring_examples(obj, globals(), name=name,
                                       optionflags=flags)
    else:
        fail, total = doctest.testmod(optionflags=flags)
        print(f"{fail} failures out of {total} tests")
```

Footnotes

Examples containing both expected output and an exception are not supported. Trying to guess where one ends and the other begins is too error-prone, and that also makes for a confusing test.

unittest— Unit testing framework

Source code: `Lib/unittest/__init__.py`

(If you are already familiar with the basic concepts of testing, you might want to skip to the list of assert methods.)

The `unittest` unit testing framework was originally inspired by JUnit and has a similar flavor as major unit testing frameworks in other languages. It supports test automation, sharing of setup and shutdown code for tests, aggregation of tests into collections, and independence of the tests from the reporting framework.

To achieve this, `unittest` supports some important concepts in an object-oriented way:

test fixture

test case

test suite

test runner

A test fixture represents the preparation needed to perform one or more tests, and any associated cleanup actions. This may involve, for example, creating temporary or proxy databases, directories, or starting a server process.

A test case is the individual unit of testing. It checks for a specific response to a particular set of inputs. `unittest` provides a base class, `TestCase`, which may be used to create new test cases.

A test suite is a collection of test cases, test suites, or both. It is used to aggregate tests that should be executed together.

A test runner is a component which orchestrates the execution of tests and provides the outcome to the user. The runner may use a graphical interface, a textual interface, or return a special value to indicate the results of executing the tests.

See also

Module `doctest`

Simple Smalltalk Testing: With Patterns

`pytest`

The Python Testing Tools Taxonomy

Testing in Python Mailing List

Another test-support module with a very different flavor.

Kent Beck's original paper on testing frameworks using the pattern shared by `unittest`.

Third-party `unittest` framework with a lighter-weight syntax for writing tests. For example, `assertfunc(10)==42`.

An extensive list of Python testing tools including functional testing frameworks and mock object libraries.

A special-interest-group for discussion of testing, and testing tools, in Python.

The script `Tools/unittestgui/unittestgui.py` in the Python source distribution is a GUI tool for test discovery and execution. This is intended largely for ease of use for those new to unit testing. For production environments it is recommended that tests be driven by a continuous integration system such as `Buildbot`, `Jenkins`, `GitHub Actions`, or `AppVeyor`.

Basic example

The `unittest` module provides a rich set of tools for constructing and running tests. This section demonstrates that a small subset of the tools suffice to meet the needs of most users.

Here is a short script to test three string methods:

```
import unittest

class TestStringMethods(unittest.TestCase):

    def test_upper(self):
        self.assertEqual('foo'.upper(), 'FOO')

    def test_isupper(self):
        self.assertTrue('FOO'.isupper())
        self.assertFalse('Foo'.isupper())

    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', 'world'])
        # check that s.split fails when the separator is not a string
        with self.assertRaises(TypeError):
            s.split(2)

if __name__ == '__main__':
    unittest.main()
```

A test case is created by subclassing `unittest.TestCase`. The three individual tests are defined with methods whose names start with the letter `test`. This naming convention informs the test runner about which methods represent tests.

The crux of each test is a call to `assertEqual()` to check for an expected result; `assertTrue()` or `assertFalse()` to verify a condition; or `assertRaises()` to verify that a specific exception gets raised. These methods are used instead of the `assert` statement so the test runner can accumulate all test results and produce a report.

The `setUp()` and `tearDown()` methods allow you to define instructions that will be executed before and after each test method. They are covered in more detail in the section [Organizing test code](#).

The final block shows a simple way to run the tests. `unittest.main()` provides a command-line interface to the test script. When run from the command line, the above script produces an output that looks like this:

```
...
-----
Ran 3 tests in 0.000s

OK
```

Passing the `-v` option to your test script will instruct `unittest.main()` to enable a higher level of verbosity, and produce the following output:

```
test_isupper (__main__.TestStringMethods.test_isupper) ... ok
test_split (__main__.TestStringMethods.test_split) ... ok
test_upper (__main__.TestStringMethods.test_upper) ... ok
-----
Ran 3 tests in 0.001s

OK
```

The above examples show the most commonly used `unittest` features which are sufficient to meet many everyday testing needs. The remainder of the documentation explores the full feature set from first principles.

Changed in version 3.11: The behavior of returning a value from a test method (other than the default `None` value), is now deprecated.

Command-Line Interface

The `unittest` module can be used from the command line to run tests from modules, classes or even individual test methods:

```
python -m unittest test_module1 test_module2
python -m unittest test_module.TestClass
python -m unittest test_module.TestClass.test_method
```

You can pass in a list with any combination of module names, and fully qualified class or method names.

Test modules can be specified by file path as well:

```
python -m unittest tests/test_something.py
```

This allows you to use the shell filename completion to specify the test module. The file specified must still be importable as a module. The path is converted to a module name by removing the `.py` and converting path separators into `.`. If you want to execute a test file that isn't importable as a module you should execute the file directly instead.

You can run tests with more detail (higher verbosity) by passing in the `-v` flag:

```
python -m unittest -v test_module
```

When executed without arguments Test Discovery is started:

```
python -m unittest
```

For a list of all the command-line options:

```
python -m unittest -h
```

Changed in version 3.2: In earlier versions it was only possible to run individual test methods and not modules or classes.

Added in version 3.14: Output is colorized by default and can be controlled using environment variables.

Command-line options

`unittest` supports these command-line options:

-b, --buffer

The standard output and standard error streams are buffered during the test run. Output during a passing test is discarded. Output is echoed normally on test fail or error and is added to the failure messages.

-c, --catch

Control-C during the test run waits for the current test to end and then reports all the results so far. A second Control-C raises the normal `KeyboardInterrupt` exception. See [Signal Handling](#) for the functions that provide this functionality.

-f, --failfast

Stop the test run on the first error or failure.

-k

Only run test methods and classes that match the pattern or substring. This option may be used multiple times, in which case all test cases that match any of the given patterns are included. Patterns that contain a wildcard character (`*`) are matched against the test name using `fnmatch.fnmatchcase()`; otherwise simple

case-sensitive substring matching is used. Patterns are matched against the fully qualified test method name as imported by the test loader. For example, `-kfoo` matches `foo_tests.SomeTest.test`.

--locals

Show local variables in tracebacks.

--durationsN

Show the N slowest test cases (N=0 for all).

Added in version 3.2: The command-line options `-b`, `-c` and `-f` were added.

Added in version 3.5: The command-line option `--locals`.

Added in version 3.7: The command-line option `-k`.

Added in version 3.12: The command-line option `--durations`.

The command line can also be used for test discovery, for running all of the tests in a project or just a subset.

Test Discovery

Added in version 3.2.

Unittest supports simple test discovery. In order to be compatible with test discovery, all of the test files must be modules or packages importable from the top-level directory of the project (this means that their filenames must be valid identifiers).

Test discovery is implemented in `TestLoader.discover()`, but can also be used from the command line. The basic command-line usage is:

```
cd project_directory
python -m unittest discover
```

■ Nota: Note As a shortcut, `python -m unittest` is the equivalent of `python -m unittest discover`. If you want to pass arguments to test discovery the `discover` sub-command must be used explicitly.

The `discover` sub-command has the following options:

-v, --verbose

Verbose output

-s, --start-directory directory

Directory to start discovery (.default)

-p, --pattern pattern

Pattern to match test files (test*.py default)

-t, --top-level-directory directory

Top level directory of project (defaults to start directory)

The `-s`, `-p`, and `-t` options can be passed in as positional arguments in that order. The following two command lines are equivalent:

```
python -m unittest discover -s project_directory -p "*_test.py"
python -m unittest discover project_directory "*_test.py"
```

As well as being a path it is possible to pass a package name, for example `myproject.subpackage.test`, as the start directory. The package name you supply will then be imported and its location on the filesystem will be used as the start directory.

Caution

Test discovery loads tests by importing them. Once test discovery has found all the test files from the start directory you specify it turns the paths into package names to import. For example `foo/bar/baz.py` will be imported as `foo.bar.baz`.

If you have a package installed globally and attempt test discovery on a different copy of the package then the import could happen from the wrong place. If this happens test discovery will warn you and exit.

If you supply the start directory as a package name rather than a path to a directory then discover assumes that whichever location it imports from is the location you intended, so you will not get the warning.

Test modules and packages can customize test loading and discovery by through the `load_tests` protocol.

Changed in version 3.4: Test discovery supports namespace packages.

Changed in version 3.11: Test discovery dropped the namespace package support. It has been broken since Python 3.7. Start directory and its subdirectories containing tests must be regular package that have `__init__.py` file.

If the start directory is the dotted name of the package, the ancestor packages can be namespace packages.

Changed in version 3.14: Test discovery supports namespace package as start directory again. To avoid scanning directories unrelated to Python, tests are not searched in subdirectories that do not contain `__init__.py`.

Organizing test code

The basic building blocks of unit testing are test cases—single scenarios that must be set up and checked for correctness. In `unittest`, test cases are represented by `unittest.TestCase` instances. To make your own test cases you must write subclasses of `TestCase` or use `FunctionTestCase`.

The testing code of a `TestCase` instance should be entirely self contained, such that it can be run either in isolation or in arbitrary combination with any number of other test cases.

The simplest `TestCase` subclass will simply implement a test method (i.e. a method whose name starts with `test`) in order to perform specific testing code:

```
import unittest

class DefaultWidgetSizeTestCase(unittest.TestCase):
    def test_default_widget_size(self):
        widget = Widget('The widget')
        self.assertEqual(widget.size(), (50, 50))
```

Note that in order to test something, we use one of the `assert*` methods provided by the `TestCase` base class. If the test fails, an exception will be raised with an explanatory message, and `unittest` will identify the test case as a failure. Any other exceptions will be treated as errors.

Tests can be numerous, and their set-up can be repetitive. Luckily, we can factor out set-up code by implementing a method called `setUp()`, which the testing framework will automatically call for every single test we run:

```
import unittest

class WidgetTestCase(unittest.TestCase):
    def setUp(self):
        self.widget = Widget('The widget')

    def test_default_widget_size(self):
        self.assertEqual(self.widget.size(), (50, 50),
                         'incorrect default size')
```

```
def test_widget_resize(self):
    self.widget.resize(100,150)
    self.assertEqual(self.widget.size(), (100,150),
                      'wrong size after resize')
```

■ **Nota:** NoteThe order in which the various tests will be run is determined by sorting the test method names with respect to the built-in ordering for strings.

If the `setUp()` method raises an exception while the test is running, the framework will consider the test to have suffered an error, and the test method will not be executed.

Similarly, we can provide a `tearDown()` method that tidies up after the test method has been run:

```
import unittest

class WidgetTestCase(unittest.TestCase):
    def setUp(self):
        self.widget = Widget('The widget')

    def tearDown(self):
        self.widget.dispose()
```

If `setUp()` succeeded, `tearDown()` will be run whether the test method succeeded or not.

Such a working environment for the testing code is called a **test fixture**. A new `TestCase` instance is created as a unique test fixture used to execute each individual test method. Thus `setUp()`, `tearDown()`, and `TestCase.__init__()` will be called once per test.

It is recommended that you use `TestCase` implementations to group tests together according to the features they test. `unittest` provides a mechanism for this: the **test suite**, represented by `unittest.TestSuite` class. In most cases, calling `unittest.main()` will do the right thing and collect all the module's test cases for you and execute them.

However, should you want to customize the building of your test suite, you can do it yourself:

```
def suite():
    suite = unittest.TestSuite()
    suite.addTest(WidgetTestCase('test_default_widget_size'))
    suite.addTest(WidgetTestCase('test_widget_resize'))
    return suite

if __name__ == '__main__':
    runner = unittest.TextTestRunner()
    runner.run(suite())
```

You can place the definitions of test cases and test suites in the same modules as the code they are to test (such as `widget.py`), but there are several advantages to placing the test code in a separate module, such as `test_widget.py`:

- The test module can be run standalone from the command line.
- The test code can more easily be separated from shipped code.
- There is less temptation to change test code to fit the code it tests without a good reason.
- Test code should be modified much less frequently than the code it tests.
- Tested code can be refactored more easily.
- Tests for modules written in C must be in separate modules anyway, so why not be consistent?
- If the testing strategy changes, there is no need to change the source code.

Re-using old test code

Some users will find that they have existing test code that they would like to run from unittest, without converting every old test function to a `TestCase` subclass.

For this reason, unittest provides a `FunctionTestCase` class. This subclass of `TestCase` can be used to wrap an existing test function. Set-up and tear-down functions can also be provided.

Given the following test function:

```
def testSomething():
    something = makeSomething()
    assert something.name is not None
    # ...
```

one can create an equivalent test case instance as follows, with optional set-up and tear-down methods:

```
testcase = unittest.FunctionTestCase(testSomething,
                                     setUp=makeSomethingDB,
                                     tearDown=deleteSomethingDB)
```

■ Nota: Note Even though `FunctionTestCase` can be used to quickly convert an existing test base over to a unittest-based system, this approach is not recommended. Taking the time to set up proper `TestCase` subclasses will make future test refactorings infinitely easier.

In some cases, the existing tests may have been written using the `doctest` module. If so, `doctest` provides a `DocTestSuite` class that can automatically build `unittest.TestSuite` instances from the existing `doctest`-based tests.

Skipping tests and expected failures

Added in version 3.1.

Unittest supports skipping individual test methods and even whole classes of tests. In addition, it supports marking a test as an “expected failure,” a test that is broken and will fail, but shouldn’t be counted as a failure on a `TestResult`.

Skipping a test is simply a matter of using the `skip()` decorator or one of its conditional variants, calling `TestCase.skipTest()` within a `setUp()` or test method, or raising `SkipTest` directly.

Basic skipping looks like this:

```
class MyTestCase(unittest.TestCase):

    @unittest.skip("demonstrating skipping")
    def test_nothing(self):
        self.fail("shouldn't happen")

    @unittest.skipIf(mylib.__version__ < (1, 3),
                    "not supported in this library version")
    def test_format(self):
        # Tests that work for only a certain version of the library.
        pass

    @unittest.skipUnless(sys.platform.startswith("win"), "requires Windows")
    def test_windows_support(self):
        # windows specific testing code
        pass

    def test_maybe_skipped(self):
        if not external_resource_available():
            self.skipTest("external resource not available")
        # test code that depends on the external resource
        pass
```

This is the output of running the example above in verbose mode:

```
test_format (__main__.MyTestCase.test_format) ... skipped 'not supported in this library version'
test_nothing (__main__.MyTestCase.test_nothing) ... skipped 'demonstrating skipping'
test_maybe_skipped (__main__.MyTestCase.test_maybe_skipped) ... skipped 'external resource not available'
test_windows_support (__main__.MyTestCase.test_windows_support) ... skipped 'requires Windows'
```

```
-----
Ran 4 tests in 0.005s
```

OK (skipped=4)

Classes can be skipped just like methods:

```
@unittest.skip("showing class skipping")
class MySkippedTestCase(unittest.TestCase):
    def test_not_run(self):
        pass
```

`TestCase.setUp()` can also skip the test. This is useful when a resource that needs to be set up is not available.

Expected failures use the `expectedFailure()` decorator.

```
class ExpectedFailureTestCase(unittest.TestCase):
    @unittest.expectedFailure
    def test_fail(self):
        self.assertEqual(1, 0, "broken")
```

It's easy to roll your own skipping decorators by making a decorator that calls `skip()` on the test when it wants it to be skipped. This decorator skips the test unless the passed object has a certain attribute:

```
def skipUnlessHasattr(obj, attr):
    if hasattr(obj, attr):
        return lambda func: func
    return unittest.skip("{!r} doesn't have {!r}".format(obj, attr))
```

The following decorators and exception implement test skipping and expected failures:

@unittest.skip(reason)

Unconditionally skip the decorated test. `reason` should describe why the test is being skipped.

@unittest.skipIf(condition, reason)

Skip the decorated test if `condition` is true.

@unittest.skipUnless(condition, reason)

Skip the decorated test unless `condition` is true.

@unittest.expectedFailure

Mark the test as an expected failure or error. If the test fails or errors in the test function itself (rather than in one of the test fixture methods) then it will be considered a success. If the test passes, it will be considered a failure.

unittest.SkipTest(reason)

This exception is raised to skip a test. Usually you can use `TestCase.skipTest()` or one of the skipping decorators instead of raising this directly.

Skipped tests will not have `setUp()` or `tearDown()` run around them. Skipped classes will not have `setUpClass()` or `tearDownClass()` run. Skipped modules will not have `setUpModule()` or `tearDownModule()` run.

Distinguishing test iterations using subtests

Added in version 3.4.

When there are very small differences among your tests, for instance some parameters, unittest allows you to distinguish them inside the body of a test method using the `subTest()` context manager.

For example, the following test:

```
class NumbersTest(unittest.TestCase):

    def test_even(self):
        """
        Test that numbers between 0 and 5 are all even.
        """
        for i in range(0, 6):
            with self.subTest(i=i):
                self.assertEqual(i % 2, 0)
```

will produce the following output:

```
=====
FAIL: test_even (__main__.NumbersTest.test_even) (i=1)
Test that numbers between 0 and 5 are all even.
-----
Traceback (most recent call last):
  File "subtests.py", line 11, in test_even
    self.assertEqual(i % 2, 0)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: 1 != 0

=====
FAIL: test_even (__main__.NumbersTest.test_even) (i=3)
Test that numbers between 0 and 5 are all even.
-----
Traceback (most recent call last):
  File "subtests.py", line 11, in test_even
    self.assertEqual(i % 2, 0)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: 1 != 0

=====
FAIL: test_even (__main__.NumbersTest.test_even) (i=5)
Test that numbers between 0 and 5 are all even.
-----
Traceback (most recent call last):
  File "subtests.py", line 11, in test_even
    self.assertEqual(i % 2, 0)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: 1 != 0
```

Without using a subtest, execution would stop after the first failure, and the error would be less easy to diagnose because the value of `i` wouldn't be displayed:

```
=====
FAIL: test_even (__main__.NumbersTest.test_even)
-----
Traceback (most recent call last):
  File "subtests.py", line 32, in test_even
    self.assertEqual(i % 2, 0)
AssertionError: 1 != 0
```

Classes and functions

This section describes in depth the API of `unittest`.

Test cases

`unittest.TestCase(methodName='runTest')`

`setUp()`

`tearDown()`

`setUpClass()`

`tearDownClass()`

`run(result=None)`

`skipTest(reason)`

`subTest(msg=None,**params)`

`debug()`

`assertEqual(first,second,msg=None)`

`assertNotEqual(first,second,msg=None)`

`assertTrue(expr,msg=None)`

`assertFalse(expr,msg=None)`

`assertIs(first,second,msg=None)`

`assertIsNot(first,second,msg=None)`

`assertIsNone(expr,msg=None)`

`assertIsNotNone(expr,msg=None)`

`assertIn(member,container,msg=None)`

`assertNotIn(member,container,msg=None)`

`assertIsInstance(obj,cls,msg=None)`

`assertNotIsInstance(obj,cls,msg=None)`

`assertIsSubclass(cls,superclass,msg=None)`

`assertNotIsSubclass(cls,superclass,msg=None)`

The `withblock` does not log on

`assertRaises(exception,callable,*args,**kwds)`

`assertRaises(exception,*args,msg=None)`

`assertRaisesRegex(exception,regex,callable,*args,**kwds)`

`assertRaisesRegex(exception,regex,*args,msg=None)`

`assertWarns(warning,callable,*args,**kwds)`

`assertWarns(warning,*args,msg=None)`

`assertWarnsRegex(warning,regex,callable,*args,**kwds)`

`assertWarnsRegex(warning,regex,*args,msg=None)`

`assertLogs(logger=None,level=None)`

`records`

`output`

`assertNoLogs(logger=None,level=None)`
`assertAlmostEqual(first,second,places=7,msg=None,delta=None)`
`assertNotAlmostEqual(first,second,places=7,msg=None,delta=None)`
`assertGreater(first,second,msg=None)`
`assertGreaterEqual(first,second,msg=None)`
`assertLess(first,second,msg=None)`
`assertLessEqual(first,second,msg=None)`
`assertRegex(text,regex,msg=None)`
`assertNotRegex(text,regex,msg=None)`
`assertCountEqual(first,second,msg=None)`
`assertStartsWith(s,prefix,msg=None)`
`assertNotStartsWith(s,prefix,msg=None)`
`assertEndsWith(s,suffix,msg=None)`
`assertNotEndsWith(s,suffix,msg=None)`
`assertHasAttr(obj,name,msg=None)`
`assertNotHasAttr(obj,name,msg=None)`
`addTypeEqualityFunc(typeobj,function)`
`assertMultiLineEqual(first,second,msg=None)`
`assertSequenceEqual(first,second,msg=None,seq_type=None)`
`assertListEqual(first,second,msg=None)`
`assertTupleEqual(first,second,msg=None)`
`assertSetEqual(first,second,msg=None)`
`assertDictEqual(first,second,msg=None)`
`fail(msg=None)`
`failureException`
`longMessage`
`maxDiff`
`countTestCases()`
`defaultTestResult()`
`id()`
`shortDescription()`
`addCleanup(function,/,*args,**kwargs)`
`enterContext(cm)`
`doCleanups()`
`classmethodaddClassCleanup(function,/,*args,**kwargs)`
`classmethodenterClassContext(cm)`

classmethoddoClassCleanups()

Instances of the `TestCase` class represent the logical test units in the `unittest` universe. This class is intended to be used as a base class, with specific tests being implemented by concrete subclasses. This class implements the interface needed by the test runner to allow it to drive the tests, and methods that the test code can use to check for and report various kinds of failure. Each instance of `TestCase` will run a single base method: the method named `methodName`. In most uses of `TestCase`, you will

Method called to prepare the test fixture. This is called immediately before calling the test method; other than `AssertionError` or `SkipTest`, any exception raised by this method will be considered an error rather than a test failure. The default implementation does nothing.

Method called immediately after the test method has been called and the result recorded. This is called even if the test method raised an exception, so the implementation in subclasses may need to be particularly careful about checking internal state. Any exception, other than `AssertionError` or `SkipTest`, raised by this method will be considered an additional error rather than a test failure (thus increasing the total number of reported errors). This method will only be called if `setUp()` succeed

A class method called before tests in an individual class are run. `setUpClass` is called with the class as the only argument and must be decorated as `@classmethod()`: `@classmethoddef setUpClass(cls):...` See `Class` and `Module Fixtures` for more details. Added in version 3.2.

A class method called after tests in an individual class have run. `tearDownClass` is called with the class as the only argument and must be decorated as `@classmethod()`: `@classmethoddef tearDownClass(cls):...` See `Class` and `Module Fixtures` for more details. Added in version 3.2.

Run the test, collecting the result into the `TestResult` object passed as `result`. If `result` is omitted or `None`, a temporary result object is created (by calling the `defaultTestResult()` method) and used. The result object is returned to `run()`'s caller. The same effect may be had by simply calling the `TestCase` instance. Changed in version 3.3: Previous versions of `run` did not return the result. Neither did calling an instance.

Calling this during a test method or `setUp()` skips the current test. See `Skipping tests and expected failures` for more information. Added in version 3.1.

Return a context manager which executes the enclosed code block as a subtest. `msg` and `params` are optional, arbitrary values which are displayed whenever a subtest fails, allowing you to identify them clearly. A test case can contain any number of subtest declarations, and they can be arbitrarily nested. See `Distinguishing test iterations using subtests` for more information. Added in version 3.4.

Run the test without collecting the result. This allows exceptions raised by the test to be propagated to the caller, and can be used to support running tests under a debugger.

Test that `first` and `second` are equal. If the values do not compare equal, the test will fail. In addition, if `first` and `second` are the exact same type and one of `list`, `tuple`, `dict`, `set`, `frozenset` or `str` or any type that a subclass registers with `addTypeEqualityFunc()` the type-specific equality function will be called in order to generate a more useful default error message (see also the list of type-specific methods). Changed in version 3.1: Added the automatic calling of type-specific equality function. Chan

Test that `first` and `second` are not equal. If the values do compare equal, the test will fail.

Test that `expr` is true (or false). Note that this is equivalent to `bool(expr) is True` and not `toExprisTrue` (use `assertIs(expr, True)` for the latter). This method should also be avoided when more specific methods are available (e.g. `assertEqual(a, b)` instead of `assertTrue(a == b)`), because they provide a better error message in case of failure.

Test that `first` and `second` are (or are not) the same object. Added in version 3.1.

Test that `expr` is (or is not) `None`. Added in version 3.1.

Test that `memberis` (or is not) in `container`. Added in version 3.1.

Test that `objis` (or is not) an instance of `cls` (which can be a class or a tuple of classes, as supported by `isinstance()`). To check for the exact type, use `assertIs(type(obj), cls)`. Added in version 3.2.

Test that `clsis` (or is not) a subclass of `superclass` (which can be a class or a tuple of classes, as supported by `issubclass()`). To check for the exact type, use `assertIs(cls, superclass)`. Added in version 3.14.

`logger` with `minimumlevel`

Test that an exception is raised when `callable` is called with any positional or keyword arguments that are also passed to `assertRaises()`. The test passes if exception is raised, is an error if another exception is raised, or fails if no exception is raised. To catch any of a group of exceptions, a tuple containing the exception classes may be passed as `exception`. If only the `exception` and possibly the `msg` arguments are given, return a context manager so that the code under test can be written inline rather

Like `assertRaises()` but also tests that `regex` matches on the string representation of the raised exception. `regex` may be a regular expression object or a string containing a regular expression suitable for use by `re.search()`. Examples: `self.assertRaisesRegex(ValueError, "invalid literal for '*XYZ'$", int, 'XYZ')` or `with self.assertRaisesRegex(ValueError, 'literal'): int('XYZ')` Added in version 3.1: Added under the name `assertRaisesRegexp`. Changed in version 3.2: Renamed to `assertRaisesRegex()`. Changed in version 3.3:

Test that a warning is triggered when `callable` is called with any positional or keyword arguments that are also passed to `assertWarns()`. The test passes if warning is triggered and fails if it isn't. Any exception is an error. To catch any of a group of warnings, a tuple containing the warning classes may be passed as `warnings`. If only the `warning` and possibly the `msg` arguments are given, return a context manager so that the code under test can be written inline rather than as a function: `with self.assertW`

Like `assertWarns()` but also tests that `regex` matches on the message of the triggered warning. `regex` may be a regular expression object or a string containing a regular expression suitable for use by `re.search()`. Example: `self.assertWarnsRegex(DeprecationWarning, r'legacy_function\(\) is deprecated', legacy_function, 'XYZ')` or `with self.assertWarnsRegex(RuntimeWarning, 'unsafe frobnicating'): frobnicate('/etc/passwd')` Added in version 3.2. Changed in version 3.3: Added the `msg` keyword argument when used as a context

A context manager to test that at least one message is logged on the `logger` or one of its children, with at least the given `level`. If given, `logger` should be `logging.Logger` object or a string giving the name of a logger. The default is the root logger, which will catch all messages that were not blocked by a non-propagating descendent logger. If given, `level` should be either a numeric logging level or its string equivalent (for example either `"ERROR"` or `logging.ERROR`). The default is `logging.INFO`. The test passes

A list of logging `LogRecord` objects of the matching log messages.

A list of `str` objects with the formatted output of matching messages.

A context manager to test that no messages are logged on the `logger` or one of its children, with at least the given `level`. If given, `logger` should be `logging.Logger` object or a string giving the name of a logger. The default is the root logger, which will catch all messages. If given, `level` should be either a numeric logging level or its string equivalent (for example either `"ERROR"` or `logging.ERROR`). The default is `logging.INFO`. Unlike `assertLogs()`, nothing will be returned by the context manager. Added in versio

Test that `first` and `second` are approximately (or not approximately) equal by computing the difference, rounding to the given number of decimal places (default 7), and comparing to zero. Note that these methods round the values to the given number of decimal places (i.e. like the `round()` function) and not significant digits. If `delta` is supplied instead of `places` then the difference between `first` and `second` must be less or equal to (or greater than) `delta`. Supplying both `delta` and `places` raises a `TypeError`. Changed in vers

Test that `first` is respectively `>`, `>=`, `<` or `<=` than `second` depending on the method name. If not, the test will fail: `>>>self.assertGreaterEqual(3,4)AssertionError: "3" unexpectedly not greater than or equal to "4"` Added in version 3.1.

Test that `aregexsearch` matches (or does not match) `text`. In case of failure, the error message will include the pattern and the `text` (or the pattern and the part of `text` that unexpectedly matched). `regex` may be a regular expression object or a string containing a regular expression suitable for use by `re.search()`. Added in version 3.1: Added under the name `assertRegexpMatches`. Changed in version 3.2: The method `assertRegexpMatches()` has been renamed to `assertRegex()`. Added in version 3.2: `assertNotRegex()`.

Test that `sequence first` contains the same elements as `second`, regardless of their order. When they don't, an error message listing the differences between the sequences will be generated. Duplicate elements are not ignored when comparing `first` and `second`. It verifies whether each element has the same count in both sequences. Equivalent to `assertEqual(Counter(list(first)),Counter(list(second)))` but works with sequences of unhashable objects as well. Added in version 3.2.

Test that the Unicode or byte strings `starts` (or does not start) with a `prefix`. `prefix` can also be a tuple of strings to try. Added in version 3.14.

Test that the Unicode or byte strings `sends` (or does not end) with a `suffix`. `suffix` can also be a tuple of strings to try. Added in version 3.14.

Test that the object `obj` has (or has not) an attribute `name`. Added in version 3.14.

Registers a type-specific method called by `assertEqual()` to check if two objects of exactly the same type `obj` (not subclasses) compare equal. `function` must take two positional arguments and a third `msg=None` keyword argument just as `assertEqual()` does. It must raise `self.failureException(msg)` when inequality between the first two parameters is detected – possibly providing useful information and explaining the inequalities in details in the error message. Added in version 3.1.

Test that the multiline string `first` is equal to the string `second`. When not equal a diff of the two strings highlighting the differences will be included in the error message. This method is used by default when comparing strings with `assertEqual()`. Added in version 3.1.

Tests that two sequences are equal. If `seq_type` is supplied, both `first` and `second` must be instances of `seq_type` or a failure will be raised. If the sequences are different an error message is constructed that shows the difference between the two. This method is not called directly by `assertEqual()`, but it's used to implement `assertListEqual()` and `assertTupleEqual()`. Added in version 3.1.

Tests that two lists or tuples are equal. If not, an error message is constructed that shows only the differences between the two. An error is also raised if either of the parameters are of the wrong type. These methods are used by default when comparing lists or tuples with `assertEqual()`. Added in version 3.1.

Tests that two sets are equal. If not, an error message is constructed that lists the differences between the sets. This method is used by default when comparing sets or frozen sets with `assertEqual()`. Fails if either of `first` or `second` does not have a `difference()` method. Added in version 3.1.

Test that two dictionaries are equal. If not, an error message is constructed that shows the differences in the dictionaries. This method will be used by default to compare dictionaries in calls to `assertEqual()`. Added in version 3.1.

Signals a test failure unconditionally, with `msg` or `None` for the error message.

This class attribute gives the exception raised by the test method. If a test framework needs to use a specialized exception, possibly to carry additional information, it must subclass this exception in order to “play fair” with the framework. The initial value of this attribute is `AssertionError`.

This class attribute determines what happens when a custom failure message is passed as the `msg` argument to an `assertXXX` call that fails. `True` is the default value. In this case, the custom message is appended to the end

of the standard failure message. When set to `False`, the custom message replaces the standard message. The class setting can be overridden in individual test methods by assigning an instance attribute, `self.longMessage`, to `True` or `False` before calling the assert methods. The class setting

This attribute controls the maximum length of diffs output by assert methods that report diffs on failure. It defaults to 80*8 characters. Assert methods affected by this attribute are `assertSequenceEqual()` (including all the sequence comparison methods that delegate to it), `assertDictEqual()` and `assertMultiLineEqual()`. Setting `maxDiff` to `None` means that there is no maximum length of diffs. Added in version 3.2.

Return the number of tests represented by this test object. For `TestCase` instances, this will always be 1.

Return an instance of the test result class that should be used for this test case class (if no other result instance is provided to the `run()` method). For `TestCase` instances, this will always be an instance of `TestResult`; subclasses of `TestCase` should override this as necessary.

Return a string identifying the specific test case. This is usually the full name of the test method, including the module and class name.

Returns a description of the test, or `None` if no description has been provided. The default implementation of this method returns the first line of the test method's docstring, if available, or `None`. Changed in version 3.1: In 3.1 this was changed to add the test name to the short description even in the presence of a docstring. This caused compatibility issues with unittest extensions and adding the test name was moved to the `TextTestResult` in Python 3.2.

Add a function to be called after `tearDown()` to cleanup resources used during the test. Functions will be called in reverse order to the order they are added (LIFO). They are called with any arguments and keyword arguments passed into `addCleanup()` when they are added. If `setUp()` fails, meaning that `tearDown()` is not called, then any cleanup functions added will still be called. Added in version 3.1.

Enter the supplied context manager. If successful, also add its `__exit__()` method as a cleanup function by `addCleanup()` and return the result of the `__enter__()` method. Added in version 3.11.

This method is called unconditionally after `tearDown()`, or after `setUp()` if `setUp()` raises an exception. It is responsible for calling all the cleanup functions added by `addCleanup()`. If you need cleanup functions to be called prior to `tearDown()` then you can call `doCleanups()` yourself. `doCleanups()` pops methods off the stack of cleanup functions one at a time, so it can be called at any time. Added in version 3.1.

Add a function to be called after `tearDownClass()` to cleanup resources used during the test class. Functions will be called in reverse order to the order they are added (LIFO). They are called with any arguments and keyword arguments passed into `addClassCleanup()` when they are added. If `setUpClass()` fails, meaning that `tearDownClass()` is not called, then any cleanup functions added will still be called. Added in version 3.8.

Enter the supplied context manager. If successful, also add its `__exit__()` method as a cleanup function by `addClassCleanup()` and return the result of the `__enter__()` method. Added in version 3.11.

This method is called unconditionally after `tearDownClass()`, or after `setUpClass()` if `setUpClass()` raises an exception. It is responsible for calling all the cleanup functions added by `addClassCleanup()`. If you need cleanup functions to be called prior to `tearDownClass()` then you can call `doClassCleanups()` yourself. `doClassCleanups()` pops methods off the stack of cleanup functions one at a time, so it can be called at any time. Added in version 3.8.

`unittest.IsolatedAsyncioTestCase(methodName='runTest')`

`loop_factory`

`asyncSetup()`

`asyncTearDown()`

addAsyncCleanup(function,/,*args,kwargs)**

asyncenterAsyncContext(cm)

run(result=None)

This class provides an API similar to `TestCase` and also accepts coroutines as test functions. Added in version 3.8. `loop_factory` The `loop_factory` passed to `asyncio.Runner`. Override in subclasses with `asyncio.EventLoop` to avoid using the `asyncio` policy system. Added in version 3.13. `async_setup()` Method called to prepare the test fixture. This is called after `TestCase.setUp()`. This is called immediately before calling the test method; other than `AssertionError` or `SkipTest`, any exception raised by this method

The `loop_factory` passed to `asyncio.Runner`. Override in subclasses with `asyncio.EventLoop` to avoid using the `asyncio` policy system. Added in version 3.13.

Method called to prepare the test fixture. This is called after `TestCase.setUp()`. This is called immediately before calling the test method; other than `AssertionError` or `SkipTest`, any exception raised by this method will be considered an error rather than a test failure. The default implementation does nothing.

Method called immediately after the test method has been called and the result recorded. This is called before `tearDown()`. This is called even if the test method raised an exception, so the implementation in subclasses may need to be particularly careful about checking internal state. Any exception, other than `AssertionError` or `SkipTest`, raised by this method will be considered an additional error rather than a test failure (thus increasing the total number of reported errors). This method will on

This method accepts a coroutine that can be used as a cleanup function.

Enter the supplied asynchronous context manager. If successful, also add its `__aexit__()` method as a cleanup function by `addAsyncCleanup()` and return the result of the `__aenter__()` method. Added in version 3.11.

Sets up a new event loop to run the test, collecting the result into the `TestResult` object passed as `result`. If `result` is omitted or `None`, a temporary result object is created (by calling the `defaultTestResult()` method) and used. The result object is returned to `run()`'s caller. At the end of the test all the tasks in the event loop are cancelled.

unittest.FunctionTestCase(testFunc,setUp=None,tearDown=None,description=None)

This class implements the portion of the `TestCase` interface which allows the test runner to drive the test, but does not provide the methods which test code can use to check and report errors. This is used to create test cases using legacy test code, allowing it to be integrated into a `unittest`-based test framework.

Grouping tests

unittest.TestSuite(tests=())

addTest(test)

addTests(tests)

run(result)

debug()

countTestCases()

__iter__()

This class represents an aggregation of individual test cases and test suites. The class presents the interface needed by the test runner to allow it to be run as any other test case. Running a `TestSuite` instance is the same as iterating over the suite, running each test individually. If `tests` is given, it must be an iterable of individual test cases or other test suites that will be used to build the suite initially. Additional methods are provided to add test cases and suites to the collection later

Add a `TestCase` or `TestSuite` to the suite.

Add all the tests from an iterable of `TestCase` and `TestSuite` instances to this test suite. This is equivalent to iterating over tests, calling `addTest()` for each element.

Run the tests associated with this suite, collecting the result into the test result object passed as `result`. Note that unlike `TestCase.run()`, `TestSuite.run()` requires the result object to be passed in.

Run the tests associated with this suite without collecting the result. This allows exceptions raised by the test to be propagated to the caller and can be used to support running tests under a debugger.

Return the number of tests represented by this test object, including all individual tests and sub-suites.

Tests grouped by a `TestSuite` are always accessed by iteration. Subclasses can lazily provide tests by overriding `__iter__()`. Note that this method may be called several times on a single suite (for example when counting tests or comparing for equality) so the tests returned by repeated iterations before `TestSuite.run()` must be the same for each call iteration. After `TestSuite.run()`, callers should not rely on the tests returned by this method unless the caller uses a subclass that overrides `TestSuite.__`

Loading and running tests

`unittest.TestLoader`

`errors`

`loadTestsFromTestCase(testCaseClass)`

`loadTestsFromModule(module,*,pattern=None)`

`loadTestsFromName(name,module=None)`

`loadTestsFromNames(names,module=None)`

`getTestCaseNames(testCaseClass)`

`discover(start_dir,pattern='test*.py',top_level_dir=None)`

`testMethodPrefix`

`sortTestMethodsUsing`

`suiteClass`

`testNamePatterns`

The `TestLoader` class is used to create test suites from classes and modules. Normally, there is no need to create an instance of this class; the `unittest` module provides an instance that can be shared as `unittest.defaultTestLoader`. Using a subclass or instance, however, allows customization of some configurable properties. `TestLoader` objects have the following attributes: `errors` A list of the non-fatal errors encountered while loading tests. Not reset by the loader at any point. Fatal errors are signal

A list of the non-fatal errors encountered while loading tests. Not reset by the loader at any point. Fatal errors are signalled by the relevant method raising an exception to the caller. Non-fatal errors are also indicated by a synthetic test that will raise the original error when run. Added in version 3.5.

Return a suite of all test cases contained in the `TestCase`-derived `testCaseClass`. A test case instance is created for each method named by `getTestCaseNames()`. By default these are the method names beginning with `test`. If `getTestCaseNames()` returns no methods, but the `runTest()` method is implemented, a single test case is created for that method instead.

Return a suite of all test cases contained in the given module. This method searches `module` for classes derived from `TestCase` and creates an instance of the class for each test method defined for the class. Note While using a hierarchy of `TestCase`-derived classes can be convenient in sharing fixtures and helper functions,

defining test methods on base classes that are not intended to be instantiated directly does not play well with this method. Doing so, however, can be useful when the fixtures are di

Return a suite of all test cases given a string specifier. The specifier is a “dotted name” that may resolve either to a module, a test case class, a test method within a test case class, a `TestSuite` instance, or a callable object which returns a `TestCase` or `TestSuite` instance. These checks are applied in the order listed here; that is, a method on a possible test case class will be picked up as “a test method within a test case class”, rather than “a callable object”. For example, if you have a mod

Similar to `loadTestsFromName()`, but takes a sequence of names rather than a single name. The return value is a test suite which supports all the tests defined for each name.

Return a sorted sequence of method names found within `testCaseClass`; this should be a subclass of `TestCase`.

Find all the test modules by recursing into subdirectories from the specified start directory, and return a `TestSuite` object containing them. Only test files that match `pattern` will be loaded. (Using shell style pattern matching.) Only module names that are importable (i.e. are valid Python identifiers) will be loaded. All test modules must be importable from the top level of the project. If the start directory is not the top level directory then `top_level_dir` must be specified separately. If import in

String giving the prefix of method names which will be interpreted as test methods. The default value is 'test'. This affects `getTestCaseNames()` and all the `loadTestsFrom*methods`.

Function to be used to compare method names when sorting them in `getTestCaseNames()` and all the `loadTestsFrom*methods`.

Callable object that constructs a test suite from a list of tests. No methods on the resulting object are needed. The default value is the `TestSuite` class. This affects all the `loadTestsFrom*methods`.

List of Unix shell-style wildcard test name patterns that test methods have to match to be included in test suites (see `-k` option). If this attribute is not `None` (the default), all test methods to be included in test suites must match one of the patterns in this list. Note that matches are always performed using `fnmatch.fnmatchcase()`, so unlike patterns passed to the `-k` option, simple substring patterns will have to be converted using `*wildcards`. This affects all the `loadTestsFrom*methods`. Added in version

`unittest.TestResult`

`errors`

`failures`

`skipped`

`expectedFailures`

`unexpectedSuccesses`

`collectedDurations`

`shouldStop`

`testsRun`

`buffer`

`failfast`

`tb_locals`

`wasSuccessful()`

`stop()`

startTest(test)
stopTest(test)
startTestRun()
stopTestRun()
addError(test,err)
addFailure(test,err)
addSuccess(test)
addSkip(test,reason)
addExpectedFailure(test,err)
addUnexpectedSuccess(test)
addSubTest(test,subtest,outcome)
addDuration(test,elapsed)

This class is used to compile information about which tests have succeeded and which have failed. A `TestResult` object stores the results of a set of tests. The `TestCase` and `TestSuite` classes ensure that results are properly recorded; test authors do not need to worry about recording the outcome of tests. Testing frameworks built on top of `unittest` may want access to the `TestResult` object generated by running a set of tests for reporting purposes; a `TestResult` instance is returned by the `TestRunner.run()` method.

A list containing 2-tuples of `TestCase` instances and strings holding formatted tracebacks. Each tuple represents a test which raised an unexpected exception.

A list containing 2-tuples of `TestCase` instances and strings holding formatted tracebacks. Each tuple represents a test where a failure was explicitly signalled using the `assert*` methods.

A list containing 2-tuples of `TestCase` instances and strings holding the reason for skipping the test. Added in version 3.1.

A list containing 2-tuples of `TestCase` instances and strings holding formatted tracebacks. Each tuple represents an expected failure or error of the test case.

A list containing `TestCase` instances that were marked as expected failures, but succeeded.

A list containing 2-tuples of test case names and floats representing the elapsed time of each test which was run. Added in version 3.12.

Set to `True` when the execution of tests should stop by `stop()`.

The total number of tests run so far.

If set to `True`, `sys.stdout` and `sys.stderr` will be buffered in between `startTest()` and `stopTest()` being called. Collected output will only be echoed onto the real `sys.stdout` and `sys.stderr` if the test fails or errors. Any output is also attached to the failure / error message. Added in version 3.2.

If set to `True` `stop()` will be called on the first failure or error, halting the test run. Added in version 3.2.

If set to `True` then local variables will be shown in tracebacks. Added in version 3.5.

Return `True` if all tests run so far have passed, otherwise returns `False`. Changed in version 3.4: Returns `False` if there were any unexpected successes from tests marked with the `expectedFailure()` decorator.

This method can be called to signal that the set of tests being run should be aborted by setting the `shouldStop` attribute to `True`. `TestRunner` objects should respect this flag and return without running any additional tests. For example, this feature is used by the `TextTestRunner` class to stop the test framework when

the user signals an interrupt from the keyboard. Interactive tools which provide `TestRunner` implementations can use this in a similar manner.

Called when the test case `test` is about to be run.

Called after the test case `test` has been executed, regardless of the outcome.

Called once before any tests are executed. Added in version 3.1.

Called once after all tests are executed. Added in version 3.1.

Called when the test case `test` raises an unexpected exception. `err` is a tuple of the form returned by `sys.exc_info():(type,value,traceback)`. The default implementation appends a tuple `(test,formatted_err)` to the instance's `errors` attribute, where `formatted_err` is a formatted traceback derived from `err`.

Called when the test case `test` signals a failure. `err` is a tuple of the form returned by `sys.exc_info():(type,value,traceback)`. The default implementation appends a tuple `(test,formatted_err)` to the instance's `failures` attribute, where `formatted_err` is a formatted traceback derived from `err`.

Called when the test case `test` succeeds. The default implementation does nothing.

Called when the test case `test` is skipped. `reason` is the reason the test gave for skipping. The default implementation appends a tuple `(test,reason)` to the instance's `skipped` attribute.

Called when the test case `test` fails or errors, but was marked with the `expectedFailure()` decorator. The default implementation appends a tuple `(test,formatted_err)` to the instance's `expectedFailures` attribute, where `formatted_err` is a formatted traceback derived from `err`.

Called when the test case `test` was marked with the `expectedFailure()` decorator, but succeeded. The default implementation appends the test to the instance's `unexpectedSuccesses` attribute.

Called when a subtest finishes. `test` is the test case corresponding to the test method. `subtest` is a `customTestCase` instance describing the subtest. If `outcome` is `None`, the subtest succeeded. Otherwise, it failed with an exception where `outcome` is a tuple of the form returned by `sys.exc_info():(type,value,traceback)`. The default implementation does nothing when the outcome is a success, and records subtest failures as normal failures. Added in version 3.4.

Called when the test case finishes. `elapsed` is the time represented in seconds, and it includes the execution of cleanup functions. Added in version 3.12.

`unittest.TextTestResult(stream, descriptions, verbosity, *, durations=None)`

A concrete implementation of `TestResult` used by the `TextTestRunner`. Subclasses should accept `**kwargs` to ensure compatibility as the interface changes. Added in version 3.2. Changed in version 3.12: Added the `durations` keyword parameter.

`unittest.defaultTestLoader`

Instance of the `TestLoader` class intended to be shared. If no customization of the `TestLoader` is needed, this instance can be used instead of repeatedly creating new instances.

`unittest.TextTestRunner(stream=None, descriptions=True, verbosity=1, failfast=False, buffer=False, resultclass=None, warnings=None, *, tb_locals=False, durations=None)`

`_makeResult()`

`run(test)`

A basic test runner implementation that outputs results to a stream. If `stream` is `None`, the default, `sys.stderr` is used as the output stream. This class has a few configurable parameters, but is essentially very simple. Graphical applications which run test suites should provide alternate implementations. Such implementations should accept `**kwargs` as the interface to construct runners changes when features are added to `unittest`. By

default this runner shows `DeprecationWarning`, `PendingDeprecationWarning`,

This method returns the instance of `TestResult` used by `run()`. It is not intended to be called directly, but can be overridden in subclasses to provide a custom `TestResult`. `_makeResult()` instantiates the class or callable passed in the `TextTestRunner` constructor as the `resultclass` argument. It defaults to `TextTestResult` if no `resultclass` is provided. The result class is instantiated with the following arguments: `stream`, `descriptions`, `verbosity`

This method is the main public interface to the `TextTestRunner`. This method takes a `TestSuite` or `TestCase` instance. A `TestResult` is created by calling `_makeResult()` and the test(s) are run and the results printed to `stdout`.

`unittest.main(module='__main__', defaultTest=None, argv=None, testRunner=None, testLoader=unittest.defaultTestLoader, exit=True, verbosity=1, failfast=None, catchbreak=None, buffer=None, warnings=None)`

A command-line program that loads a set of tests from a module and runs them; this is primarily for making test modules conveniently executable. The simplest use for this function is to include the following line at the end of a test script: `if __name__ == '__main__': unittest.main()` You can run tests with more detailed information by passing in the `verbosity` argument: `if __name__ == '__main__': unittest.main(verbosity=2)` The default `test` argument is either the name of a single test or an iterable of test names to

load_tests Protocol

Added in version 3.2.

Modules or packages can customize how tests are loaded from them during normal test runs or test discovery by implementing a function called `load_tests`.

If a test module defines `load_tests` it will be called by `TestLoader.loadTestsFromModule()` with the following arguments:

```
load_tests(loader, standard_tests, pattern)
```

where `pattern` is passed straight through from `loadTestsFromModule`. It defaults to `None`.

It should return a `TestSuite`.

`loader` is the instance of `TestLoader` doing the loading. `standard_tests` are the tests that would be loaded by default from the module. It is common for test modules to only want to add or remove tests from the standard set of tests. The third argument is used when loading packages as part of test discovery.

A typical `load_tests` function that loads tests from a specific set of `TestCase` classes may look like:

```
test_cases = (TestCase1, TestCase2, TestCase3)
```

```
def load_tests(loader, tests, pattern):
    suite = TestSuite()
    for test_class in test_cases:
        tests = loader.loadTestsFromTestCase(test_class)
        suite.addTests(tests)
    return suite
```

If discovery is started in a directory containing a package, either from the command line or by calling `TestLoader.discover()`, then the package `__init__.py` will be checked for `load_tests`. If that function does not exist, discovery will recurse into the package as though it were just another directory. Otherwise, discovery of the package's tests will be left up to `load_tests` which is called with the following arguments:

```
load_tests(loader, standard_tests, pattern)
```

This should return a `TestSuite` representing all the tests from the package. (`standard_tests` will only contain tests collected from `__init__.py`.)

Because the pattern is passed into `load_tests` the package is free to continue (and potentially modify) test discovery. A 'do nothing' `load_tests` function for a test package would look like:

```
def load_tests(loader, standard_tests, pattern):
    # top level directory cached on loader instance
    this_dir = os.path.dirname(__file__)
    package_tests = loader.discover(start_dir=this_dir, pattern=pattern)
    standard_tests.addTests(package_tests)
    return standard_tests
```

Changed in version 3.5: Discovery no longer checks package names for matching pattern due to the impossibility of package names matching the default pattern.

Class and Module Fixtures

Class and module level fixtures are implemented in `TestSuite`. When the test suite encounters a test from a new class then `tearDownClass()` from the previous class (if there is one) is called, followed by `setUpClass()` from the new class.

Similarly if a test is from a different module from the previous test then `tearDownModule` from the previous module is run, followed by `setUpModule` from the new module.

After all the tests have run the final `tearDownClass` and `tearDownModule` are run.

Note that shared fixtures do not play well with [potential] features like test parallelization and they break test isolation. They should be used with care.

The default ordering of tests created by the unittest test loaders is to group all tests from the same modules and classes together. This will lead to `setUpClass`/`setUpModule`(etc) being called exactly once per class and module. If you randomize the order, so that tests from different modules and classes are adjacent to each other, then these shared fixture functions may be called multiple times in a single test run.

Shared fixtures are not intended to work with suites with non-standard ordering. `ABaseTestSuite` still exists for frameworks that don't want to support shared fixtures.

If there are any exceptions raised during one of the shared fixture functions the test is reported as an error. Because there is no corresponding test instance an `_ErrorHolder` object (that has the same interface as a `TestCase`) is created to represent the error. If you are just using the standard unittest test runner then this detail doesn't matter, but if you are a framework author it may be relevant.

setUpClass and tearDownClass

These must be implemented as class methods:

```
import unittest

class Test(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls._connection = createExpensiveConnectionObject()

    @classmethod
    def tearDownClass(cls):
        cls._connection.destroy()
```

If you want the `setUpClass` and `tearDownClass` on base classes called then you must call up to them yourself. The implementations in `TestCase` are empty.

If an exception is raised during `setUpClass` then the tests in the class are not run and the `tearDownClass` is not run. Skipped classes will not have `setUpClass` or `tearDownClass` run. If the exception is a `SkipTest` exception then

the class will be reported as having been skipped instead of as an error.

setUpModule and tearDownModule

These should be implemented as functions:

```
def setUpModule():
    createConnection()

def tearDownModule():
    closeConnection()
```

If an exception is raised in `setUpModule` then none of the tests in the module will be run and `tearDownModule` will not be run. If the exception is a `SkipTestException` then the module will be reported as having been skipped instead of as an error.

To add cleanup code that must be run even in the case of an exception, use `addModuleCleanup`:

`unittest.addModuleCleanup(function,/, *args, **kwargs)`

Add a function to be called after `tearDownModule()` to cleanup resources used during the test class. Functions will be called in reverse order to the order they are added (LIFO). They are called with any arguments and keyword arguments passed into `addModuleCleanup()` when they are added. If `setUpModule()` fails, meaning that `tearDownModule()` is not called, then any cleanup functions added will still be called. Added in version 3.8.

`unittest.enterModuleContext(cm)`

Enter the supplied context manager. If successful, also add its `__exit__()` method as a cleanup function by `addModuleCleanup()` and return the result of the `__enter__()` method. Added in version 3.11.

`unittest.doModuleCleanups()`

This function is called unconditionally after `tearDownModule()`, or after `setUpModule()` if `setUpModule()` raises an exception. It is responsible for calling all the cleanup functions added by `addModuleCleanup()`. If you need cleanup functions to be called prior to `tearDownModule()` then you can call `doModuleCleanups()` yourself. `doModuleCleanups()` pops methods off the stack of cleanup functions one at a time, so it can be called at any time. Added in version 3.8.

Signal Handling

Added in version 3.2.

The `-c/--catch` command-line option to `unittest`, along with the `catchbreak` parameter to `unittest.main()`, provide more friendly handling of control-C during a test run. With `catch break` behavior enabled control-C will allow the currently running test to complete, and the test run will then end and report all the results so far. A second control-c will raise a `KeyboardInterrupt` in the usual way.

The control-c handling signal handler attempts to remain compatible with code or tests that install their own `signal.SIGINT` handler. If the `unittest` handler is called but isn't the installed `signal.SIGINT` handler, i.e. it has been replaced by the system under test and delegated to, then it calls the default handler. This will normally be the expected behavior by code that replaces an installed handler and delegates to it. For individual tests that need `unittest` control-c handling disabled the `removeHandler()` decorator can be used.

There are a few utility functions for framework authors to enable control-c handling functionality within test frameworks.

`unittest.installHandler()`

Install the control-c handler. When a `signal.SIGINT` is received (usually in response to the user pressing control-c) all registered results have `stop()` called.

unittest.registerResult(result)

Register a `TestResult` object for control-c handling. Registering a result stores a weak reference to it, so it doesn't prevent the result from being garbage collected. Registering a `TestResult` object has no side-effects if control-c handling is not enabled, so test frameworks can unconditionally register all results they create independently of whether or not handling is enabled.

unittest.removeResult(result)

Remove a registered result. Once a result has been removed then `stop()` will no longer be called on that result object in response to a control-c.

unittest.removeHandler(function=None)

When called without arguments this function removes the control-c handler if it has been installed. This function can also be used as a test decorator to temporarily remove the handler while the test is being executed: `@unittest.removeHandlerdef test_signal_handling(self):...`

unittest.mock— mock object library

Added in version 3.3.

Source code: [Lib/unittest/mock.py](#)

`unittest.mock` is a library for testing in Python. It allows you to replace parts of your system under test with mock objects and make assertions about how they have been used.

`unittest.mock` provides a `coreMock` class removing the need to create a host of stubs throughout your test suite. After performing an action, you can make assertions about which methods / attributes were used and arguments they were called with. You can also specify return values and set needed attributes in the normal way.

Additionally, mock provides a `patch()` decorator that handles patching module and class level attributes within the scope of a test, along with `sentinel` for creating unique objects. See the [quick guide](#) for some examples of how to use `Mock`, `MagicMock` and `patch()`.

Mock is designed for use with `unittest` and is based on the 'action -> assertion' pattern instead of 'record -> replay' used by many mocking frameworks.

There is a backport of `unittest.mock` for earlier versions of Python, available as `mock` on PyPI.

Quick Guide

`Mock` and `MagicMock` objects create all attributes and methods as you access them and store details of how they have been used. You can configure them, to specify return values or limit what attributes are available, and then make assertions about how they have been used:

```
>>> from unittest.mock import MagicMock
>>> thing = ProductionClass()
>>> thing.method = MagicMock(return_value=3)
>>> thing.method(3, 4, 5, key='value')
3
>>> thing.method.assert_called_with(3, 4, 5, key='value')
```

`side_effect` allows you to perform side effects, including raising an exception when a mock is called:

```
>>> from unittest.mock import Mock
>>> mock = Mock(side_effect=KeyError('foo'))
>>> mock()
Traceback (most recent call last):
...
KeyError: 'foo'

>>> values = {'a': 1, 'b': 2, 'c': 3}
>>> def side_effect(arg):
...     return values[arg]
...
>>> mock.side_effect = side_effect
>>> mock('a'), mock('b'), mock('c')
(1, 2, 3)
>>> mock.side_effect = [5, 4, 3, 2, 1]
>>> mock(), mock(), mock()
(5, 4, 3)
```

Mock has many other ways you can configure it and control its behaviour. For example the `spec` argument configures the mock to take its specification from another object. Attempting to access attributes or methods on the mock that don't exist on the spec will fail with an `AttributeError`.

The `patch()` decorator / context manager makes it easy to mock classes or objects in a module under test. The object you specify will be replaced with a mock (or other object) during the test and restored when the test ends:

```
>>> from unittest.mock import patch
>>> @patch('module.ClassName2')
... @patch('module.ClassName1')
... def test(MockClass1, MockClass2):
...     module.ClassName1()
...     module.ClassName2()
...     assert MockClass1 is module.ClassName1
...     assert MockClass2 is module.ClassName2
...     assert MockClass1.called
...     assert MockClass2.called
...
>>> test()
```

■ Nota: Note When you nest patch decorators the mocks are passed in to the decorated function in the same order they applied (the normal Python order that decorators are applied). This means from the bottom up, so in the example above the mock for `module.ClassName1` is passed in first. With `patch()` it matters that you patch objects in the namespace where they are looked up. This is normally straightforward, but for a quick guide read [where to patch](#).

As well as a decorator `patch()` can be used as a context manager in a `with` statement:

```
>>> with patch.object(ProductionClass, 'method', return_value=None) as mock_method:
...     thing = ProductionClass()
...     thing.method(1, 2, 3)
...
>>> mock_method.assert_called_once_with(1, 2, 3)
```

There is also `patch.dict()` for setting values in a dictionary just during a scope and restoring the dictionary to its original state when the test ends:

```
>>> foo = {'key': 'value'}
>>> original = foo.copy()
>>> with patch.dict(foo, {'newkey': 'newvalue'}, clear=True):
...     assert foo == {'newkey': 'newvalue'}
...
>>> assert foo == original
```

Mock supports the mocking of Python magic methods. The easiest way of using magic methods is with the `MagicMock` class. It allows you to do things like:

```
>>> mock = MagicMock()
>>> mock.__str__.return_value = 'foobarbaz'
>>> str(mock)
'foobarbaz'
>>> mock.__str__.assert_called_with()
```

Mock allows you to assign functions (or other Mock instances) to magic methods and they will be called appropriately. The `MagicMock` class is just a `Mock` variant that has all of the magic methods pre-created for you (well, all the useful ones anyway).

The following is an example of using magic methods with the ordinary `Mock` class:

```
>>> mock = Mock()
>>> mock.__str__ = Mock(return_value='whewheeee')
>>> str(mock)
'whewheeee'
```

For ensuring that the mock objects in your tests have the same api as the objects they are replacing, you can use auto-specing. Auto-specing can be done through the `autospec` argument to `patch`, or the `create_autospec()` function. Auto-specing creates mock objects that have the same attributes and methods as the objects they are replacing, and any functions and methods (including constructors) have the same call

signature as the real object.

This ensures that your mocks will fail in the same way as your production code if they are used incorrectly:

```
>>> from unittest.mock import create_autospec
>>> def function(a, b, c):
...     pass
...
>>> mock_function = create_autospec(function, return_value='fishy')
>>> mock_function(1, 2, 3)
'fishy'
>>> mock_function.assert_called_once_with(1, 2, 3)
>>> mock_function('wrong arguments')
Traceback (most recent call last):
...
TypeError: missing a required argument: 'b'
```

`create_autospec()` can also be used on classes, where it copies the signature of the `__init__` method, and on callable objects where it copies the signature of the `__call__` method.

The Mock Class

`Mock` is a flexible mock object intended to replace the use of stubs and test doubles throughout your code.

Mocks are callable and create attributes as new mocks when you access them[1]. Accessing the same attribute will always return the same mock. Mocks record how you use them, allowing you to make assertions about what your code has done to them.

`MagicMock` is a subclass of `Mock` with all the magic methods pre-created and ready to use. There are also non-callable variants, useful when you are mocking out objects that aren't callable: `NonCallableMock` and `NonCallableMagicMock`

The `patch()` decorator makes it easy to temporarily replace classes in a particular module with a `Mock` object. By default `patch()` will create a `MagicMock` for you. You can specify an alternative class of `Mock` using the `new_callable` argument to `patch()`.

`unittest.mock.Mock(spec=None, side_effect=None, return_value=DEFAULT, wraps=None, name=None, spec_set=None, unsafe=False, **kwargs)`

`assert_called()`

`assert_called_once()`

`assert_called_with(*args, **kwargs)`

`assert_called_once_with(*args, **kwargs)`

`assert_any_call(*args, **kwargs)`

`assert_has_calls(calls, any_order=False)`

`assert_not_called()`

`reset_mock(*, return_value=False, side_effect=False)`

`mock_add_spec(spec, spec_set=False)`

`attach_mock(mock, attribute)`

`configure_mock(**kwargs)`

`__dir__()`

`_get_child_mock(**kw)`

called

call_count

return_value

side_effect

call_args

call_args_list

method_calls

mock_calls

__class__

Create a new `Mock` object. `Mock` takes several optional arguments that specify the behaviour of the `Mock` object: `spec`: This can be either a list of strings or an existing object (a class or instance) that acts as the specification for the mock object. If you pass in an object then a list of strings is formed by calling `dir` on the object (excluding unsupported magic attributes and methods). Accessing any attribute not in this list will raise an `AttributeError`. If `spec` is an object (rather than a list of str

Assert that the mock was called at least once. `>>> mock = Mock() >>> mock.method() <Mock name='mock.method()' id='...'>>> mock.method.assert_called()` Added in version 3.6.

Assert that the mock was called exactly once. `>>> mock = Mock() >>> mock.method() <Mock name='mock.method()' id='...'>>> mock.method.assert_called_once() >>> mock.method() <Mock name='mock.method()' id='...'>>> mock.method.assert_called_once()` Traceback (most recent call last): ... `AssertionError: Expected 'method' to have been called once. Called 2 times. Calls: [call(), call()].` Added in version 3.6.

This method is a convenient way of asserting that the last call has been made in a particular way: `>>> mock = Mock() >>> mock.method(1, 2, 3, test='wow') <Mock name='mock.method()' id='...'>>> mock.method.assert_called_with(1, 2, 3, test='wow')`

Assert that the mock was called exactly once and that call was with the specified arguments. `>>> mock = Mock(return_value=None) >>> mock('foo', bar='baz') >>> mock.assert_called_once_with('foo', bar='baz') >>> mock('other', bar='values') >>> mock.assert_called_once_with('other', bar='values')` Traceback (most recent call last): ... `AssertionError: Expected 'mock' to be called once. Called 2 times. Calls: [call('foo', bar='baz'), call('other', bar='values')].`

assert the mock has been called with the specified arguments. The assert passes if the mock has ever been called, unlike `assert_called_with()` and `assert_called_once_with()` that only pass if the call is the most recent one, and in the case of `assert_called_once_with()` it must also be the only call. `>>> mock = Mock(return_value=None) >>> mock(1, 2, arg='thing') >>> mock('some', 'thing', 'else') >>> mock.assert_any_call(1, 2, arg='thing')`

assert the mock has been called with the specified calls. The `mock_calls` list is checked for the calls. If `any_order` is false then the calls must be sequential. There can be extra calls before or after the specified calls. If `any_order` is true then the calls can be in any order, but they must all appear in `mock_calls`. `>>> mock = Mock(return_value=None) >>> mock(1) >>> mock(2) >>> mock(3) >>> mock(4) >>> calls = [call(2), call(3)] >>> mock.assert_has_calls(calls) >>> calls = [call(4), call(2), call(3)] >>> mock.assert_has_calls(cal`

Assert the mock was never called. `>>> m = Mock() >>> m.hello.assert_not_called() >>> obj = m.hello() >>> m.hello.assert_not_called()` Traceback (most recent call last): ... `AssertionError: Expected 'hello' to not have been called. Called 1 times. Calls: [call()].` Added in version 3.5.

The `reset_mock` method resets all the call attributes on a mock object:

```
>>> mock = Mock(return_value=None)
>>> mock('hello')
>>> mock.called
True
>>> mock.reset_mock()
>>> mock.called
False
```

This can be useful where you want to make a series of assertions that reuse the same object. `return_value` parameter when set to `True` resets `return_value`:

```
>>> mock = Mock(return_value=5)
>>> mock('hello')
5
>>> mock.reset_mock(return_value=True)
>>> mock('hello')
<Mock name='mock()' id='...'>
```

`side_effect` parameter when set to `True` resets `side_effect`

Add a spec to a mock. `spec` can either be an object or a list of strings. Only attributes on the `spec` can be fetched as attributes from the mock. If `spec_set` is `true` then only attributes on the `spec` can be set.

Attach a mock as an attribute of this one, replacing its name and parent. Calls to the attached mock will be recorded in the `method_calls` and `mock_calls` attributes of this one.

Set attributes on the mock through keyword arguments. Attributes plus return values and side effects can be set on child mocks using standard dot notation and unpacking a dictionary in the method call:

```
>>> mock = Mock()
>>> attrs = {'method.return_value': 3, 'other.side_effect': KeyError}
>>> mock.configure_mock(**attrs)
>>> mock.method(3)
3
>>> mock.other()
Traceback (most recent call last):
...
KeyError
```

The same thing can be achieved in the constructor call to mocks:

```
>>> attrs = {'method.return_value': 3, 'other.side_effect':
```

Mock objects limit the results of `dir(some_mock)` to useful results. For mocks with aspect this includes all the permitted attributes for the mock. See `FILTER_DIR` for what this filtering does, and how to switch it off.

Create the child mocks for attributes and return value. By default child mocks will be the same type as the parent. Subclasses of `Mock` may want to override this to customize the way child mocks are made. For non-callable mocks the callable variant will be used (rather than any custom subclass).

A boolean representing whether or not the mock object has been called:

```
>>> mock = Mock(return_value=None)
>>> mock.called
False
>>> mock()
>>> mock.called
True
```

An integer telling you how many times the mock object has been called:

```
>>> mock = Mock(return_value=None)
>>> mock.call_count
0
>>> mock()
>>> mock()
>>> mock.call_count
2
```

Set this to configure the value returned by calling the mock:

```
>>> mock = Mock()
>>> mock.return_value = 'fish'
>>> mock()
'fish'
```

The default return value is a mock object and you can configure it in the normal way:

```
>>> mock = Mock()
>>> mock.return_value.attribute = sentinel.Attribute
>>> mock.return_value
<Mock name='mock()' id='...'>
```

`assert_called_with()` `return_value` can also be set in the constructor:

```
>>> mock = Mock(return_value=3)
>>> mock.return_value
3
>>> mock()
3
```

This can either be a function to be called when the mock is called, an iterable or an exception (class or instance) to be raised. If you pass in a function it will be called with same arguments as the mock and unless the function returns the `DEFAULT` singleton the call to the mock will then return whatever the function returns. If the function returns `DEFAULT` then the mock will return its normal value (from the `return_value`). If you pass in an iterable, it is used to retrieve an iterator which must yield

This is either `None` (if the mock hasn't been called), or the arguments that the mock was last called with. This will be in the form of a tuple: the first member, which can also be accessed through the `args` property, is any positional arguments the mock was called with (or an empty tuple) and the second member, which can also be accessed through the `kwargs` property, is any keyword arguments (or an empty dictionary).

```
>>> mock = Mock(return_value=None)
>>> print(mock.call_args)
None
>>> mock()
>>> mock.call_args
call_args
```

This is a list of all the calls made to the mock object in sequence (so the length of the list is the number of times it has been called). Before any calls have been made it is an empty list. The `call` object can be used for conveniently constructing lists of calls to compare with `call_args_list`.

```
>>> mock = Mock(return_value=None)
>>> mock()
>>> mock(3, 4)
>>> mock(key='fish', next='w00t!')
>>> mock.call_args_list
[call(), call(3, 4), call(key='fish', next='w00t!')]
>>> expected = [], ((3, 4),), ({'key': 'fish', 'next': 'w00t!'})
```

As well as tracking calls to themselves, mocks also track calls to methods and attributes, and their methods and attributes:

```
>>> mock = Mock()
>>> mock.method()
<Mock name='mock.method()'>
```

id='...'>>>mock.property.method.attribute()<Mock name='mock.property.method.attribute()' id='...'>>>mock.method_calls[call.method(), call.property.method.attribute()])Members ofmethod_callsarecallableobjects. These can be unpacked as tuples to get at the individual arguments. See`calls` as tuples.

`mock_calls` records all calls to the mock object, its methods, magic methods and return value

```
mocks.>>>mock=MagicMock()>>>result=mock(1,2,3)>>>mock.first(a=3)<MagicMock name='mock.first()' id='...'>>>mock.second()<MagicMock name='mock.second()' id='...'>>>int(mock)1>>>result(1)<MagicMock name='mock()' id='...'>>>expected=[call(1,2,3),call.first(a=3),call.second(),...call.__int__(),call()(1)]>>>mock.mock_calls==expectedTrueMembers ofmock_callsarecallableobjects. These can be unpacked as tuples to get
```

Normally the `__class__` attribute of an object will return its type. For a mock object with `aspec`, `__class__` returns the spec class instead. This allows mock objects to pass `isinstance()` tests for the object they are replacing / masquerading as:

```
>>>mock=Mock(spec=3)>>>isinstance(mock,int)True__class__is assignable to, this allows a mock to pass anisinstance()check without forcing you to use a spec:>>>mock=Mock()>>>mock.__class__=dict>>>isinstance(mock,dict)True
```

`unittest.mock.NonCallableMock(spec=None, wraps=None, name=None, spec_set=None, **kwargs)`

A non-callable version of `Mock`. The constructor parameters have the same meaning of `Mock`, with the exception of `return_value` and `side_effect` which have no meaning on a non-callable mock.

Mock objects that use a class or an instance as `spec` or `spec_set` are able to pass `isinstance()` tests:

```
>>> mock = Mock(spec=SomeClass)
>>> isinstance(mock, SomeClass)
True
>>> mock = Mock(spec_set=SomeClass())
>>> isinstance(mock, SomeClass)
True
```

The `Mock` classes have support for mocking magic methods. See `magic methods` for the full details.

The mock classes and the `patch()` decorators all take arbitrary keyword arguments for configuration. For the `patch()` decorators the keywords are passed to the constructor of the mock being created. The keyword arguments are for configuring attributes of the mock:

```
>>> m = MagicMock(attribute=3, other='fish')
>>> m.attribute
3
>>> m.other
'fish'
```

The return value and side effect of child mocks can be set in the same way, using dotted notation. As you can't use dotted names directly in a call you have to create a dictionary and unpack it using `**`:

```
>>> attrs = {'method.return_value': 3, 'other.side_effect': KeyError}
>>> mock = Mock(some_attribute='eggs', **attrs)
>>> mock.some_attribute
'eggs'
>>> mock.method()
3
>>> mock.other()
Traceback (most recent call last):
...
KeyError
```

A callable mock which was created with `aspec` (or `aspec_set`) will introspect the specification object's signature when matching calls to the mock. Therefore, it can match the actual call's arguments regardless of whether they were passed positionally or by name:

```
>>> def f(a, b, c): pass
...

```

```
>>> mock = Mock(spec=f)
>>> mock(1, 2, c=3)
<Mock name='mock()' id='140161580456576'>
>>> mock.assert_called_with(1, 2, 3)
>>> mock.assert_called_with(a=1, b=2, c=3)
```

This applies to `assert_called_with()`, `assert_called_once_with()`, `assert_has_calls()` and `assert_any_call()`. When Autospeccing, it will also apply to method calls on the mock object.

Changed in version 3.4: Added signature introspection on specced and autospecced mock objects.

class unittest.mock.PropertyMock(*args,kwargs)**

A mock intended to be used as a property, or other descriptor, on a class. `PropertyMock` provides `__get__()` and `__set__()` methods so you can specify a return value when it is fetched. Fetching a `PropertyMock` instance from an object calls the mock, with no args. Setting it calls the mock with the value being set. >>> class Foo: ... @property ... def foo(self): ... return 'something' ... @foo.setter ... def foo(self, value): ... pass ... >>> withpatch('__main__.Foo.foo', new_callable=PropertyMock) as mock_foo: ... mock_foo.return_value

Because of the way mock attributes are stored you can't directly attach a `PropertyMock` to a mock object. Instead you can attach it to the mock type object:

```
>>> m = MagicMock()
>>> p = PropertyMock(return_value=3)
>>> type(m).foo = p
>>> m.foo
3
>>> p.assert_called_once_with()
```

Caution

If an `AttributeError` is raised by `PropertyMock`, it will be interpreted as a missing descriptor and `__getattr__()` will be called on the parent mock:

```
>>> m = MagicMock()
>>> no_attribute = PropertyMock(side_effect=AttributeError)
>>> type(m).my_property = no_attribute
>>> m.my_property
<MagicMock name='mock.my_property' id='140165240345424'>
```

See `__getattr__()` for details.

class unittest.mock.AsyncMock(spec=None, side_effect=None, return_value=DEFAULT, wraps=None, name=None, spec_set=None, unsafe=False, **kwargs)

assert_awaited()

assert_awaited_once()

assert_awaited_with(*args,kwargs)**

assert_awaited_once_with(*args,kwargs)**

assert_any_await(*args,kwargs)**

assert_has_awaits(calls, any_order=False)

assert_not_awaited()

reset_mock(*args,kwargs)**

await_count

await_args

await_args_list

An asynchronous version of `MagicMock`. The `AsyncMock` object will behave so the object is recognized as an async function, and the result of a call is an awaitable. `>>> mock = AsyncMock() >>> inspect.iscoroutinefunction(mock) True >>> inspect.isawaitable(mock()) True` The result of `mock()` is an async function which will have the outcome of `side_effect` or `return_value` after it has been awaited: if `side_effect` is a function, the async function will return the result of that function, if `side_effect` is an exception, the async

Assert that the mock was awaited at least once. Note that this is separate from the object having been called, the `await` keyword must be used: `>>> mock = AsyncMock() >>> async def main(coroutine_mock): ... await coroutine_mock ... >>> coroutine_mock = mock() >>> mock.called True >>> mock.assert_awaited() Traceback (most recent call last): ... AssertionError: Expected mock to have been awaited. >>> asyncio.run(main(coroutine_mock)) >>> mock.assert_awaited()`

Assert that the mock was awaited exactly once. `>>> mock = AsyncMock() >>> async def main(): ... await mock() ... >>> asyncio.run(main()) >>> mock.assert_awaited_once() >>> asyncio.run(main()) >>> mock.assert_awaited_once() Traceback (most recent call last): ... AssertionError: Expected mock to have been awaited once. Awaited 2 times.`

Assert that the last await was with the specified arguments. `>>> mock = AsyncMock() >>> async def main(*args, **kwargs): ... await mock(*args, **kwargs) ... >>> asyncio.run(main('foo', bar='bar')) >>> mock.assert_awaited_with('foo', bar='bar') >>> mock.assert_awaited_with('other') Traceback (most recent call last): ... AssertionError: expected await not found. Expected: mock('other') Actual: mock('foo', bar='bar')`

Assert that the mock was awaited exactly once and with the specified arguments. `>>> mock = AsyncMock() >>> async def main(*args, **kwargs): ... await mock(*args, **kwargs) ... >>> asyncio.run(main('foo', bar='bar')) >>> mock.assert_awaited_once_with('foo', bar='bar') >>> asyncio.run(main('foo', bar='bar')) >>> mock.assert_awaited_once_with('foo', bar='bar') Traceback (most recent call last): ... AssertionError: Expected mock to have been awaited once. Awaited 2 times.`

Assert the mock has ever been awaited with the specified arguments. `>>> mock = AsyncMock() >>> async def main(*args, **kwargs): ... await mock(*args, **kwargs) ... >>> asyncio.run(main('foo', bar='bar')) >>> asyncio.run(main('hello')) >>> mock.assert_any_await('foo', bar='bar') >>> mock.assert_any_await('other') Traceback (most recent call last): ... AssertionError: mock('other') await not found`

Assert the mock has been awaited with the specified calls. The `await_args_list` is checked for the awaits. If `any_order` is false then the awaits must be sequential. There can be extra calls before or after the specified awaits. If `any_order` is true then the awaits can be in any order, but they must all appear in `await_args_list`. `>>> mock = AsyncMock() >>> async def main(*args, **kwargs): ... await mock(*args, **kwargs) ... >>> calls = [call("foo"), call("bar")] >>> mock.assert_has_awaits(calls) Traceback (most recent call`

Assert that the mock was never awaited. `>>> mock = AsyncMock() >>> mock.assert_not_awaited()`

See `Mock.reset_mock()`. Also sets `await_count` to 0, `await_args` to None, and clears the `await_args_list`.

An integer keeping track of how many times the mock object has been awaited. `>>> mock = AsyncMock() >>> async def main(): ... await mock() ... >>> asyncio.run(main()) >>> mock.await_count 1 >>> asyncio.run(main()) >>> mock.await_count 2`

This is either None (if the mock hasn't been awaited), or the arguments that the mock was last awaited with. Functions the same as `Mock.call_args`. `>>> mock = AsyncMock() >>> async def main(*args): ... await mock(*args) ... >>> mock.await_args >>> asyncio.run(main('foo')) >>> mock.await_argscall('foo') >>> asyncio.run(main('bar')) >>> mock.await_argscall('bar')`

This is a list of all the awaits made to the mock object in sequence (so the length of the list is the number of times it has been awaited). Before any awaits have been made it is an empty list. `>>> mock = AsyncMock() >>> async def main(*args): ... await mock(*args) ... >>> mock.await_args_list >>> asyncio.run(main('foo')) >>> mock.await_args_list[call('foo')] >>> asyncio.run(main('bar')) >>> mock.await_args_list[call('foo'), call('bar')]`

```
class unittest.mock.ThreadingMock(spec=None, side_effect=None, return_value=DEFAULT, wraps=None,
name=None, spec_set=None, unsafe=False, *, timeout=UNSET, **kwargs)
```

```
wait_until_called(*, timeout=UNSET)
```

```
wait_until_any_call_with(*args, **kwargs)
```

DEFAULT_TIMEOUT

A version of `MagicMock` for multithreading tests. The `ThreadingMock` object provides extra methods to wait for a call to be invoked, rather than assert on it immediately. The default timeout is specified by the `timeout` argument, or if unset by the `ThreadingMock.DEFAULT_TIMEOUT` attribute, which defaults to blocking (`None`). You can configure the global default timeout by setting `ThreadingMock.DEFAULT_TIMEOUT`. `wait_until_called(*, timeout=UNSET)` Waits until the mock is called. If a timeout was passed at the creation

Waits until the mock is called. If a timeout was passed at the creation of the mock or if a timeout argument is passed to this function, the function raises an `AssertionError` if the call is not performed in time. >>> mock = ThreadingMock() >>> thread = threading.Thread(target=mock) >>> thread.start() >>> mock.wait_until_called(timeout=1) >>> thread.join()

Waits until the mock is called with the specified arguments. If a timeout was passed at the creation of the mock the function raises an `AssertionError` if the call is not performed in time. >>> mock = ThreadingMock() >>> thread = threading.Thread(target=mock, args=("arg1", "arg2"), kwargs={"arg": "thing"}) >>> thread.start() >>> mock.wait_until_any_call_with("arg1", "arg2", arg="thing") >>> thread.join()

Global default timeout in seconds to create instances of `ThreadingMock`.

Calling

Mock objects are callable. The call will return the value set as the `return_value` attribute. The default return value is a new `Mock` object; it is created the first time the return value is accessed (either explicitly or by calling the `Mock`) - but it is stored and the same one returned each time.

Calls made to the object will be recorded in the attributes like `call_args` and `call_args_list`.

If `side_effect` is set then it will be called after the call has been recorded, so if `side_effect` raises an exception the call is still recorded.

The simplest way to make a mock raise an exception when called is to make `side_effect` an exception class or instance:

```
>>> m = MagicMock(side_effect=IndexError)
>>> m(1, 2, 3)
Traceback (most recent call last):
...
IndexError
>>> m.mock_calls
[call(1, 2, 3)]
>>> m.side_effect = KeyError('Bang!')
>>> m('two', 'three', 'four')
Traceback (most recent call last):
...
KeyError: 'Bang!'
>>> m.mock_calls
[call(1, 2, 3), call('two', 'three', 'four')]
```

If `side_effect` is a function then whatever that function returns is what calls to the mock return.

The `side_effect` function is called with the same arguments as the mock. This allows you to vary the return value of the call dynamically, based on the input:

```
>>> def side_effect(value):
```

```

...     return value + 1
...
>>> m = MagicMock(side_effect=side_effect)
>>> m(1)
2
>>> m(2)
3
>>> m.mock_calls
[call(1), call(2)]

```

If you want the mock to still return the default return value (a new mock), or any set return value, then there are two ways of doing this. Either return `return_value` from inside `side_effect`, or return `DEFAULT`:

```

>>> m = MagicMock()
>>> def side_effect(*args, **kwargs):
...     return m.return_value
...
>>> m.side_effect = side_effect
>>> m.return_value = 3
>>> m()
3
>>> def side_effect(*args, **kwargs):
...     return DEFAULT
...
>>> m.side_effect = side_effect
>>> m()
3

```

To remove `side_effect`, and return to the default behaviour, set the `side_effect` to `None`:

```

>>> m = MagicMock(return_value=6)
>>> def side_effect(*args, **kwargs):
...     return 3
...
>>> m.side_effect = side_effect
>>> m()
3
>>> m.side_effect = None
>>> m()
6

```

The `side_effect` can also be any iterable object. Repeated calls to the mock will return values from the iterable (until the iterable is exhausted and a `StopIteration` is raised):

```

>>> m = MagicMock(side_effect=[1, 2, 3])
>>> m()
1
>>> m()
2
>>> m()
3
>>> m()
Traceback (most recent call last):
...
StopIteration

```

If any members of the iterable are exceptions they will be raised instead of returned:

```

>>> iterable = (33, ValueError, 66)
>>> m = MagicMock(side_effect=iterable)
>>> m()
33
>>> m()
Traceback (most recent call last):
...
ValueError

```



```
>>> m()
66
```

Deleting Attributes

Mock objects create attributes on demand. This allows them to pretend to be objects of any type.

You may want a mock object to return `False` to a `hasattr()` call, or raise an `AttributeError` when an attribute is fetched. You can do this by providing an object as `aspec` for a mock, but that isn't always convenient.

You “block” attributes by deleting them. Once deleted, accessing an attribute will raise an `AttributeError`.

```
>>> mock = MagicMock()
>>> hasattr(mock, 'm')
True
>>> del mock.m
>>> hasattr(mock, 'm')
False
>>> del mock.f
>>> mock.f
Traceback (most recent call last):
...
AttributeError: f
```

Mock names and the name attribute

Since “name” is an argument to the `Mock` constructor, if you want your mock object to have a “name” attribute you can't just pass it in at creation time. There are two alternatives. One option is to use `configure_mock()`:

```
>>> mock = MagicMock()
>>> mock.configure_mock(name='my_name')
>>> mock.name
'my_name'
```

A simpler option is to simply set the “name” attribute after mock creation:

```
>>> mock = MagicMock()
>>> mock.name = "foo"
```

Attaching Mocks as Attributes

When you attach a mock as an attribute of another mock (or as the return value) it becomes a “child” of that mock. Calls to the child are recorded in the `method_calls` and `mock_calls` attributes of the parent. This is useful for configuring child mocks and then attaching them to the parent, or for attaching mocks to a parent that records all calls to the children and allows you to make assertions about the order of calls between mocks:

```
>>> parent = MagicMock()
>>> child1 = MagicMock(return_value=None)
>>> child2 = MagicMock(return_value=None)
>>> parent.child1 = child1
>>> parent.child2 = child2
>>> child1(1)
>>> child2(2)
>>> parent.mock_calls
[call.child1(1), call.child2(2)]
```

The exception to this is if the mock has a name. This allows you to prevent the “parenting” if for some reason you don't want it to happen.

```
>>> mock = MagicMock()
>>> not_a_child = MagicMock(name='not-a-child')
>>> mock.attribute = not_a_child
>>> mock.attribute()
<MagicMock name='not-a-child()' id='...'>
>>> mock.mock_calls
[]
```

Mocks created for you by `patch()` are automatically given names. To attach mocks that have names to a parent you use the `attach_mock()` method:

```
>>> thing1 = object()
>>> thing2 = object()
>>> parent = MagicMock()
>>> with patch('__main__.thing1', return_value=None) as child1:
...     with patch('__main__.thing2', return_value=None) as child2:
...         parent.attach_mock(child1, 'child1')
...         parent.attach_mock(child2, 'child2')
...         child1('one')
...         child2('two')
...
>>> parent.mock_calls
[call.child1('one'), call.child2('two')]
```

The only exceptions are magic methods and attributes (those that have leading and trailing double underscores). Mock doesn't create these but instead raises an `AttributeError`. This is because the interpreter will often implicitly request these methods, and gets very confused to get a new Mock object when it expects a magic method. If you need magic method support see magic methods.

The patchers

The patch decorators are used for patching objects only within the scope of the function they decorate. They automatically handle the unpatching for you, even if exceptions are raised. All of these functions can also be used in with statements or as class decorators.

patch

■ Nota: Note The key is to do the patching in the right namespace. See the section where to patch.

`unittest.mock.patch(target, new=DEFAULT, spec=None, create=False, spec_set=None, autospec=None, new_callable=None, **kwargs)`

`patch()` acts as a function decorator, class decorator or a context manager. Inside the body of the function or with statement, the `target` is patched with a new object. When the function/with statement exits the patch is undone. If `new` is omitted, then the target is replaced with an `AsyncMock` if the patched object is an async function or a `MagicMock` otherwise. If `patch()` is used as a decorator and `new` is omitted, the created mock is passed in as an extra argument to the decorated function. If `patch()` is used as a

`patch()` as function decorator, creating the mock for you and passing it into the decorated function:

```
>>> @patch('__main__.SomeClass')
... def function(normal_argument, mock_class):
...     print(mock_class is SomeClass)
...
>>> function(None)
True
```

Patching a class replaces the class with a `MagicMock` instance. If the class is instantiated in the code under test then it will be the `return_value` of the mock that will be used.

If the class is instantiated multiple times you could use `side_effect` to return a new mock each time. Alternatively you can set the `return_value` to be anything you want.

To configure return values on methods of instances on the patched class you must do this on the `return_value`. For example:

```
>>> class Class:
...     def method(self):
...         pass
```

```
...
>>> with patch('__main__.Class') as MockClass:
...     instance = MockClass.return_value
...     instance.method.return_value = 'foo'
...     assert Class() is instance
...     assert Class().method() == 'foo'
...

```

If you use `spec` or `spec_set` and `patch()` is replacing a class, then the return value of the created mock will have the same spec.

```
>>> Original = Class
>>> patcher = patch('__main__.Class', spec=True)
>>> MockClass = patcher.start()
>>> instance = MockClass()
>>> assert isinstance(instance, Original)
>>> patcher.stop()

```

The `new_callable` argument is useful where you want to use an alternative class to the default `MagicMock` for the created mock. For example, if you wanted a `NonCallableMock` to be used:

```
>>> thing = object()
>>> with patch('__main__.thing', new_callable=NonCallableMock) as mock_thing:
...     assert thing is mock_thing
...     thing()
...
Traceback (most recent call last):
...
TypeError: 'NonCallableMock' object is not callable

```

Another use case might be to replace an object with an `io.StringIO` instance:

```
>>> from io import StringIO
>>> def foo():
...     print('Something')
...
>>> @patch('sys.stdout', new_callable=StringIO)
... def test(mock_stdout):
...     foo()
...     assert mock_stdout.getvalue() == 'Something\n'
...
>>> test()

```

When `patch()` is creating a mock for you, it is common that the first thing you need to do is to configure the mock. Some of that configuration can be done in the call to `patch`. Any arbitrary keywords you pass into the call will be used to set attributes on the created mock:

```
>>> patcher = patch('__main__.thing', first='one', second='two')
>>> mock_thing = patcher.start()
>>> mock_thing.first
'one'
>>> mock_thing.second
'two'

```

As well as attributes on the created mock attributes, like the `return_value` and `side_effect`, of child mocks can also be configured. These aren't syntactically valid to pass in directly as keyword arguments, but a dictionary with these as keys can still be expanded into a `patch()` call using `**`:

```
>>> config = {'method.return_value': 3, 'other.side_effect': KeyError}
>>> patcher = patch('__main__.thing', **config)
>>> mock_thing = patcher.start()
>>> mock_thing.method()
3
>>> mock_thing.other()
Traceback (most recent call last):

```

```
...
KeyError
```

By default, attempting to patch a function in a module (or a method or an attribute in a class) that does not exist will fail with `AttributeError`:

```
>>> @patch('sys.non_existing_attribute', 42)
... def test():
...     assert sys.non_existing_attribute == 42
...
>>> test()
Traceback (most recent call last):
...
AttributeError: <module 'sys' (built-in)> does not have the attribute 'non_existing_attribute'
```

but adding `create=True` in the call to `patch()` will make the previous example work as expected:

```
>>> @patch('sys.non_existing_attribute', 42, create=True)
... def test(mock_stdout):
...     assert sys.non_existing_attribute == 42
...
>>> test()
```

Changed in version 3.8: `patch()` now returns an `AsyncMock` if the target is an async function.

patch.object

`patch.object(target, attribute, new=DEFAULT, spec=None, create=False, spec_set=None, autospec=None, new_callable=None, **kwargs)`

patch the named member (attribute) on an object (target) with a mock object. `patch.object()` can be used as a decorator, class decorator or a context manager.

Arguments `new`, `spec`, `create`, `spec_set`, `autospec` and `new_callable` have the same meaning as for `patch()`. Like `patch()`, `patch.object()` takes arbitrary keyword arguments for configuring the mock object it creates. When used as a class decorator `patch.object()` honours `patch.TEST_PREFIX` for choosing which methods to wrap.

You can either call `patch.object()` with three arguments or two arguments. The three argument form takes the object to be patched, the attribute name and the object to replace the attribute with.

When calling with the two argument form you omit the replacement object, and a mock is created for you and passed in as an extra argument to the decorated function:

```
>>> @patch.object(SomeClass, 'class_method')
... def test(mock_method):
...     SomeClass.class_method(3)
...     mock_method.assert_called_with(3)
...
>>> test()
```

`spec`, `create` and the other arguments to `patch.object()` have the same meaning as they do for `patch()`.

patch.dict

`patch.dict(in_dict, values={}, clear=False, **kwargs)`

Patch a dictionary, or dictionary like object, and restore the dictionary to its original state after the test, where the restored dictionary is a copy of the dictionary as it was before the test. `in_dict` can be a dictionary or a mapping like container. If it is a mapping then it must at least support getting, setting and deleting items plus iterating over keys. `in_dict` can also be a string specifying the name of the dictionary, which will then be fetched by importing it. `values` can be a dictionary of

`patch.dict()` can be used as a context manager, decorator or class decorator:

```
>>> foo = {}
```

```
>>> @patch.dict(foo, {'newkey': 'newvalue'})
... def test():
...     assert foo == {'newkey': 'newvalue'}
...
>>> test()
>>> assert foo == {}
```

When used as a class decorator `patch.dict()` honours `patch.TEST_PREFIX` (default to 'test') for choosing which methods to wrap:

```
>>> import os
>>> import unittest
>>> from unittest.mock import patch
>>> @patch.dict('os.environ', {'newkey': 'newvalue'})
... class TestSample(unittest.TestCase):
...     def test_sample(self):
...         self.assertEqual(os.environ['newkey'], 'newvalue')
```

If you want to use a different prefix for your test, you can inform the patchers of the different prefix by setting `patch.TEST_PREFIX`. For more details about how to change the value of see `TEST_PREFIX`.

`patch.dict()` can be used to add members to a dictionary, or simply let a test change a dictionary, and ensure the dictionary is restored when the test ends.

```
>>> foo = {}
>>> with patch.dict(foo, {'newkey': 'newvalue'}) as patched_foo:
...     assert foo == {'newkey': 'newvalue'}
...     assert patched_foo == {'newkey': 'newvalue'}
...     # You can add, update or delete keys of foo (or patched_foo, it's the same dict)
...     patched_foo['spam'] = 'eggs'
...
>>> assert foo == {}
>>> assert patched_foo == {}

>>> import os
>>> with patch.dict('os.environ', {'newkey': 'newvalue'}):
...     print(os.environ['newkey'])
...
newvalue
>>> assert 'newkey' not in os.environ
```

Keywords can be used in the `patch.dict()` call to set values in the dictionary:

```
>>> mymodule = MagicMock()
>>> mymodule.function.return_value = 'fish'
>>> with patch.dict('sys.modules', mymodule=mymodule):
...     import mymodule
...     mymodule.function('some', 'args')
...
'fish'
```

`patch.dict()` can be used with dictionary like objects that aren't actually dictionaries. At the very minimum they must support item getting, setting, deleting and either iteration or membership test. This corresponds to the magic methods `__getitem__()`, `__setitem__()`, `__delitem__()` and either `__iter__()` or `__contains__()`.

```
>>> class Container:
...     def __init__(self):
...         self.values = {}
...     def __getitem__(self, name):
...         return self.values[name]
...     def __setitem__(self, name, value):
...         self.values[name] = value
...     def __delitem__(self, name):
...         del self.values[name]
...     def __iter__(self):
...         return iter(self.values)
```

```
...
>>> thing = Container()
>>> thing['one'] = 1
>>> with patch.dict(thing, one=2, two=3):
...     assert thing['one'] == 2
...     assert thing['two'] == 3
...
>>> assert thing['one'] == 1
>>> assert list(thing) == ['one']
```

patch.multiple

patch.multiple(target,spec=None,create=False,spec_set=None,autospec=None,new_callable=None,kw args)**

Perform multiple patches in a single call. It takes the object to be patched (either as an object or a string to fetch the object by importing) and keyword arguments for the patches: `with patch.multiple(settings, FIRST_PATCH='one', SECOND_PATCH='two'):`... Use `DEFAULT` as the value if you want `patch.multiple()` to create mocks for you. In this case the created mocks are passed into a decorated function by keyword, and a dictionary is returned when `patch.multiple()` is used as a context manager.

If you want `patch.multiple()` to create mocks for you, then you can use `DEFAULT` as the value. If you use `patch.multiple()` as a decorator then the created mocks are passed into the decorated function by keyword.

```
>>> thing = object()
>>> other = object()

>>> @patch.multiple('__main__', thing=DEFAULT, other=DEFAULT)
... def test_function(thing, other):
...     assert isinstance(thing, MagicMock)
...     assert isinstance(other, MagicMock)
...
>>> test_function()
```

`patch.multiple()` can be nested with other patch decorators, but put arguments passed by keyword after any of the standard arguments created by `patch()`:

```
>>> @patch('sys.exit')
... @patch.multiple('__main__', thing=DEFAULT, other=DEFAULT)
... def test_function(mock_exit, other, thing):
...     assert 'other' in repr(other)
...     assert 'thing' in repr(thing)
...     assert 'exit' in repr(mock_exit)
...
>>> test_function()
```

If `patch.multiple()` is used as a context manager, the value returned by the context manager is a dictionary where created mocks are keyed by name:

```
>>> with patch.multiple('__main__', thing=DEFAULT, other=DEFAULT) as values:
...     assert 'other' in repr(values['other'])
...     assert 'thing' in repr(values['thing'])
...     assert values['thing'] is thing
...     assert values['other'] is other
...
>>>
```

patch methods: start and stop

All the patchers have `start()` and `stop()` methods. These make it simpler to do patching in `setUp` methods or where you want to do multiple patches without nesting decorators or with statements.

To use them call `patch()`, `patch.object()` or `patch.dict()` as normal and keep a reference to the returned patcher object. You can then call `start()` to put the patch in place and `stop()` to undo it.

If you are using `patch()` to create a mock for you then it will be returned by the call `patcher.start`.

```
>>> patcher = patch('package.module.ClassName')
>>> from package import module
>>> original = module.ClassName
>>> new_mock = patcher.start()
>>> assert module.ClassName is not original
>>> assert module.ClassName is new_mock
>>> patcher.stop()
>>> assert module.ClassName is original
>>> assert module.ClassName is not new_mock
```

A typical use case for this might be for doing multiple patches in the `setUp` method of a `TestCase`:

```
>>> class MyTest(unittest.TestCase):
...     def setUp(self):
...         self.patcher1 = patch('package.module.Class1')
...         self.patcher2 = patch('package.module.Class2')
...         self.MockClass1 = self.patcher1.start()
...         self.MockClass2 = self.patcher2.start()
...
...     def tearDown(self):
...         self.patcher1.stop()
...         self.patcher2.stop()
...
...     def test_something(self):
...         assert package.module.Class1 is self.MockClass1
...         assert package.module.Class2 is self.MockClass2
...
>>> MyTest('test_something').run()
```

Caution

If you use this technique you must ensure that the patching is “undone” by calling `stop`. This can be fiddlier than you might think, because if an exception is raised in the `setUp` then the `tearDown` is not called. `unittest.TestCase.addCleanup()` makes this easier:

```
>>> class MyTest(unittest.TestCase):
...     def setUp(self):
...         patcher = patch('package.module.Class')
...         self.MockClass = patcher.start()
...         self.addCleanup(patcher.stop)
...
...     def test_something(self):
...         assert package.module.Class is self.MockClass
...
... 
```

As an added bonus you no longer need to keep a reference to the `patcher` object.

It is also possible to stop all patches which have been started by using `patch.stopall()`.

patch.stopall()

Stop all active patches. Only stops patches started with `start`.

patch builtins

You can patch any builtins within a module. The following example patches `builtinord()`:

```
>>> @patch('__main__.ord')
... def test(mock_ord):
...     mock_ord.return_value = 101
...     print(ord('c'))
...
>>> test()
101
```

TEST_PREFIX

All of the patchers can be used as class decorators. When used in this way they wrap every test method on the class. The patchers recognise methods that start with 'test' as being test methods. This is the same way that the unittest.TestLoader finds test methods by default.

It is possible that you want to use a different prefix for your tests. You can inform the patchers of the different prefix by setting patch.TEST_PREFIX:

```
>>> patch.TEST_PREFIX = 'foo'
>>> value = 3
>>>
>>> @patch('__main__.value', 'not three')
... class Thing:
...     def foo_one(self):
...         print(value)
...     def foo_two(self):
...         print(value)
...
>>>
>>> Thing().foo_one()
not three
>>> Thing().foo_two()
not three
>>> value
3
```

Nesting Patch Decorators

If you want to perform multiple patches then you can simply stack up the decorators.

You can stack up multiple patch decorators using this pattern:

```
>>> @patch.object(SomeClass, 'class_method')
... @patch.object(SomeClass, 'static_method')
... def test(mock1, mock2):
...     assert SomeClass.static_method is mock1
...     assert SomeClass.class_method is mock2
...     SomeClass.static_method('foo')
...     SomeClass.class_method('bar')
...     return mock1, mock2
...
>>> mock1, mock2 = test()
>>> mock1.assert_called_once_with('foo')
>>> mock2.assert_called_once_with('bar')
```

Note that the decorators are applied from the bottom upwards. This is the standard way that Python applies decorators. The order of the created mocks passed into your test function matches this order.

Where to patch

patch() works by (temporarily) changing the object that a name points to with another one. There can be many names pointing to any individual object, so for patching to work you must ensure that you patch the name used by the system under test.

The basic principle is that you patch where an object is looked up, which is not necessarily the same place as where it is defined. A couple of examples will help to clarify this.

Imagine we have a project that we want to test with the following structure:

```
a.py
    -> Defines SomeClass

b.py
    -> from a import SomeClass
```



```
-> some_function instantiates SomeClass
```

Now we want to test `some_function` but we want to mock out `SomeClass` using `patch()`. The problem is that when we import module `b`, which we will have to do when it imports `SomeClass` from module `a`. If we use `patch()` to mock out `a.SomeClass` then it will have no effect on our test; module `b` already has a reference to the real `SomeClass` and it looks like our patching had no effect.

The key is to patch out `SomeClass` where it is used (or where it is looked up). In this case `some_function` will actually look up `SomeClass` in module `b`, where we have imported it. The patching should look like:

```
@patch('b.SomeClass')
```

However, consider the alternative scenario where instead of from `a` import `SomeClass` module `b` does `import a` and `some_function` uses `a.SomeClass`. Both of these import forms are common. In this case the class we want to patch is being looked up in the module and so we have to patch `a.SomeClass` instead:

```
@patch('a.SomeClass')
```

Patching Descriptors and Proxy Objects

Both `patch` and `patch.object` correctly patch and restore descriptors: class methods, static methods and properties. You should patch these on the class rather than an instance. They also work with some objects that proxy attribute access, like the `django` settings object.

MagicMock and magic method support

Mocking Magic Methods

`Mock` supports mocking the Python protocol methods, also known as “magic methods”. This allows mock objects to replace containers or other objects that implement Python protocols.

Because magic methods are looked up differently from normal methods[2], this support has been specially implemented. This means that only specific magic methods are supported. The supported list includes almost all of them. If there are any missing that you need please let us know.

You mock magic methods by setting the method you are interested in to a function or a mock instance. If you are using a function then it must take `self` as the first argument[3].

```
>>> def __str__(self):
...     return 'fooble'
...
>>> mock = Mock()
>>> mock.__str__ = __str__
>>> str(mock)
'fooble'

>>> mock = Mock()
>>> mock.__str__ = Mock()
>>> mock.__str__.return_value = 'fooble'
>>> str(mock)
'fooble'

>>> mock = Mock()
>>> mock.__iter__ = Mock(return_value=iter([]))
>>> list(mock)
[]
```

One use case for this is for mocking objects used as context managers in a `with` statement:

```
>>> mock = Mock()
>>> mock.__enter__ = Mock(return_value='foo')
>>> mock.__exit__ = Mock(return_value=False)
```

```
>>> with mock as m:
...     assert m == 'foo'
...
>>> mock.__enter__.assert_called_with()
>>> mock.__exit__.assert_called_with(None, None, None)
```

Calls to magic methods do not appear in `method_calls`, but they are recorded in `mock_calls`.

■ **Nota:** Note if you use the `spec` keyword argument to create a mock then attempting to set a magic method that isn't in the spec will raise an `AttributeError`.

The full list of supported magic methods is:

- `__hash__`, `__sizeof__`, `__repr__` and `__str__`
- `__dir__`, `__format__` and `__subclasses__`
- `__round__`, `__floor__`, `__trunc__` and `__ceil__`
- Comparisons: `__lt__`, `__gt__`, `__le__`, `__ge__`, `__eq__` and `__ne__`
- Container methods: `__getitem__`, `__setitem__`, `__delitem__`, `__contains__`, `__len__`, `__iter__`, `__reversed__` and `__missing__`
- Context manager: `__enter__`, `__exit__`, `__aenter__` and `__aexit__`
- Unary numeric methods: `__neg__`, `__pos__` and `__invert__`
- The numeric methods (including right hand and in-place variants): `__add__`, `__sub__`, `__mul__`, `__matmul__`, `__truediv__`, `__floordiv__`, `__mod__`, `__divmod__`, `__lshift__`, `__rshift__`, `__and__`, `__xor__`, `__or__`, and `__pow__`
- Numeric conversion methods: `__complex__`, `__int__`, `__float__` and `__index__`
- Descriptor methods: `__get__`, `__set__` and `__delete__`
- Pickling: `__reduce__`, `__reduce_ex__`, `__getinitargs__`, `__getnewargs__`, `__getstate__` and `__setstate__`
- File system path representation: `__fspath__`
- Asynchronous iteration methods: `__aiter__` and `__anext__`

Changed in version 3.8: Added support for `os.PathLike.__fspath__()`.

Changed in version 3.8: Added support for `__aenter__`, `__aexit__`, `__aiter__` and `__anext__`.

The following methods exist but are not supported as they are either in use by mock, can't be set dynamically, or can cause problems:

- `__getattr__`, `__setattr__`, `__init__` and `__new__`
- `__prepare__`, `__instancecheck__`, `__subclasscheck__`, `__del__`

Magic Mock

There are two `MagicMock` variants: `MagicMock` and `NonCallableMagicMock`.

`unittest.mock.MagicMock(*args, **kw)`

`MagicMock` is a subclass of `Mock` with default implementations of most of the magic methods. You can use `MagicMock` without having to configure the magic methods yourself. The constructor parameters have the same meaning as for `Mock`. If you use the `spec` or `spec_set` arguments then only magic methods that exist in the spec will be created.

`unittest.mock.NonCallableMagicMock(*args, **kw)`

A non-callable version of `MagicMock`. The constructor parameters have the same meaning as for `MagicMock`, with the exception of `return_value` and `side_effect` which have no meaning on a non-callable mock.

The magic methods are setup with `MagicMock` objects, so you can configure them and use them in the usual way:

```
>>> mock = MagicMock()
>>> mock[3] = 'fish'
>>> mock.__getitem__.assert_called_with(3, 'fish')
>>> mock.__getitem__.return_value = 'result'
>>> mock[2]
'result'
```

By default many of the protocol methods are required to return objects of a specific type. These methods are preconfigured with a default return value, so that they can be used without you having to do anything if you aren't interested in the return value. You can still set the return value manually if you want to change the default.

Methods and their defaults:

- `__lt__`: `NotImplemented`
- `__gt__`: `NotImplemented`
- `__le__`: `NotImplemented`
- `__ge__`: `NotImplemented`
- `__int__`: `1`
- `__contains__`: `False`
- `__len__`: `0`
- `__iter__`: `iter([])`
- `__exit__`: `False`
- `__aexit__`: `False`
- `__complex__`: `1j`
- `__float__`: `1.0`
- `__bool__`: `True`
- `__index__`: `1`
- `__hash__`: default hash for the mock
- `__str__`: default str for the mock
- `__sizeof__`: default sizeof for the mock

For example:

```
>>> mock = MagicMock()
>>> int(mock)
1
>>> len(mock)
0
>>> list(mock)
[]
>>> object() in mock
False
```

The two equality methods, `__eq__()` and `__ne__()`, are special. They do the default equality comparison on identity, using the `side_effect` attribute, unless you change their return value to return something else:

```
>>> MagicMock() == 3
False
>>> MagicMock() != 3
```

```
True
>>> mock = MagicMock()
>>> mock.__eq__.return_value = True
>>> mock == 3
True
```

The return value of `MagicMock.__iter__()` can be any iterable object and isn't required to be an iterator:

```
>>> mock = MagicMock()
>>> mock.__iter__.return_value = ['a', 'b', 'c']
>>> list(mock)
['a', 'b', 'c']
>>> list(mock)
['a', 'b', 'c']
```

If the return value is an iterator, then iterating over it once will consume it and subsequent iterations will result in an empty list:

```
>>> mock.__iter__.return_value = iter(['a', 'b', 'c'])
>>> list(mock)
['a', 'b', 'c']
>>> list(mock)
[]
```

`MagicMock` has all of the supported magic methods configured except for some of the obscure and obsolete ones. You can still set these up if you want.

Magic methods that are supported but not setup by default in `MagicMock` are:

- `__subclasses__`
- `__dir__`
- `__format__`
- `__get__`, `__set__` and `__delete__`
- `__reversed__` and `__missing__`
- `__reduce__`, `__reduce_ex__`, `__getinitargs__`, `__getnewargs__`, `__getstate__` and `__setstate__`
- `__getformat__`

Magic methods should be looked up on the class rather than the instance. Different versions of Python are inconsistent about applying this rule. The supported protocol methods should work with all supported versions of Python.

The function is basically hooked up to the class, but each `Mock` instance is kept isolated from the others.

Helpers

sentinel

unittest.mock.sentinel

The `sentinel` object provides a convenient way of providing unique objects for your tests. Attributes are created on demand when you access them by name. Accessing the same attribute will always return the same object. The objects returned have a sensible `repr` so that test failure messages are readable. Changed in version 3.7: The `sentinel` attributes now preserve their identity when they are copied or pickled.

Sometimes when testing you need to test that a specific object is passed as an argument to another method, or returned. It can be common to create named sentinel objects to test this. `sentinel` provides a convenient way of creating and testing the identity of objects like this.

In this example we monkey patch `method` to return `sentinel.some_object`:

```
>>> real = ProductionClass()
>>> real.method = Mock(name="method")
>>> real.method.return_value = sentinel.some_object
>>> result = real.method()
>>> assert result is sentinel.some_object
>>> result
sentinel.some_object
```

DEFAULT

`unittest.mock.DEFAULT`

The `DEFAULT` object is a pre-created sentinel (actually `sentinel.DEFAULT`). It can be used by side effect functions to indicate that the normal return value should be used.

call

`unittest.mock.call(*args,**kwargs)`

`call()` is a helper object for making simpler assertions, for comparing with `call_args`, `call_args_list`, `mock_calls` and `method_calls`. `call()` can also be used with `assert_has_calls()`.

```
>>> m = MagicMock(return_value=None)
>>> m(1,2,a='foo',b='bar')
>>> m()
>>> m.call_args_list == [call(1,2,a='foo',b='bar'), call()]
True
```

`call.call_list()`

For a `call` object that represents multiple calls, `call_list()` returns a list of all the intermediate calls as well as the final call.

`call_list` is particularly useful for making assertions on “chained calls”. A chained call is multiple calls on a single line of code. This results in multiple entries in `mock_calls` on a mock. Manually constructing the sequence of calls can be tedious.

`call_list()` can construct the sequence of calls from the same chained call:

```
>>> m = MagicMock()
>>> m(1).method(arg='foo').other('bar')(2.0)
<MagicMock name='mock().method().other()' id='...'>
>>> kall = call(1).method(arg='foo').other('bar')(2.0)
>>> kall.call_list()
[call(1),
 call().method(arg='foo'),
 call().method().other('bar'),
 call().method().other()(2.0)]
>>> m.mock_calls == kall.call_list()
True
```

A `call` object is either a tuple of (positional args, keyword args) or (name, positional args, keyword args) depending on how it was constructed. When you construct them yourself this isn't particularly interesting, but the `call` objects that are in the `Mock.call_args`, `Mock.call_args_list` and `Mock.mock_calls` attributes can be introspected to get at the individual arguments they contain.

The `call` objects in `Mock.call_args` and `Mock.call_args_list` are two-tuples of (positional args, keyword args) whereas the `call` objects in `Mock.mock_calls`, along with ones you construct yourself, are three-tuples of (name, positional args, keyword args).

You can use their “tupleness” to pull out the individual arguments for more complex introspection and assertions. The positional arguments are a tuple (an empty tuple if there are no positional arguments) and the keyword arguments are a dictionary:

```
>>> m = MagicMock(return_value=None)
```

```

>>> m(1, 2, 3, arg='one', arg2='two')
>>> kall = m.call_args
>>> kall.args
(1, 2, 3)
>>> kall.kwargs
{'arg': 'one', 'arg2': 'two'}
>>> kall.args is kall[0]
True
>>> kall.kwargs is kall[1]
True

>>> m = MagicMock()
>>> m.foo(4, 5, 6, arg='two', arg2='three')
<MagicMock name='mock.foo()' id='...'>
>>> kall = m.mock_calls[0]
>>> name, args, kwargs = kall
>>> name
'foo'
>>> args
(4, 5, 6)
>>> kwargs
{'arg': 'two', 'arg2': 'three'}
>>> name is m.mock_calls[0][0]
True

```

create_autospec

unittest.mock.create_autospec(spec, spec_set=False, instance=False, **kwargs)

Create a mock object using another object as a spec. Attributes on the mock will use the corresponding attribute on the spec object as their spec. Functions or methods being mocked will have their arguments checked to ensure that they are called with the correct signature. If `spec_set` is `True` then attempting to set attributes that don't exist on the spec object will raise an `AttributeError`. If a class is used as a spec then the return value of the mock (the instance of the class) will have the same spec.

See [Autospeccing](#) for examples of how to use auto-speccing with `create_autospec()` and the `autospec` argument to `patch()`.

Changed in version 3.8: `create_autospec()` now returns an `AsyncMock` if the target is an async function.

ANY

unittest.mock.ANY

Sometimes you may need to make assertions about some of the arguments in a call to mock, but either not care about some of the arguments or want to pull them individually out of `call_args` and make more complex assertions on them.

To ignore certain arguments you can pass in objects that compare equal to everything. Calls to `assert_called_with()` and `assert_called_once_with()` will then succeed no matter what was passed in.

```

>>> mock = Mock(return_value=None)
>>> mock('foo', bar=object())
>>> mock.assert_called_once_with('foo', bar=ANY)

```

`ANY` can also be used in comparisons with call lists like `mock_calls`:

```

>>> m = MagicMock(return_value=None)
>>> m(1)
>>> m(1, 2)
>>> m(object())
>>> m.mock_calls == [call(1), call(1, 2), ANY]
True

```

`ANY` is not limited to comparisons with call objects and so can also be used in test assertions:

```
class TestStringMethods(unittest.TestCase):

    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', ANY])
```

FILTER_DIR

unittest.mock.FILTER_DIR

`FILTER_DIR` is a module level variable that controls the way mock objects respond to `dir()`. The default is `True`, which uses the filtering described below, to only show useful members. If you dislike this filtering, or need to switch it off for diagnostic purposes, then set `mock.FILTER_DIR=False`.

With filtering on, `dir(some_mock)` shows only useful attributes and will include any dynamically created attributes that wouldn't normally be shown. If the mock was created with `aspec` (or `autospec` of course) then all the attributes from the original are shown, even if they haven't been accessed yet:

```
>>> dir(Mock())
['assert_any_call',
 'assert_called',
 'assert_called_once',
 'assert_called_once_with',
 'assert_called_with',
 'assert_has_calls',
 'assert_not_called',
 'attach_mock',
 ...
>>> from urllib import request
>>> dir(Mock(spec=request))
['AbstractBasicAuthHandler',
 'AbstractDigestAuthHandler',
 'AbstractHTTPHandler',
 'BaseHandler',
 ...]
```

Many of the not-very-useful (private to `Mock` rather than the thing being mocked) underscore and double underscore prefixed attributes have been filtered from the result of calling `dir()` on a `Mock`. If you dislike this behaviour you can switch it off by setting the module level switch `FILTER_DIR`:

```
>>> from unittest import mock
>>> mock.FILTER_DIR = False
>>> dir(mock.Mock())
['_NonCallableMock__get_return_value',
 '_NonCallableMock__get_side_effect',
 '_NonCallableMock__return_value_doc',
 '_NonCallableMock__set_return_value',
 '_NonCallableMock__set_side_effect',
 '__call__',
 '__class__',
 ...]
```

Alternatively you can just use `vars(my_mock)` (instance members) and `dir(type(my_mock))` (type members) to bypass the filtering irrespective of `FILTER_DIR`.

mock_open

unittest.mock.mock_open(mock=None, read_data=None)

A helper function to create a mock to replace the use of `open()`. It works for `open()` called directly or used as a context manager. The `mock` argument is the mock object to configure. If `None` (the default) then a `MagicMock` will be created for you, with the API limited to methods or attributes available on standard file handles. `read_data` is a string for the `read()`, `readline()`, and `readlines()` methods of the file handle to return. Calls to those methods will

take data from `read_data` until it is depleted. The mock

Using `open()` as a context manager is a great way to ensure your file handles are closed properly and is becoming common:

```
with open('/some/path', 'w') as f:
    f.write('something')
```

The issue is that even if you mock out the call to `open()` it is the returned object that is used as a context manager (and has `__enter__()` and `__exit__()` called).

Mocking context managers with `aMagicMock` is common enough and fiddly enough that a helper function is useful.

```
>>> m = mock_open()
>>> with patch('__main__.open', m):
...     with open('foo', 'w') as h:
...         h.write('some stuff')
...
>>> m.mock_calls
[call('foo', 'w'),
 call().__enter__(),
 call().write('some stuff'),
 call().__exit__(None, None, None)]
>>> m.assert_called_once_with('foo', 'w')
>>> handle = m()
>>> handle.write.assert_called_once_with('some stuff')
```

And for reading files:

```
>>> with patch('__main__.open', mock_open(read_data='bibble')) as m:
...     with open('foo') as h:
...         result = h.read()
...
>>> m.assert_called_once_with('foo')
>>> assert result == 'bibble'
```

Autospeccing

Autospeccing is based on the existing `spec` feature of `mock`. It limits the api of mocks to the api of an original object (the spec), but it is recursive (implemented lazily) so that attributes of mocks only have the same api as the attributes of the spec. In addition mocked functions / methods have the same call signature as the original so they raise a `TypeError` if they are called incorrectly.

Before I explain how auto-speccing works, here's why it is needed.

`Mock` is a very powerful and flexible object, but it suffers from a flaw which is general to mocking. If you refactor some of your code, rename members and so on, any tests for code that is still using the old api but uses mocks instead of the real objects will still pass. This means your tests can all pass even though your code is broken.

Changed in version 3.5: Before 3.5, tests with a typo in the word `assert` would silently pass when they should raise an error. You can still achieve this behavior by passing `unsafe=True` to `Mock`.

Note that this is another reason why you need integration tests as well as unit tests. Testing everything in isolation is all fine and dandy, but if you don't test how your units are "wired together" there is still lots of room for bugs that tests might have caught.

`unittest.mock` already provides a feature to help with this, called `spec`. If you use a class or instance as the spec for a mock then you can only access attributes on the mock that exist on the real class:

```
>>> from urllib import request
>>> mock = Mock(spec=request.Request)
>>> mock.assert_called_with # Intentional typo!
Traceback (most recent call last):
```



```
...
AttributeError: Mock object has no attribute 'assret_called_with'
```

The spec only applies to the mock itself, so we still have the same issue with any methods on the mock:

```
>>> mock.header_items()
<mock.Mock object at 0x...>
>>> mock.header_items.assret_called_with() # Intentional typo!
```

Auto-spec'ing solves this problem. You can either pass `autospec=True` to `patch()`/`patch.object()` or use the `create_autospec()` function to create a mock with a spec. If you use the `autospec=True` argument to `patch()` then the object that is being replaced will be used as the spec object. Because the spec'ing is done “lazily” (the spec is created as attributes on the mock are accessed) you can use it with very complex or deeply nested objects (like modules that import modules that import modules) without a big performance hit.

Here's an example of it in use:

```
>>> from urllib import request
>>> patcher = patch('__main__.request', autospec=True)
>>> mock_request = patcher.start()
>>> request is mock_request
True
>>> mock_request.Request
<MagicMock name='request.Request' spec='Request' id='... '>
```

You can see that `request.Request` has a spec. `request.Request` takes two arguments in the constructor (one of which is self). Here's what happens if we try to call it incorrectly:

```
>>> req = request.Request()
Traceback (most recent call last):
...
TypeError: <lambda>() takes at least 2 arguments (1 given)
```

The spec also applies to instantiated classes (i.e. the return value of specced mocks):

```
>>> req = request.Request('foo')
>>> req
<NonCallableMagicMock name='request.Request()' spec='Request' id='... '>
```

`Request` objects are not callable, so the return value of instantiating our mocked out `request.Request` is a non-callable mock. With the spec in place any typos in our asserts will raise the correct error:

```
>>> req.add_header('spam', 'eggs')
<MagicMock name='request.Request().add_header()' id='... '>
>>> req.add_header.assret_called_with # Intentional typo!
Traceback (most recent call last):
...
AttributeError: Mock object has no attribute 'assret_called_with'
>>> req.add_header.assert_called_with('spam', 'eggs')
```

In many cases you will just be able to add `autospec=True` to your existing `patch()` calls and then be protected against bugs due to typos and api changes.

As well as using `autospec` through `patch()` there is a `create_autospec()` for creating autospecced mocks directly:

```
>>> from urllib import request
>>> mock_request = create_autospec(request)
>>> mock_request.Request('foo', 'bar')
<NonCallableMagicMock name='mock.Request()' spec='Request' id='... '>
```

This isn't without caveats and limitations however, which is why it is not the default behaviour. In order to know what attributes are available on the spec object, `autospec` has to introspect (access attributes) the spec. As you traverse attributes on the mock a corresponding traversal of the original object is happening under the hood. If any of your specced objects have properties or descriptors that can trigger code execution then you may not be able to use `autospec`. On the other hand it is much better to design your objects so that introspection is safe[4].

A more serious problem is that it is common for instance attributes to be created in the `__init__()` method and not to exist on the class at all. `autospec` can't know about any dynamically created attributes and restricts the api to visible attributes.

```
>>> class Something:
...     def __init__(self):
...         self.a = 33
...
>>> with patch('__main__.Something', autospec=True):
...     thing = Something()
...     thing.a
...
Traceback (most recent call last):
...
AttributeError: Mock object has no attribute 'a'
```

There are a few different ways of resolving this problem. The easiest, but not necessarily the least annoying, way is to simply set the required attributes on the mock after creation. Just because `autospec` doesn't allow you to fetch attributes that don't exist on the spec it doesn't prevent you setting them:

```
>>> with patch('__main__.Something', autospec=True):
...     thing = Something()
...     thing.a = 33
...
```

There is a more aggressive version of both `spec` and `autospec` that does prevent you setting non-existent attributes. This is useful if you want to ensure your code only sets valid attributes too, but obviously it prevents this particular scenario:

```
>>> with patch('__main__.Something', autospec=True, spec_set=True):
...     thing = Something()
...     thing.a = 33
...
Traceback (most recent call last):
...
AttributeError: Mock object has no attribute 'a'
```

Probably the best way of solving the problem is to add class attributes as default values for instance members initialised in `__init__()`. Note that if you are only setting default attributes in `__init__()` then providing them via class attributes (shared between instances of course) is faster too. e.g.

```
class Something:
    a = 33
```

This brings up another issue. It is relatively common to provide a default value of `None` for members that will later be an object of a different type. `None` would be useless as a spec because it wouldn't let you access any attributes or methods on it. As `None` is never going to be useful as a spec, and probably indicates a member that will normally be of some other type, `autospec` doesn't use a spec for members that are set to `None`. These will just be ordinary mocks (well - `MagicMocks`):

```
>>> class Something:
...     member = None
...
>>> mock = create_autospec(Something)
>>> mock.member.foo.bar.baz()
<MagicMock name='mock.member.foo.bar.baz()' id='...'>
```

If modifying your production classes to add defaults isn't to your liking then there are more options. One of these is simply to use an instance as the spec rather than the class. The other is to create a subclass of the production class and add the defaults to the subclass without affecting the production class. Both of these require you to use an alternative object as the spec. Thankfully `patch()` supports this - you can simply pass the alternative object as the `autospec` argument:

```
>>> class Something:
...     def __init__(self):
...         self.a = 33
...
>>> class SomethingForTest(Something):
...     a = 33
...
>>> p = patch('__main__.Something', autospec=SomethingForTest)
>>> mock = p.start()
>>> mock.a
<NonCallableMagicMock name='Something.a' spec='int' id='...'>
```

This only applies to classes or already instantiated objects. Calling a mocked class to create a mock instance does not create a real instance. It is only attribute lookups - along with calls to `dir()` - that are done.

Sealing mocks

`unittest.mock.seal(mock)`

Seal will disable the automatic creation of mocks when accessing an attribute of the mock being sealed or any of its attributes that are already mocks recursively. If a mock instance with a name or a spec is assigned to an attribute it won't be considered in the sealing chain. This allows one to prevent seal from fixing part of the mock object. `>>> mock = Mock() >>> mock.submock.attribute1 = 2 >>> mock.not_submock = mock.Mock(name="sample_name") >>> seal(mock) >>> mock.new_attribute` # This will raise `AttributeError`

Order of precedence of `side_effect`, `return_value` and `wraps`

The order of their precedence is:

- `side_effect`
- `return_value`
- `wraps`

If all three are set, mock will return the value from `side_effect`, ignoring `return_value` and the wrapped object altogether. If any two are set, the one with the higher precedence will return the value. Regardless of the order of which was set first, the order of precedence remains unchanged.

```
>>> from unittest.mock import Mock
>>> class Order:
...     @staticmethod
...     def get_value():
...         return "third"
...
>>> order_mock = Mock(spec=Order, wraps=Order)
>>> order_mock.get_value.side_effect = ["first"]
>>> order_mock.get_value.return_value = "second"
>>> order_mock.get_value()
'first'
```

As `None` is the default value of `side_effect`, if you reassign its value back to `None`, the order of precedence will be checked between `return_value` and the wrapped object, ignoring `side_effect`.

```
>>> order_mock.get_value.side_effect = None
>>> order_mock.get_value()
'second'
```

If the value being returned by `side_effect` is `DEFAULT`, it is ignored and the order of precedence moves to the successor to obtain the value to return.

```
>>> from unittest.mock import DEFAULT
>>> order_mock.get_value.side_effect = [DEFAULT]
```

```
>>> order_mock.get_value()
'second'
```

When Mock wraps an object, the default value of return_value will be DEFAULT.

```
>>> order_mock = Mock(spec=Order, wraps=Order)
>>> order_mock.return_value
sentinel.DEFAULT
>>> order_mock.get_value.return_value
sentinel.DEFAULT
```

The order of precedence will ignore this value and it will move to the last successor which is the wrapped object.

As the real call is being made to the wrapped object, creating an instance of this mock will return the real instance of the class. The positional arguments, if any, required by the wrapped object must be passed.

```
>>> order_mock_instance = order_mock()
>>> isinstance(order_mock_instance, Order)
True
>>> order_mock_instance.get_value()
'third'

>>> order_mock.get_value.return_value = DEFAULT
>>> order_mock.get_value()
'third'

>>> order_mock.get_value.return_value = "second"
>>> order_mock.get_value()
'second'
```

But if you assign None to it, this will not be ignored as it is an explicit assignment. So, the order of precedence will not move to the wrapped object.

```
>>> order_mock.get_value.return_value = None
>>> order_mock.get_value() is None
True
```

Even if you set all three at once when initializing the mock, the order of precedence remains the same:

```
>>> order_mock = Mock(spec=Order, wraps=Order,
...                   **{"get_value.side_effect": ["first"],
...                   "get_value.return_value": "second"})
...
>>> order_mock.get_value()
'first'
>>> order_mock.get_value.side_effect = None
>>> order_mock.get_value()
'second'
>>> order_mock.get_value.return_value = DEFAULT
>>> order_mock.get_value()
'third'
```

If side_effect is exhausted, the order of precedence will not cause a value to be obtained from the successors. Instead, StopIteration exception is raised.

```
>>> order_mock = Mock(spec=Order, wraps=Order)
>>> order_mock.get_value.side_effect = ["first side effect value",
...                                   "another side effect value"]
>>> order_mock.get_value.return_value = "second"

>>> order_mock.get_value()
'first side effect value'
>>> order_mock.get_value()
'another side effect value'

>>> order_mock.get_value()
```

```
Traceback (most recent call last):
...
StopIteration
```

unittest.mock— getting started

Added in version 3.3.

Using Mock

Mock patching methods

Common uses for `Mock` objects include:

- Patching methods
- Recording method calls on objects

You might want to replace a method on an object to check that it is called with the correct arguments by another part of the system:

```
>>> real = SomeClass()
>>> real.method = MagicMock(name='method')
>>> real.method(3, 4, 5, key='value')
<MagicMock name='method()' id='...'>
```

Once our mock has been used (`real.method` in this example) it has methods and attributes that allow you to make assertions about how it has been used.

■ **Nota:** In most of these examples the `Mock` and `MagicMock` classes are interchangeable. As the `MagicMock` is the more capable class it makes a sensible one to use by default.

Once the mock has been called its `called` attribute is set to `True`. More importantly we can use the `assert_called_with()` or `assert_called_once_with()` method to check that it was called with the correct arguments.

This example tests that calling `ProductionClass().method` results in a call to `something` method:

```
>>> class ProductionClass:
...     def method(self):
...         self.something(1, 2, 3)
...     def something(self, a, b, c):
...         pass
...
>>> real = ProductionClass()
>>> real.something = MagicMock()
>>> real.method()
>>> real.something.assert_called_once_with(1, 2, 3)
```

Mock for method calls on an object

In the last example we patched a method directly on an object to check that it was called correctly. Another common use case is to pass an object into a method (or some part of the system under test) and then check that it is used in the correct way.

The simple `ProductionClass` below has a `closer` method. If it is called with an object then it calls `close` on it.

```
>>> class ProductionClass:
...     def closer(self, something):
...         something.close()
...
>>>
```

So to test it we need to pass in an object with a `close` method and check that it was called correctly.

```
>>> real = ProductionClass()
>>> mock = Mock()
```

```
>>> real.closer(mock)
>>> mock.close.assert_called_with()
```

We don't have to do any work to provide the 'close' method on our mock. Accessing close creates it. So, if 'close' hasn't already been called then accessing it in the test will create it, but `assert_called_with()` will raise a failure exception.

Mocking classes

A common use case is to mock out classes instantiated by your code under test. When you patch a class, then that class is replaced with a mock. Instances are created by calling the class. This means you access the "mock instance" by looking at the return value of the mocked class.

In the example below we have a function `some_function` that instantiates `Foo` and calls a method on it. The call to `patch()` replaces the class `Foo` with a mock. The `Foo` instance is the result of calling the mock, so it is configured by modifying the mock `return_value`.

```
>>> def some_function():
...     instance = module.Foo()
...     return instance.method()
...
>>> with patch('module.Foo') as mock:
...     instance = mock.return_value
...     instance.method.return_value = 'the result'
...     result = some_function()
...     assert result == 'the result'
```

Naming your mocks

It can be useful to give your mocks a name. The name is shown in the repr of the mock and can be helpful when the mock appears in test failure messages. The name is also propagated to attributes or methods of the mock:

```
>>> mock = MagicMock(name='foo')
>>> mock
<MagicMock name='foo' id='...'>
>>> mock.method
<MagicMock name='foo.method' id='...'>
```

Tracking all calls

Often you want to track more than a single call to a method. The `mock_calls` attribute records all calls to child attributes of the mock - and also to their children.

```
>>> mock = MagicMock()
>>> mock.method()
<MagicMock name='mock.method()' id='...'>
>>> mock.attribute.method(10, x=53)
<MagicMock name='mock.attribute.method()' id='...'>
>>> mock.mock_calls
[call.method(), call.attribute.method(10, x=53)]
```

If you make an assertion about `mock_calls` and any unexpected methods have been called, then the assertion will fail. This is useful because as well as asserting that the calls you expected have been made, you are also checking that they were made in the right order and with no additional calls:

You use the `call` object to construct lists for comparing with `mock_calls`:

```
>>> expected = [call.method(), call.attribute.method(10, x=53)]
>>> mock.mock_calls == expected
True
```

However, parameters to calls that return mocks are not recorded, which means it is not possible to track nested calls where the parameters used to create ancestors are important:

```
>>> m = Mock()
>>> m.factory(important=True).deliver()
<Mock name='mock.factory().deliver()' id='...'>
>>> m.mock_calls[-1] == call.factory(important=False).deliver()
True
```

Setting return values and attributes

Setting the return values on a mock object is trivially easy:

```
>>> mock = Mock()
>>> mock.return_value = 3
>>> mock()
3
```

Of course you can do the same for methods on the mock:

```
>>> mock = Mock()
>>> mock.method.return_value = 3
>>> mock.method()
3
```

The return value can also be set in the constructor:

```
>>> mock = Mock(return_value=3)
>>> mock()
3
```

If you need an attribute setting on your mock, just do it:

```
>>> mock = Mock()
>>> mock.x = 3
>>> mock.x
3
```

Sometimes you want to mock up a more complex situation, like for `examplemock.connection.cursor().execute("SELECT 1")`. If we wanted this call to return a list, then we have to configure the result of the nested call.

We can use `call` to construct the set of calls in a “chained call” like this for easy assertion afterwards:

```
>>> mock = Mock()
>>> cursor = mock.connection.cursor.return_value
>>> cursor.execute.return_value = ['foo']
>>> mock.connection.cursor().execute("SELECT 1")
['foo']
>>> expected = call.connection.cursor().execute("SELECT 1").call_list()
>>> mock.mock_calls
[call.connection.cursor(), call.connection.cursor().execute('SELECT 1')]
>>> mock.mock_calls == expected
True
```

It is the call to `call_list()` that turns our call object into a list of calls representing the chained calls.

Raising exceptions with mocks

A useful attribute is `side_effect`. If you set this to an exception class or instance then the exception will be raised when the mock is called.

```
>>> mock = Mock(side_effect=Exception('Boom!'))
>>> mock()
Traceback (most recent call last):
...
Exception: Boom!
```

Side effect functions and iterables

`side_effect` can also be set to a function or an iterable. The use case for `side_effect` as an iterable is where your mock is going to be called several times, and you want each call to return a different value. When you set `side_effect` to an iterable every call to the mock returns the next value from the iterable:

```
>>> mock = MagicMock(side_effect=[4, 5, 6])
>>> mock()
4
>>> mock()
5
>>> mock()
6
```

For more advanced use cases, like dynamically varying the return values depending on what the mock is called with, `side_effect` can be a function. The function will be called with the same arguments as the mock. Whatever the function returns is what the call returns:

```
>>> vals = {(1, 2): 1, (2, 3): 2}
>>> def side_effect(*args):
...     return vals[args]
...
>>> mock = MagicMock(side_effect=side_effect)
>>> mock(1, 2)
1
>>> mock(2, 3)
2
```

Mocking asynchronous iterators

Since Python 3.8, `AsyncMock` and `MagicMock` have support to mock asynchronous iterators through `__aiter__`. The `return_value` attribute of `__aiter__` can be used to set the return values to be used for iteration.

```
>>> mock = MagicMock() # AsyncMock also works here
>>> mock.__aiter__.return_value = [1, 2, 3]
>>> async def main():
...     return [i async for i in mock]
...
>>> asyncio.run(main())
[1, 2, 3]
```

Mocking asynchronous context manager

Since Python 3.8, `AsyncMock` and `MagicMock` have support to mock asynchronous Context Manager through `__aenter__` and `__aexit__`. By default, `__aenter__` and `__aexit__` are `AsyncMock` instances that return an async function.

```
>>> class AsyncContextManager:
...     async def __aenter__(self):
...         return self
...     async def __aexit__(self, exc_type, exc, tb):
...         pass
...
>>> mock_instance = MagicMock(AsyncContextManager()) # AsyncMock also works here
>>> async def main():
...     async with mock_instance as result:
...         pass
...
>>> asyncio.run(main())
>>> mock_instance.__aenter__.assert_awaited_once()
>>> mock_instance.__aexit__.assert_awaited_once()
```

Creating a mock from an existing object

One problem with over use of mocking is that it couples your tests to the implementation of your mocks rather than your real code. Suppose you have a class that implements `some_method`. In a test for another class, you

provide a mock of this object that also provides some_method. If later you refactor the first class, so that it no longer has some_method- then your tests will continue to pass even though your code is now broken!

Mock allows you to provide an object as a specification for the mock, using the spec keyword argument.

Accessing methods / attributes on the mock that don't exist on your specification object will immediately raise an attribute error. If you change the implementation of your specification, then tests that use that class will start failing immediately without you having to instantiate the class in those tests.

```
>>> mock = Mock(spec=SomeClass)
>>> mock.old_method()
Traceback (most recent call last):
...
AttributeError: Mock object has no attribute 'old_method'. Did you mean: 'class_method'?
```

Using a specification also enables a smarter matching of calls made to the mock, regardless of whether some parameters were passed as positional or named arguments:

```
>>> def f(a, b, c): pass
...
>>> mock = Mock(spec=f)
>>> mock(1, 2, 3)
<Mock name='mock()' id='140161580456576'>
>>> mock.assert_called_with(a=1, b=2, c=3)
```

If you want this smarter matching to also work with method calls on the mock, you can use auto-specing.

If you want a stronger form of specification that prevents the setting of arbitrary attributes as well as the getting of them then you can use spec_set instead of spec.

Using side_effect to return per file content

mock_open() is used to patch open() method. side_effect can be used to return a new Mock object per call. This can be used to return different contents per file stored in a dictionary:

```
DEFAULT = "default"
data_dict = {"file1": "data1",
             "file2": "data2"}

def open_side_effect(name):
    return mock_open(read_data=data_dict.get(name, DEFAULT))()

with patch("builtins.open", side_effect=open_side_effect):
    with open("file1") as file1:
        assert file1.read() == "data1"

    with open("file2") as file2:
        assert file2.read() == "data2"

    with open("file3") as file2:
        assert file2.read() == "default"
```

Patch decorators

■ Nota: NoteWithpatch() it matters that you patch objects in the namespace where they are looked up. This is normally straightforward, but for a quick guide read where to patch.

A common need in tests is to patch a class attribute or a module attribute, for example patching a builtin or patching a class in a module to test that it is instantiated. Modules and classes are effectively global, so patching on them has to be undone after the test or the patch will persist into other tests and cause hard to diagnose problems.

mock provides three convenient decorators for this: `patch()`, `patch.object()` and `patch.dict()`. `patch` takes a single string, of the form `package.module.Class.attribute` to specify the attribute you are patching. It also optionally takes a value that you want the attribute (or class or whatever) to be replaced with. `'patch.object'` takes an object and the name of the attribute you would like patched, plus optionally the value to patch it with.

`patch.object`:

```
>>> original = SomeClass.attribute
>>> @patch.object(SomeClass, 'attribute', sentinel.attribute)
... def test():
...     assert SomeClass.attribute == sentinel.attribute
...
>>> test()
>>> assert SomeClass.attribute == original

>>> @patch('package.module.attribute', sentinel.attribute)
... def test():
...     from package.module import attribute
...     assert attribute is sentinel.attribute
...
>>> test()
```

If you are patching a module (including builtins) then use `patch()` instead of `patch.object()`:

```
>>> mock = MagicMock(return_value=sentinel.file_handle)
>>> with patch('builtins.open', mock):
...     handle = open('filename', 'r')
...
>>> mock.assert_called_with('filename', 'r')
>>> assert handle == sentinel.file_handle, "incorrect file handle returned"
```

The module name can be 'dotted', in the form `package.module` if needed:

```
>>> @patch('package.module.ClassName.attribute', sentinel.attribute)
... def test():
...     from package.module import ClassName
...     assert ClassName.attribute == sentinel.attribute
...
>>> test()
```

A nice pattern is to actually decorate test methods themselves:

```
>>> class MyTest(unittest.TestCase):
...     @patch.object(SomeClass, 'attribute', sentinel.attribute)
...     def test_something(self):
...         self.assertEqual(SomeClass.attribute, sentinel.attribute)
...
>>> original = SomeClass.attribute
>>> MyTest('test_something').test_something()
>>> assert SomeClass.attribute == original
```

If you want to patch with a Mock, you can use `patch()` with only one argument (or `patch.object()` with two arguments). The mock will be created for you and passed into the test function / method:

```
>>> class MyTest(unittest.TestCase):
...     @patch.object(SomeClass, 'static_method')
...     def test_something(self, mock_method):
...         SomeClass.static_method()
...         mock_method.assert_called_with()
...
>>> MyTest('test_something').test_something()
```

You can stack up multiple patch decorators using this pattern:

```
>>> class MyTest(unittest.TestCase):
...     @patch('package.module.ClassName1')
```

```

...     @patch('package.module.ClassName2')
...     def test_something(self, MockClass2, MockClass1):
...         self.assertIs(package.module.ClassName1, MockClass1)
...         self.assertIs(package.module.ClassName2, MockClass2)
...
>>> MyTest('test_something').test_something()

```

When you nest patch decorators the mocks are passed in to the decorated function in the same order they applied (the normal Python order that decorators are applied). This means from the bottom up, so in the example above the mock for `test_module.ClassName2` is passed in first.

There is also `patch.dict()` for setting values in a dictionary just during a scope and restoring the dictionary to its original state when the test ends:

```

>>> foo = {'key': 'value'}
>>> original = foo.copy()
>>> with patch.dict(foo, {'newkey': 'newvalue'}, clear=True):
...     assert foo == {'newkey': 'newvalue'}
...
>>> assert foo == original

```

`patch`, `patch.object` and `patch.dict` can all be used as context managers.

Where you use `patch()` to create a mock for you, you can get a reference to the mock using the “as” form of the `with` statement:

```

>>> class ProductionClass:
...     def method(self):
...         pass
...
>>> with patch.object(ProductionClass, 'method') as mock_method:
...     mock_method.return_value = None
...     real = ProductionClass()
...     real.method(1, 2, 3)
...
>>> mock_method.assert_called_with(1, 2, 3)

```

As an alternative `patch`, `patch.object` and `patch.dict` can be used as class decorators. When used in this way it is the same as applying the decorator individually to every method whose name starts with “test”.

Further examples

Here are some more examples for some slightly more advanced scenarios.

Mocking chained calls

Mocking chained calls is actually straightforward with mock once you understand the `return_value` attribute. When a mock is called for the first time, or you fetch its `return_value` before it has been called, a new `Mock` is created.

This means that you can see how the object returned from a call to a mocked object has been used by interrogating the `return_value` mock:

```

>>> mock = Mock()
>>> mock().foo(a=2, b=3)
<Mock name='mock().foo()' id='...'>
>>> mock.return_value.foo.assert_called_with(a=2, b=3)

```

From here it is a simple step to configure and then make assertions about chained calls. Of course another alternative is writing your code in a more testable way in the first place...

So, suppose we have some code that looks a little bit like this:

```
>>> class Something:
...     def __init__(self):
...         self.backend = BackendProvider()
...     def method(self):
...         response = self.backend.get_endpoint('foobar').create_call('spam', 'eggs').start_call()
...         # more code
```

Assuming that `BackendProvider` is already well tested, how do we test `method()`? Specifically, we want to test that the code section `#morecode` uses the response object in the correct way.

As this chain of calls is made from an instance attribute we can monkey patch the `backend` attribute on a `Something` instance. In this particular case we are only interested in the return value from the final call `start_call` so we don't have much configuration to do. Let's assume the object it returns is 'file-like', so we'll ensure that our response object uses the builtin `open()` as its spec.

To do this we create a mock instance as our mock backend and create a mock response object for it. To set the response as the return value for that final `start_call` we could do this:

```
mock_backend.get_endpoint.return_value.create_call.return_value.start_call.return_value = mock_response
```

We can do that in a slightly nicer way using the `configure_mock()` method to directly set the return value for us:

```
>>> something = Something()
>>> mock_response = Mock(spec=open)
>>> mock_backend = Mock()
>>> config = {'get_endpoint.return_value.create_call.return_value.start_call.return_value': mock_response}
>>> mock_backend.configure_mock(**config)
```

With these we monkey patch the "mock backend" in place and can make the real call:

```
>>> something.backend = mock_backend
>>> something.method()
```

Using `mock_calls` we can check the chained call with a single assert. A chained call is several calls in one line of code, so there will be several entries in `mock_calls`. We can use `call.call_list()` to create this list of calls for us:

```
>>> chained = call.get_endpoint('foobar').create_call('spam', 'eggs').start_call()
>>> call_list = chained.call_list()
>>> assert mock_backend.mock_calls == call_list
```

Partial mocking

For some tests, you may want to mock out a call to `datetime.date.today()` to return a known date, but don't want to prevent the code under test from creating new date objects. Unfortunately `datetime.date` is written in C, so you cannot just monkey-patch out the static `datetime.date.today()` method.

Instead, you can effectively wrap the date class with a mock, while passing through calls to the constructor to the real class (and returning real instances).

The `patch_decorator` is used here to mock out the `date` class in the module under test. The `side_effect` attribute on the mock date class is then set to a lambda function that returns a real date. When the mock date class is called a real date will be constructed and returned by `side_effect`.

```
>>> import datetime as dt
>>> with patch('mymodule.date') as mock_date:
...     mock_date.today.return_value = dt.date(2010, 10, 8)
...     mock_date.side_effect = lambda *args, **kw: dt.date(*args, **kw)
...
...     assert mymodule.date.today() == dt.date(2010, 10, 8)
...     assert mymodule.date(2009, 6, 8) == dt.date(2009, 6, 8)
```

Note that we don't patch `datetime.date` globally, we patch `date` in the module that uses it. See where to patch.

When `date.today()` is called a known date is returned, but calls to the `date(...)` constructor still return normal dates. Without this you can find yourself having to calculate an expected result using exactly the same algorithm as the

code under test, which is a classic testing anti-pattern.

Calls to the date constructor are recorded in `themock_dateattributes` (`call_count` and friends) which may also be useful for your tests.

An alternative way of dealing with mocking dates, or other builtin classes, is discussed in [this blog entry](#).

Mocking a generator method

A Python generator is a function or method that uses the `yield` statement to return a series of values when iterated over[1].

A generator method / function is called to return the generator object. It is the generator object that is then iterated over. The protocol method for iteration is `__iter__()`, so we can mock this using a `MagicMock`.

Here's an example class with an "iter" method implemented as a generator:

```
>>> class Foo:
...     def iter(self):
...         for i in [1, 2, 3]:
...             yield i
...
>>> foo = Foo()
>>> list(foo.iter())
[1, 2, 3]
```

How would we mock this class, and in particular its "iter" method?

To configure the values returned from the iteration (implicit in the call to `list`), we need to configure the object returned by the call to `foo.iter()`.

```
>>> mock_foo = MagicMock()
>>> mock_foo.iter.return_value = iter([1, 2, 3])
>>> list(mock_foo.iter())
[1, 2, 3]
```

There are also generator expressions and more advanced uses of generators, but we aren't concerned about them here. A very good introduction to generators and how powerful they are is: [Generator Tricks for Systems Programmers](#).

Applying the same patch to every test method

If you want several patches in place for multiple test methods the obvious way is to apply the patch decorators to every method. This can feel like unnecessary repetition. Instead, you can use `patch()` (in all its various forms) as a class decorator. This applies the patches to all test methods on the class. A test method is identified by methods whose names start with `test`:

```
>>> @patch('mymodule.SomeClass')
... class MyTest(unittest.TestCase):
...
...     def test_one(self, MockSomeClass):
...         self.assertIs(mymodule.SomeClass, MockSomeClass)
...
...     def test_two(self, MockSomeClass):
...         self.assertIs(mymodule.SomeClass, MockSomeClass)
...
...     def not_a_test(self):
...         return 'something'
...
>>> MyTest('test_one').test_one()
>>> MyTest('test_two').test_two()
>>> MyTest('test_two').not_a_test()
'something'
```

An alternative way of managing patches is to use the `patch` methods: `start` and `stop`. These allow you to move the patching into your `setUp` and `tearDown` methods.

```
>>> class MyTest(unittest.TestCase):
...     def setUp(self):
...         self.patcher = patch('mymodule.foo')
...         self.mock_foo = self.patcher.start()
...
...     def test_foo(self):
...         self.assertIs(mymodule.foo, self.mock_foo)
...
...     def tearDown(self):
...         self.patcher.stop()
...
>>> MyTest('test_foo').run()
```

If you use this technique you must ensure that the patching is “undone” by calling `stop`. This can be fiddlier than you might think, because if an exception is raised in the `setUp` then `tearDown` is not called. `unittest.TestCase.addCleanup()` makes this easier:

```
>>> class MyTest(unittest.TestCase):
...     def setUp(self):
...         patcher = patch('mymodule.foo')
...         self.addCleanup(patcher.stop)
...         self.mock_foo = patcher.start()
...
...     def test_foo(self):
...         self.assertIs(mymodule.foo, self.mock_foo)
...
>>> MyTest('test_foo').run()
```

Mocking unbound methods

Sometimes a test needs to patch an unbound method, which means patching the method on the class rather than on the instance. In order to make assertions about which objects were calling this particular method, you need to pass `self` as the first argument. The issue is that you can't patch with a mock for this, because if you replace an unbound method with a mock it doesn't become a bound method when fetched from the instance, and so it doesn't get `self` passed in. The workaround is to patch the unbound method with a real function instead. The `patch()` decorator makes it so simple to patch out methods with a mock that having to create a real function becomes a nuisance.

If you pass `autospec=True` to patch then it does the patching with a real function object. This function object has the same signature as the one it is replacing, but delegates to a mock under the hood. You still get your mock auto-created in exactly the same way as before. What it means though, is that if you use it to patch out an unbound method on a class the mocked function will be turned into a bound method if it is fetched from an instance. It will have `self` passed in as the first argument, which is exactly what was needed:

```
>>> class Foo:
...     def foo(self):
...         pass
...
>>> with patch.object(Foo, 'foo', autospec=True) as mock_foo:
...     mock_foo.return_value = 'foo'
...     foo = Foo()
...     foo.foo()
...
'foo'
>>> mock_foo.assert_called_once_with(foo)
```

If we don't use `autospec=True` then the unbound method is patched out with a `Mock` instance instead, and isn't called with `self`.

Checking multiple calls with mock

mock has a nice API for making assertions about how your mock objects are used.

```
>>> mock = Mock()
>>> mock.foo_bar.return_value = None
>>> mock.foo_bar('baz', spam='eggs')
>>> mock.foo_bar.assert_called_with('baz', spam='eggs')
```

If your mock is only being called once you can use the `assert_called_once_with()` method that also asserts that the `call_count` is one.

```
>>> mock.foo_bar.assert_called_once_with('baz', spam='eggs')
>>> mock.foo_bar()
>>> mock.foo_bar.assert_called_once_with('baz', spam='eggs')
Traceback (most recent call last):
...
AssertionError: Expected 'foo_bar' to be called once. Called 2 times.
Calls: [call('baz', spam='eggs'), call()].
```

Both `assert_called_with` and `assert_called_once_with` make assertions about the most recent call. If your mock is going to be called several times, and you want to make assertions about all those calls you can use `call_args_list`:

```
>>> mock = Mock(return_value=None)
>>> mock(1, 2, 3)
>>> mock(4, 5, 6)
>>> mock()
>>> mock.call_args_list
[call(1, 2, 3), call(4, 5, 6), call()]
```

The `call` helper makes it easy to make assertions about these calls. You can build up a list of expected calls and compare it to `call_args_list`. This looks remarkably similar to the repr of `call_args_list`:

```
>>> expected = [call(1, 2, 3), call(4, 5, 6), call()]
>>> mock.call_args_list == expected
True
```

Coping with mutable arguments

Another situation is rare, but can bite you, is when your mock is called with mutable arguments. `call_args` and `call_args_list` store references to the arguments. If the arguments are mutated by the code under test then you can no longer make assertions about what the values were when the mock was called.

Here's some example code that shows the problem. Imagine the following functions defined in 'mymodule':

```
def frob(val):
    pass

def grob(val):
    "First frob and then clear val"
    frob(val)
    val.clear()
```

When we try to test that `grob` calls `frob` with the correct argument look what happens:

```
>>> with patch('mymodule.frob') as mock_frob:
...     val = {6}
...     mymodule.grob(val)
...
>>> val
set()
>>> mock_frob.assert_called_with({6})
Traceback (most recent call last):
...
AssertionError: Expected: ({6},), {}
Called with: ({set(),}, {})
```


One possibility would be for mock to copy the arguments you pass in. This could then cause problems if you do assertions that rely on object identity for equality.

Here's one solution that uses the `side_effect` functionality. If you provide a `side_effect` function for a mock then `side_effect` will be called with the same args as the mock. This gives us an opportunity to copy the arguments and store them for later assertions. In this example I'm using another mock to store the arguments so that I can use the mock methods for doing the assertion. Again a helper function sets this up for me.

```
>>> from copy import deepcopy
>>> from unittest.mock import Mock, patch, DEFAULT
>>> def copy_call_args(mock):
...     new_mock = Mock()
...     def side_effect(*args, **kwargs):
...         args = deepcopy(args)
...         kwargs = deepcopy(kwargs)
...         new_mock(*args, **kwargs)
...         return DEFAULT
...     mock.side_effect = side_effect
...     return new_mock
...
>>> with patch('mymodule.frob') as mock_frob:
...     new_mock = copy_call_args(mock_frob)
...     val = {6}
...     mymodule.frob(val)
...
>>> new_mock.assert_called_with({6})
>>> new_mock.call_args
call({6})
```

`copy_call_args` is called with the mock that will be called. It returns a new mock that we do the assertion on. The `side_effect` function makes a copy of the args and calls our `new_mock` with the copy.

■ **Nota:** Note! If your mock is only going to be used once there is an easier way of checking arguments at the point they are called. You can simply do the checking inside `side_effect` function. `>>> def side_effect(arg): ... assert arg == {6} ... >>> mock = Mock(side_effect=side_effect) >>> mock({6}) >>> mock(set()) Traceback (most recent call last): ... AssertionError`

An alternative approach is to create a subclass of `Mock` or `MagicMock` that copies (using `copy.deepcopy()`) the arguments. Here's an example implementation:

```
>>> from copy import deepcopy
>>> class CopyingMock(MagicMock):
...     def __call__(self, /, *args, **kwargs):
...         args = deepcopy(args)
...         kwargs = deepcopy(kwargs)
...         return super().__call__(*args, **kwargs)
...
>>> c = CopyingMock(return_value=None)
>>> arg = set()
>>> c(arg)
>>> arg.add(1)
>>> c.assert_called_with(set())
>>> c.assert_called_with(arg)
Traceback (most recent call last):
...
AssertionError: expected call not found.
Expected: mock({1})
Actual: mock(set())
>>> c.foo
<CopyingMock name='mock.foo' id='...'>
```

When you subclass `Mock` or `MagicMock` all dynamically created attributes, and the `return_value` will use your subclass automatically. That means all children of a `CopyingMock` will also have the type `CopyingMock`.

Nesting patches

Using patch as a context manager is nice, but if you do multiple patches you can end up with nested with statements indenting further and further to the right:

```
>>> class MyTest(unittest.TestCase):
...     def test_foo(self):
...         with patch('mymodule.Foo') as mock_foo:
...             with patch('mymodule.Bar') as mock_bar:
...                 with patch('mymodule.Spam') as mock_spam:
...                     assert mymodule.Foo is mock_foo
...                     assert mymodule.Bar is mock_bar
...                     assert mymodule.Spam is mock_spam
...
>>> original = mymodule.Foo
>>> MyTest('test_foo').test_foo()
>>> assert mymodule.Foo is original
```

With unittestcleanupfunctions and thepatch methods: start and stopwe can achieve the same effect without the nested indentation. A simple helper method,create_patch, puts the patch in place and returns the created mock for us:

```
>>> class MyTest(unittest.TestCase):
...     def create_patch(self, name):
...         patcher = patch(name)
...         thing = patcher.start()
...         self.addCleanup(patcher.stop)
...         return thing
...
...     def test_foo(self):
...         mock_foo = self.create_patch('mymodule.Foo')
...         mock_bar = self.create_patch('mymodule.Bar')
...         mock_spam = self.create_patch('mymodule.Spam')
...
...         assert mymodule.Foo is mock_foo
...         assert mymodule.Bar is mock_bar
...         assert mymodule.Spam is mock_spam
...
>>> original = mymodule.Foo
>>> MyTest('test_foo').run()
>>> assert mymodule.Foo is original
```

Mocking a dictionary with MagicMock

You may want to mock a dictionary, or other container object, recording all access to it whilst having it still behave like a dictionary.

We can do this withMagicMock, which will behave like a dictionary, and usingside_effectto delegate dictionary access to a real underlying dictionary that is under our control.

When the__getitem__()and__setitem__()methods of ourMagicMockare called (normal dictionary access) thenside_effectis called with the key (and in the case of__setitem__the value too). We can also control what is returned.

After theMagicMockhas been used we can use attributes likecall_args_listto assert about how the dictionary was used:

```
>>> my_dict = {'a': 1, 'b': 2, 'c': 3}
>>> def getitem(name):
...     return my_dict[name]
...
>>> def setitem(name, val):
```

```

...     my_dict[name] = val
...
>>> mock = MagicMock()
>>> mock.__getitem__.side_effect = getitem
>>> mock.__setitem__.side_effect = setitem

```

■ Nota: NoteAn alternative to using `MagicMock` is to use `Mock` and only provide the magic methods you specifically want: `>>> mock = Mock() >>> mock.__getitem__ = Mock(side_effect=getitem) >>> mock.__setitem__ = Mock(side_effect=setitem)` A third option is to use `MagicMock` but passing in `dict` as the `spec` (or `spec_set`) argument so that the `MagicMock` created only has dictionary magic methods available: `>>> mock = MagicMock(spec=dict) >>> mock.__getitem__.side_effect=getitem >>> mock.__setitem__.side_effect=setitem`

With these side effect functions in place, the mock will behave like a normal dictionary but recording the access. It even raises a `KeyError` if you try to access a key that doesn't exist.

```

>>> mock['a']
1
>>> mock['c']
3
>>> mock['d']
Traceback (most recent call last):
...
KeyError: 'd'
>>> mock['b'] = 'fish'
>>> mock['d'] = 'eggs'
>>> mock['b']
'fish'
>>> mock['d']
'eggs'

```

After it has been used you can make assertions about the access using the normal mock methods and attributes:

```

>>> mock.__getitem__.call_args_list
[call('a'), call('c'), call('d'), call('b'), call('d')]
>>> mock.__setitem__.call_args_list
[call('b', 'fish'), call('d', 'eggs')]
>>> my_dict
{'a': 1, 'b': 'fish', 'c': 3, 'd': 'eggs'}

```

Mock subclasses and their attributes

There are various reasons why you might want to subclass `Mock`. One reason might be to add helper methods. Here's a silly example:

```

>>> class MyMock(MagicMock):
...     def has_been_called(self):
...         return self.called
...
>>> mymock = MyMock(return_value=None)
>>> mymock
<MyMock id='...'>
>>> mymock.has_been_called()
False
>>> mymock()
>>> mymock.has_been_called()
True

```

The standard behaviour for `Mock` instances is that attributes and the return value mocks are of the same type as the mock they are accessed on. This ensures that `Mock` attributes are `Mocks` and `MagicMock` attributes are `MagicMocks`[2]. So if you're subclassing to add helper methods then they'll also be available on the attributes and return value mock of instances of your subclass.

```

>>> mymock.foo
<MyMock name='mock.foo' id='...'>

```

```
>>> mymock.foo.has_been_called()
False
>>> mymock.foo()
<MyMock name='mock.foo()' id='...'>
>>> mymock.foo.has_been_called()
True
```

Sometimes this is inconvenient. For example, one user is subclassing mock to create a Twisted adaptor. Having this applied to attributes too actually causes errors.

Mock (in all its flavours) uses a method called `_get_child_mock` to create these “sub-mocks” for attributes and return values. You can prevent your subclass being used for attributes by overriding this method. The signature is that it takes arbitrary keyword arguments (`**kwargs`) which are then passed onto the mock constructor:

```
>>> class Subclass(MagicMock):
...     def _get_child_mock(self, /, **kwargs):
...         return MagicMock(**kwargs)
...
>>> mymock = Subclass()
>>> mymock.foo
<MagicMock name='mock.foo' id='...'>
>>> assert isinstance(mymock, Subclass)
>>> assert not isinstance(mymock.foo, Subclass)
>>> assert not isinstance(mymock(), Subclass)
```

An exception to this rule are the non-callable mocks. Attributes use the callable variant because otherwise non-callable mocks couldn't have callable methods.

Mocking imports with patch.dict

One situation where mocking can be hard is where you have a local import inside a function. These are harder to mock because they aren't using an object from the module namespace that we can patch out.

Generally local imports are to be avoided. They are sometimes done to prevent circular dependencies, for which there is usually a much better way to solve the problem (refactor the code) or to prevent “up front costs” by delaying the import. This can also be solved in better ways than an unconditional local import (store the module as a class or module attribute and only do the import on first use).

That aside there is a way to use `mock` to affect the results of an import. Importing fetches an object from `sys.modules`. Note that it fetches an object, which need not be a module. Importing a module for the first time results in a module object being put in `sys.modules`, so usually when you import something you get a module back. This need not be the case however.

This means you can use `patch.dict()` to temporarily put a mock in place in `sys.modules`. Any imports whilst this patch is active will fetch the mock. When the patch is complete (the decorated function exits, the `with` statement body is complete or `patcher.stop()` is called) then whatever was there previously will be restored safely.

Here's an example that mocks out the 'fooble' module.

```
>>> import sys
>>> mock = Mock()
>>> with patch.dict('sys.modules', {'fooble': mock}):
...     import fooble
...     fooble.blob()
...
<Mock name='mock.blob()' id='...'>
>>> assert 'fooble' not in sys.modules
>>> mock.blob.assert_called_once_with()
```

As you can see the import `fooble` succeeds, but on exit there is no 'fooble' left in `sys.modules`.

This also works for the `from module import name` form:

```
>>> mock = Mock()
```

```
>>> with patch.dict('sys.modules', {'fooble': mock}):
...     from fooble import blob
...     blob.blip()
...
<Mock name='mock.blob.blip()' id='...'>
>>> mock.blob.blip.assert_called_once_with()
```

With slightly more work you can also mock package imports:

```
>>> mock = Mock()
>>> modules = {'package': mock, 'package.module': mock.module}
>>> with patch.dict('sys.modules', modules):
...     from package.module import fooble
...     fooble()
...
<Mock name='mock.module.fooble()' id='...'>
>>> mock.module.fooble.assert_called_once_with()
```

Tracking order of calls and less verbose call assertions

The `Mock` class allows you to track the order of method calls on your mock objects through the `method_calls` attribute. This doesn't allow you to track the order of calls between separate mock objects, however we can use `mock_call`s to achieve the same effect.

Because mocks track calls to child mocks in `mock_calls`, and accessing an arbitrary attribute of a mock creates a child mock, we can create our separate mocks from a parent one. Calls to those child mock will then all be recorded, in order, in the `mock_calls` of the parent:

```
>>> manager = Mock()
>>> mock_foo = manager.foo
>>> mock_bar = manager.bar

>>> mock_foo.something()
<Mock name='mock.foo.something()' id='...'>
>>> mock_bar.other.thing()
<Mock name='mock.bar.other.thing()' id='...'>

>>> manager.mock_calls
[call.foo.something(), call.bar.other.thing()]
```

We can then assert about the calls, including the order, by comparing with the `mock_calls` attribute on the manager mock:

```
>>> expected_calls = [call.foo.something(), call.bar.other.thing()]
>>> manager.mock_calls == expected_calls
True
```

If `patch` is creating, and putting in place, your mocks then you can attach them to a manager mock using the `attach_mock()` method. After attaching calls will be recorded in `mock_calls` of the manager.

```
>>> manager = MagicMock()
>>> with patch('mymodule.Class1') as MockClass1:
...     with patch('mymodule.Class2') as MockClass2:
...         manager.attach_mock(MockClass1, 'MockClass1')
...         manager.attach_mock(MockClass2, 'MockClass2')
...         MockClass1().foo()
...         MockClass2().bar()
<MagicMock name='mock.MockClass1().foo()' id='...'>
<MagicMock name='mock.MockClass2().bar()' id='...'>
>>> manager.mock_calls
[call.MockClass1(),
call.MockClass1().foo(),
call.MockClass2(),
call.MockClass2().bar()]
```

If many calls have been made, but you're only interested in a particular sequence of them then an alternative is to use the `assert_has_calls()` method. This takes a list of calls (constructed with the `call` object). If that sequence of calls are in `mock_call` then the assert succeeds.

```
>>> m = MagicMock()
>>> m().foo().bar().baz()
<MagicMock name='mock().foo().bar().baz()' id='...'>
>>> m.one().two().three()
<MagicMock name='mock.one().two().three()' id='...'>
>>> calls = call.one().two().three().call_list()
>>> m.assert_has_calls(calls)
```

Even though the chained call `m.one().two().three()` aren't the only calls that have been made to the mock, the assert still succeeds.

Sometimes a mock may have several calls made to it, and you are only interested in asserting about some of those calls. You may not even care about the order. In this case you can pass `any_order=True` to `assert_has_calls`:

```
>>> m = MagicMock()
>>> m(1), m.two(2, 3), m.seven(7), m.fifty('50')
(...)
>>> calls = [call.fifty('50'), call(1), call.seven(7)]
>>> m.assert_has_calls(calls, any_order=True)
```

More complex argument matching

Using the same basic concept as `ANY` we can implement matchers to do more complex assertions on objects used as arguments to mocks.

Suppose we expect some object to be passed to a mock that by default compares equal based on object identity (which is the Python default for user defined classes). To use `assert_called_with()` we would need to pass in the exact same object. If we are only interested in some of the attributes of this object then we can create a matcher that will check these attributes for us.

You can see in this example how a 'standard' call to `assert_called_with` isn't sufficient:

```
>>> class Foo:
...     def __init__(self, a, b):
...         self.a, self.b = a, b
...
>>> mock = Mock(return_value=None)
>>> mock(Foo(1, 2))
>>> mock.assert_called_with(Foo(1, 2))
Traceback (most recent call last):
...
AssertionError: expected call not found.
Expected: mock(<__main__.Foo object at 0x...>)
Actual: mock(<__main__.Foo object at 0x...>)
```

A comparison function for our `Foo` class might look something like this:

```
>>> def compare(self, other):
...     if not type(self) == type(other):
...         return False
...     if self.a != other.a:
...         return False
...     if self.b != other.b:
...         return False
...     return True
...
```

And a matcher object that can use comparison functions like this for its equality operation would look something like this:

```
>>> class Matcher:
...     def __init__(self, compare, some_obj):
...         self.compare = compare
...         self.some_obj = some_obj
...     def __eq__(self, other):
...         return self.compare(self.some_obj, other)
... 
```

Putting all this together:

```
>>> match_foo = Matcher(compare, Foo(1, 2))
>>> mock.assert_called_with(match_foo)
```

The `Matcher` is instantiated with our compare function and the `Foo` object we want to compare against.

In `assert_called_with` the `Matcher` equality method will be called, which compares the object the mock was called with against the one we created our matcher with. If they match then `assert_called_with` passes, and if they don't an `AssertionError` is raised:

```
>>> match_wrong = Matcher(compare, Foo(3, 4))
>>> mock.assert_called_with(match_wrong)
Traceback (most recent call last):
...
AssertionError: Expected: ((<Matcher object at 0x...>,), {})
Called with: ((<Foo object at 0x...>,), {})
```

With a bit of tweaking you could have the comparison function raise the `AssertionError` directly and provide a more useful failure message.

As of version 1.5, the Python testing library `PyHamcrest` provides similar functionality, that may be useful here, in the form of its equality matcher (`hamcrest.library.integration.match_equality`).

test— Regression tests package for Python

■ Nota: NoteThetestpackage is meant for internal use by Python only. It is documented for the benefit of the core developers of Python. Any use of this package outside of Python's standard library is discouraged as code mentioned here can change or be removed without notice between releases of Python.

Thetestpackage contains all regression tests for Python as well as the moduletest.supportandtest.regrtest.test.supportis used to enhance your tests whiletest.regrtestdrives the testing suite.

Each module in thetestpackage whose name starts withtest_is a testing suite for a specific module or feature. All new tests should be written using theunittestordoctestmodule. Some older tests are written using a “traditional” testing style that compares output printed tosys.stdout; this style of test is considered deprecated.

See also

Moduleunittest

Moduledoctest

Writing PyUnit regression tests.

Tests embedded in documentation strings.

Writing Unit Tests for thetestpackage

It is preferred that tests that use theunittestmodule follow a few guidelines. One is to name the test module by starting it withtest_and end it with the name of the module being tested. The test methods in the test module should start withtest_and end with a description of what the method is testing. This is needed so that the methods are recognized by the test driver as test methods. Also, no documentation string for the method should be included. A comment (such as#TestsfunctionreturnsonlyTrueorFalse) should be used to provide documentation for test methods. This is done because documentation strings get printed out if they exist and thus what test is being run is not stated.

A basic boilerplate is often used:

```
import unittest
from test import support

class MyTestCase1(unittest.TestCase):

    # Only use setUp() and tearDown() if necessary

    def setUp(self):
        ... code to execute in preparation for tests ...

    def tearDown(self):
        ... code to execute to clean up after tests ...

    def test_feature_one(self):
        # Test feature one.
        ... testing code ...

    def test_feature_two(self):
        # Test feature two.
        ... testing code ...

    ... more test methods ...

class MyTestCase2(unittest.TestCase):
```



```

... same structure as MyTestCase1 ...

... more test classes ...

if __name__ == '__main__':
    unittest.main()

```

This code pattern allows the testing suite to be run bytest.regrtest, on its own as a script that supports theunittestCLI, or via thepython-munittestCLI.

The goal for regression testing is to try to break code. This leads to a few guidelines to be followed:

- The testing suite should exercise all classes, functions, and constants. This includes not just the external API that is to be presented to the outside world but also “private” code.
- Whitebox testing (examining the code being tested when the tests are being written) is preferred. Blackbox testing (testing only the published user interface) is not complete enough to make sure all boundary and edge cases are tested.
- Make sure all possible values are tested including invalid ones. This makes sure that not only all valid values are acceptable but also that improper values are handled correctly.
- Exhaust as many code paths as possible. Test where branching occurs and thus tailor input to make sure as many different paths through the code are taken.
- Add an explicit test for any bugs discovered for the tested code. This will make sure that the error does not crop up again if the code is changed in the future.
- Make sure to clean up after your tests (such as close and remove all temporary files).
- If a test is dependent on a specific condition of the operating system then verify the condition already exists before attempting the test.
- Import as few modules as possible and do it as soon as possible. This minimizes external dependencies of tests and also minimizes possible anomalous behavior from side-effects of importing a module.
- Try to maximize code reuse. On occasion, tests will vary by something as small as what type of input is used. Minimize code duplication by subclassing a basic test class with a class that specifies the input:


```

class TestFuncAcceptsSequencesMixin:
    func = mySuperWhammyFunction
    def test_func(self):
        self.func(self.arg)
class AcceptLists(TestFuncAcceptsSequencesMixin, unittest.TestCase):
    arg = [1, 2, 3]
class AcceptStrin

```

See also

Test Driven Development

A book by Kent Beck on writing tests before code.

Running tests using the command-line interface

Thetestpackage can be run as a script to drive Python’s regression test suite, thanks to the-moption:python -m test. Under the hood, it uses test.regrtest; the callpython -m test.regrtestused in previous Python versions still works. Running the script by itself automatically starts running all regression tests in thetestpackage. It does this by finding all modules in the package whose name starts withtest_, importing them, and executing the functiontest_main()if present or loading the tests via unittest.TestLoader.loadTestsFromModule iftest_maindoes not exist. The names of tests to execute may also be passed to the script. Specifying a single regression test (python -m test test_spam) will minimize output and only print whether the test passed or failed.

Runningtestdirectly allows what resources are available for tests to use to be set. You do this by using the-ucommand-line option. Specifyingallas the value for the-uoption enables all possible resources:python -m test -uall. If all but one resource is desired (a more common case), a comma-separated list of resources that are

not desired may be listed after all. The command `python -m test -uall,-audio,-largefile` will run tests with all resources except the audio and large file resources. For a list of all resources and more command-line options, run `python -m test -h`.

Some other ways to execute the regression tests depend on what platform the tests are being executed on. On Unix, you can run `make test` at the top-level directory where Python was built. On Windows, executing `rt.bat` from your `PCbuild` directory will run all regression tests.

Added in version 3.14: Output is colored by default and can be controlled using environment variables.

test.support— Utilities for the Python test suite

The `test.support` module provides support for Python's regression test suite.

■ Nota: Not `test.support` is not a public module. It is documented here to help Python developers write tests. The API of this module is subject to change without backwards compatibility concerns between releases.

This module defines the following exceptions:

exceptiontest.support.TestFailed

Exception to be raised when a test fails. This is deprecated in favor of `unittest.TestCase`'s assertion methods.

exceptiontest.support.ResourceDenied

Subclass of `unittest.SkipTest`. Raised when a resource (such as a network connection) is not available. Raised by `therequires()` function.

The `test.support` module defines the following constants:

test.support.verbose

True when verbose output is enabled. Should be checked when more detailed information is desired about a running test. `verbose` is set by `test.regrtest`.

test.support.is_jython

True if the running interpreter is Jython.

test.support.is_android

True if `sys.platform` is `android`.

test.support.is_emscripten

True if `sys.platform` is `emscripten`.

test.support.is_wasi

True if `sys.platform` is `wasi`.

test.support.is_apple_mobile

True if `sys.platform` is `ios`, `tvos`, or `watchos`.

test.support.is_apple

True if `sys.platform` is `darwin` or `is_apple_mobile` is True.

test.support.unix_shell

Path for shell if not on Windows; otherwise None.

test.support.LOOPBACK_TIMEOUT

Timeout in seconds for tests using a network server listening on the network local loopback interface like 127.0.0.1. The timeout is long enough to prevent test failure: it takes into account that the client and the server can run in different threads or even different processes. The timeout should be long enough for `connect()`, `recv()` and `send()` methods of `socket.socket`. Its default value is 5 seconds. See also `INTERNET_TIMEOUT`.

test.support.INTERNET_TIMEOUT

Timeout in seconds for network requests going to the internet. The timeout is short enough to prevent a test to wait for too long if the internet request is blocked for whatever reason. Usually, a timeout using `INTERNET_TIMEOUT` should not mark a test as failed, but skip the test instead: see `transient_internet()`. Its default value is 1 minute. See also `LOOPBACK_TIMEOUT`.

test.support.SHORT_TIMEOUT

Timeout in seconds to mark a test as failed if the test takes “too long”. The timeout value depends on the `regtest--timeout` command line option. If a test using `SHORT_TIMEOUT` starts to fail randomly on slow buildbots, use `LONG_TIMEOUT` instead. Its default value is 30 seconds.

test.support.LONG_TIMEOUT

Timeout in seconds to detect when a test hangs. It is long enough to reduce the risk of test failure on the slowest Python buildbots. It should not be used to mark a test as failed if the test takes “too long”. The timeout value depends on the `regtest--timeout` command line option. Its default value is 5 minutes. See also `LOOPBACK_TIMEOUT`, `INTERNET_TIMEOUT` and `SHORT_TIMEOUT`.

test.support.PGO

Set when tests can be skipped when they are not useful for PGO.

test.support.PIPE_MAX_SIZE

A constant that is likely larger than the underlying OS pipe buffer size, to make writes blocking.

test.support.Py_DEBUG

True if Python was built with the `Py_DEBUG` macro defined, that is, if Python was built in debug mode. Added in version 3.12.

test.support.SOCK_MAX_SIZE

A constant that is likely larger than the underlying OS socket buffer size, to make writes blocking.

test.support.TEST_SUPPORT_DIR

Set to the top level directory that contains `test.support`.

test.support.TEST_HOME_DIR

Set to the top level directory for the test package.

test.support.TEST_DATA_DIR

Set to the `data` directory within the test package.

test.support.MAX_Py_ssize_t

Set to `sys.maxsize` for big memory tests.

test.support.max_memuse

Set by `set_memlimit()` as the memory limit for big memory tests. Limited by `MAX_Py_ssize_t`.

test.support.real_max_memuse

Set by `set_memlimit()` as the memory limit for big memory tests. Not limited by `MAX_Py_ssize_t`.

test.support.MISSING_C_DOCSTRINGS

Set to `True` if Python is built without docstrings (the `WITH_DOC_STRINGS` macro is not defined). See the `configure--without-doc-strings` option. See also the `HAVE_DOCSTRINGS` variable.

test.support.HAVE_DOCSTRINGS

Set to `True` if function docstrings are available. See the `python-O` option, which strips docstrings of functions implemented in Python. See also the `MISSING_C_DOCSTRINGS` variable.

test.support.TEST_HTTP_URL

Define the URL of a dedicated HTTP server for the network tests.

test.support.ALWAYS_EQ

Object that is equal to anything. Used to test mixed type comparison.

test.support.NEVER_EQ

Object that is not equal to anything (even to `ALWAYS_EQ`). Used to test mixed type comparison.

test.support.LARGEST

Object that is greater than anything (except itself). Used to test mixed type comparison.

test.support.SMALLEST

Object that is less than anything (except itself). Used to test mixed type comparison.

The `test.support` module defines the following functions:

test.support.busy_retry(timeout, err_msg=None, /, *, error=True)

Run the loop body until `break` stops the loop. After `timeout` seconds, raise an `AssertionError` if `error` is `True`, or just stop the loop if `error` is `False`. Example: `for _ in support.busy_retry(support.SHORT_TIMEOUT): if check(): break` Example of `error=False` usage: `for _ in support.busy_retry(support.SHORT_TIMEOUT, error=False): if check(): break else: raise RuntimeError('my custom error')`

test.support.sleeping_retry(timeout, err_msg=None, /, *, init_delay=0.010, max_delay=1.0, error=True)

Wait strategy that applies exponential backoff. Run the loop body until `break` stops the loop. Sleep at each loop iteration, but not at the first iteration. The sleep delay is doubled at each iteration (up to `max_delay` seconds). See `busy_retry()` documentation for the parameters usage. Example raising an exception after `SHORT_TIMEOUT` seconds: `for _ in support.sleeping_retry(support.SHORT_TIMEOUT): if check(): break` Example of `error=False` usage: `for _ in support.sleeping_retry(support.SHORT_TIMEOUT, error=False): if check(): break`

test.support.is_resource_enabled(resource)

Return `True` if `resource` is enabled and available. The list of available resources is only set when `test.regrtest` is executing the tests.

test.support.get_resource_value(resource)

Return the value specified for `resource` (as `resource=value`). Return `None` if `resource` is disabled or no value is specified.

test.support.python_is_optimized()

Return `True` if Python was not built with `-O0` or `-Og`.

test.support.with_pymalloc()

Return `testcapi.WITH_PYMALLOC`.

test.support.requires(resource,msg=None)

Raises `ResourceDenied` if `resource` is not available. `msg` is the argument to `ResourceDenied` if it is raised. Always returns `True` if called by a function whose `__name__` is `'__main__'`. Used when tests are executed by `test.regrtest`.

test.support.sortdict(dict)

Return a repr of `dict` with keys sorted.

test.support.findfile(filename,subdir=None)

Return the path to the file named `filename`. If no match is found `filename` is returned. This does not equal a failure since it could be the path to the file. `subdir` indicates a relative path to use to find the file rather than looking directly in the path directories.

test.support.get_pagesize()

Get size of a page in bytes. Added in version 3.12.

test.support.setswitchinterval(interval)

Set the `sys.setswitchinterval()` to the given `interval`. Defines a minimum interval for Android systems to prevent the system from hanging.

test.support.check_impl_detail(guards)**

Use this check to guard CPython's implementation-specific tests or to run them only on the implementations guarded by the arguments. This function returns `True` or `False` depending on the host platform. Example usage: `check_impl_detail()` # Only on CPython (default). `check_impl_detail(jython=True)` # Only on Jython. `check_impl_detail(cpython=False)` # Everywhere except CPython.

test.support.set_memlimit(limit)

Set the values for `max_memuse` and `real_max_memuse` for big memory tests.

test.support.record_original_stdout(stdout)

Store the value from `stdout`. It is meant to hold the `stdout` at the time the `regrtest` began.

test.support.get_original_stdout()

Return the original `stdout` set by `record_original_stdout()` or `sys.stdout` if it's not set.

test.support.args_from_interpreter_flags()

Return a list of command line arguments reproducing the current settings in `sys.flags` and `sys.warnoptions`.

test.support.optim_args_from_interpreter_flags()

Return a list of command line arguments reproducing the current optimization settings in `sys.flags`.

test.support.captured_stdin()**test.support.captured_stdout()****test.support.captured_stderr()**

A context managers that temporarily replaces the named stream with `io.StringIO` object. Example use with output streams: `with captured_stdout() as stdout, captured_stderr() as stderr: print("hello") print("error", file=sys.stderr)` `assert stdout.getvalue() == "hello\n"` `assert stderr.getvalue() == "error\n"` Example use with input stream: `with captured_stdin() as stdin: stdin.write('hello\n') stdin.seek(0)` # call test code that consumes from `sys.stdin` `captured = input()` `self.assertEqual(captured, "hello")`

test.support.disable_fault_handler()

A context manager that temporary disables `fault_handler`.

test.support.gc_collect()

Force as many objects as possible to be collected. This is needed because timely deallocation is not guaranteed by the garbage collector. This means that `__del__` methods may be called later than expected and weakrefs may remain alive for longer than expected.

test.support.disable_gc()

A context manager that disables the garbage collector on entry. On exit, the garbage collector is restored to its prior state.

test.support.swap_attr(obj,attr,new_val)

Context manager to swap out an attribute with a new object. Usage: `with swap_attr(obj, "attr", 5): ...`. This will set `obj.attr` to 5 for the duration of the `with` block, restoring the old value at the end of the block. If `attr` doesn't exist on `obj`, it will be created and then deleted at the end of the block. The old value (or `None` if it doesn't exist) will be assigned to the target of the "as" clause, if there is one.

test.support.swap_item(obj,attr,new_val)

Context manager to swap out an item with a new object. Usage: `with swap_item(obj, "item", 5): ...`. This will set `obj["item"]` to 5 for the duration of the `with` block, restoring the old value at the end of the block. If `item` doesn't exist on `obj`, it will be created and then deleted at the end of the block. The old value (or `None` if it doesn't exist) will be assigned to the target of the "as" clause, if there is one.

test.support.flush_std_streams()

Call the `flush()` method on `sys.stdout` and then `sys.stderr`. It can be used to make sure that the logs order is consistent before writing into `stderr`. Added in version 3.11.

test.support.print_warning(msg)

Print a warning into `sys.__stderr__`. Format the message as: `f"Warning--{msg}"`. If `msg` is made of multiple lines, add `"Warning--"` prefix to each line. Added in version 3.9.

test.support.wait_process(pid,*,exitcode,timeout=None)

Wait until process `pid` completes and check that the process exit code is `exitcode`. Raise an `AssertionError` if the process exit code is not equal to `exitcode`. If the process runs longer than `timeout` seconds (`SHORT_TIMEOUT` by default), kill the process and raise an `AssertionError`. The timeout feature is not available on Windows. Added in version 3.9.

test.support.calcobjsize(fmt)

Return the size of the `PyObject` whose structure members are defined by `fmt`. The returned value includes the size of the Python object header and alignment.

test.support.calcvobjsize(fmt)

Return the size of the `PyVarObject` whose structure members are defined by `fmt`. The returned value includes the size of the Python object header and alignment.

test.support.checksizeof(test,o,size)

For `testcasetest`, assert that `sys.getsizeof(o)` plus the GC header size equals `size`.

@test.support.anticipate_failure(condition)

A decorator to conditionally mark tests with `unittest.expectedFailure()`. Any use of this decorator should have an associated comment identifying the relevant tracker issue.

test.support.system_must_validate_cert(f)

A decorator that skips the decorated test on TLS certification validation failures.

@test.support.run_with_locale(catstr,*locales)

A decorator for running a function in a different locale, correctly resetting it after it has finished. `catstr` is the locale category as a string (for example "LC_ALL"). The `locales` passed will be tried sequentially, and the first valid locale will be used.

@test.support.run_with_tz(tz)

A decorator for running a function in a specific timezone, correctly resetting it after it has finished.

@test.support.requires_freebsd_version(*min_version)

Decorator for the minimum version when running test on FreeBSD. If the FreeBSD version is less than the minimum, the test is skipped.

@test.support.requires_linux_version(*min_version)

Decorator for the minimum version when running test on Linux. If the Linux version is less than the minimum, the test is skipped.

@test.support.requires_mac_version(*min_version)

Decorator for the minimum version when running test on macOS. If the macOS version is less than the minimum, the test is skipped.

@test.support.requires_gil_enabled

Decorator for skipping tests on the free-threaded build. If the GIL is disabled, the test is skipped.

@test.support.requires_IEEE_754

Decorator for skipping tests on non-IEEE 754 platforms.

@test.support.requires_zlib

Decorator for skipping tests if `zlib` doesn't exist.

@test.support.requires_gzip

Decorator for skipping tests if `gzip` doesn't exist.

@test.support.requires_bz2

Decorator for skipping tests if `bz2` doesn't exist.

@test.support.requires_lzma

Decorator for skipping tests if `lzma` doesn't exist.

@test.support.requires_resource(resource)

Decorator for skipping tests if `resource` is not available.

@test.support.requires_docstrings

Decorator for only running the test if `HAVE_DOCSTRINGS`.

@test.support.requires_limited_api

Decorator for only running the test if `Limited C API` is available.

@test.support.cpython_only

Decorator for tests only applicable to CPython.

@test.support.impl_detail(msg=None,guards)**

Decorator for invoking `check_impl_detail()` on guards. If that returns `False`, then uses `msg` as the reason for skipping the test.

@test.support.thread_unsafe(reason=None)

Decorator for marking tests as thread-unsafe. This test always runs in one thread even when invoked with `--parallel-threads`.

@test.support.no_tracing

Decorator to temporarily turn off tracing for the duration of the test.

@test.support.refcount_test

Decorator for tests which involve reference counting. The decorator does not run the test if it is not run by CPython. Any trace function is unset for the duration of the test to prevent unexpected refcounts caused by the trace function.

@test.support.bigmemtest(size, memuse, dry_run=True)

Decorator for bigmem tests. `size` is a requested size for the test (in arbitrary, test-interpreted units.) `memuse` is the number of bytes per unit for the test, or a good estimate of it. For example, a test that needs two byte buffers, of 4 GiB each, could be decorated with `@bigmemtest(size=_4G, memuse=2)`. The `size` argument is normally passed to the decorated test method as an extra argument. If `dry_run` is `True`, the value passed to the test method may be less than the requested value. If `dry_run` is `False`, it

@test.support.bigaddrspace_test

Decorator for tests that fill the address space.

test.support.linked_to_musl()

Return `False` if there is no evidence the interpreter was compiled with `musl`, otherwise return a version triple, either `(0,0,0)` if the version is unknown, or the actual version if it is known. Intended for use in skip decorators. `emscripten` and `wasi` are assumed to be compiled with `musl`; otherwise `platform.libc_ver` is checked.

test.support.check_syntax_error(testcase, statement, errtext="*", lineno=None, offset=None)

Test for syntax errors in `statement` by attempting to compile `statement`. `testcase` is the `unittest` instance for the test. `errtext` is the regular expression which should match the string representation of the raised `SyntaxError`. If `lineno` is not `None`, compares to the line of the exception. If `offset` is not `None`, compares to the offset of the exception.

test.support.open_urlresource(url, *args, **kw)

Open `url`. If open fails, raises `TestFailed`.

test.support.reap_children()

Use this at the end of `test_main` whenever sub-processes are started. This will help ensure that no extra children (zombies) stick around to hog resources and create problems when looking for ref leaks.

test.support.get_attribute(obj, name)

Get an attribute, raising `unittest.SkipTest` if `AttributeError` is raised.

test.support.catch_unraisable_exception()

Context manager catching unraisable exception using `sys.unraisablehook()`. Storing the exception value (`cm.unraisable.exc_value`) creates a reference cycle. The reference cycle is broken explicitly when the context manager exits. Storing the object (`cm.unraisable.object`) can resurrect it if it is set to an object which is being finalized. Exiting the context manager clears the stored

object.Usage:withsupport.catch_unraisable_exception()ascm:# code creating an "unraisable exception"...#
check the unra

test.support.load_package_tests(pkg_dir,loader,standard_tests,pattern)

Generic implementation of the unittestload_testsprotocol for use in test packages.pkg_diris the root directory of the package;loader,standard_tests, andpatternare the arguments expected byload_tests. In simple cases, the test package's__init__.pycan be the following:importosfromtest.supportimportload_package_testsdefload_tests(*args):returnload_package_tests(os.path.dirname(__file__),*args)

test.support.detect_api_mismatch(ref_api,other_api,*ignore=())

Returns the set of attributes, functions or methods ofref_apinot found onother_api, except for a defined list of items to be ignored in this check specified inignore.By default this skips private attributes beginning with '_' but includes all magic methods, i.e. those starting and ending in '__'.Added in version 3.5.

test.support.patch(test_instance,object_to_patch,attr_name,new_value)

Overrideobject_to_patch.attr_namewithnew_value. Also add cleanup procedure totest_instanceto restoreobject_to_patchforattr_name. Theattr_nameshould be a valid attribute forobject_to_patch.

test.support.run_in_subinterp(code)

Runcodein subinterpreter. Raiseunittest.SkipTestiftracemallocis enabled.

test.support.check_free_after_iterating(test,iter,cls,args=())

Assert instances ofclsare deallocated after iterating.

test.support.missing_compiler_executable(cmd_names=[])

Check for the existence of the compiler executables whose names are listed incmd_namesor all the compiler executables whencmd_namesis empty and return the first missing executable orNonewhen none is found missing.

test.support.check_all__(test_case,module,name_of_module=None,extra=(),not_exported=())

Assert that the__all__variable ofmodulecontains all public names.The module's public names (its API) are detected automatically based on whether they match the public name convention and were defined inmodule.Thename_of_moduleargument can specify (as a string or tuple thereof) what module(s) an API could be defined in order to be detected as a public API. One case for this is whenmoduleimports part of its public API from other modules, possibly a C backend (likecsvand its_csv).Theextraargument c

test.support.skip_if_broken_multiprocessing_synchronize()

Skip tests if themultiprocessing.synchronizemodule is missing, if there is no available semaphore implementation, or if creating a lock raises anOSError.Added in version 3.10.

test.support.check_disallow_instantiation(test_case,tp,*args,kwds)**

Assert that typetpcannot be instantiated usingargsandkwds.Added in version 3.10.

test.support.adjust_int_max_str_digits(max_digits)

This function returns a context manager that will change the globalsys.set_int_max_str_digits()setting for the duration of the context to allow execution of test code that needs a different limit on the number of digits when converting between an integer and string.Added in version 3.11.

The test.supportmodule defines the following classes:

classtest.support.SuppressCrashReport

A context manager used to try to prevent crash dialog popups on tests that are expected to crash a subprocess.On Windows, it disables Windows Error Reporting dialogs usingSetErrorMode.On

UNIX, `resource.setrlimit()` is used to set `resource.RLIMIT_CORE`'s soft limit to 0 to prevent core dump file creation. On both platforms, the old value is restored by `__exit__()`.

classtest.support.SaveSignals

save(self)

restore(self)

Class to save and restore signal handlers registered by the Python signal handler. `save(self)` Save the signal handlers to a dictionary mapping signal numbers to the current signal handler. `restore(self)` Set the signal numbers from the `save()` dictionary to the saved handler.

Save the signal handlers to a dictionary mapping signal numbers to the current signal handler.

Set the signal numbers from the `save()` dictionary to the saved handler.

classtest.support.Matcher

matches(self,d,kwargs)**

match_value(self,k,dv,v)

`matches(self,d,**kwargs)` Try to match a single dict with the supplied arguments. `match_value(self,k,dv,v)` Try to match a single stored value (dv) with a supplied value (v).

Try to match a single dict with the supplied arguments.

Try to match a single stored value (dv) with a supplied value (v).

test.support.socket_helper— Utilities for socket tests

The `test.support.socket_helper` module provides support for socket tests.

Added in version 3.9.

test.support.socket_helper.IPV6_ENABLED

Set to `True` if IPv6 is enabled on this host, `False` otherwise.

test.support.socket_helper.find_unused_port(family=socket.AF_INET, socktype=socket.SOCK_STREAM)

Returns an unused port that should be suitable for binding. This is achieved by creating a temporary socket with the same family and type as the `sock` parameter (default is `AF_INET, SOCK_STREAM`), and binding it to the specified host address (defaults to `0.0.0.0`) with the port set to 0, eliciting an unused ephemeral port from the OS. The temporary socket is then closed and deleted, and the ephemeral port is returned. Either this method or `bind_port()` should be used for any tests where a server socket is needed.

test.support.socket_helper.bind_port(sock, host=HOST)

Bind the socket to a free port and return the port number. Relies on ephemeral ports in order to ensure we are using an unbound port. This is important as many tests may be running simultaneously, especially in a buildbot environment. This method raises an exception if the `sock.family` is `AF_INET` and `sock.type` is `SOCK_STREAM`, and the socket has `SO_REUSEADDR` or `SO_REUSEPORT` set on it. Tests should never set these socket options for TCP/IP sockets. The only case for setting these options is testing multicasting.

test.support.socket_helper.bind_unix_socket(sock, addr)

Bind a Unix socket, raising `unittest.SkipTest` if `PermissionError` is raised.

@test.support.socket_helper.skip_unless_bind_unix_socket

A decorator for running tests that require a functional `bind()` for Unix sockets.

test.support.socket_helper.transient_internet(resource_name,*,timeout=30.0,errno=())

A context manager that raises `ResourceDenied` when various issues with the internet connection manifest themselves as exceptions.

test.support.script_helper— Utilities for the Python execution tests

The `test.support.script_helper` module provides support for Python's script execution tests.

test.support.script_helper.interpreter_requires_environment()

Return `True` if `sys.executable` interpreter requires environment variables in order to be able to run at all. This is designed to be used with `@unittest.skipIf()` to annotate tests that need to use an `assert_python*()` function to launch an isolated mode (-I) or no environment mode (-E) sub-interpreter process. A normal build & test does not run into this situation but it can happen when trying to run the standard library test suite from an interpreter that doesn't have an obvious home with Python's current `ho`

test.support.script_helper.run_python_until_end(*args,env_vars)**

Set up the environment based on `env_vars` for running the interpreter in a subprocess. The values can include `__isolated`, `__cleanenv`, `__cwd`, and `TERM`. Changed in version 3.9: The function no longer strips whitespaces from `stderr`.

test.support.script_helper.assert_python_ok(*args,env_vars)**

Assert that running the interpreter with `args` and optional environment variables `env_vars` succeeds (`rc==0`) and return a `(returncode,stdout,stderr)` tuple. If the `__cleanenv` keyword-only parameter is set, `env_vars` is used as a fresh environment. Python is started in isolated mode (command line option -I), except if the `__isolated` keyword-only parameter is set to `False`. Changed in version 3.9: The function no longer strips whitespaces from `stderr`.

test.support.script_helper.assert_python_failure(*args,env_vars)**

Assert that running the interpreter with `args` and optional environment variables `env_vars` fails (`rc!=0`) and return a `(returncode,stdout,stderr)` tuple. See `assert_python_ok()` for more options. Changed in version 3.9: The function no longer strips whitespaces from `stderr`.

test.support.script_helper.spawn_python(*args,stdout=subprocess.PIPE,stderr=subprocess.STDOUT,kw)**

Run a Python subprocess with the given arguments. `kw` is extra keyword args to pass to `subprocess.Popen()`. Returns a `subprocess.Popen` object.

test.support.script_helper.kill_python(p)

Run the given `subprocess.Popen` process until completion and return `stdout`.

test.support.script_helper.make_script(script_dir,script_basename,source,omit_suffix=False)

Create script containing `source` in path `script_dir` and `script_basename`. If `omit_suffix` is `False`, append `.py` to the name. Return the full script path.

test.support.script_helper.make_zip_script(zip_dir,zip_basename,script_name,name_in_zip=None)

Create zip file at `zip_dir` and `zip_basename` with extension `zip` which contains the files in `script_name`. `name_in_zip` is the archive name. Return a tuple containing `(fullpath,fullpathofarchive)`.

test.support.script_helper.make_pkg(pkg_dir,init_source="")

Create a directory named `pkg_dir` containing an `__init__` file with `init_source` as its contents.

test.support.script_helper.make_zip_pkg(zip_dir,zip_basename,pkg_name,script_basename,source,depth=1,compiled=False)

Create a zip package directory with a path of `zip_dir` and `zip_basename` containing an empty `__init__` file and a file `script_basename` containing the `source`. If `compiled` is `True`, both source files will be compiled and added to the zip package. Return a tuple of the full zip path and the archive name for the zip file.

test.support.bytecode_helper— Support tools for testing correct bytecode generation

The `test.support.bytecode_helper` module provides support for testing and inspecting bytecode generation.

Added in version 3.9.

The module defines the following class:

class test.support.bytecode_helper.BytecodeTestCase(unittest.TestCase)

This class has custom assertion methods for inspecting bytecode.

BytecodeTestCase.get_disassembly_as_string(co)

Return the disassembly of `co` as string.

BytecodeTestCase.assertInBytecode(x,opname,argval=_UNSPECIFIED)

Return `instr` if `opname` is found, otherwise throws `AssertionError`.

BytecodeTestCase.assertNotInBytecode(x,opname,argval=_UNSPECIFIED)

Throws `AssertionError` if `opname` is found.

test.support.threading_helper— Utilities for threading tests

The `test.support.threading_helper` module provides support for threading tests.

Added in version 3.10.

test.support.threading_helper.join_thread(thread,timeout=None)

Join a thread with `timeout`. Raise an `AssertionError` if thread is still alive after `timeout` seconds.

@test.support.threading_helper.reap_threads

Decorator to ensure the threads are cleaned up even if the test fails.

test.support.threading_helper.start_threads(threads,unlock=None)

Context manager to start `threads`, which is a sequence of threads. `unlock` is a function called after the threads are started, even if an exception was raised; an example would be `threading.Event.set().start_threads` will attempt to join the started threads upon exit.

test.support.threading_helper.threading_cleanup(*original_values)

Cleanup up threads not specified in `original_values`. Designed to emit a warning if a test leaves running threads in the background.

test.support.threading_helper.threading_setup()

Return current thread count and copy of dangling threads.

test.support.threading_helper.wait_threads_exit(timeout=None)

Context manager to wait until all threads created in the with statement exit.

test.support.threading_helper.catch_threading_exception()

Context manager catching `threading.ThreadException` using `threading.excepthook()`. Attributes set when an exception is caught: `exc_type`, `exc_value`, `exc_traceback`, `thread`. See `threading.excepthook()` documentation. These attributes are deleted at the context manager exit.

Usage: `with threading_helper.catch_threading_exception() as cm:` # code spawning a thread which raises an exception... # check the thread exception, use cm attributes: # `exc_type`, `exc_value`, `exc_traceback`, `thread`... # `exc_type`, `exc_value`, `exc_traceback`, `th`

test.support.threading_helper.run_concurrently(worker_func, nthreads, args=(), kwargs={})

Run the worker function concurrently in multiple threads. Re-raises an exception if any thread raises one, after all threads have finished.

test.support.os_helper— Utilities for os tests

The `test.support.os_helper` module provides support for os tests.

Added in version 3.10.

test.support.os_helper.FS_NONASCII

A non-ASCII character encodable by `os.fsencode()`.

test.support.os_helper.SAVEDCWD

Set to `os.getcwd()`.

test.support.os_helper.TESTFN

Set to a name that is safe to use as the name of a temporary file. Any temporary file that is created should be closed and unlinked (removed).

test.support.os_helper.TESTFN_NONASCII

Set to a filename containing the `FS_NONASCII` character, if it exists. This guarantees that if the filename exists, it can be encoded and decoded with the default filesystem encoding. This allows tests that require a non-ASCII filename to be easily skipped on platforms where they can't work.

test.support.os_helper.TESTFN_UNENCODABLE

Set to a filename (str type) that should not be able to be encoded by file system encoding in strict mode. It may be `None` if it's not possible to generate such a filename.

test.support.os_helper.TESTFN_UNDECODABLE

Set to a filename (bytes type) that should not be able to be decoded by file system encoding in strict mode. It may be `None` if it's not possible to generate such a filename.

test.support.os_helper.TESTFN_UNICODE

Set to a non-ASCII name for a temporary file.

class test.support.os_helper.EnvironmentVarGuard

Class used to temporarily set or unset environment variables. Instances can be used as a context manager and have a complete dictionary interface for querying/modifying the underlying `os.environ`. After exit from the context manager all changes to environment variables done through this instance will be rolled back. Changed in version 3.1: Added dictionary interface.

classtest.support.os_helper.FakePath(path)

Simplepath-like object. It implements the `__fspath__()` method which just returns the `path` argument. If `path` is an exception, it will be raised in `__fspath__()`.

EnvironmentVarGuard.set(envvar,value)

Temporarily set the environment variable `envvar` to the value of `value`.

EnvironmentVarGuard.unset(envvar,*others)

Temporarily unset one or more environment variables. Changed in version 3.14: More than one environment variable can be unset.

test.support.os_helper.can_symlink()

Return `True` if the OS supports symbolic links, `False` otherwise.

test.support.os_helper.can_xattr()

Return `True` if the OS supports `xattr`, `False` otherwise.

test.support.os_helper.change_cwd(path,quiet=False)

A context manager that temporarily changes the current working directory to `path` and yields the directory. If `quiet` is `False`, the context manager raises an exception on error. Otherwise, it issues only a warning and keeps the current working directory the same.

test.support.os_helper.create_empty_file(filename)

Create an empty file with `filename`. If it already exists, truncate it.

test.support.os_helper.fd_count()

Count the number of open file descriptors.

test.support.os_helper.fs_is_case_insensitive(directory)

Return `True` if the file system for `directory` is case-insensitive.

test.support.os_helper.make_bad_fd()

Create an invalid file descriptor by opening and closing a temporary file, and returning its descriptor.

test.support.os_helper.rmdir(filename)

Call `os.rmdir()` on `filename`. On Windows platforms, this is wrapped with a wait loop that checks for the existence of the file, which is needed due to antivirus programs that can hold files open and prevent deletion.

test.support.os_helper.rmtree(path)

Call `shutil.rmtree()` on `path` or `callos.lstat()` and `os.rmdir()` to remove a path and its contents. As with `rmdir()`, on Windows platforms this is wrapped with a wait loop that checks for the existence of the files.

@test.support.os_helper.skip_unless_symlink

A decorator for running tests that require support for symbolic links.

@test.support.os_helper.skip_unless_xattr

A decorator for running tests that require support for `xattr`.

test.support.os_helper.temp_cwd(name='tempcwd',quiet=False)

A context manager that temporarily creates a new directory and changes the current working directory (CWD). The context manager creates a temporary directory in the current directory with name `name` before temporarily changing the current working directory. If `name` is `None`, the temporary directory is created

using `tempfile.mkdtemp()`. If `quiet` is `False` and it is not possible to create or change the CWD, an error is raised. Otherwise, only a warning is raised and the original CWD is used.

`test.support.os_helper.temp_dir(path=None, quiet=False)`

A context manager that creates a temporary directory at `path` and yields the directory. If `path` is `None`, the temporary directory is created using `tempfile.mkdtemp()`. If `quiet` is `False`, the context manager raises an exception on error. Otherwise, if `path` is specified and cannot be created, only a warning is issued.

`test.support.os_helper.temp_umask(umask)`

A context manager that temporarily sets the process `umask`.

`test.support.os_helper.unlink(filename)`

Call `os.unlink()` on `filename`. As with `rmdir()`, on Windows platforms, this is wrapped with a wait loop that checks for the existence of the file.

test.support.import_helper— Utilities for import tests

The `test.support.import_helper` module provides support for import tests.

Added in version 3.10.

`test.support.import_helper.forget(module_name)`

Remove the module named `module_name` from `sys.modules` and delete any byte-compiled files of the module.

`test.support.import_helper.import_fresh_module(name, fresh=(), blocked=(), deprecated=False)`

This function imports and returns a fresh copy of the named Python module by removing the named module from `sys.modules` before doing the import. Note that unlike `reload()`, the original module is not affected by this operation. `fresh` is an iterable of additional module names that are also removed from the `sys.modules` cache before doing the import. `blocked` is an iterable of module names that are replaced with `None` in the module cache during the import to ensure that attempts to import them raise `ImportError`. `T`

`test.support.import_helper.import_module(name, deprecated=False, *, required_on=())`

This function imports and returns the named module. Unlike a normal import, this function raises `unittest.SkipTest` if the module cannot be imported. Module and package deprecation messages are suppressed during this import if `deprecated` is `True`. If a module is required on a platform but optional for others, set `required_on` to an iterable of platform prefixes which will be compared against `sys.platform`. Added in version 3.1.

`test.support.import_helper.modules_setup()`

Return a copy of `sys.modules`.

`test.support.import_helper.modules_cleanup(oldmodules)`

Remove modules except for `oldmodules` and `encodings` in order to preserve internal cache.

`test.support.import_helper.unload(name)`

Delete `name` from `sys.modules`.

`test.support.import_helper.make_legacy_pyc(source)`

Move a PEP 3147/PEP 488 pyc file to its legacy pyc location and return the file system path to the legacy pyc file. The `source` value is the file system path to the source file. It does not need to exist, however the PEP 3147/488 pyc file must exist.

`class test.support.import_helper.CleanImport(*module_names)`

A context manager to force import to return a new module reference. This is useful for testing module-level behaviors, such as the emission of a `DeprecationWarning` on import. Example usage: `with CleanImport('foo'): importlib.import_module('foo') # New reference.`

`class test.support.import_helper.DirsOnSysPath(*paths)`

A context manager to temporarily add directories to `sys.path`. This makes a copy of `sys.path`, appends any directories given as positional arguments, then reverts `sys.path` to the copied settings when the context ends. Note that all `sys.path` modifications in the body of the context manager, including replacement of the object, will be reverted at the end of the block.

test.support.warnings_helper— Utilities for warnings tests

The `test.support.warnings_helper` module provides support for warnings tests.

Added in version 3.10.

`test.support.warnings_helper.ignore_warnings(*, category)`

Suppress warnings that are instances of `category`, which must be `Warning` or a subclass. Roughly equivalent to `warnings.catch_warnings()` with `warnings.simplefilter('ignore', category=category)`. For example: `@warning_helper.ignore_warnings(category=DeprecationWarning) def test_suppress_warning(): # do something` Added in version 3.8.

`test.support.warnings_helper.check_no_resource_warning(testcase)`

Context manager to check that no `ResourceWarning` was raised. You must remove the object which may emit `ResourceWarning` before the end of the context manager.

`test.support.warnings_helper.check_syntax_warning(testcase, statement, errtext="", *, lineno=1, offset=None)`

Test for syntax warning in `statement` by attempting to compile `statement`. Test also that the `SyntaxWarning` is emitted only once, and that it will be converted to a `SyntaxError` when turned into error. `testcase` is the `unittest` instance for the test. `errtext` is the regular expression which should match the string representation of the emitted `SyntaxWarning` and raised `SyntaxError`. If `lineno` is not `None`, compares to the line of the warning and exception. If `offset` is not `None`, compares to the offset of the exception. Added

`test.support.warnings_helper.check_warnings(*filters, quiet=True)`

A convenience wrapper for `warnings.catch_warnings()` that makes it easier to test that a warning was correctly raised. It is approximately equivalent to calling `warnings.catch_warnings(record=True)` with `warnings.simplefilter()` set to `always` and with the option to automatically validate the results that are recorded. `check_warnings` accepts 2-tuples of the form `("message regexp", WarningCategory)` as positional arguments. If one or more filters are provided, or if the optional keyword argument `quiet` is `False`, it checks

`class test.support.warnings_helper.WarningsRecorder`

Class used to record warnings for unit tests. See documentation of `check_warnings()` above for more details.

Debugging and Profiling

These libraries help you with Python development: the debugger enables you to step through code, analyze stack frames and set breakpoints etc., and the profilers run code and give you a detailed breakdown of execution times, allowing you to identify bottlenecks in your programs. Auditing events provide visibility into runtime behaviors that would otherwise require intrusive debugging or patching.

- `Audit` events table
- `bdb`— Debugger framework
- `faulthandler`— Dump the Python tracebackDumping the tracebackDumping the C stackC Stack CompatibilityFault handler stateDumping the tracebacks after a timeoutDumping the traceback on a user signalIssue with file descriptorsExample
- `pdb`— The Python DebuggerCommand-line interfaceDebugger commands
- The Python ProfilersIntroduction to the profilersInstant User's ManualprofileandcProfileModule ReferenceTheStatsClassWhat Is Deterministic Profiling?LimitationsCalibrationUsing a custom timer
- `timeit`— Measure execution time of small code snippetsBasic ExamplesPython InterfaceCommand-Line InterfaceExamples
- `trace`— Trace or track Python statement executionCommand-Line UsageMain optionsModifiersFiltersProgrammatic Interface
- `tracemalloc`— Trace memory allocationsExamplesDisplay the top 10Compute differencesGet the traceback of a memory blockPretty topRecord the current and peak size of all traced memory blocksAPIFunctionsDomainFilterFilterFrameSnapshotStatisticStatisticDiffTraceTraceback

Audit events table

This table contains all events raised by `sys.audit()` or `PySys_Audit()` calls throughout the CPython runtime and the standard library. These calls were added in 3.8 or later (see PEP 578).

See `sys.addaudithook()` and `PySys_AddAuditHook()` for information on handling these events.

CPython implementation detail: This table is generated from the CPython documentation, and may not represent events raised by other implementations. See your runtime specific documentation for actual events raised.

Audit event	Arguments	References
<code>_thread.start_new_thread</code>	<code>function</code> <code>args</code> <code>kwargs</code> [1]	<code>array.__new__</code> <code>typecode</code> <code>initializer</code> [1] <code>builtins.breakpoint</code> <code>breakpointhook</code> [1] <code>builtins.id</code> <code>id</code> [1] <code>builtins.input</code> <code>prompt</code> [1] <code>builtins.input/result</code> <code>result</code> [1] <code>code.__new__</code> <code>code</code> <code>filename</code> <code>name</code> <code>argcount</code> <code>posonlyargcount</code> <code>kwnonlyargcount</code> <code>nlocals</code> <code>stacksize</code> <code>flags</code> [1] <code>compile</code> <code>source</code> <code>filename</code> [1] <code>cpython.PyConfig_Set</code> <code>name</code> <code>value</code> [1] <code>cpython.PyInterpreterState_Clear</code> [1] <code>cpython.PyInterpreterState_New</code> [1] <code>cpython._PySys_ClearAuditHooks</code> [1] <code>cpython.remote_debugger_script</code> <code>script_path</code> [1] <code>cpython.run_command</code> <code>command</code> [1] <code>cpython.run_file</code> <code>filename</code> [1] <code>cpython.run_i</code>

The following events are raised internally and do not correspond to any public API of CPython:

Audit event	Arguments
<code>_winapi.CreateFile</code>	<code>file_name</code> <code>desired_access</code> <code>share_mode</code> <code>creation_disposition</code> <code>flags_and_attributes</code>
<code>_winapi.CreateJunction</code>	<code>src_path</code> <code>dst_path</code>
<code>_winapi.CreateNamedPipe</code>	<code>name</code> <code>open_mode</code> <code>pipe_mode</code>
<code>_winapi.CreatePipe</code>	<code>application_name</code> <code>command_line</code> <code>current_directory</code>
<code>_winapi.CreateProcess</code>	<code>process_id</code> <code>desired_access</code>
<code>_winapi.TerminateProcess</code>	<code>handle</code> <code>exit_code</code>
<code>_posixsubprocess.fork_exec</code>	<code>exec_list</code> <code>args</code> <code>env</code>

Added in version 3.14: `The _posixsubprocess.fork_exec` internal audit event.

bdb— Debugger framework

Source code: `Lib/bdb.py`

The `bdb` module handles basic debugger functions, like setting breakpoints or managing execution via the debugger.

The following exception is defined:

exception `bdb.BdbQuit`

Exception raised by the `Bdb` class for quitting the debugger.

The `bdb` module also defines two classes:

`class bdb.Breakpoint(self, file, line, temporary=False, cond=None, funcname=None)`

`deleteMe()`

`enable()`

`disable()`

`bpformat()`

`bpprint(out=None)`

`file`

`line`

`temporary`

`cond`

`funcname`

`enabled`

`bpbynumber`

`bplist`

`ignore`

`hits`

This class implements temporary breakpoints, ignore counts, disabling and (re-)enabling, and conditionals. Breakpoints are indexed by number through a list called `bpbynumber` and by `(file, line)` pairs through `bplist`. The former points to a single instance of class `Breakpoint`. The latter points to a list of such instances since there may be more than one breakpoint per line. When creating a breakpoint, its associated filename should be in canonical form. If a `funcname` is defined, a breakpoint hit will be coun

Delete the breakpoint from the list associated to a file/line. If it is the last breakpoint in that position, it also deletes the entry for the file/line.

Mark the breakpoint as enabled.

Mark the breakpoint as disabled.

Return a string with all the information about the breakpoint, nicely formatted: Breakpoint number. Temporary status (del or keep). File/line position. Break condition. Number of times to ignore. Number of times hit. Added in version 3.2.

Print the output of `bpformat()` to the `fileout`, or if it is `None`, to standard output.

File name of theBreakpoint.

Line number of theBreakpointwithinfile.

Trueif aBreakpointat (file, line) is temporary.

Condition for evaluating aBreakpointat (file, line).

Function name that defines whether aBreakpointis hit upon entering the function.

TrueifBreakpointis enabled.

Numeric index for a single instance of aBreakpoint.

Dictionary ofBreakpointinstances indexed by (file,line) tuples.

Number of times to ignore aBreakpoint.

Count of the number of times aBreakpointhas been hit.

classbdb.Bdb(skip=None,backend='settrace')

canonic(filename)

start_trace(self)

stop_trace(self)

reset()

trace_dispatch(frame,event,arg)

dispatch_line(frame)

dispatch_call(frame,arg)

dispatch_return(frame,arg)

dispatch_exception(frame,arg)

is_skipped_module(module_name)

stop_here(frame)

break_here(frame)

break_anywhere(frame)

user_call(frame,argument_list)

user_line(frame)

user_return(frame,return_value)

user_exception(frame,exc_info)

do_clear(arg)

set_step()

set_next(frame)

set_return(frame)

set_until(frame,lineno=None)

set_trace([frame])

set_continue()

set_quit()

```

set_break(filename,lineno,temporary=False,cond=None,funcname=None)
clear_break(filename,lineno)
clear_bpbynumber(arg)
clear_all_file_breaks(filename)
clear_all_breaks()
get_bpbynumber(arg)
get_break(filename,lineno)
get_breaks(filename,lineno)
get_file_breaks(filename)
get_all_breaks()
disable_current_event()
restart_events()
get_stack(f,t)
format_stack_entry(frame_lineno,lprefix=':')
run(cmd,globals=None,locals=None)
runeval(expr,globals=None,locals=None)
runctx(cmd,globals,locals)
runcall(func,/,*args,**kwds)

```

The `Bdb` class acts as a generic Python debugger base class. This class takes care of the details of the trace facility; a derived class should implement user interaction. The standard debugger class (`pdb.Pdb`) is an example. The `skip` argument, if given, must be an iterable of glob-style module name patterns. The debugger will not step into frames that originate in a module that matches one of these patterns. Whether a frame is considered to originate in a certain module is determined by the `__name__` in

Return canonical form of `filename`. For real file names, the canonical form is an operating-system-dependent, case-normalized absolute path. A filename with angle brackets, such as `<stdin>` generated in interactive mode, is returned unchanged.

Start tracing. For 'settrace' backend, this method is equivalent to `sys.settrace(self.trace_dispatch)`. Added in version 3.14.

Stop tracing. For 'settrace' backend, this method is equivalent to `sys.settrace(None)`. Added in version 3.14.

Set the `botframe`, `stopframe`, `returnframe` and `quitting` attributes with values ready to start debugging.

This function is installed as the trace function of debugged frames. Its return value is the new trace function (in most cases, that is, itself). The default implementation decides how to dispatch a frame, depending on the type of event (passed as a string) that is about to be executed. `event` can be one of the following: "line": A new line of code is going to be executed. "call": A function is about to be called, or another code block entered. "return": A function or other code block is about to return.

If the debugger should stop on the current line, invoke the `user_line()` method (which should be overridden in subclasses). Raise `ABdbQuit` exception if the `quitting` flag is set (which can be set from `user_line()`). Return a reference to the `trace_dispatch()` method for further tracing in that scope.

If the debugger should stop on this function call, invoke the `user_call()` method (which should be overridden in subclasses). Raise `ABdbQuit` exception if the `quitting` flag is set (which can be set from `user_call()`). Return a

reference to `thetrace_dispatch()` method for further tracing in that scope.

If the debugger should stop on this function return, invoke `thetrace_return()` method (which should be overridden in subclasses). Raise `ABdbQuitException` if `thetrace_quitflag` is set (which can be set from `thetrace_return()`). Return a reference to `thetrace_dispatch()` method for further tracing in that scope.

If the debugger should stop at this exception, invokes `thetrace_exception()` method (which should be overridden in subclasses). Raise `ABdbQuitException` if `thetrace_quitflag` is set (which can be set from `thetrace_exception()`). Return a reference to `thetrace_dispatch()` method for further tracing in that scope.

Return `True` if `module_name` matches any skip pattern.

Return `True` if `frame` is below the starting frame in the stack.

Return `True` if there is an effective breakpoint for this line. Check whether a line or function breakpoint exists and is in effect. Delete temporary breakpoints based on information from `effective()`.

Return `True` if any breakpoint exists for `frame`'s filename.

Called from `dispatch_call()` if a break might stop inside the called function. `argument_list` is not used anymore and will always be `None`. The argument is kept for backwards compatibility.

Called from `dispatch_line()` when either `stop_here()` or `break_here()` returns `True`.

Called from `dispatch_return()` when `stop_here()` returns `True`.

Called from `dispatch_exception()` when `stop_here()` returns `True`.

Handle how a breakpoint must be removed when it is a temporary one. This method must be implemented by derived classes.

Stop after one line of code.

Stop on the next line in or below the given frame.

Stop when returning from the given frame.

Stop when the line with the `lineno` greater than the current one is reached or when returning from current frame.

Start debugging from `frame`. If `frame` is not specified, debugging starts from caller's frame. Changed in version 3.13: `set_trace()` will enter the debugger immediately, rather than on the next line of code to be executed.

Stop only at breakpoints or when finished. If there are no breakpoints, set the system trace function to `None`.

Set the `quitting` attribute to `True`. This raises `BdbQuit` in the next call to one of the `dispatch_*` methods.

Set a new breakpoint. If the `lineno` doesn't exist for the `filename` passed as argument, return an error message. The `filename` should be in canonical form, as described in the `canonic()` method.

Delete the breakpoints in `filename` and `lineno`. If none were set, return an error message.

Delete the breakpoint which has the `index` in the `Breakpoint.bp` by `number`. If `arg` is not numeric or out of range, return an error message.

Delete all breakpoints in `filename`. If none were set, return an error message.

Delete all existing breakpoints. If none were set, return an error message.

Return a breakpoint specified by the given number. If `arg` is a string, it will be converted to a number. If `arg` is a non-numeric string, if the given breakpoint never existed or has been deleted, a `ValueError` is raised. Added in version 3.2.

Return `True` if there is a breakpoint for `lineno` in `filename`.

Return all breakpoints for `lineno` in `filename`, or an empty list if none are set.

Return all breakpoints in filename, or an empty list if none are set.

Return all breakpoints that are set.

Disable the current event until the next `timerestart_events()` is called. This is helpful when the debugger is not interested in the current line. Added in version 3.14.

Restart all the disabled events. This function is automatically called in `dispatch_*` methods after `user_*` methods are called. If the `dispatch_*` methods are not overridden, the disabled events will be restarted after each user interaction. Added in version 3.14.

Return a list of (frame, lineno) tuples in a stack trace, and a size. The most recently called frame is last in the list. The size is the number of frames below the frame where the debugger was invoked.

Return a string with information about a stack entry, which is a (frame, lineno) tuple. The return string contains: The canonical filename which contains the frame. The function name or "<lambda>". The input arguments. The return value. The line of code (if it exists).

Debug a statement executed via the `exec()` function. `globals` defaults to `__main__.__dict__`, `locals` defaults to `globals`.

Debug an expression executed via the `eval()` function. `globals` and `local` have the same meaning as in `run()`.

For backwards compatibility. Calls the `run()` method.

Debug a single function call, and return its result.

Finally, the module defines the following functions:

bdb.checkfuncname(b, frame)

Return `True` if we should break here, depending on the way the breakpoint was set. If it was set via line number, it checks if `b.line` is the same as the one in `frame`. If the breakpoint was set via function name, we have to check we are in the right `frame` (the right function) and if we are on its first executable line.

bdb.effective(file, line, frame)

Return (active breakpoint, delete temporary flag) or (None, None) as the breakpoint to act upon. The active breakpoint is the first entry in `bplist` for the (file, line) (which must exist) that is enabled, for which `checkfuncname()` is true, and that has neither a false condition nor positive ignore count. The flag, meaning that a temporary breakpoint should be deleted, is `False` only when the condition cannot be evaluated (in which case, ignore count is ignored). If no such entry exists, then (None, None) is returned.

bdb.set_trace()

Start debugging with a `Bdb` instance from caller's frame.

faulthandler— Dump the Python traceback

Added in version 3.3.

This module contains functions to dump Python tracebacks explicitly, on a fault, after a timeout, or on a user signal. Call `faulthandler.enable()` to install fault handlers for the `SIGSEGV`, `SIGFPE`, `SIGABRT`, `SIGBUS`, and `SIGILL` signals. You can also enable them at startup by setting the `PYTHONFAULTHANDLER` environment variable or by using the `-Xfaulthandler` command line option.

The fault handler is compatible with system fault handlers like Apport or the Windows fault handler. The module uses an alternative stack for signal handlers if the `sigaltstack()` function is available. This allows it to dump the traceback even on a stack overflow.

The fault handler is called on catastrophic cases and therefore can only use signal-safe functions (e.g. it cannot allocate memory on the heap). Because of this limitation traceback dumping is minimal compared to normal Python tracebacks:

- Only ASCII is supported. The `backslashreplace` error handler is used on encoding.
- Each string is limited to 500 characters.
- Only the filename, the function name and the line number are displayed. (no source code)
- It is limited to 100 frames and 100 threads.
- The order is reversed: the most recent call is shown first.

By default, the Python traceback is written to `sys.stderr`. To see tracebacks, applications must be run in the terminal. A log file can alternatively be passed to `faulthandler.enable()`.

The module is implemented in C, so tracebacks can be dumped on a crash or when Python is deadlocked.

The Python Development Mode calls `faulthandler.enable()` at Python startup.

See also

Module `pdb`

Module `traceback`

Interactive source code debugger for Python programs.

Standard interface to extract, format and print stack traces of Python programs.

Dumping the traceback

`faulthandler.dump_traceback(file=sys.stderr, all_threads=True)`

Dump the tracebacks of all threads into `file`. If `all_threads` is `False`, dump only the current thread. See also `traceback.print_tb()`, which can be used to print a traceback object. Changed in version 3.5: Added support for passing file descriptor to this function.

Dumping the C stack

Added in version 3.14.

`faulthandler.dump_c_stack(file=sys.stderr)`

Dump the C stack trace of the current thread into file. If the Python build does not support it or the operating system does not provide a stack trace, then this prints an error in place of a dumped C stack.

C Stack Compatibility

If the system does not support the C-level `backtrace(3)` or `laddr1(3)`, then C stack dumps will not work. An error will be printed instead of the stack.

Additionally, some compilers do not support CPython's implementation of C stack dumps. As a result, a different error may be printed instead of the stack, even if the operating system supports dumping stacks.

■ Nota: Note Dumping C stacks can be arbitrarily slow, depending on the DWARF level of the binaries in the call stack.

Fault handler state

`faulthandler.enable(file=sys.stderr, all_threads=True, c_stack=True)`

Enable the fault handler: install handlers for the `SIGSEGV`, `SIGFPE`, `SIGABRT`, `SIGBUS` and `SIGILL` signals to dump the Python traceback. If `all_threads` is `True`, produce tracebacks for every running thread. Otherwise, dump only the current thread. The file must be kept open until the fault handler is disabled: see issue with file descriptors. If `c_stack` is `True`, then the C stack trace is printed after the Python traceback, unless the system does not support it. See `dump_c_stack()` for more information on compatibility. Ch

`faulthandler.disable()`

Disable the fault handler: uninstall the signal handlers installed by `enable()`.

`faulthandler.is_enabled()`

Check if the fault handler is enabled.

Dumping the tracebacks after a timeout

`faulthandler.dump_traceback_later(timeout, repeat=False, file=sys.stderr, exit=False)`

Dump the tracebacks of all threads, after a timeout of `timeout` seconds, or every `timeout` seconds if `repeat` is `True`. If `exit` is `True`, call `_exit()` with `status=1` after dumping the tracebacks. (Note `_exit()` exits the process immediately, which means it doesn't do any cleanup like flushing file buffers.) If the function is called twice, the new call replaces previous parameters and resets the timeout. The timer has a sub-second resolution. The file must be kept open until the traceback is dumped or `cancel_dump_traceback_later()`

`faulthandler.cancel_dump_traceback_later()`

Cancel the last call to `dump_traceback_later()`.

Dumping the traceback on a user signal

`faulthandler.register(signum, file=sys.stderr, all_threads=True, chain=False)`

Register a user signal: install a handler for the `signum` signal to dump the traceback of all threads, or of the current thread if `all_threads` is `False`, into file. Call the previous handler if `chain` is `True`. The file must be kept open until the signal is unregistered by `unregister()`: see issue with file descriptors. Not available on Windows. Changed in version 3.5: Added support for passing file descriptor to this function.

`faulthandler.unregister(signum)`

Unregister a user signal: `uninstall the handler of the signal installed by register()`. Return `True` if the signal was registered, `False` otherwise. Not available on Windows.

Issue with file descriptors

`enable()`, `dump_traceback_later()` and `register()` keep the file descriptor of their `file` argument. If the file is closed and its file descriptor is reused by a new file, or `ios.dup2()` is used to replace the file descriptor, the traceback will be written into a different file. Call these functions again each time that the file is replaced.

Example

Example of a segmentation fault on Linux with and without enabling the fault handler:

```
$ python -c "import ctypes; ctypes.string_at(0)"
Segmentation fault
```

```
$ python -q -X faulthandler
>>> import ctypes
>>> ctypes.string_at(0)
Fatal Python error: Segmentation fault
```

```
Current thread 0x00007fb899f39700 (most recent call first):
  File "/opt/python/Lib/ctypes/__init__.py", line 486 in string_at
  File "<stdin>", line 1 in <module>
```

```
Current thread's C stack trace (most recent call first):
  Binary file "/opt/python/python", at _Py_DumpStack+0x42 [0x5b27f7d7147e]
  Binary file "/opt/python/python", at +0x32dcdb [0x5b27f7d85cbd]
  Binary file "/opt/python/python", at +0x32df8a [0x5b27f7d85f8a]
  Binary file "/usr/lib/libc.so.6", at +0x3def0 [0x77b73226bef0]
  Binary file "/usr/lib/libc.so.6", at +0x17ef9c [0x77b7323acf9c]
  Binary file "/opt/python/build/lib.linux-x86_64-3.14/_ctypes.cpython-314d-x86_64-linux-gnu.so", at +0xcdcf6 [0x5b27f7d85f8a]
  Binary file "/usr/lib/libffi.so.8", at +0x7976 [0x77b73158f976]
  Binary file "/usr/lib/libffi.so.8", at +0x413c [0x77b73158c13c]
  Binary file "/usr/lib/libffi.so.8", at ffi_call+0x12e [0x77b73158ef0e]
  Binary file "/opt/python/build/lib.linux-x86_64-3.14/_ctypes.cpython-314d-x86_64-linux-gnu.so", at +0x15a33 [0x5b27f7d85f8a]
  Binary file "/opt/python/build/lib.linux-x86_64-3.14/_ctypes.cpython-314d-x86_64-linux-gnu.so", at +0x164fa [0x5b27f7d85f8a]
  Binary file "/opt/python/build/lib.linux-x86_64-3.14/_ctypes.cpython-314d-x86_64-linux-gnu.so", at +0xc624 [0x5b27f7d85f8a]
  Binary file "/opt/python/python", at _PyObject_MakeTpCall+0xce [0x5b27f7b73883]
  Binary file "/opt/python/python", at +0x11bab6 [0x5b27f7b73ab6]
  Binary file "/opt/python/python", at PyObject_Vectorcall+0x23 [0x5b27f7b73b04]
  Binary file "/opt/python/python", at _PyEval_EvalFrameDefault+0x490c [0x5b27f7cbb302]
  Binary file "/opt/python/python", at +0x2818e6 [0x5b27f7cd98e6]
  Binary file "/opt/python/python", at +0x281aab [0x5b27f7cd9aab]
  Binary file "/opt/python/python", at PyEval_EvalCode+0xc5 [0x5b27f7cd9ba3]
  Binary file "/opt/python/python", at +0x255957 [0x5b27f7cad957]
  Binary file "/opt/python/python", at +0x255ab4 [0x5b27f7cadab4]
  Binary file "/opt/python/python", at _PyEval_EvalFrameDefault+0x6c3e [0x5b27f7cbd634]
  Binary file "/opt/python/python", at +0x2818e6 [0x5b27f7cd98e6]
  Binary file "/opt/python/python", at +0x281aab [0x5b27f7cd9aab]
  Binary file "/opt/python/python", at +0x11b6e1 [0x5b27f7b736e1]
  Binary file "/opt/python/python", at +0x11d348 [0x5b27f7b75348]
  Binary file "/opt/python/python", at +0x11d626 [0x5b27f7b75626]
  Binary file "/opt/python/python", at PyObject_Call+0x20 [0x5b27f7b7565e]
  Binary file "/opt/python/python", at +0x32a67a [0x5b27f7d8267a]
  Binary file "/opt/python/python", at +0x32a7f8 [0x5b27f7d827f8]
  Binary file "/opt/python/python", at +0x32ac1b [0x5b27f7d82c1b]
  Binary file "/opt/python/python", at Py_RunMain+0x31 [0x5b27f7d82ebe]
  <truncated rest of calls>
Segmentation fault
```

pdb— The Python Debugger

Source code: `Lib/pdb.py`

The module `pdb` defines an interactive source code debugger for Python programs. It supports setting (conditional) breakpoints and single stepping at the source line level, inspection of stack frames, source code listing, and evaluation of arbitrary Python code in the context of any stack frame. It also supports post-mortem debugging and can be called under program control.

The debugger is extensible – it is actually defined as the class `Pdb`. This is currently undocumented but easily understood by reading the source. The extension interface uses the modules `bdb` and `cmd`.

See also

Module `faulthandler`

Module `traceback`

Used to dump Python tracebacks explicitly, on a fault, after a timeout, or on a user signal.

Standard interface to extract, format and print stack traces of Python programs.

The typical usage to break into the debugger is to insert:

```
import pdb; pdb.set_trace()
```

Or:

```
breakpoint()
```

at the location you want to break into the debugger, and then run the program. You can then step through the code following this statement, and continue running without the debugger using the `continue` command.

Changed in version 3.7: The built-in `breakpoint()`, when called with defaults, can be used instead of `import pdb; pdb.set_trace()`.

```
def double(x):
    breakpoint()
    return x * 2
val = 3
print(f"{val} * 2 is {double(val)}")
```

The debugger's prompt is `(Pdb)`, which is the indicator that you are in debug mode:

```
> ... (2) double()
-> breakpoint()
(Pdb) p x
3
(Pdb) continue
3 * 2 is 6
```

Changed in version 3.3: Tab-completion via the `readline` module is available for commands and command arguments, e.g. the current global and local names are offered as arguments of the `p` command.

Command-line interface

You can also invoke `pdb` from the command line to debug other scripts. For example:

```
python -m pdb [-c command] (-m module | -p pid | pyfile) [args ...]
```

When invoked as a module, `pdb` will automatically enter post-mortem debugging if the program being debugged exits abnormally. After post-mortem debugging (or after normal exit of the program), `pdb` will restart the

program. Automatic restarting preserves pdb's state (such as breakpoints) and in most cases is more useful than quitting the debugger upon program's exit.

-c,--command<command>

To execute commands as if given in a.pdbrcfile; seeDebugger commands.Changed in version 3.2:Added the-coption.

-m<module>

To execute modules similar to the waypython-mdoes. As with a script, the debugger will pause execution just before the first line of the module.Changed in version 3.7:Added the-moption.

-p,--pid<pid>

Attach to the process with the specified PID.Added in version 3.14.

To attach to a running Python process for remote debugging, use the-por--pidoption with the target process's PID:

```
python -m pdb -p 1234
```

■ Nota: NoteAttaching to a process that is blocked in a system call or waiting for I/O will only work once the next bytecode instruction is executed or when the process receives a signal.

Typical usage to execute a statement under control of the debugger is:

```
>>> import pdb
>>> def f(x):
...     print(1 / x)
>>> pdb.run("f(2)")
> <string>(1)<module>()
(Pdb) continue
0.5
>>>
```

The typical usage to inspect a crashed program is:

```
>>> import pdb
>>> def f(x):
...     print(1 / x)
...
>>> f(0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in f
ZeroDivisionError: division by zero
>>> pdb.pm()
> <stdin>(2)f()
(Pdb) p x
0
(Pdb)
```

Changed in version 3.13:The implementation ofPEP 667means that name assignments made viapdbwill immediately affect the active scope, even when running inside anoptimized scope.

The module defines the following functions; each enters the debugger in a slightly different way:

pdb.run(statement, globals=None, locals=None)

Execute thestatement(given as a string or a code object) under debugger control. The debugger prompt appears before any code is executed; you can set breakpoints and typecontinue, or you can step through the statement usingstepornext(all these commands are explained below). The optionalglobalsandlocalsarguments specify the environment in which the code is executed; by default the dictionary of the module__main__is used. (See the explanation of the built-inexec()oreval()functions.)

pdb.runeval(expression, globals=None, locals=None)

Evaluate the expression (given as a string or a code object) under debugger control. When `runeval()` returns, it returns the value of the expression. Otherwise this function is similar to `run()`.

pdb.runcall(function, *args, **kwargs)

Call the function (a function or method object, not a string) with the given arguments. When `runcall()` returns, it returns whatever the function call returned. The debugger prompt appears as soon as the function is entered.

pdb.set_trace(*, header=None, commands=None)

Enter the debugger at the calling stack frame. This is useful to hard-code a breakpoint at a given point in a program, even if the code is not otherwise being debugged (e.g. when an assertion fails). If given, `header` is printed to the console just before debugging begins. The `commands` argument, if given, is a list of commands to execute when the debugger starts. Changed in version 3.7: The keyword-only argument `header`. Changed in version 3.13: `set_trace()` will enter the debugger immediately, rather than

awaitable pdb.set_trace_async(*, header=None, commands=None)

async version of `set_trace()`. This function should be used inside an async function with `await`. `asyncio.run()`: `await pdb.set_trace_async()` `await` statements are supported if the debugger is invoked by this function. Added in version 3.14.

pdb.post_mortem(t=None)

Enter post-mortem debugging of the given exception or `traceback` object. If no value is given, it uses the exception that is currently being handled, or raises `ValueError` if there isn't one. Changed in version 3.13: Support for exception objects was added.

pdb.pm()

Enter post-mortem debugging of the exception found in `sys.last_exc`.

pdb.set_default_backend(backend)

There are two supported backends for `pdb`: 'settrace' and 'monitoring'. See `bdb.Bdb` for details. The user can set the default backend to use if none is specified when instantiating `Pdb`. If no backend is specified, the default is 'settrace'. Note `breakpoint()` and `set_trace()` will not be affected by this function. They always use 'monitoring' backend. Added in version 3.14.

pdb.get_default_backend()

Returns the default backend for `pdb`. Added in version 3.14.

The `run()` functions and `set_trace()` are aliases for instantiating the `Pdb` class and calling the method of the same name. If you want to access further features, you have to do this yourself:

```
class Pdb(completekey='tab', stdin=None, stdout=None, skip=None, nosigint=False, readrc=True, mode=None, backend=None, colorize=False)
```

```
run(statement, globals=None, locals=None)
```

```
runeval(expression, globals=None, locals=None)
```

```
runcall(function, *args, **kwargs)
```

```
set_trace()
```

`Pdb` is the debugger class. The `completekey`, `stdin` and `stdout` arguments are passed to the underlying `cmd.Cmd` class; see the description there. The `skip` argument, if given, must be an iterable of glob-style module name patterns. The debugger will not step into frames that originate in a module that matches one of these patterns.[1] By default, `Pdb` sets a handler for the SIGINT signal (which is sent when the user

presses Ctrl-C on the console) when you give a `continue` command. This allows you to break into the deb

See the documentation for the functions explained above.

Debugger commands

The commands recognized by the debugger are listed below. Most commands can be abbreviated to one or two letters as indicated; e.g. `h(elp)` means that either `h` or `help` can be used to enter the help command (but not `he` or `hel`, nor `h` or `help` or `HELP`). Arguments to commands must be separated by whitespace (spaces or tabs). Optional arguments are enclosed in square brackets (`[]`) in the command syntax; the square brackets must not be typed. Alternatives in the command syntax are separated by a vertical bar (`|`).

Entering a blank line repeats the last command entered. Exception: if the last command was a `list` command, the next 11 lines are listed.

Commands that the debugger doesn't recognize are assumed to be Python statements and are executed in the context of the program being debugged. Python statements can also be prefixed with an exclamation point (`!`). This is a powerful way to inspect the program being debugged; it is even possible to change a variable or call a function. When an exception occurs in such a statement, the exception name is printed but the debugger's state is not changed.

Changed in version 3.13: Expressions/Statements whose prefix is a `pdb` command are now correctly identified and executed.

The debugger supports aliases. Aliases can have parameters which allows one a certain level of adaptability to the context under examination.

Multiple commands may be entered on a single line, separated by `;;`. (A single `;` is not used as it is the separator for multiple commands in a line that is passed to the Python parser.) No intelligence is applied to separating the commands; the input is split at the first `;;` pair, even if it is in the middle of a quoted string. A workaround for strings with double semicolons is to use implicit string concatenation `'";'or'";'";'`.

To set a temporary global variable, use a convenience variable. A convenience variable is a variable whose name starts with `$`. For example, `$foo=1` sets a global variable `$foo` which you can use in the debugger session.

The convenience variables are cleared when the program resumes execution so it's less likely to interfere with your program compared to using normal variables like `foo=1`.

There are four preset convenience variables:

- `$_frame`: the current frame you are debugging
- `$_retval`: the return value if the frame is returning
- `$_exception`: the exception if the frame is raising an exception
- `$_asynctask`: the asyncio task if `pdb` stops in an async function

Added in version 3.12: Added the convenience variable feature.

Added in version 3.14: Added the `$_asynctask` convenience variable.

If a file `.pdbrc` exists in the user's home directory or in the current directory, it is read with 'utf-8' encoding and executed as if it had been typed at the debugger prompt, with the exception that empty lines and lines starting with `#` are ignored. This is particularly useful for aliases. If both files exist, the one in the home directory is read first and aliases defined there can be overridden by the local file.

Changed in version 3.2: `.pdbrc` can now contain commands that continue debugging, such as `continue` or `next`. Previously, these commands had no effect.

Changed in version 3.11: `.pdbrc` is now read with 'utf-8' encoding. Previously, it was read with the system locale encoding.

`h(elp)[command]`

Without argument, print the list of available commands. With a `command` argument, print help about that command. `help.pdb` displays the full documentation (the `docstring` of the `pdb` module). Since the `command` argument must be an identifier, `help` must be entered to get help on the `!command`.

`w(here)[count]`

Print a stack trace, with the most recent frame at the bottom. If `count` is 0, print the current frame entry. If `count` is negative, print the least recent `-count` frames. If `count` is positive, print the most recent `count` frames. An arrow (`>`) indicates the current frame, which determines the context of most commands. Changed in version 3.14: `count` argument is added.

`d(own)[count]`

Move the current frame `count` (default one) levels down in the stack trace (to a newer frame).

`u(p)[count]`

Move the current frame `count` (default one) levels up in the stack trace (to an older frame).

`b(reak)[([filename:]lineno|function)[,condition]]`

With a `lineno` argument, set a break at `lineno` in the current file. The line number may be prefixed with a `filename` and a colon, to specify a breakpoint in another file (possibly one that hasn't been loaded yet). The file is searched on `sys.path`. Acceptable forms of `filename` are `abspath/to/file.py`, `relpath/file.py`, `module` and `package.module`. With a `function` argument, set a break at the first executable statement within that function. `function` can be any expression that evaluates to a function in the current

`tbreak([([filename:]lineno|function)[,condition])]`

Temporary breakpoint, which is removed automatically when it is first hit. The arguments are the same as for `break`.

`cl(ear)[filename:lineno|bpnumber...]`

With a `filename:lineno` argument, clear all the breakpoints at this line. With a space separated list of breakpoint numbers, clear those breakpoints. Without argument, clear all breaks (but first ask confirmation).

`disablebpnumber[bpnumber...]`

Disable the breakpoints given as a space separated list of breakpoint numbers. Disabling a breakpoint means it cannot cause the program to stop execution, but unlike clearing a breakpoint, it remains in the list of breakpoints and can be (re-)enabled.

`enablebpnumber[bpnumber...]`

Enable the breakpoints specified.

`ignorebpnumber[count]`

Set the ignore count for the given breakpoint number. If `count` is omitted, the ignore count is set to 0. A breakpoint becomes active when the ignore count is zero. When non-zero, the `count` is decremented each time the breakpoint is reached and the breakpoint is not disabled and any associated condition evaluates to true.

`conditionbpnumber[condition]`

Set a new `condition` for the breakpoint, an expression which must evaluate to true before the breakpoint is honored. If `condition` is absent, any existing condition is removed; i.e., the breakpoint is made unconditional.

commands[bpnumber]

Specify a list of commands for breakpoint number `bpnumber`. The commands themselves appear on the following lines. Type a line containing `justend` to terminate the commands. An example: `(Pdb)commands1(com)psome_variable(com)end(Pdb)` To remove all commands from a breakpoint, type `commands` and follow it immediately with `end`; that is, give no commands. With no `bpnumber` argument, `commands` refers to the last breakpoint set. You can use breakpoint commands to start your program up again. Simply use `thecontinuecom`

s(step)

Execute the current line, stop at the first possible occasion (either in a function that is called or on the next line in the current function).

n(ext)

Continue execution until the next line in the current function is reached or it returns. (The difference between `next` and `step` is that `step` stops inside a called function, while `next` executes called functions at (nearly) full speed, only stopping at the next line in the current function.)

unt(il)[lineno]

Without argument, continue execution until the line with a number greater than the current one is reached. With `lineno`, continue execution until a line with a number greater or equal to `lineno` is reached. In both cases, also stop when the current frame returns. Changed in version 3.2: Allow giving an explicit line number.

r(eturn)

Continue execution until the current function returns.

c(ontinue)

Continue execution, only stop when a breakpoint is encountered.

j(ump)lineno

Set the next line that will be executed. Only available in the bottom-most frame. This lets you jump back and execute code again, or jump forward to skip code that you don't want to run. It should be noted that not all jumps are allowed – for instance it is not possible to jump into the middle of a `for` loop or out of a `finally` clause.

l(ist)[first[,last]]

List source code for the current file. Without arguments, list 11 lines around the current line or continue the previous listing. With `as` argument, list 11 lines around the current line. With one argument, list 11 lines around at that line. With two arguments, list the given range; if the second argument is less than the first, it is interpreted as a count. The current line in the current frame is indicated by `>`. If an exception is being debugged, the line where the exception was originally

ll|longlist

List all source code for the current function or frame. Interesting lines are marked as `forlist`. Added in version 3.2.

a(rgs)

Print the arguments of the current function and their current values.

pexpression

Evaluate `expression` in the current context and print its value. Note `print()` can also be used, but is not a debugger command — this executes the Python `print()` function.

ppexpression

Like the `pprint` command, except the value of `expression` is pretty-printed using the `pprint` module.

what is expression

Print the type of `expression`.

source expression

Try to get source code of `expression` and display it. Added in version 3.2.

display[expression]

Display the value of `expression` if it changed, each time execution stops in the current frame. Without `expression`, list all display expressions for the current frame. Note `Display` evaluates `expression` and compares to the result of the previous evaluation of `expression`, so when the result is mutable, display may not be able to pick up the changes. Example: `lst=[]; breakpoint(); pass; lst.append(1); print(lst)`. `Display` won't realize `lst` has been changed because the result of evaluation is modified in place by `lst.append()`.

undisplay[expression]

Do not display `expression` anymore in the current frame. Without `expression`, clear all display expressions for the current frame. Added in version 3.2.

interact

Start an interactive interpreter (using the `code` module) in a new global namespace initialised from the local and global namespaces for the current scope. Use `exit()` or `quit()` to exit the interpreter and return to the debugger. Note `AsInteract` creates a new dedicated namespace for code execution, assignments to variables will not affect the original namespaces. However, modifications to any referenced mutable objects will be reflected in the original namespaces as usual. Added in version 3.2. Changed in version 3.3.

alias[name[command]]

Create an alias called `name` that executes `command`. The `command` must not be enclosed in quotes. Replaceable parameters can be indicated by `%1`, `%2`, ... and `%9`, while `%*` is replaced by all the parameters. If `command` is omitted, the current alias for `name` is shown. If no arguments are given, all aliases are listed. Aliases may be nested and can contain anything that can be legally typed at the `pdb` prompt. Note that internal `pdb` commands can be overridden by aliases. Such a command is then hidden until the alias is removed.

unalias name

Delete the specified alias `name`.

!statement

Execute the (one-line) `statement` in the context of the current stack frame. The exclamation point can be omitted unless the first word of the statement resembles a debugger command, e.g.: `(Pdb) ! n=42` (`Pdb`) To set a global variable, you can prefix the assignment command with `global` statement on the same line, e.g.: `(Pdb) global list_options; list_options = ['-l']` (`Pdb`)

run[args...]

restart[args...]

Restart the debugged Python program. If `args` is supplied, it is split with `shlex` and the result is used as the `sys.argv`. History, breakpoints, actions and debugger options are preserved. `restart` is an alias for `run`. Changed in version 3.14: `run` and `restart` commands are disabled when the debugger is invoked in 'inline' mode.

q(uit)

Quit from the debugger. The program being executed is aborted. An end-of-file input is equivalent to `quit`. A confirmation prompt will be shown if the debugger is invoked in 'inline' mode. Either `Y`, `<Enter>` or `EOF` will confirm.

the quit. Changed in version 3.14: A confirmation prompt will be shown if the debugger is invoked in 'inline' mode. After the confirmation, the debugger will call `sys.exit()` immediately, instead of raising `bdb.BdbQuit` in the next trace event.

debugcode

Enter a recursive debugger that steps through `code` (which is an arbitrary expression or statement to be executed in the current environment).

retval

Print the return value for the last return of the current function.

exceptions[excnumber]

List or jump between chained exceptions. When using `pdb.pm()` or `Pdb.post_mortem(...)` with a chained exception instead of a traceback, it allows the user to move between the chained exceptions using `exceptions` command to list exceptions, and `exceptions<number>` to switch to that exception. Example:

```
def out():
    try:
        middle()
    except Exception as e:
        raise ValueError("reraise middle() error")
from def middle():
    try:
        return inner(0)
    except Exception as e:
        raise ValueError("Middle fail")
def inner(x):
    1/x
out()
calling pdb.pm() will allow to
```

Footnotes

Whether a frame is considered to originate in a certain module is determined by the `__name__` in the frame `globals`.

The Python Profilers

Source code: `Lib/profile.py` and `Lib/pstats.py`

Introduction to the profilers

`cProfile` and `profile` provide deterministic profiling of Python programs. A profile is a set of statistics that describes how often and for how long various parts of the program executed. These statistics can be formatted into reports via the `pstats` module.

The Python standard library provides two different implementations of the same profiling interface:

- `cProfile` is recommended for most users; it's a C extension with reasonable overhead that makes it suitable for profiling long-running programs. Based on `sprof`, contributed by Brett Rosen and Ted Czotter.
- `profile`, a pure Python module whose interface is imitated by `cProfile`, but which adds significant overhead to profiled programs. If you're trying to extend the profiler in some way, the task might be easier with this module. Originally designed and written by Jim Roskind.

■ **Nota:** Note The profiler modules are designed to provide an execution profile for a given program, not for benchmarking purposes (for that, there is `timeit` for reasonably accurate results). This particularly applies to benchmarking Python code against C code: the profilers introduce overhead for Python code, but not for C-level functions, and so the C code would seem faster than any Python one.

Instant User's Manual

This section is provided for users that "don't want to read the manual." It provides a very brief overview, and allows a user to rapidly perform profiling on an existing application.

To profile a function that takes a single argument, you can do:

```
import cProfile
import re
cProfile.run('re.compile("foo|bar")')
```

(Use `profile` instead of `cProfile` if the latter is not available on your system.)

The above action would run `re.compile()` and print profile results like the following:

```
214 function calls (207 primitive calls) in 0.002 seconds
```

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	0.002	0.002	{built-in method builtins.exec}
1	0.000	0.000	0.001	0.001	<string>:1(<module>)
1	0.000	0.000	0.001	0.001	__init__.py:250(compile)
1	0.000	0.000	0.001	0.001	__init__.py:289(_compile)
1	0.000	0.000	0.000	0.000	_compiler.py:759(compile)
1	0.000	0.000	0.000	0.000	_parser.py:937(parse)
1	0.000	0.000	0.000	0.000	_compiler.py:598(_code)
1	0.000	0.000	0.000	0.000	_parser.py:435(_parse_sub)

The first line indicates that 214 calls were monitored. Of those calls, 207 were primitive, meaning that the call was not induced via recursion. The next line: `Ordered by: cumulative time` indicates the output is sorted by the `cumtime` values. The column headings include:

ncalls

tottime

percall

cumtime

percall

filename:lineno(function)

for the number of calls.

for the total time spent in the given function (and excluding time made in calls to sub-functions)

is the quotient of tottime divided by nccalls

is the cumulative time spent in this and all subfunctions (from invocation till exit). This figure is accurate even for recursive functions.

is the quotient of cumtime divided by primitive calls

provides the respective data of each function

When there are two numbers in the first column (for example 3/1), it means that the function recursed. The second value is the number of primitive calls and the former is the total number of calls. Note that when the function does not recurse, these two values are the same, and only the single figure is printed.

Instead of printing the output at the end of the profile run, you can save the results to a file by specifying a filename to the `run()` function:

```
import cProfile
import re
cProfile.run('re.compile("foo|bar")', 'restats')
```

The `pstats.Stats` class reads profile results from a file and formats them in various ways.

The files `cProfile` and `profile` can also be invoked as a script to profile another script. For example:

```
python -m cProfile [-o output_file] [-s sort_order] (-m module | myscript.py)
```

-o<output_file>

Writes the profile results to a file instead of to stdout.

-s<sort_order>

Specifies one of the `sort_stats()` sort values to sort the output by. This only applies when -o is not supplied.

-m<module>

Specifies that a module is being profiled instead of a script. Added in version 3.7: Added the -m option to `cProfile`. Added in version 3.8: Added the -m option to `profile`.

The `pstats` module's `Stats` class has a variety of methods for manipulating and printing the data saved into a profile results file:

```
import pstats
from pstats import SortKey
p = pstats.Stats('restats')
p.strip_dirs().sort_stats(-1).print_stats()
```

The `strip_dirs()` method removed the extraneous path from all the module names. The `sort_stats()` method sorted all the entries according to the standard module/line/name string that is printed. The `print_stats()` method printed out all the statistics. You might try the following sort calls:

```
p.sort_stats(SortKey.NAME)
p.print_stats()
```

The first call will actually sort the list by function name, and the second call will print out the statistics. The following are some interesting calls to experiment with:

```
p.sort_stats(SortKey.CUMULATIVE).print_stats(10)
```

This sorts the profile by cumulative time in a function, and then only prints the ten most significant lines. If you want to understand what algorithms are taking time, the above line is what you would use.

If you were looking to see what functions were looping a lot, and taking a lot of time, you would do:

```
p.sort_stats(SortKey.TIME).print_stats(10)
```

to sort according to time spent within each function, and then print the statistics for the top ten functions.

You might also try:

```
p.sort_stats(SortKey.FILENAME).print_stats('__init__')
```

This will sort all the statistics by file name, and then print out statistics for only the class init methods (since they are spelled with `__init__` in them). As one final example, you could try:

```
p.sort_stats(SortKey.TIME, SortKey.CUMULATIVE).print_stats(.5, 'init')
```

This line sorts statistics with a primary key of time, and a secondary key of cumulative time, and then prints out some of the statistics. To be specific, the list is first culled down to 50% (re:.5) of its original size, then only lines containing `init` are maintained, and that sub-sub-list is printed.

If you wondered what functions called the above functions, you could now (pis still sorted according to the last criteria) do:

```
p.print_callers(.5, 'init')
```

and you would get a list of callers for each of the listed functions.

If you want more functionality, you're going to have to read the manual, or guess what the following functions do:

```
p.print_callees()  
p.add('restats')
```

Invoked as a script, the `statsmodule` is a statistics browser for reading and examining profile dumps. It has a simple line-oriented interface (implemented using `cmd`) and interactive help.

profile and cProfile Module Reference

Both the `profile` and `cProfile` modules provide the following functions:

profile.run(command,filename=None,sort=-1)

This function takes a single argument that can be passed to the `exec()` function, and an optional file name. In all cases this routine executes: `exec(command,__main__.__dict__,__main__.__dict__)` and gathers profiling statistics from the execution. If no file name is present, then this function automatically creates a `Stats` instance and prints a simple profiling report. If the sort value is specified, it is passed to this `Stats` instance to control how the results are sorted.

profile.runctx(command,globals,locals,filename=None,sort=-1)

This function is similar to `run()`, with added arguments to supply the `globals` and `locals` mappings for the `command` string. This routine executes: `exec(command,globals,locals)` and gathers profiling statistics as in the `run()` function above.

class profile.Profile(timer=None,timeunit=0.0,subcalls=True,builtins=True)

enable()

disable()
create_stats()
print_stats(sort=-1)
dump_stats(filename)
run(cmd)
runcx(cmd,globals,locals)
runcall(func,/,*args,kwargs)**

This class is normally only used if more precise control over profiling is needed than what the `Profile.run()` function provides. A custom timer can be supplied for measuring how long code takes to run via the `timer` argument. This must be a function that returns a single number representing the current time. If the number is an integer, the `timeunits` specifies a multiplier that specifies the duration of each unit of time. For example, if the timer returns times measured in thousands of seconds, the time

Start collecting profiling data. Only `incProfile`.

Stop collecting profiling data. Only `incProfile`.

Stop collecting profiling data and record the results internally as the current profile.

Create a `Stats` object based on the current profile and print the results to `stdout`. The `sort` parameter specifies the sorting order of the displayed statistics. It accepts a single key or a tuple of keys to enable multi-level sorting, as in `Stats.sort_stats`. Added in version 3.13: `print_stats()` now accepts a tuple of keys.

Write the results of the current profile to `filename`.

Profile the `cmd` via `exec()`.

Profile the `cmd` via `exec()` with the specified global and local environment.

Profile `func(*args,**kwargs)`

Note that profiling will only work if the called command/function actually returns. If the interpreter is terminated (e.g. via `sys.exit()` call during the called command/function execution) no profiling results will be printed.

TheStatsClass

Analysis of the profiler data is done using the `Stats` class.

`class Stats(*filenamesorprofile,stream=sys.stdout)`

`strip_dirs()`
`add(*filenames)`
`dump_stats(filename)`
`sort_stats(*keys)`
`reverse_order()`
`print_stats(*restrictions)`
`print_callers(*restrictions)`
`print_callees(*restrictions)`
`get_stats_profile()`

This class constructor creates an instance of a “statistics object” from a filename (or list of filenames) or from a Profile instance. Output will be printed to the stream specified by stream. The file selected by the above constructor must have been created by the corresponding version of profile or cProfile. To be specific, there is no file compatibility guaranteed with future versions of this profiler, and there is no compatibility with files produced by other profilers, or the same profiler run on a d

This method for the Stats class removes all leading path information from file names. It is very useful in reducing the size of the printout to fit within (close to) 80 columns. This method modifies the object, and the stripped information is lost. After performing a strip operation, the object is considered to have its entries in a “random” order, as it was just after object initialization and loading. If strip_dirs() causes two function names to be indistinguishable (they are on the same line o

This method of the Stats class accumulates additional profiling information into the current profiling object. Its arguments should refer to filenames created by the corresponding version of profile.run() or cProfile.run(). Statistics for identically named (re: file, line, name) functions are automatically accumulated into single function statistics.

Save the data loaded into the Stats object to a file named filename. The file is created if it does not exist, and is overwritten if it already exists. This is equivalent to the method of the same name on the profile.Profile and cProfile.Profile classes.

This method modifies the Stats object by sorting it according to the supplied criteria. The argument can be either a string or a SortKey enum identifying the basis of a sort (example: 'time', 'name', SortKey.TIME or SortKey.NAME). The SortKey enums argument have advantage over the string argument in that it is more robust and less error prone. When more than one key is provided, then additional keys are used as secondary criteria when there is equality in all keys selected before them. For example, sor

This method for the Stats class reverses the ordering of the basic list within the object. Note that by default ascending vs descending order is properly selected based on the sort key of choice.

This method for the Stats class prints out a report as described in the profile.run() definition. The order of the printing is based on the last sort_stats() operation done on the object (subject to caveats in add() and strip_dirs()). The arguments provided (if any) can be used to limit the list down to the significant entries. Initially, the list is taken to be the complete set of profiled functions. Each restriction is either an integer (to select a count of lines), or a decimal fraction between 0.0 an

This method for the Stats class prints a list of all functions that called each function in the profiled database. The ordering is identical to that provided by print_stats(), and the definition of the restricting argument is also identical. Each caller is reported on its own line. The format differs slightly depending on the profiler that produced the stats: With profile, a number is shown in parentheses after each caller to show how many times this specific call was made. For convenience, a sec

This method for the Stats class prints a list of all function that were called by the indicated function. Aside from this reversal of direction of calls (re: called vs was called by), the arguments and ordering are identical to the print_callers() method.

This method returns an instance of StatsProfile, which contains a mapping of function names to instances of FunctionProfile. Each FunctionProfile instance holds information related to the function's profile such as how long the function took to run, how many times it was called, etc... Added in version 3.9: Added the following dataclasses: StatsProfile, FunctionProfile. Added the following function: get_stats_profile.

What Is Deterministic Profiling?

Deterministic profiling is meant to reflect the fact that all function call, function return, and exception events are monitored, and precise timings are made for the intervals between these events (during which time the user's code is executing). In contrast, statistical profiling (which is not done by this module) randomly samples the effective instruction pointer, and deduces where time is being spent. The latter technique traditionally involves less overhead (as the code does not need to be instrumented), but provides only relative indications of where time is being spent.

In Python, since there is an interpreter active during execution, the presence of instrumented code is not required in order to do deterministic profiling. Python automatically provides a hook (optional callback) for each event. In addition, the interpreted nature of Python tends to add so much overhead to execution, that deterministic profiling tends to only add small processing overhead in typical applications. The result is that deterministic profiling is not that expensive, yet provides extensive run time statistics about the execution of a Python program.

Call count statistics can be used to identify bugs in code (surprising counts), and to identify possible inline-expansion points (high call counts). Internal time statistics can be used to identify "hot loops" that should be carefully optimized. Cumulative time statistics should be used to identify high level errors in the selection of algorithms. Note that the unusual handling of cumulative times in this profiler allows statistics for recursive implementations of algorithms to be directly compared to iterative implementations.

Limitations

One limitation has to do with accuracy of timing information. There is a fundamental problem with deterministic profilers involving accuracy. The most obvious restriction is that the underlying "clock" is only ticking at a rate (typically) of about .001 seconds. Hence no measurements will be more accurate than the underlying clock. If enough measurements are taken, then the "error" will tend to average out. Unfortunately, removing this first error induces a second source of error.

The second problem is that it "takes a while" from when an event is dispatched until the profiler's call to get the time actually gets the state of the clock. Similarly, there is a certain lag when exiting the profiler event handler from the time that the clock's value was obtained (and then squirreled away), until the user's code is once again executing. As a result, functions that are called many times, or call many functions, will typically accumulate this error. The error that accumulates in this fashion is typically less than the accuracy of the clock (less than one clock tick), but it can accumulate and become very significant.

The problem is more important with `profile` than with the lower-overhead `cProfile`. For this reason, `profile` provides a means of calibrating itself for a given platform so that this error can be probabilistically (on the average) removed. After the profiler is calibrated, it will be more accurate (in a least square sense), but it will sometimes produce negative numbers (when call counts are exceptionally low, and the gods of probability work against you :-).) Do not be alarmed by negative numbers in the profile. They should only appear if you have calibrated your profiler, and the results are actually better than without calibration.

Calibration

The profiler of the `profile` module subtracts a constant from each event handling time to compensate for the overhead of calling the time function, and socking away the results. By default, the constant is 0. The following procedure can be used to obtain a better constant for a given platform (see `Limitations`).

```
import profile
pr = profile.Profile()
for i in range(5):
    print(pr.calibrate(10000))
```


The method executes the number of Python calls given by the argument, directly and again under the profiler, measuring the time for both. It then computes the hidden overhead per profiler event, and returns that as a float. For example, on a 1.8Ghz Intel Core i5 running macOS, and using Python's `time.process_time()` as the timer, the magical number is about $4.04\text{e-}6$.

The object of this exercise is to get a fairly consistent result. If your computer is very fast, or your timer function has poor resolution, you might have to pass 100000, or even 1000000, to get consistent results.

When you have a consistent answer, there are three ways you can use it:

```
import profile

# 1. Apply computed bias to all Profile instances created hereafter.
profile.Profile.bias = your_computed_bias

# 2. Apply computed bias to a specific Profile instance.
pr = profile.Profile()
pr.bias = your_computed_bias

# 3. Specify computed bias in instance constructor.
pr = profile.Profile(bias=your_computed_bias)
```

If you have a choice, you are better off choosing a smaller constant, and then your results will “less often” show up as negative in profile statistics.

Using a custom timer

If you want to change how current time is determined (for example, to force use of wall-clock time or elapsed process time), pass the timing function you want to the `Profile` class constructor:

```
pr = profile.Profile(your_time_func)
```

The resulting profiler will then call `your_time_func`. Depending on whether you are using `profile.Profile` or `cProfile.Profile`, `your_time_func`'s return value will be interpreted differently:

profile.Profile

cProfile.Profile

`your_time_func` should return a single number, or a list of numbers whose sum is the current time (like `whatos.times()` returns). If the function returns a single time number, or the list of returned numbers has length 2, then you will get an especially fast version of the dispatch routine. Be warned that you should calibrate the profiler class for the timer function that you choose (see Calibration). For most machines, a timer that returns a lone integer value will provide the best results in terms

`your_time_func` should return a single number. If it returns integers, you can also invoke the class constructor with a second argument specifying the real duration of one unit of time. For example, if `your_integer_time_func` returns times measured in thousands of seconds, you would construct the `Profile` instance as follows: `pr=cProfile.Profile(your_integer_time_func,0.001)` As the `cProfile.Profile` class cannot be calibrated, custom timer functions should be used with care and should be as fast as possible

Python 3.3 adds several new functions that can be used to make precise measurements of process or wall-clock time. For example, see `time.perf_counter()`.

timeit— Measure execution time of small code snippets

Source code: `Lib/timeit.py`

This module provides a simple way to time small bits of Python code. It has both a Command-Line Interface as well as a callable one. It avoids a number of common traps for measuring execution times. See also Tim Peters' introduction to the "Algorithms" chapter in the second edition of *Python Cookbook*, published by O'Reilly.

Basic Examples

The following example shows how the Command-Line Interface can be used to compare three different expressions:

```
$ python -m timeit "'-'.join(str(n) for n in range(100))"
10000 loops, best of 5: 30.2 usec per loop
$ python -m timeit "'-'.join([str(n) for n in range(100)])"
10000 loops, best of 5: 27.5 usec per loop
$ python -m timeit "'-'.join(map(str, range(100)))"
10000 loops, best of 5: 23.2 usec per loop
```

This can be achieved from the Python Interface with:

```
>>> import timeit
>>> timeit.timeit('"-'.join(str(n) for n in range(100))', number=10000)
0.3018611848820001
>>> timeit.timeit('"-'.join([str(n) for n in range(100)])', number=10000)
0.2727368790656328
>>> timeit.timeit('"-'.join(map(str, range(100)))', number=10000)
0.23702679807320237
```

A callable can also be passed from the Python Interface:

```
>>> timeit.timeit(lambda: "-".join(map(str, range(100))), number=10000)
0.19665591977536678
```

Note however that `timeit()` will automatically determine the number of repetitions only when the command-line interface is used. In the `Examples` section you can find more advanced examples.

Python Interface

The module defines three convenience functions and a public class:

`timeit.timeit(stmt='pass', setup='pass', timer=<defaulttimer>, number=1000000, globals=None)`

Create a `Timer` instance with the given statement, setup code and timer function and run its `timeit()` method with `number` executions. The optional `globals` argument specifies a namespace in which to execute the code. Changed in version 3.5: The optional `globals` parameter was added.

`timeit.repeat(stmt='pass', setup='pass', timer=<defaulttimer>, repeat=5, number=1000000, globals=None)`

Create a `Timer` instance with the given statement, setup code and timer function and run its `repeat()` method with the given `repeatcount` and `number` executions. The optional `globals` argument specifies a namespace in which to execute the code. Changed in version 3.5: The optional `globals` parameter was added. Changed in version 3.7: Default value of `repeat` changed from 3 to 5.

`timeit.default_timer()`

The default timer, which is always `time.perf_counter()`, returns float seconds. An alternative, `time.perf_counter_ns`, returns integer nanoseconds. Changed in version 3.3: `time.perf_counter()` is now the default timer.

`class timeit.Timer(stmt='pass', setup='pass', timer=<timerfunction>, globals=None)`

`timeit(number=1000000)`

`autorange(callback=None)`

`repeat(repeat=5, number=1000000)`

`print_exc(file=None)`

Class for timing execution speed of small code snippets. The constructor takes a statement to be timed, an additional statement used for setup, and a timer function. Both statements default to 'pass'; the timer function is platform-dependent (see the module doc string). `stmt` and `setup` may also contain multiple statements separated by; or newlines, as long as they don't contain multi-line string literals. The statement will by default be executed within `timeit`'s namespace; this behavior can be control

`Timer` number of executions of the main statement. This executes the setup statement once, and then returns the time it takes to execute the main statement a number of times. The default timer returns seconds as a float. The argument is the number of times through the loop, defaulting to one million. The main statement, the setup statement and the timer function to be used are passed to the constructor. Note By default, `timeit()` temporarily turns off garbage collection during the timing. The advantage of

Automatically determine how many times to call `timeit()`. This is a convenience function that call `timeit()` repeatedly so that the total time ≥ 0.2 second, returning the eventual (number of loops, time taken for that number of loops). It call `timeit()` with increasing numbers from the sequence 1, 2, 5, 10, 20, 50, ... until the time taken is at least 0.2 seconds. If `callback` is given and is not `None`, it will be called after each trial with two arguments: `callback(number, time_taken)`. Added in version 3.6.

`Calltimeit()` a few times. This is a convenience function that calls `timeit()` repeatedly, returning a list of results. The first argument specifies how many times to call `timeit()`. The second argument specifies the `number` argument for `timeit()`. Note It's tempting to calculate mean and standard deviation from the result vector and report these. However, this is not very useful. In a typical case, the lowest value gives a lower bound for how fast your machine can run the given code snippet; higher val

Helper to print a traceback from the timed code. Typical use: `t=Timer(...)` # outside the try/except `try: t.timeit(...)` # or `t.repeat(...)` except `Exception: t.print_exc()` The advantage over the standard traceback is that source lines in the compiled template will be displayed. The optional `file` argument directs where the traceback is sent; it defaults to `sys.stderr`.

Command-Line Interface

When called as a program from the command line, the following form is used:

```
python -m timeit [-n N] [-r N] [-u U] [-s S] [-p] [-v] [-h] [statement ...]
```

Where the following options are understood:

`-nN, --number=N`

how many times to execute 'statement'

`-rN, --repeat=N`

how many times to repeat the timer (default 5)

`-sS, --setup=S`

statement to be executed once initially (defaultpass)

-p,--process

measure process time, not wallclock time, using `time.process_time()` instead of `time.perf_counter()`, which is the default. Added in version 3.3.

-u,--unit=U

specify a time unit for timer output; can select `nsec`, `usec`, `msec`, or `sec`. Added in version 3.5.

-v,--verbose

print raw timing results; repeat for more digits precision

-h,--help

print a short usage message and exit

A multi-line statement may be given by specifying each line as a separate statement argument; indented lines are possible by enclosing an argument in quotes and using leading spaces. Multiple-options are treated similarly.

If `-n` is not given, a suitable number of loops is calculated by trying increasing numbers from the sequence 1, 2, 5, 10, 20, 50, ... until the total time is at least 0.2 seconds.

`default_timer()` measurements can be affected by other programs running on the same machine, so the best thing to do when accurate timing is necessary is to repeat the timing a few times and use the best time.

The `-o` option is good for this; the default of 5 repetitions is probably enough in most cases. You can use `time.process_time()` to measure CPU time.

■ Nota: Note There is a certain baseline overhead associated with executing a pass statement. The code here doesn't try to hide it, but you should be aware of it. The baseline overhead can be measured by invoking the program without arguments, and it might differ between Python versions.

Examples

It is possible to provide a setup statement that is executed only once at the beginning:

```
$ python -m timeit -s "text = 'sample string'; char = 'g'" "char in text"
5000000 loops, best of 5: 0.0877 usec per loop
$ python -m timeit -s "text = 'sample string'; char = 'g'" "text.find(char)"
1000000 loops, best of 5: 0.342 usec per loop
```

In the output, there are three fields. The loop count, which tells you how many times the statement body was run per timing loop repetition. The repetition count ('best of 5') which tells you how many times the timing loop was repeated, and finally the time the statement body took on average within the best repetition of the timing loop. That is, the time the fastest repetition took divided by the loop count.

```
>>> import timeit
>>> timeit.timeit('char in text', setup='text = "sample string"; char = "g"')
0.41440500499993504
>>> timeit.timeit('text.find(char)', setup='text = "sample string"; char = "g"')
1.7246671520006203
```

The same can be done using the `Timer` class and its methods:

```
>>> import timeit
>>> t = timeit.Timer('char in text', setup='text = "sample string"; char = "g"')
>>> t.timeit()
0.3955516149999312
>>> t.repeat()
[0.40183617287970225, 0.37027556854118704, 0.38344867356679524, 0.3712595970846668, 0.37866875250654886]
```

The following examples show how to time expressions that contain multiple lines. Here we compare the cost of using `hasattr()` vs. `try/except` to test for missing and present object attributes:

```
$ python -m timeit "try: " " str.__bool__ " except AttributeError: " " pass"
20000 loops, best of 5: 15.7 usec per loop
$ python -m timeit "if hasattr(str, '__bool__'): pass"
50000 loops, best of 5: 4.26 usec per loop
```

```
$ python -m timeit "try: " " int.__bool__ " except AttributeError: " " pass"
200000 loops, best of 5: 1.43 usec per loop
$ python -m timeit "if hasattr(int, '__bool__'): pass"
100000 loops, best of 5: 2.23 usec per loop
```

```
>>> import timeit
>>> # attribute is missing
>>> s = """\
... try:
...     str.__bool__
... except AttributeError:
...     pass
... """
>>> timeit.timeit(stmt=s, number=100000)
0.9138244460009446
>>> s = "if hasattr(str, '__bool__'): pass"
>>> timeit.timeit(stmt=s, number=100000)
0.5829014980008651
>>>
>>> # attribute is present
>>> s = """\
... try:
...     int.__bool__
... except AttributeError:
...     pass
... """
>>> timeit.timeit(stmt=s, number=100000)
0.04215312199994514
>>> s = "if hasattr(int, '__bool__'): pass"
>>> timeit.timeit(stmt=s, number=100000)
0.08588060699912603
```

To give the `timeit` module access to functions you define, you can pass a `setup` parameter which contains an import statement:

```
def test():
    """Stupid test function"""
    L = [i for i in range(100)]

if __name__ == '__main__':
    import timeit
    print(timeit.timeit("test()", setup="from __main__ import test"))
```

Another option is to pass `globals()` to the `globals` parameter, which will cause the code to be executed within your current global namespace. This can be more convenient than individually specifying imports:

```
def f(x):
    return x**2
def g(x):
    return x**4
def h(x):
    return x**8

import timeit
print(timeit.timeit('[func(42) for func in (f,g,h)]', globals=globals()))
```

trace— Trace or track Python statement execution

Source code:Lib/trace.py

The `trace` module allows you to trace program execution, generate annotated statement coverage listings, print caller/callee relationships and list functions executed during a program run. It can be used in another program or from the command line.

See also

Coverage.py

A popular third-party coverage tool that provides HTML output along with advanced features such as branch coverage.

Command-Line Usage

The `trace` module can be invoked from the command line. It can be as simple as

```
python -m trace --count -C . somefile.py ...
```

The above will execute `somefile.py` and generate annotated listings of all Python modules imported during the execution into the current directory.

--help

Display usage and exit.

--version

Display the version of the module and exit.

Added in version 3.8: Added `--module` option that allows running an executable module.

Main options

At least one of the following options must be specified when invoking `trace`. The `--listfuncs` option is mutually exclusive with the `--trace` and `--count` options. When `--listfuncs` is provided, neither `--count` nor `--trace` are accepted, and vice versa.

-c,--count

Produce a set of annotated listing files upon program completion that shows how many times each statement was executed. See also `--coverdir`, `--file` and `--no-report` below.

-t,--trace

Display lines as they are executed.

-l,--listfuncs

Display the functions executed by running the program.

-r,--report

Produce an annotated list from an earlier program run that used the `--count` and `--file` option. This does not execute any code.

-T,--trackcalls

Display the calling relationships exposed by running the program.

Modifiers

-f,--file=<file>

Name of a file to accumulate counts over several tracing runs. Should be used with the `--count` option.

-C,--coverdir=<dir>

Directory where the report files go. The coverage report for `package.module` is written to `filedir/package/module.cover`.

-m,--missing

When generating annotated listings, mark lines which were not executed with `>>>>>`.

-s,--summary

When using `--count` or `--report`, write a brief summary to stdout for each file processed.

-R,--no-report

Do not generate annotated listings. This is useful if you intend to make several runs with `--count`, and then produce a single set of annotated listings at the end.

-g,--timing

Prefix each line with the time since the program started. Only used while tracing.

Filters

These options may be repeated multiple times.

--ignore-module=<mod>

Ignore each of the given module names and its submodules (if it is a package). The argument can be a list of names separated by a comma.

--ignore-dir=<dir>

Ignore all modules and packages in the named directory and subdirectories. The argument can be a list of directories separated by `os.pathsep`.

Programmatic Interface

`classtrace.Trace(count=1,trace=1,countfuncs=0,countcallers=0,ignoremods=(),ignoredirs=(),infile=None,outfile=None,timing=False)`

`run(cmd)`

`runctx(cmd,globals=None,locals=None)`

`runfunc(func,/,*args,**kwds)`

`results()`

Create an object to trace execution of a single statement or expression. All parameters are optional. `countenables` counting of line numbers. `traceenables` line execution tracing. `countfuncs` enables listing of the functions called during the run. `countcallers` enables call relationship tracking. `ignoremods` is a list of modules or packages to ignore. `ignoredirs` is a list of directories whose modules or packages should be ignored. `infile` is the name of the file from which to read stored count information. `outfil`

Execute the command and gather statistics from the execution with the current tracing parameters. `cmd` must be a string or code object, suitable for passing into `exec()`.

Execute the command and gather statistics from the execution with the current tracing parameters, in the defined global and local environments. If not defined, `globals` and `locals` default to empty dictionaries.

Call `func` with the given arguments under control of the `Trace` object with the current tracing parameters.

Return a `CoverageResults` object that contains the cumulative results of all previous calls to `run`, `runctx` and `runfunc` for the given `Trace` instance. Does not reset the accumulated trace results.

class `trace.CoverageResults`

`update(other)`

`write_results(show_missing=True, summary=False, coverdir=None, *, ignore_missing_files=False)`

A container for coverage results, created by `Trace.results()`. Should not be created directly by the user. `update(other)` Merge in data from another `CoverageResults` object. `write_results(show_missing=True, summary=False, coverdir=None, *, ignore_missing_files=False)` Write coverage results. Set `show_missing` to show lines that had no hits. Set `summary` to include in the output the coverage summary per module. `coverdir` specifies the directory into which the coverage result files will be output. If `None`, the results

Merge in data from another `CoverageResults` object.

Write coverage results. Set `show_missing` to show lines that had no hits. Set `summary` to include in the output the coverage summary per module. `coverdir` specifies the directory into which the coverage result files will be output. If `None`, the results for each source file are placed in its directory. If `ignore_missing_files` is `True`, coverage counts for files that no longer exist are silently ignored. Otherwise, a missing file will raise a `FileNotFoundError`. Changed in version 3.13: Added `ignore_missing_files`

A simple example demonstrating the use of the programmatic interface:

```
import sys
import trace

# create a Trace object, telling it what to ignore, and whether to
# do tracing or line-counting or both.
tracer = trace.Trace(
    ignoredirs=[sys.prefix, sys.exec_prefix],
    trace=0,
    count=1)

# run the new command using the given tracer
tracer.run('main()')

# make a report, placing output in the current directory
r = tracer.results()
r.write_results(show_missing=True, coverdir=".")
```

tracemalloc— Trace memory allocations

Added in version 3.4.

Source code: `Lib/tracemalloc.py`

The `tracemalloc` module is a debug tool to trace memory blocks allocated by Python. It provides the following information:

- Traceback where an object was allocated
- Statistics on allocated memory blocks per filename and per line number: total size, number and average size of allocated memory blocks
- Compute the differences between two snapshots to detect memory leaks

To trace most memory blocks allocated by Python, the module should be started as early as possible by setting the `PYTHONTRACEMALLOC` environment variable to 1, or by using `-Xtracemalloc` command line option. The `tracemalloc.start()` function can be called at runtime to start tracing Python memory allocations.

By default, a trace of an allocated memory block only stores the most recent frame (1 frame). To store 25 frames at startup: set the `PYTHONTRACEMALLOC` environment variable to 25, or use the `-Xtracemalloc=25` command line option.

Examples

Display the top 10

Display the 10 files allocating the most memory:

```
import tracemalloc

tracemalloc.start()

# ... run your application ...

snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics('lineno')

print("[ Top 10 ]")
for stat in top_stats[:10]:
    print(stat)
```

Example of output of the Python test suite:

```
[ Top 10 ]
<frozen importlib._bootstrap>:716: size=4855 KiB, count=39328, average=126 B
<frozen importlib._bootstrap>:284: size=521 KiB, count=3199, average=167 B
/usr/lib/python3.4/collections/__init__.py:368: size=244 KiB, count=2315, average=108 B
/usr/lib/python3.4/unittest/case.py:381: size=185 KiB, count=779, average=243 B
/usr/lib/python3.4/unittest/case.py:402: size=154 KiB, count=378, average=416 B
/usr/lib/python3.4/abc.py:133: size=88.7 KiB, count=347, average=262 B
<frozen importlib._bootstrap>:1446: size=70.4 KiB, count=911, average=79 B
<frozen importlib._bootstrap>:1454: size=52.0 KiB, count=25, average=2131 B
<string>:5: size=49.7 KiB, count=148, average=344 B
/usr/lib/python3.4/sysconfig.py:411: size=48.0 KiB, count=1, average=48.0 KiB
```

We can see that Python loaded 4855 KiB data (bytecode and constants) from modules and that the `collections` module allocated 244 KiB to build named tuple types.

See `Snapshot.statistics()` for more options.

Compute differences

Take two snapshots and display the differences:

```
import tracemalloc
tracemalloc.start()
# ... start your application ...

snapshot1 = tracemalloc.take_snapshot()
# ... call the function leaking memory ...
snapshot2 = tracemalloc.take_snapshot()

top_stats = snapshot2.compare_to(snapshot1, 'lineno')

print("[ Top 10 differences ]")
for stat in top_stats[:10]:
    print(stat)
```

Example of output before/after running some tests of the Python test suite:

```
[ Top 10 differences ]
<frozen importlib._bootstrap>:716: size=8173 KiB (+4428 KiB), count=71332 (+39369), average=117 B
/usr/lib/python3.4/linecache.py:127: size=940 KiB (+940 KiB), count=8106 (+8106), average=119 B
/usr/lib/python3.4/unittest/case.py:571: size=298 KiB (+298 KiB), count=589 (+589), average=519 B
<frozen importlib._bootstrap>:284: size=1005 KiB (+166 KiB), count=7423 (+1526), average=139 B
/usr/lib/python3.4/mimetypes.py:217: size=112 KiB (+112 KiB), count=1334 (+1334), average=86 B
/usr/lib/python3.4/http/server.py:848: size=96.0 KiB (+96.0 KiB), count=1 (+1), average=96.0 KiB
/usr/lib/python3.4/inspect.py:1465: size=83.5 KiB (+83.5 KiB), count=109 (+109), average=784 B
/usr/lib/python3.4/unittest/mock.py:491: size=77.7 KiB (+77.7 KiB), count=143 (+143), average=557 B
/usr/lib/python3.4/urllib/parse.py:476: size=71.8 KiB (+71.8 KiB), count=969 (+969), average=76 B
/usr/lib/python3.4/contextlib.py:38: size=67.2 KiB (+67.2 KiB), count=126 (+126), average=546 B
```

We can see that Python has loaded 8173 KiB of module data (bytecode and constants), and that this is 4428 KiB more than had been loaded before the tests, when the previous snapshot was taken. Similarly, the linecache module has cached 940 KiB of Python source code to format tracebacks, all of it since the previous snapshot.

If the system has little free memory, snapshots can be written on disk using the `Snapshot.dump()` method to analyze the snapshot offline. Then use the `Snapshot.load()` method to reload the snapshot.

Get the traceback of a memory block

Code to display the traceback of the biggest memory block:

```
import tracemalloc

# Store 25 frames
tracemalloc.start(25)

# ... run your application ...

snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics('traceback')

# pick the biggest memory block
stat = top_stats[0]
print("%s memory blocks: %.1f KiB" % (stat.count, stat.size / 1024))
for line in stat.traceback.format():
    print(line)
```

Example of output of the Python test suite (traceback limited to 25 frames):

```
903 memory blocks: 870.1 KiB
File "<frozen importlib._bootstrap>", line 716
File "<frozen importlib._bootstrap>", line 1036
File "<frozen importlib._bootstrap>", line 934
```

```

File "<frozen importlib._bootstrap>", line 1068
File "<frozen importlib._bootstrap>", line 619
File "<frozen importlib._bootstrap>", line 1581
File "<frozen importlib._bootstrap>", line 1614
File "/usr/lib/python3.4/doctest.py", line 101
    import pdb
File "<frozen importlib._bootstrap>", line 284
File "<frozen importlib._bootstrap>", line 938
File "<frozen importlib._bootstrap>", line 1068
File "<frozen importlib._bootstrap>", line 619
File "<frozen importlib._bootstrap>", line 1581
File "<frozen importlib._bootstrap>", line 1614
File "/usr/lib/python3.4/test/support/__init__.py", line 1728
    import doctest
File "/usr/lib/python3.4/test/test_pickletools.py", line 21
    support.run_doctest(pickletools)
File "/usr/lib/python3.4/test/regtest.py", line 1276
    test_runner()
File "/usr/lib/python3.4/test/regtest.py", line 976
    display_failure=not verbose)
File "/usr/lib/python3.4/test/regtest.py", line 761
    match_tests=ns.match_tests)
File "/usr/lib/python3.4/test/regtest.py", line 1563
    main()
File "/usr/lib/python3.4/test/__main__.py", line 3
    regtest.main_in_temp_cwd()
File "/usr/lib/python3.4/runpy.py", line 73
    exec(code, run_globals)
File "/usr/lib/python3.4/runpy.py", line 160
    "__main__", fname, loader, pkg_name)

```

We can see that the most memory was allocated in the `importlib` module to load data (bytecode and constants) from modules: 870.1 KiB. The traceback is where the `importlib` loaded data most recently: on the `import pdb` line of the `doctest` module. The traceback may change if a new module is loaded.

Pretty top

Code to display the 10 lines allocating the most memory with a pretty output, ignoring `<frozen importlib._bootstrap>` and `<unknown>` files:

```

import linecache
import os
import tracemalloc

def display_top(snapshot, key_type='lineno', limit=10):
    snapshot = snapshot.filter_traces((
        tracemalloc.Filter(False, "<frozen importlib._bootstrap>"),
        tracemalloc.Filter(False, "<unknown>"),
    ))
    top_stats = snapshot.statistics(key_type)

    print("Top %s lines" % limit)
    for index, stat in enumerate(top_stats[:limit], 1):
        frame = stat.traceback[0]
        print("#%s: %s:%s: %.1f KiB"
              % (index, frame.filename, frame.lineno, stat.size / 1024))
        line = linecache.getline(frame.filename, frame.lineno).strip()
        if line:
            print('    %s' % line)

    other = top_stats[limit:]
    if other:
        size = sum(stat.size for stat in other)
        print("%s other: %.1f KiB" % (len(other), size / 1024))

```

```

    total = sum(stat.size for stat in top_stats)
    print("Total allocated size: %.1f KiB" % (total / 1024))

tracemalloc.start()

# ... run your application ...

snapshot = tracemalloc.take_snapshot()
display_top(snapshot)

```

Example of output of the Python test suite:

```

Top 10 lines
#1: Lib/base64.py:414: 419.8 KiB
    _b85chars2 = [(a + b) for a in _b85chars for b in _b85chars]
#2: Lib/base64.py:306: 419.8 KiB
    _a85chars2 = [(a + b) for a in _a85chars for b in _a85chars]
#3: collections/__init__.py:368: 293.6 KiB
    exec(class_definition, namespace)
#4: Lib/abc.py:133: 115.2 KiB
    cls = super().__new__(mcls, name, bases, namespace)
#5: unittest/case.py:574: 103.1 KiB
    testMethod()
#6: Lib/linecache.py:127: 95.4 KiB
    lines = fp.readlines()
#7: urllib/parse.py:476: 71.8 KiB
    for a in _hexdig for b in _hexdig}
#8: <string>:5: 62.0 KiB
#9: Lib/_weakrefset.py:37: 60.0 KiB
    self.data = set()
#10: Lib/base64.py:142: 59.8 KiB
    _b32tab2 = [a + b for a in _b32tab for b in _b32tab]
6220 other: 3602.8 KiB
Total allocated size: 5303.1 KiB

```

See `Snapshot.statistics()` for more options.

Record the current and peak size of all traced memory blocks

The following code computes two sums like $0+1+2+\dots$ inefficiently, by creating a list of those numbers. This list consumes a lot of memory temporarily. We can use `get_traced_memory()` and `reset_peak()` to observe the small memory usage after the sum is computed as well as the peak memory usage during the computations:

```

import tracemalloc

tracemalloc.start()

# Example code: compute a sum with a large temporary list
large_sum = sum(list(range(100000)))

first_size, first_peak = tracemalloc.get_traced_memory()

tracemalloc.reset_peak()

# Example code: compute a sum with a small temporary list
small_sum = sum(list(range(1000)))

second_size, second_peak = tracemalloc.get_traced_memory()

print(f"{first_size=}, {first_peak=}")
print(f"{second_size=}, {second_peak=}")

```

Output:

```
first_size=664, first_peak=3592984
```

second_size=804, second_peak=29704

Using `reset_peak()` ensured we could accurately record the peak during the computation of `small_sum`, even though it is much smaller than the overall peak size of memory blocks since the `start()` call. Without the call to `reset_peak()`, `second_peak` would still be the peak from the computation `large_sum` (that is, equal to `first_peak`). In this case, both peaks are much higher than the final memory usage, and which suggests we could optimise (by removing the unnecessary call to `list`, and writing `sum(range(...))`).

API

Functions

`tracemalloc.clear_traces()`

Clear traces of memory blocks allocated by Python. See also `stop()`.

`tracemalloc.get_object_traceback(obj)`

Get the traceback where the Python object `obj` was allocated. Return a `Traceback` instance, or `None` if the `tracemalloc` module is not tracing memory allocations or did not trace the allocation of the object. See also `gc.get_referrers()` and `sys.getsizeof()` functions.

`tracemalloc.get_traceback_limit()`

Get the maximum number of frames stored in the traceback of a trace. The `tracemalloc` module must be tracing memory allocations to get the limit, otherwise an exception is raised. The limit is set by the `start()` function.

`tracemalloc.get_traced_memory()`

Get the current size and peak size of memory blocks traced by the `tracemalloc` module as a tuple: (current:int, peak:int).

`tracemalloc.reset_peak()`

Set the peak size of memory blocks traced by the `tracemalloc` module to the current size. Do nothing if the `tracemalloc` module is not tracing memory allocations. This function only modifies the recorded peak size, and does not modify or clear any traces, unlike `clear_traces()`. Snapshots taken with `take_snapshot()` before a call to `reset_peak()` can be meaningfully compared to snapshots taken after the call. See also `get_traced_memory()`. Added in version 3.9.

`tracemalloc.get_tracemalloc_memory()`

Get the memory usage in bytes of the `tracemalloc` module used to store traces of memory blocks. Return an int.

`tracemalloc.is_tracing()`

True if the `tracemalloc` module is tracing Python memory allocations, False otherwise. See also `start()` and `stop()` functions.

`tracemalloc.start(nframe:int=1)`

Start tracing Python memory allocations: install hooks on Python memory allocators. Collected tracebacks of traces will be limited to `nframe` frames. By default, a trace of a memory block only stores the most recent frame: the limit is 1. `nframe` must be greater or equal to 1. You can still read the original number of total frames that composed the traceback by looking at the `Traceback.total_nframe` attribute. Storing more than 1 frame is only useful to compute statistics grouped by 'traceback' or to compute cum

`tracemalloc.stop()`

Stop tracing Python memory allocations: uninstall hooks on Python memory allocators. Also clears all previously collected traces of memory blocks allocated by Python. Call `take_snapshot()` function to take a

snapshot of traces before clearing them. See also `start()`, `is_tracing()` and `clear_traces()` functions.

tracemalloc.take_snapshot()

Take a snapshot of traces of memory blocks allocated by Python. Return a new `Snapshot` instance. The snapshot does not include memory blocks allocated before the `tracemalloc` module started to trace memory allocations. Tracebacks of traces are limited to `get_traceback_limit()` frames. Use the `frame` parameter of the `start()` function to store more frames. The `tracemalloc` module must be tracing memory allocations to take a snapshot, see the `start()` function. See also the `get_object_traceback()` function.

DomainFilter

class tracemalloc.DomainFilter(inclusive:bool, domain:int)

inclusive

domain

Filter traces of memory blocks by their address space (domain). Added in version 3.6. If `inclusive` is `True` (include), match memory blocks allocated in the address space `domain`. If `inclusive` is `False` (exclude), match memory blocks not allocated in the address space `domain`. `domain` Address space of a memory block (int). Read-only property.

If `inclusive` is `True` (include), match memory blocks allocated in the address space `domain`. If `inclusive` is `False` (exclude), match memory blocks not allocated in the address space `domain`.

Address space of a memory block (int). Read-only property.

Filter

class tracemalloc.Filter(inclusive:bool, filename_pattern:str, lineno:int=None, all_frames:bool=False, domain:int=None)

domain

inclusive

lineno

filename_pattern

all_frames

Filter on traces of memory blocks. See the `fnmatch.fnmatch()` function for the syntax of `filename_pattern`. The `'pyc'` file extension is replaced with `'py'`. Examples: `Filter(True, subprocess.__file__)` only includes traces of the `subprocess` module. `Filter(False, tracemalloc.__file__)` excludes traces of the `tracemalloc` module. `Filter(False, "<unknown>")` excludes empty tracebacks. Changed in version 3.5: The `'pyo'` file extension is no longer replaced with `'py'`. Changed in version 3.6: Added the `domain` attribute. `domain` Address space of a memory block (int or `None`). `tracemalloc` uses the `domain 0` to trace memory allocations made by Python. C extensions can use other domains to trace other resources.

Address space of a memory block (int or `None`). `tracemalloc` uses the `domain 0` to trace memory allocations made by Python. C extensions can use other domains to trace other resources.

If `inclusive` is `True` (include), only match memory blocks allocated in a file with a name matching `filename_pattern` at line number `lineno`. If `inclusive` is `False` (exclude), ignore memory blocks allocated in a file with a name matching `filename_pattern` at line number `lineno`.

Line number (int) of the filter. If `lineno` is `None`, the filter matches any line number.

Filename pattern of the filter (str). Read-only property.

If `all_frames` is `True`, all frames of the traceback are checked. If `all_frames` is `False`, only the most recent frame is checked. This attribute has no effect if the traceback limit is 1. See the `get_traceback_limit()` function and `Snapshot.traceback_limit` attribute.

Frame

class tracemalloc.Frame

filename

lineno

Frame of a traceback. The `Traceback` class is a sequence of `Frame` instances. `filename` Filename (str). `lineno` Line number (int).

Filename (str).

Line number (int).

Snapshot

class tracemalloc.Snapshot

compare_to(old_snapshot: Snapshot, key_type: str, cumulative: bool=False)

dump(filename)

filter_traces(filters)

classmethod load(filename)

statistics(key_type: str, cumulative: bool=False)

traceback_limit

traces

Snapshot of traces of memory blocks allocated by Python. The `take_snapshot()` function creates a snapshot instance. `compare_to(old_snapshot: Snapshot, key_type: str, cumulative: bool=False)` Compute the differences with an old snapshot. Get statistics as a sorted list of `StatisticDiff` instances grouped by `key_type`. See the `Snapshot.statistics()` method for `key_type` and `cumulative` parameters. The result is sorted from the biggest to the smallest by: absolute value of `StatisticDiff.size_diff`, `StatisticDiff.size`, absolute value

Compute the differences with an old snapshot. Get statistics as a sorted list of `StatisticDiff` instances grouped by `key_type`. See the `Snapshot.statistics()` method for `key_type` and `cumulative` parameters. The result is sorted from the biggest to the smallest by: absolute value of `StatisticDiff.size_diff`, `StatisticDiff.size`, absolute value of `StatisticDiff.count_diff`, `StatisticDiff.count` and then by `StatisticDiff.traceback`.

Write the snapshot into a file. Use `load()` to reload the snapshot.

Create a new `Snapshot` instance with a filtered `traces` sequence, `filters` is a list of `DomainFilter` and `Filter` instances. If `filters` is an empty list, return a new `Snapshot` instance with a copy of the traces. All inclusive filters are applied at once, a trace is ignored if no inclusive filters match it. A trace is ignored if at least one exclusive filter matches it. Changed in version 3.6: `DomainFilter` instances are now also accepted in filters.

Load a snapshot from a file. See also `dump()`.

Get statistics as a sorted list of `Statistic` instances grouped by `key_type`: `key_type` description 'filename' filename 'lineno' lineno 'filename and line number' 'traceback' 'traceback' If `cumulative` is `True`, cumulate size and count of memory blocks of all frames of the traceback of a trace, not only the most recent frame. The cumulative mode can only be used with `key_type` equal to 'filename' and 'lineno'. The result is sorted from the biggest to the smallest by: `Statistic.size`, `Statistic.count` and then by `Statistic.traceback`.

Maximum number of frames stored in the traceback of traces: result of the `get_traceback_limit()` when the snapshot was taken.

Traces of all memory blocks allocated by Python: sequence of `Trace` instances. The sequence has an undefined order. Use the `Snapshot.statistics()` method to get a sorted list of statistics.

Statistic

`class tracemalloc.Statistic`

`count`

`size`

`traceback`

Statistic on memory allocations. `Snapshot.statistics()` returns a list of `Statistic` instances. See also the `StatisticDiff` class. `count` Number of memory blocks (int). `size` Total size of memory blocks in bytes (int). `traceback` `Traceback` where the memory block was allocated, `Traceback` instance.

Number of memory blocks (int).

Total size of memory blocks in bytes (int).

Traceback where the memory block was allocated, `Traceback` instance.

StatisticDiff

`class tracemalloc.StatisticDiff`

`count`

`count_diff`

`size`

`size_diff`

`traceback`

Statistic difference on memory allocations between an old and a new `Snapshot` instance. `Snapshot.compare_to()` returns a list of `StatisticDiff` instances. See also the `Statistic` class. `count` Number of memory blocks in the new snapshot (int): 0 if the memory blocks have been released in the new snapshot. `count_diff` Difference of number of memory blocks between the old and the new snapshots (int): 0 if the memory blocks have been allocated in the new snapshot. `size` Total size of memory blocks in bytes in the new snapshot.

Number of memory blocks in the new snapshot (int): 0 if the memory blocks have been released in the new snapshot.

Difference of number of memory blocks between the old and the new snapshots (int): 0 if the memory blocks have been allocated in the new snapshot.

Total size of memory blocks in bytes in the new snapshot (int): 0 if the memory blocks have been released in the new snapshot.

Difference of total size of memory blocks in bytes between the old and the new snapshots (int): 0 if the memory blocks have been allocated in the new snapshot.

Traceback where the memory blocks were allocated, `Traceback` instance.

Trace

`class tracemalloc.Trace`

`domain`

`size`

traceback

Trace of a memory block. The `Snapshot.traces` attribute is a sequence of `Trace` instances. Changed in version 3.6: Added the `domain` attribute. `domain` Address space of a memory block (int). Read-only property. `tracemalloc` uses the `domain 0` to trace memory allocations made by Python. C extensions can use other domains to trace other resources. `size` Size of the memory block in bytes (int). `traceback` `Traceback` where the memory block was allocated, `Traceback` instance.

Address space of a memory block (int). Read-only property. `tracemalloc` uses the `domain 0` to trace memory allocations made by Python. C extensions can use other domains to trace other resources.

Size of the memory block in bytes (int).

Traceback where the memory block was allocated, `Traceback` instance.

Traceback

class `tracemalloc.Traceback`

`total_nframe`

`format(limit=None, most_recent_first=False)`

Sequence of `Frame` instances sorted from the oldest frame to the most recent frame. A traceback contains at least 1 frame. If the `tracemalloc` module failed to get a frame, the filename "`<unknown>`" at line number 0 is used. When a snapshot is taken, tracebacks of traces are limited to `get_traceback_limit()` frames. See the `take_snapshot()` function. The original number of frames of the traceback is stored in the `Traceback.total_nframe` attribute. That allows one to know if a traceback has been truncated by the `traceba`

Total number of frames that composed the traceback before truncation. This attribute can be set to `None` if the information is not available.

Format the traceback as a list of lines. Use the `linecache` module to retrieve lines from the source code. If `limit` is set, format the `limit` most recent frames if `limit` is positive. Otherwise, format the `abs(limit)` oldest frames. If `most_recent_first` is `True`, the order of the formatted frames is reversed, returning the most recent frame first instead of last. Similar to the `traceback.format_tb()` function, except that `format()` does not include newlines. Example: `print("Traceback (most recent call first):")` for `line` in `tr`

Software Packaging and Distribution

These libraries help you with publishing and installing Python software. While these modules are designed to work in conjunction with the Python Package Index, they can also be used with a local index server, or without any index server at all.

- `ensurepip`— Bootstrapping the `pip` installer
Command-line interface
Module API
- `venv`— Creation of virtual environments
Creating virtual environments
How `venvs` work
API
An example of extending `EnvBuilder`
- `zipapp`— Manage executable Python zip archives
Basic Example
Command-Line Interface
Python API
Examples
Specifying the Interpreter
Creating Standalone Applications with `zipapp`
Caveats
The Python Zip Application Archive Format

ensurepip— Bootstrapping the pip installer

Added in version 3.4.

Source code: [Lib/ensurepip](#)

The `ensurepip` package provides support for bootstrapping the pip installer into an existing Python installation or virtual environment. This bootstrapping approach reflects the fact that pip is an independent project with its own release cycle, and the latest available stable version is bundled with maintenance and feature releases of the CPython reference interpreter.

In most cases, end users of Python shouldn't need to invoke this module directly (as pip should be bootstrapped by default), but it may be needed if installing pip was skipped when installing Python (or when creating a virtual environment) or after explicitly uninstalling pip.

■ **Note:** This module does not access the internet. All of the components needed to bootstrap pip are included as internal parts of the package.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements for optional modules](#).

See also

Installing Python Modules

PEP 453: Explicit bootstrapping of pip in Python installations

The end user guide for installing Python packages

The original rationale and specification for this module.

Availability: not Android, not iOS, not WASI.

This module is not supported on mobile platforms or WebAssembly platforms.

Command-line interface

The command line interface is invoked using the interpreter's `-m` switch.

The simplest possible invocation is:

```
python -m ensurepip
```

This invocation will install pip if it is not already installed, but otherwise does nothing. To ensure the installed version of pip is at least as recent as the one available in `ensurepip`, pass the `--upgrade` option:

```
python -m ensurepip --upgrade
```

By default, pip is installed into the current virtual environment (if one is active) or into the system site packages (if there is no active virtual environment). The installation location can be controlled through two additional command line options:

--root<dir>

Install pip relative to the given root directory rather than the root of the currently active virtual environment (if any) or the default root for the current Python installation.

--user

Install pip into the user site packages directory rather than globally for the current Python installation (this option is not permitted inside an active virtual environment).

By default, the scripts `pipX` and `pipX.Y` will be installed (where `X.Y` stands for the version of Python used to invoke `ensurepip`). The scripts installed can be controlled through two additional command line options:

--altinstall

If an alternate installation is requested, the `pipX` script will not be installed.

--default-pip

If a “default pip” installation is requested, the `pip` script will be installed in addition to the two regular scripts.

Providing both of the script selection options will trigger an exception.

Module API

`ensurepip` exposes two functions for programmatic use:

`ensurepip.version()`

Returns a string specifying the available version of pip that will be installed when bootstrapping an environment.

`ensurepip.bootstrap(root=None, upgrade=False, user=False, altinstall=False, default_pip=False, verbosity=0)`

Bootstrap pip into the current or designated environment. `root` specifies an alternative root directory to install relative to. If `root` is `None`, then installation uses the default install location for the current environment. `upgrade` indicates whether or not to upgrade an existing installation of an earlier version of pip to the available version. `user` indicates whether to use the user scheme rather than installing globally. By default, the scripts `pipX` and `pipX.Y` will be installed (where `X.Y` stands for the current

venv— Creation of virtual environments

Added in version 3.3.

Source code: [Lib/venv/](#)

The `venv` module supports creating lightweight “virtual environments”, each with their own independent set of Python packages installed in their site directories. A virtual environment is created on top of an existing Python installation, known as the virtual environment’s “base” Python, and by default is isolated from the packages in the base environment, so that only those explicitly installed in the virtual environment are available. See [Virtual Environments](#) and [site’s virtual environments documentation](#) for more information.

When used from within a virtual environment, common installation tools such as `pip` will install Python packages into a virtual environment without needing to be told to do so explicitly.

A virtual environment is (amongst other things):

- Used to contain a specific Python interpreter and software libraries and binaries which are needed to support a project (library or application). These are by default isolated from software in other virtual environments and Python interpreters and libraries installed in the operating system.
- Contained in a directory, conventionally named `.venv` or `venv` in the project directory, or under a container directory for lots of virtual environments, such as `~/virtualenvs`.
- Not checked into source control systems such as Git.
- Considered as disposable – it should be simple to delete and recreate it from scratch. You don’t place any project code in the environment.
- Not considered as movable or copyable – you just recreate the same environment in the target location.

See [PEP 405](#) for more background on Python virtual environments.

See also

[Python Packaging User Guide: Creating and using virtual environments](#)

Availability: not Android, not iOS, not WASI.

This module is not supported on mobile platforms or WebAssembly platforms.

Creating virtual environments

Virtual environments are created by executing the `venv` module:

```
python -m venv /path/to/new/virtual/environment
```

This creates the target directory (including parent directories as needed) and places a `pyvenv.cfg` file in it with a `home` key pointing to the Python installation from which the command was run. It also creates a `bin` (or `Scripts` on Windows) subdirectory containing a copy or symlink of the Python executable (as appropriate for the platform or arguments used at environment creation time). It also creates a `lib/pythonX.Y/site-packages` subdirectory (on Windows, this is `Lib\site-packages`). If an existing directory is specified, it will be re-used.

Changed in version 3.5: The use of `venv` is now recommended for creating virtual environments.

Deprecated since version 3.6, removed in version 3.8: `pyvenv` was the recommended tool for creating virtual environments for Python 3.3 and 3.4, and replaced in 3.5 by executing `venv` directly.

On Windows, invoke the `venv` command as follows:

```
PS> python -m venv C:\path\to\new\virtual\environment
```

The command, if run with -h, will show the available options:

```
usage: venv [-h] [--system-site-packages] [--symlinks | --copies] [--clear]
           [--upgrade] [--without-pip] [--prompt PROMPT] [--upgrade-deps]
           [--without-scm-ignore-files]
           ENV_DIR [ENV_DIR ...]
```

Creates virtual Python environments in one or more target directories.

Once an environment has been created, you may wish to activate it, e.g. by sourcing an activate script in its bin directory.

ENV_DIR

A required argument specifying the directory to create the environment in.

--system-site-packages

Give the virtual environment access to the system site-packages directory.

--symlinks

Try to use symlinks rather than copies, when symlinks are not the default for the platform.

--copies

Try to use copies rather than symlinks, even when symlinks are the default for the platform.

--clear

Delete the contents of the environment directory if it already exists, before environment creation.

--upgrade

Upgrade the environment directory to use this version of Python, assuming Python has been upgraded in-place.

--without-pip

Skips installing or upgrading pip in the virtual environment (pip is bootstrapped by default).

--prompt<PROMPT>

Provides an alternative prompt prefix for this environment.

--upgrade-deps

Upgrade core dependencies (pip) to the latest version in PyPI.

--without-scm-ignore-files

Skips adding SCM ignore files to the environment directory (Git is supported by default).

Changed in version 3.4: Installs pip by default, added the --without-pip and --copies options.

Changed in version 3.4: In earlier versions, if the target directory already existed, an error was raised, unless the --clear or --upgrade option was provided.

Changed in version 3.9: Add --upgrade-deps option to upgrade pip + setuptools to the latest on PyPI.

Changed in version 3.12: setuptools is no longer a core venv dependency.

Changed in version 3.13: Added the --without-scm-ignore-files option.

Changed in version 3.13: venv now creates a .gitignore file for Git by default.

■ Nota: Note While symlinks are supported on Windows, they are not recommended. Of particular note is that double-clicking python.exe in File Explorer will resolve the symlink eagerly and ignore the virtual environment.

■ Nota: NoteOn Microsoft Windows, it may be required to enable theActivate.ps1script by setting the execution policy for the user. You can do this by issuing the following PowerShell command:PSC:\>Set-ExecutionPolicy-ExecutionPolicyRemoteSigned-ScopeCurrentUserSeeAbout Execution Policiesfor more information.

The createdpyvenv.cfgfile also includes theinclude-system-site-packageskey, set tottrueifvenvis run with the--system-site-packagesoption,falseotherwise.

Unless the--without-pipoption is given,ensurepipwill be invoked to bootstrappipinto the virtual environment.

Multiple paths can be given tovenv, in which case an identical virtual environment will be created, according to the given options, at each provided path.

How venvs work

When a Python interpreter is running from a virtual environment,sys.prefixandsys.exec_prefixpoint to the directories of the virtual environment, whereassys.base_prefixandsys.base_exec_prefixpoint to those of the base Python used to create the environment. It is sufficient to checksys.prefix!=sys.base_prefixto determine if the current interpreter is running from a virtual environment.

A virtual environment may be “activated” using a script in its binary directory (binon POSIX;Scriptson Windows). This will prepend that directory to yourPATH, so that runningpythonwill invoke the environment’s Python interpreter and you can run installed scripts without having to use their full path. The invocation of the activation script is platform-specific (<venv>must be replaced by the path to the directory containing the virtual environment):

Platform	Shell	Command to activate virtual environment
POSIX	bash/zsh	\$ source <venv> /bin/activate
	fish	\$ source <venv> /bin/activate.fish
	csh/tcsh	\$ source <venv> /bin/activate.csh
Windows	pwsh	\$ <venv> /bin/Activate.ps1
	cmd.exe	C:\> <venv> \Scripts\activate.bat
	PowerShell	PS C:\> <venv> \Scripts\Activate.ps1

Added in version 3.4:fishandcshactivation scripts.

Added in version 3.8:PowerShell activation scripts installed under POSIX for PowerShell Core support.

You don’t specificallyneedto activate a virtual environment, as you can just specify the full path to that environment’s Python interpreter when invoking Python. Furthermore, all scripts installed in the environment should be runnable without activating it.

In order to achieve this, scripts installed into virtual environments have a “shebang” line which points to the environment’s Python interpreter,#!/<path-to-venv>/bin/python. This means that the script will run with that interpreter regardless of the value ofPATH. On Windows, “shebang” line processing is supported if you have thePython install managerinstalled. Thus, double-clicking an installed script in a Windows Explorer window should run it with the correct interpreter without the environment needing to be activated or on thePATH.

When a virtual environment has been activated, theVIRTUAL_ENVenvironment variable is set to the path of the environment. Since explicitly activating a virtual environment is not required to use it,VIRTUAL_ENVcannot be relied upon to determine whether a virtual environment is being used.

■■ Advertencia: WarningBecause scripts installed in environments should not expect the environment to be activated, their shebang lines contain the absolute paths to their environment’s interpreters. Because of this, environments are inherently non-portable, in the general case. You should always have a simple means of recreating an environment (for example, if you have a requirements filerequirements.txt, you can invokepipinstall-rrequirements.txtusing the environment’spipto install all of the packages needed by the environment). If for any reason you need to move the environment to a new location, you should

You can deactivate a virtual environment by typingdeactivatein your shell. The exact mechanism is platform-specific and is an internal implementation detail (typically, a script or shell function will be used).

API

The high-level method described above makes use of a simple API which provides mechanisms for third-party virtual environment creators to customize environment creation according to their needs, the `EnvBuilder` class.

```
class venv.EnvBuilder(system_site_packages=False, clear=False, symlinks=False, upgrade=False, with_pip=False, prompt=None, upgrade_deps=False, *, scm_ignore_files=frozenset())
```

```
create(env_dir)
```

```
ensure_directories(env_dir)
```

```
create_configuration(context)
```

```
setup_python(context)
```

```
setup_scripts(context)
```

```
upgrade_dependencies(context)
```

```
post_setup(context)
```

```
install_scripts(context, path)
```

```
create_git_ignore_file(context)
```

The `EnvBuilder` class accepts the following keyword arguments on instantiation: `system_site_packages`— a boolean value indicating that the system Python site-packages should be available to the environment (defaults to `False`). `clear`— a boolean value which, if `True`, will delete the contents of any existing target directory, before creating the environment. `symlinks`— a boolean value indicating whether to attempt to symlink the Python binary rather than copying. `upgrade`— a boolean value which, if `True`, will

Create a virtual environment by specifying the target directory (absolute or relative to the current directory) which is to contain the virtual environment. The `create` method will either create the environment in the specified directory, or raise an appropriate exception. The `create` method of the `EnvBuilder` class illustrates the hooks available for subclass customization:

```
def create(self, env_dir): """Create a virtualized Python environment in a directory. env_dir is the target directory to create an envir
```

Creates the environment directory and all necessary subdirectories that don't already exist, and returns a context object. This context object is just a holder for attributes (such as paths) for use by the other methods. If the `EnvBuilder` is created with the arg `clear=True`, contents of the environment directory will be cleared and then all necessary subdirectories will be recreated. The returned context object is a `types.SimpleNamespace` with the following attributes: `env_dir`— The location of the virt

Creates the `pyvenv.cfg` configuration file in the environment.

Creates a copy or symlink to the Python executable in the environment. On POSIX systems, if a specific executable `python3.x` was used, symlinks to `python` and `python3` will be created pointing to that executable, unless files with those names already exist.

Installs activation scripts appropriate to the platform into the virtual environment.

Upgrades the core `venv` dependency packages (currently `pip`) in the environment. This is done by shelling out to the `pip` executable in the environment. Added in version 3.9. Changed in version 3.12: `setuptools` is no longer a core `venv` dependency.

A placeholder method which can be overridden in third party implementations to pre-install packages in the virtual environment or perform other post-creation steps.

This method can be called from `setup_scripts()` or `post_setup()` in subclasses to assist in installing custom scripts into the virtual environment. `path` is the path to a directory that should contain subdirectories `common`, `posix`, `nt`; each containing scripts destined for the `bin` directory in the environment. The contents of `common` and the

directory corresponding to `toos.name` are copied after some text replacement of placeholders: `__VENV_DIR__` is replaced with the absolute path of the environment directory, `__VENV_NAME__`

Creates a `.gitignore` file within the virtual environment that causes the entire directory to be ignored by the Git source control manager. Added in version 3.13.

There is also a module-level convenience function:

`venv.create(env_dir, system_site_packages=False, clear=False, symlinks=False, with_pip=False, prompt=None, upgrade_deps=False, *, scm_ignore_files=frozenset())`

Create an `EnvBuilder` with the given keyword arguments, and call its `create()` method with the `env_dir` argument. Added in version 3.3. Changed in version 3.4: Added the `with_pip` parameter. Changed in version 3.6: Added the `prompt` parameter. Changed in version 3.9: Added the `upgrade_deps` parameter. Changed in version 3.13: Added the `scm_ignore_files` parameter

An example of extending `EnvBuilder`

The following script shows how to extend `EnvBuilder` by implementing a subclass which installs `setuptools` and `pip` into a created virtual environment:

```
import os
import os.path
from subprocess import Popen, PIPE
import sys
from threading import Thread
from urllib.parse import urlsplit
from urllib.request import urlretrieve
import venv

class ExtendedEnvBuilder(venv.EnvBuilder):
    """
    This builder installs setuptools and pip so that you can pip or
    easy_install other packages into the created virtual environment.

    :param nodist: If true, setuptools and pip are not installed into the
        created virtual environment.
    :param nopip: If true, pip is not installed into the created
        virtual environment.
    :param progress: If setuptools or pip are installed, the progress of the
        installation can be monitored by passing a progress
        callable. If specified, it is called with two
        arguments: a string indicating some progress, and a
        context indicating where the string is coming from.
        The context argument can have one of three values:
        'main', indicating that it is called from virtualize()
        itself, and 'stdout' and 'stderr', which are obtained
        by reading lines from the output streams of a subprocess
        which is used to install the app.

        If a callable is not specified, default progress
        information is output to sys.stderr.
    """

    def __init__(self, *args, **kwargs):
        self.nodist = kwargs.pop('nodist', False)
        self.nopip = kwargs.pop('nopip', False)
        self.progress = kwargs.pop('progress', None)
        self.verbose = kwargs.pop('verbose', False)
        super().__init__(*args, **kwargs)

    def post_setup(self, context):
```

```

"""
Set up any packages which need to be pre-installed into the
virtual environment being created.

:param context: The information for the virtual environment
                 creation request being processed.
"""
os.environ['VIRTUAL_ENV'] = context.env_dir
if not self.nodist:
    self.install_setuptools(context)
# Can't install pip without setuptools
if not self.nopip and not self.nodist:
    self.install_pip(context)

def reader(self, stream, context):
    """
    Read lines from a subprocess' output stream and either pass to a progress
    callable (if specified) or write progress information to sys.stderr.
    """
    progress = self.progress
    while True:
        s = stream.readline()
        if not s:
            break
        if progress is not None:
            progress(s, context)
        else:
            if not self.verbose:
                sys.stderr.write('.')
            else:
                sys.stderr.write(s.decode('utf-8'))
            sys.stderr.flush()
    stream.close()

def install_script(self, context, name, url):
    _, _, path, _, _ = urlsplit(url)
    fn = os.path.split(path)[-1]
    binpath = context.bin_path
    distpath = os.path.join(binpath, fn)

```

This script is also available for [downloadonline](#).

zipapp— Manage executable Python zip archives

Added in version 3.5.

Source code: `Lib/zipapp.py`

This module provides tools to manage the creation of zip files containing Python code, which can be executed directly by the Python interpreter. The module provides both a Command-Line Interface and a Python API.

Basic Example

The following example shows how the Command-Line Interface can be used to create an executable archive from a directory containing Python code. When run, the archive will execute the `main` function from the module `myapp` in the archive.

```
$ python -m zipapp myapp -m "myapp:main"
$ python myapp.pyz
<output from myapp>
```

Command-Line Interface

When called as a program from the command line, the following form is used:

```
$ python -m zipapp source [options]
```

If `source` is a directory, this will create an archive from the contents of `source`. If `source` is a file, it should be an archive, and it will be copied to the target archive (or the contents of its shebang line will be displayed if the `--info` option is specified).

The following options are understood:

-o<output>,--output=<output>

Write the output to a file named `output`. If this option is not specified, the output filename will be the same as the input `source`, with the extension `.pyz` added. If an explicit filename is given, it is used as is (so a `.pyz` extension should be included if required). An output filename must be specified if `source` is an archive (and in that case, `output` must not be the same as `source`).

-p<interpreter>,--python=<interpreter>

Add a `#!` line to the archive specifying `interpreter` as the command to run. Also, on POSIX, make the archive executable. The default is to write no `#!` line, and not make the file executable.

-m<mainfn>,--main=<mainfn>

Write a `__main__.py` file to the archive that executes `mainfn`. The `mainfn` argument should have the form `"pkg.mod:fn"`, where `"pkg.mod"` is a package/module in the archive, and `"fn"` is a callable in the given module. The `__main__.py` file will execute that callable. `--main` cannot be specified when copying an archive.

-c,--compress

Compress files with the deflate method, reducing the size of the output file. By default, files are stored uncompressed in the archive. `--compress` has no effect when copying an archive. Added in version 3.7.

--info

Display the interpreter embedded in the archive, for diagnostic purposes. In this case, any other options are ignored and `SOURCE` must be an archive, not a directory.

-h,--help

Print a short usage message and exit.

Python API

The module defines two convenience functions:

zipapp.create_archive(source,target=None,interpreter=None,main=None,filter=None,compressed=False)

Create an application archive from source. The source can be any of the following: The name of a directory, or a path-like object referring to a directory, in which case a new application archive will be created from the content of that directory. The name of an existing application archive file, or a path-like object referring to such a file, in which case the file is copied to the target (modifying it to reflect the value given for the interpreter argument). The file name should include the .pyz extension

zipapp.get_interpreter(archive)

Return the interpreter specified in the #! line at the start of the archive. If there is no #! line, return None. The archive argument can be a filename or a file-like object open for reading in bytes mode. It is assumed to be at the start of the archive.

Examples

Pack up a directory into an archive, and run it.

```
$ python -m zipapp myapp
$ python myapp.pyz
<output from myapp>
```

The same can be done using the create_archive() function:

```
>>> import zipapp
>>> zipapp.create_archive('myapp', 'myapp.pyz')
```

To make the application directly executable on POSIX, specify an interpreter to use.

```
$ python -m zipapp myapp -p "/usr/bin/env python"
$ ./myapp.pyz
<output from myapp>
```

To replace the shebang line on an existing archive, create a modified archive using the create_archive() function:

```
>>> import zipapp
>>> zipapp.create_archive('old_archive.pyz', 'new_archive.pyz', '/usr/bin/python3')
```

To update the file in place, do the replacement in memory using a BytesIO object, and then overwrite the source afterwards. Note that there is a risk when overwriting a file in place that an error will result in the loss of the original file. This code does not protect against such errors, but production code should do so. Also, this method will only work if the archive fits in memory:

```
>>> import zipapp
>>> import io
>>> temp = io.BytesIO()
>>> zipapp.create_archive('myapp.pyz', temp, '/usr/bin/python2')
>>> with open('myapp.pyz', 'wb') as f:
>>>     f.write(temp.getvalue())
```

Specifying the Interpreter

Note that if you specify an interpreter and then distribute your application archive, you need to ensure that the interpreter used is portable. The Python launcher for Windows supports most common forms of `POSIX#!` line, but there are other issues to consider:

- If you use `“/usr/bin/env python”` (or other forms of the “python” command, such as `“/usr/bin/python”`), you need to consider that your users may have either Python 2 or Python 3 as their default, and write your code to work under both versions.
- If you use an explicit version, for example `“/usr/bin/env python3”` your application will not work for users who do not have that version. (This may be what you want if you have not made your code Python 2 compatible).
- There is no way to say “python X.Y or later”, so be careful of using an exact version like `“/usr/bin/env python3.4”` as you will need to change your shebang line for users of Python 3.5, for example.

Typically, you should use an `“/usr/bin/env python2”` or `“/usr/bin/env python3”`, depending on whether your code is written for Python 2 or 3.

Creating Standalone Applications with zipapp

Using the `zipapp` module, it is possible to create self-contained Python programs, which can be distributed to end users who only need to have a suitable version of Python installed on their system. The key to doing this is to bundle all of the application’s dependencies into the archive, along with the application code.

The steps to create a standalone archive are as follows:

- Create your application in a directory as normal, so you have `myapp` directory containing a `__main__.py` file, and any supporting application code.
- Install all of your application’s dependencies into `myapp` directory, using `pip: $python -mpip install -r requirements.txt --target myapp` (this assumes you have your project requirements in `requirements.txt` file - if not, you can just list the dependencies manually on the pip command line).
- Package the application using: `$python -m zipapp -p “interpreter” myapp`

This will produce a standalone executable, which can be run on any machine with the appropriate interpreter available. See [Specifying the Interpreter](#) for details. It can be shipped to users as a single file.

On Unix, `myapp.pyz` file is executable as it stands. You can rename the file to remove the `.pyz` extension if you prefer a “plain” command name. On Windows, `myapp.pyz[w]` file is executable by virtue of the fact that the Python interpreter registers the `.pyz` and `.pyzw` extensions when installed.

Caveats

If your application depends on a package that includes a C extension, that package cannot be run from a zip file (this is an OS limitation, as executable code must be present in the filesystem for the OS loader to load it). In this case, you can exclude that dependency from the zipfile, and either require your users to have it installed, or ship it alongside your zipfile and add code to your `__main__.py` to include the directory containing the unzipped module in `sys.path`. In this case, you will need to make sure to ship appropriate binaries for your target architecture(s) (and potentially pick the correct version to add to `sys.path` at runtime, based on the user’s machine).

The Python Zip Application Archive Format

Python has been able to execute zip files which contain a `__main__.py` file since version 2.6. In order to be executed by Python, an application archive simply has to be a standard zip file containing a `__main__.py` file which will be run as the entry point for the application. As usual for any Python script, the parent of the script (in

this case the zip file) will be placed on `sys.path` and thus further modules can be imported from the zip file.

The zip file format allows arbitrary data to be prepended to a zip file. The zip application format uses this ability to prepend a standard POSIX “shebang” line to the file (`#!/path/to/interpreter`).

Formally, the Python zip application format is therefore:

- An optional shebang line, containing the characters `#!/` followed by an interpreter name, and then a newline (`\n`) character. The interpreter name can be anything acceptable to the OS “shebang” processing, or the Python launcher on Windows. The interpreter should be encoded in UTF-8 on Windows, and `sys.getfilesystemencoding()` on POSIX.
- Standard zipfile data, as generated by the `zipfile` module. The zipfile content must include a file called `__main__.py` (which must be in the “root” of the zipfile - i.e., it cannot be in a subdirectory). The zipfile data can be compressed or uncompressed.

If an application archive has a shebang line, it may have the executable bit set on POSIX systems, to allow it to be executed directly.

There is no requirement that the tools in this module are used to create application archives - the module is a convenience, but archives in the above format created by any means are acceptable to Python.

Python Runtime Services

The modules described in this chapter provide a wide range of services related to the Python interpreter and its interaction with its environment. Here's an overview:

- `sys`— System-specific parameters and functions
- `sys.monitoring`— Execution event monitoringTool identifiersRegistering and using toolsEventsLocal eventsDeprecated eventAncillary eventsOther eventsThe `STOP_ITERATION` eventTurning events on and offSetting events globallyPer code object eventsDisabling eventsRegistering callback functionsCallback function arguments
- `sysconfig`— Provide access to Python's configuration informationConfiguration variablesInstallation pathsUser schemeposix_usernt_userosx_framework_userHome schemeposix_homePrefix schemeposix_prefixntInstallation path functionsOther functionsCommand-line usage
- `builtins`— Built-in objects
- `__main__`— Top-level code environment `__name__ == '__main__'`What is the “top-level code environment”?Idiomatic UsagePackaging Considerations `__main__.py`in Python PackagesIdiomatic Usageimport `__main__`
- `warnings`— Warning controlWarning CategoriesThe Warnings FilterRepeated Warning Suppression CriteriaDescribing Warning FiltersDefault Warning FilterOverriding the default filterTemporarily Suppressing WarningsTesting WarningsUpdating Code For New Versions of DependenciesAvailable FunctionsAvailable Context ManagersConcurrent safety of Context Managers
- `dataclasses`— Data ClassesModule contentsPost-init processingClass variablesInit-only variablesFrozen instancesInheritanceRe-ordering of keyword-only parameters in `__init__()`Default factory functionsMutable default valuesDescriptor-typed fields
- `contextlib`— Utilities forwith-statement contextsUtilitiesExamples and RecipesSupporting a variable number of context managersCatching exceptions from `__enter__` methodsCleaning up in an `__enter__` implementationReplacing any use oftry-finallyand flag variablesUsing a context manager as a function decoratorSingle use, reusable and reentrant context managersReentrant context managersReusable context manag
- `abc`— Abstract Base Classes
- `atexit`— Exit handlersatexitExample
- `traceback`— Print or retrieve a stack tracebackModule-Level FunctionsTracebackExceptionObjectsStackSummaryObjectsFrameSummaryObjectsExamples of Using the Module-Level FunctionsExamples of UsingTracebackException
- `__future__`— Future statement definitionsModule Contents
- `gc`— Garbage Collector interface
- `inspect`— Inspect live objectsTypes and membersRetrieving source codeIntrospecting callables with the Signature objectClasses and functionsThe interpreter stackFetching attributes staticallyCurrent State of Generators, Coroutines, and Asynchronous GeneratorsCode Objects Bit FlagsBuffer flagsCommand-line interface
- `annotationlib`— Functionality for introspecting annotationsAnnotation semanticsClassesFunctionsRecipesUsing annotations in a metaclassLimitations of theSTRINGformatLimitations of theFORWARDREFformatSecurity implications of introspecting annotations

- `site`— Site-specific configuration hooks `sitecustomize` `usercustomize` `Readline` configuration Module contents Command-line interface

See also

- See `concurrent.interpreters` module, which similarly exposes core runtime functionality.

sys— System-specific parameters and functions

This module provides access to some variables used or maintained by the interpreter and to functions that interact strongly with the interpreter. It is always available. Unless explicitly noted otherwise, all variables are read-only.

sys.abiflags

On POSIX systems where Python was built with the standard C library, this contains the ABI flags as specified by PEP 3149. Added in version 3.2. Changed in version 3.8: Default flags became an empty string (mflag for pymalloc has been removed). Availability: Unix.

sys.addaudithook(hook)

Append the callable `hook` to the list of active auditing hooks for the current (sub)interpreter. When an auditing event is raised through the `sys.audit()` function, each hook will be called in the order it was added with the event name and the tuple of arguments. Native hooks added by `PySys_AddAuditHook()` are called first, followed by hooks added in the current (sub)interpreter. Hooks can then log the event, raise an exception to abort the operation, or terminate the process entirely. Note that audit hoo

sys.argv

The list of command line arguments passed to a Python script. `argv[0]` is the script name (it is operating system dependent whether this is a full pathname or not). If the command was executed using the `-c` command line option to the interpreter, `argv[0]` is set to the string `'-c'`. If no script name was passed to the Python interpreter, `argv[0]` is the empty string. To loop over the standard input, or the list of files given on the command line, see the `fileinput` module. See also `sys.orig_argv`. Note On Unix, comm

sys.audit(event,*args)

Raise an auditing event and trigger any active auditing hooks. `event` is a string identifying the event, and `args` may contain optional arguments with more information about the event. The number and types of arguments for a given event are considered a public and stable API and should not be modified between releases. For example, one auditing event is named `os.chdir`. This event has one argument called `path` that will contain the requested new working directory. `sys.audit()` will call the existing auditing

sys.base_exec_prefix

Equivalent to `exec_prefix`, but referring to the base Python installation. When running under Virtual Environments, `exec_prefix` gets overwritten to the virtual environment prefix. `base_exec_prefix`, conversely, does not change, and always points to the base Python installation. Refer to Virtual Environments for more information. Added in version 3.3.

sys.base_prefix

Equivalent to `prefix`, but referring to the base Python installation. When running under virtual environment, `prefix` gets overwritten to the virtual environment prefix. `base_prefix`, conversely, does not change, and always points to the base Python installation. Refer to Virtual Environments for more information. Added in version 3.3.

sys.byteorder

An indicator of the native byte order. This will have the value `'big'` on big-endian (most-significant byte first) platforms, and `'little'` on little-endian (least-significant byte first) platforms.

sys.builtin_module_names

A tuple of strings containing the names of all modules that are compiled into this Python interpreter. (This information is not available in any other way — `modules.keys()` only lists the imported modules.) See also

thesys.stdlib_module_nameslist.

sys.call_tracing(func,args)

Callfunc(*args), while tracing is enabled. The tracing state is saved, and restored afterwards. This is intended to be called from a debugger from a checkpoint, to recursively debug or profile some other code. Tracing is suspended while calling a tracing function set by `settrace()` or `setprofile()` to avoid infinite recursion. `call_tracing()` enables explicit recursion of the tracing function.

sys.copyright

A string containing the copyright pertaining to the Python interpreter.

sys._clear_type_cache()

Clear the internal type cache. The type cache is used to speed up attribute and method lookups. Use the function only to drop unnecessary references during reference leak debugging. This function should be used for internal and specialized purposes only. Deprecated since version 3.13: Use the more general `_clear_internal_caches()` function instead.

sys._clear_internal_caches()

Clear all internal performance-related caches. Use this function only to release unnecessary references and memory blocks when hunting for leaks. Added in version 3.13.

sys._current_frames()

Return a dictionary mapping each thread's identifier to the topmost stack frame currently active in that thread at the time the function is called. Note that functions in the `traceback` module can build the call stack given such a frame. This is most useful for debugging deadlock: this function does not require the deadlocked threads' cooperation, and such threads' call stacks are frozen for as long as they remain deadlocked. The frame returned for a non-deadlocked thread may bear no relationship

sys._current_exceptions()

Return a dictionary mapping each thread's identifier to the topmost exception currently active in that thread at the time the function is called. If a thread is not currently handling an exception, it is not included in the result dictionary. This is most useful for statistical profiling. This function should be used for internal and specialized purposes only. Raises a `RuntimeError` if `sys._current_exceptions` is called with no arguments. Changed in version 3.12: Each value in the dictionary is now a single exception

sys.breakpointhook()

This hook function is called by `builtins.breakpoint()`. By default, it drops you into the `pdb` debugger, but it can be set to any other function so that you can choose which debugger gets used. The signature of this function is dependent on what it calls. For example, the default binding (e.g. `pdb.set_trace()`) expects no arguments, but you might bind it to a function that expects additional arguments (positional and/or keyword). The `builtins.breakpoint()` function passes its `*args` and `**kwargs` straight through

sys._debugmallocstats()

Print low-level information to `stderr` about the state of CPython's memory allocator. If Python is built in debug mode (configure with `--with-pydebug` option), it also performs some expensive internal consistency checks. Added in version 3.3. CPython implementation detail: This function is specific to CPython. The exact output format is not defined here, and may change.

sys.dllhandle

Integer specifying the handle of the Python DLL. Availability: Windows.

sys.displayhook(value)

If `value` is not `None`, this function prints `repr(value)` to `sys.stdout`, and saves `value` in `builtins._`. If `repr(value)` is not encodable to `sys.stdout.encoding` with `sys.stdout.errorhandler` (which is probably 'strict'), encode it to `sys.stdout.encoding` with 'backslashreplace' error handler. `sys.displayhook` is called on the result of evaluating an expression entered in an interactive Python session. The display of these values can be customized by assigning another one-argument function to `sys.displayhook`. Pseudo-code

`sys.dont_write_bytecode`

If this is true, Python won't try to write .pyc files on the import of source modules. This value is initially set to `True` or `False` depending on the `-B` command line option and the `PYTHONDONTWRITEBYTECODE` environment variable, but you can set it yourself to control bytecode file generation.

`sys._emscripten_info`

`_emscripten_info.emscripten_version`

`_emscripten_info.runtime`

`_emscripten_info.threads`

`_emscripten_info.shared_memory`

A named tuple holding information about the environment on the `wasms32-emscripten` platform. The named tuple is provisional and may change in the future. `_emscripten_info.emscripten_version` Emscripten version as tuple of ints (major, minor, micro), e.g. (3,1,8). `_emscripten_info.runtime` Runtime string, e.g. browser user agent, 'Node.js v14.18.2', or 'UNKNOWN'. `_emscripten_info.threads` True if Python is compiled with Emscripten threads support. `_emscripten_info.shared_memory` True if Python is compiled with shared

Emscripten version as tuple of ints (major, minor, micro), e.g. (3,1,8).

Runtime string, e.g. browser user agent, 'Node.js v14.18.2', or 'UNKNOWN'.

True if Python is compiled with Emscripten threads support.

True if Python is compiled with shared memory support.

`sys.pycache_prefix`

If this is set (not `None`), Python will write bytecode-cache .pyc files to (and read them from) a parallel directory tree rooted at this directory, rather than from `__pycache__` directories in the source code tree. Any `__pycache__` directories in the source code tree will be ignored and new .pyc files written within the `pycache` prefix. Thus if you use `compileall` as a pre-build step, you must ensure you run it with the same `pycache` prefix (if any) that you will use at runtime. A relative path is interpreted `rel`

`sys.excepthook(type,value,traceback)`

This function prints out a given traceback and exception to `sys.stderr`. When an exception other than `SystemExit` is raised and uncaught, the interpreter calls `sys.excepthook` with three arguments, the exception class, exception instance, and a traceback object. In an interactive session this happens just before control is returned to the prompt; in a Python program this happens just before the program exits. The handling of such top-level exceptions can be customized by assigning another three-argumen

`sys.__breakpointhook__`

`sys.__displayhook__`

`sys.__excepthook__`

`sys.__unraisablehook__`

These objects contain the original values of `breakpointhook`, `displayhook`, `excepthook`, and `unraisablehook` at the start of the program. They are saved so that `breakpointhook`, `displayhook` and `excepthook`, `unraisablehook` can be restored in case they happen to get replaced with broken or alternative objects. Added in version

3.7: `__breakpointhook__` Added in version 3.8: `__unraisablehook__`

sys.exception()

This function, when called while an exception handler is executing (such as `except except*clause`), returns the exception instance that was caught by this handler. When exception handlers are nested within one another, only the exception handled by the innermost handler is accessible. If no exception handler is executing, this function returns `None`. Added in version 3.11.

sys.exc_info()

This function returns the old-style representation of the handled exception. If an exception is currently handled (`sys.exception()` would return `e`), `exc_info()` returns the tuple `(type(e), e, e.__traceback__)`. That is, a tuple containing the type of the exception (a subclass of `BaseException`), the exception itself, and a `traceback` object which typically encapsulates the call stack at the point where the exception last occurred. If no exception is being handled anywhere on the stack, this function returns a tuple

sys.exec_prefix

A string giving the site-specific directory prefix where the platform-dependent Python files are installed; by default, this is also `'usr/local'`. This can be set at build time with the `--exec-prefix` argument to the `configure` script. Specifically, all configuration files (e.g. the `pyconfig.h` header file) are installed in the directory `exec_prefix/lib/pythonX.Y/config`, and shared library modules are installed in `exec_prefix/lib/pythonX.Y/lib-dynload`, where `X.Y` is the version number of Python, for example 3

sys.executable

A string giving the absolute path of the executable binary for the Python interpreter, on systems where this makes sense. If Python is unable to retrieve the real path to its executable, `sys.executable` will be an empty string or `None`.

sys.exit([arg])

Raise a `SystemExit` exception, signaling an intention to exit the interpreter. The optional argument `arg` can be an integer giving the exit status (defaulting to zero), or another type of object. If it is an integer, zero is considered “successful termination” and any nonzero value is considered “abnormal termination” by shells and the like. Most systems require it to be in the range 0–127, and produce undefined results otherwise. Some systems have a convention for assigning specific meanings to spe

sys.flags

flags.debug

flags.inspect

flags.interactive

flags.isolated

flags.optimize

flags.dont_write_bytecode

flags.no_user_site

flags.no_site

flags.ignore_environment

flags.verbose

flags.bytes_warning

flags.quiet

flags.hash_randomization

flags.dev_mode

flags.utf8_mode

flags.safe_path

flags.int_max_str_digits

flags.warn_default_encoding

flags.gil

flags.thread_inherit_context

flags.context_aware_warnings

The named tuple `flagsexposes` the status of command line flags. Flags should only be accessed only by name and not by index. The attributes are read only.
`flags.debug-dflags.inspect-iflags.interactive-iflags.isolated-lflags.optimize-Oor-OOflags.dont_write_bytecode-Bflags.no_user_site-sflags.no_site-Sflags.ignore_environment-Eflags.verbose-vflags.bytes_warning-bflags.quiet-qflags.hash_randomization-Rflags.dev_mode-Xdev(Python Development Mode)flags.utf8_mode-Xutf8flags.safe_path-Pflags.int_max_str_`

sys.float_info

float_info.epsilon

float_info.dig

float_info.mant_dig

float_info.max

float_info.max_exp

float_info.max_10_exp

float_info.min

float_info.min_exp

float_info.min_10_exp

float_info.radix

float_info.rounds

A named tuple holding information about the float type. It contains low level information about the precision and internal representation. The values correspond to the various floating-point constants defined in the standard header file `float.h` for the 'C' programming language; see section 5.2.4.2.2 of the 1999 ISO/IEC C standard [C99], 'Characteristics of floating types', for details. Attributes of the `float_info` named tuple: attribute `float.h` macro explanation `float_info.epsilon` DBL_EPSILON difference between

sys.float_repr_style

A string indicating how the `repr()` function behaves for floats. If the string has value 'short' then for a finite float `x`, `repr(x)` aims to produce a short string with the property that `float(repr(x)) == x`. This is the usual behaviour in Python 3.1 and later. Otherwise, `float_repr_style` has value 'legacy' and `repr(x)` behaves in the same way as it did in versions of Python prior to 3.1. Added in version 3.1.

sys.getallocatedblocks()

Return the number of memory blocks currently allocated by the interpreter, regardless of their size. This function is mainly useful for tracking and debugging memory leaks. Because of the interpreter's internal caches, the result can vary from call to call; you may have to call `clear_internal_caches()` and `gc.collect()` to get more

predictable results. If a Python build or implementation cannot reasonably compute this information, `getallocatedblocks()` is allowed to return 0 instead. Added in version 3.

`sys.getunicodeinternedsize()`

Return the number of unicode objects that have been interned. Added in version 3.12.

`sys.getandroidapilevel()`

Return the build-time API level of Android as an integer. This represents the minimum version of Android this build of Python can run on. For runtime version information, see `platform.android_ver()`. Availability: Android. Added in version 3.7.

`sys.getdefaultencoding()`

Return 'utf-8'. This is the name of the default string encoding, used in methods like `str.encode()`.

`sys.getdlopenflags()`

Return the current value of the flags that are used for `dlopen()` calls. Symbolic names for the flag values can be found in the `os` module (RTLD_XXX constants, e.g. `os.RTLD_LAZY`). Availability: Unix.

`sys.getfilesystemencoding()`

Get the filesystem encoding: the encoding used with the filesystem error handler to convert between Unicode filenames and bytes filenames. The filesystem error handler is returned from `getfilesystemencodeerrors()`. For best compatibility, `str` should be used for filenames in all cases, although representing filenames as bytes is also supported. Functions accepting or returning filenames should support either `str` or `bytes` and internally convert to the system's preferred representation. `os.fsencode()` and `os`

`sys.getfilesystemencodeerrors()`

Get the filesystem error handler: the error handler used with the filesystem encoding to convert between Unicode filenames and bytes filenames. The filesystem encoding is returned from `getfilesystemencoding().os.fsencode()` and `os.fsdecode()` should be used to ensure that the correct encoding and errors mode are used. The filesystem encoding and error handler are configured at Python startup by the `PyConfig_Read()` function: see `filesystem_encoding` and `filesystem_errors` members of `PyConfig`. Added in version 3.6.

`sys.get_int_max_str_digits()`

Returns the current value for the integer string conversion length limitation. See `sys.set_int_max_str_digits()`. Added in version 3.11.

`sys.getrefcount(object)`

Return the reference count of the object. The count returned is generally one higher than you might expect, because it includes the (temporary) reference as an argument to `getrefcount()`. Note that the returned value may not actually reflect how many references to the object are actually held. For example, some objects are immortal and have a very high refcount that does not reflect the actual number of references. Consequently, do not rely on the returned value to be accurate, other than a value 0.

`sys.getrecursionlimit()`

Return the current value of the recursion limit, the maximum depth of the Python interpreter stack. This limit prevents infinite recursion from causing an overflow of the C stack and crashing Python. It can be set by `sys.setrecursionlimit()`.

`sys.getsizeof(object[, default])`

Return the size of an object in bytes. The object can be any type of object. All built-in objects will return correct results, but this does not have to hold true for third-party extensions as it is implementation specific. Only the

memory consumption directly attributed to the object is accounted for, not the memory consumption of objects it refers to. If given, default will be returned if the object does not provide means to retrieve the size. Otherwise a `TypeError` will be raised. `getsizeof()` calls `t`

`sys.getswitchinterval()`

Return the interpreter's "thread switch interval" in seconds; see `setswitchinterval()`. Added in version 3.2.

`sys._getframe([depth])`

Return a frame object from the call stack. If optional integer `depth` is given, return the frame object that many calls below the top of the stack. If that is deeper than the call stack, `ValueError` is raised. The default for `depth` is zero, returning the frame at the top of the call stack. Raises an auditing event `sys._getframe` with argument `frame`. CPython implementation detail: This function should be used for internal and specialized purposes only. It is not guaranteed to exist in all implementations of P

`sys._getframemodulename([depth])`

Return the name of a module from the call stack. If optional integer `depth` is given, return the module that many calls below the top of the stack. If that is deeper than the call stack, or if the module is unidentifiable, `None` is returned. The default for `depth` is zero, returning the module at the top of the call stack. Raises an auditing event `sys._getframemodulename` with argument `depth`. CPython implementation detail: This function should be used for internal and specialized purposes only. It is not guar

`sys.getobjects(limit[,type])`

This function only exists if CPython was built using the specialized configure option `--with-trace-refs`. It is intended only for debugging garbage-collection issues. Return a list of up to `limit` dynamically allocated Python objects. If `type` is given, only objects of that exact type (not subtypes) are included. Objects from the list are not safe to use. Specifically, the result will include objects from all interpreters that share their object allocator state (that is, ones created with `PyInterpreterConf`

`sys.getprofile()`

Get the profiler function as set by `setprofile()`.

`sys.gettrace()`

Get the trace function as set by `settrace()`. CPython implementation detail: The `gettrace()` function is intended only for implementing debuggers, profilers, coverage tools and the like. Its behavior is part of the implementation platform, rather than part of the language definition, and thus may not be available in all Python implementations.

`sys.getwindowsversion()`

Return a named tuple describing the Windows version currently running. The named elements are `major`, `minor`, `build`, `platform`, `service_pack`, `service_pack_minor`, `service_pack_major`, `suite_mask`, `product_type` and `platform_version`. `service_pack` contains a string, `platform_version` a 3-tuple and all other values are integers. The components can also be accessed by name, so `sys.getwindowsversion()[0]` is equivalent to `sys.getwindowsversion().major`. For compatibility with prior versions, only the first 5 elements are retri

`sys.get_asyncgen_hooks()`

Returns an `asyncgen_hooks` object, which is similar to a named tuple of the form `(firstiter, finalizer)`, where `firstiter` and `finalizer` are expected to be either `None` or functions which take an asynchronous generator iterator as an argument, and are used to schedule finalization of an asynchronous generator by an event loop. Added in version 3.6: See PEP 525 for more details. Note This function has been added on a provisional basis (see PEP 411 for details.)

`sys.get_coroutine_origin_tracking_depth()`

Get the current coroutine origin tracking depth, as set by `set_coroutine_origin_tracking_depth()`. Added in version 3.7. **Note** This function has been added on a provisional basis (see PEP 411 for details.) Use it only for debugging purposes.

sys.hash_info

hash_info.width

hash_info.modulus

hash_info.inf

hash_info.nan

hash_info.imag

hash_info.algorithm

hash_info.hash_bits

hash_info.seed_bits

hash_info.cutoff

A named tuple giving parameters of the numeric hash implementation. For more details about hashing of numeric types, see [Hashing of numeric types](#). **hash_info.width** The width in bits used for hash values **hash_info.modulus** The prime modulus P used for numeric hash scheme **hash_info.inf** The hash value returned for a positive infinity **hash_info.nan** (This attribute is no longer used) **hash_info.imag** The multiplier used for the imaginary part of a complex number **hash_info.algorithm** The name of the algorithm for hashing

The width in bits used for hash values

The prime modulus P used for numeric hash scheme

The hash value returned for a positive infinity

(This attribute is no longer used)

The multiplier used for the imaginary part of a complex number

The name of the algorithm for hashing of str, bytes, and memoryview

The internal output size of the hash algorithm

The size of the seed key of the hash algorithm

Cutoff for small string DJBX33A optimization in range[1, cutoff).

sys.hexversion

The version number encoded as a single integer. This is guaranteed to increase with each version, including proper support for non-production releases. For example, to test that the Python interpreter is at least version 1.5.2, use: `if sys.hexversion >= 0x010502F0: # use some advanced feature... else: # use an alternative implementation or warn the user...` This is called `hexversion` since it only really looks meaningful when viewed as the result of passing it to the built-in `hex()` function. Then a named tuple

sys.implementation

An object containing information about the implementation of the currently running Python interpreter. The following attributes are required to exist in all Python implementations. **name** is the implementation's identifier, e.g. 'cpython'. The actual string is defined by the Python implementation, but it is guaranteed to be lower case. **version** is a named tuple, in the same format as `sys.version_info`. It represents the version of the Python implementation. This has a distinct meaning from the specific

sys.int_info

int_info.bits_per_digit

int_info.sizeof_digit

int_info.default_max_str_digits

int_info.str_digits_check_threshold

Anamed tuplethat holds information about Python's internal representation of integers. The attributes are read only.
int_info.bits_per_digitThe number of bits held in each digit. Python integers are stored internally in $\text{base}2^{**}\text{int_info.bits_per_digit}$.
int_info.sizeof_digitThe size in bytes of the C type used to represent a digit.
int_info.default_max_str_digitsThe default value for `sys.get_int_max_str_digits()` when it is not otherwise explicitly configured.
int_info.str_digits_check_thresholdThe mini

The number of bits held in each digit. Python integers are stored internally in $\text{base}2^{**}\text{int_info.bits_per_digit}$.

The size in bytes of the C type used to represent a digit.

The default value for `sys.get_int_max_str_digits()` when it is not otherwise explicitly configured.

The minimum non-zero value for `sys.set_int_max_str_digits()`, `PYTHONINTMAXSTRDIGITS`, or `Xint_max_str_digits`.

sys.__interactivehook__

When this attribute exists, its value is automatically called (with no arguments) when the interpreter is launched in interactive mode. This is done after the `PYTHONSTARTUP` file is read, so that you can set this hook there. The `site` module sets this. Raises an `auditing` event `cpython.run_interactivehook` with the hook object as the argument when the hook is called on startup. Added in version 3.4.

sys.intern(string)

Enter `string` in the table of "interned" strings and return the interned string – which is `string` itself or a copy. Interning strings is useful to gain a little performance on dictionary lookup – if the keys in a dictionary are interned, and the lookup key is interned, the key comparisons (after hashing) can be done by a pointer compare instead of a string compare. Normally, the names used in Python programs are automatically interned, and the dictionaries used to hold module, class or instance attr

sys.is_gil_enabled()

Return `True` if the GIL is enabled and `False` if it is disabled. Added in version 3.13. CPython implementation detail: It is not guaranteed to exist in all implementations of Python.

sys.is_finalizing()

Return `True` if the main Python interpreter is shutting down. Return `False` otherwise. See also the `PythonFinalizationError` exception. Added in version 3.5.

sys._jit

_jit.is_available()

_jit.is_enabled()

_jit.is_active()

Utilities for observing just-in-time compilation. CPython implementation detail: JIT compilation is an experimental implementation detail of CPython. `sys._jit` is not guaranteed to exist or behave the same way in all Python implementations, versions, or build configurations. Added in version 3.14. `_jit.is_available()` Return `True` if the current Python executable supports JIT compilation, and `False` otherwise. This can be controlled by building CPython with the `--experimental-jit` option on Windows, and the `--enabl`

Return True if the current Python executable supports JIT compilation, and False otherwise. This can be controlled by building CPython with the `--experimental-jit` option on Windows, and the `--enable-experimental-jit` option on all other platforms.

Return True if JIT compilation is enabled for the current Python process (implies `sys._jit.is_available()`), and False otherwise. If JIT compilation is available, this can be controlled by setting the `PYTHON_JIT` environment variable to 0 (disabled) or 1 (enabled) at interpreter startup.

Return True if the topmost Python frame is currently executing JIT code (implies `sys._jit.is_enabled()`), and False otherwise. Note This function is intended for testing and debugging the JIT itself. It should be avoided for any other purpose. Note Due to the nature of tracing JIT compilers, repeated calls to this function may give surprising results. For example, branching on its return value will likely lead to unexpected behavior (if doing so causes JIT code to be entered or exited):
>>> for warmup in range

sys.last_exc

This variable is not always defined; it is set to the exception instance when an exception is not handled and the interpreter prints an error message and a stack traceback. Its intended use is to allow an interactive user to import a debugger module and engage in post-mortem debugging without having to re-execute the command that caused the error. (Typical use is `import pdb; pdb.pm()` to enter the post-mortem debugger; see `pdb` module for more information.) Added in version 3.12.

sys._is_immortal(op)

Return True if the given object is immortal, False otherwise. Note Objects that are immortal (and thus return True upon being passed to this function) are not guaranteed to be immortal in future versions, and vice versa for mortal objects. Added in version 3.14. CPython implementation detail: This function should be used for specialized purposes only. It is not guaranteed to exist in all implementations of Python.

sys._is_interned(string)

Return True if the given string is “interned”, False otherwise. Added in version 3.13. CPython implementation detail: It is not guaranteed to exist in all implementations of Python.

sys.last_type

sys.last_value

sys.last_traceback

These three variables are deprecated; use `sys.last_exc` instead. They hold the legacy representation of `sys.last_exc`, as returned from `exc_info()` above.

sys.maxsize

An integer giving the maximum value a variable of type `Py_ssize_t` can take. It's usually $2^{31}-1$ on a 32-bit platform and $2^{63}-1$ on a 64-bit platform.

sys.maxunicode

An integer giving the value of the largest Unicode code point, i.e. `0x10FFFF` in hexadecimal). Changed in version 3.3: Before PEP 393, `sys.maxunicode` used to be either `0xFFFF` or `0x10FFFF`, depending on the configuration option that specified whether Unicode characters were stored as UCS-2 or UCS-4.

sys.meta_path

importlib.abc.MetaPathFinder

importlib.machinery.ModuleSpec

A list of meta path finder objects that have their `find_spec()` methods called to see if one of the objects can find the module to be imported. By default, it holds entries that implement Python's default import semantics.

The `find_spec()` method is called with at least the absolute name of the module being imported. If the module to be imported is contained in a package, then the parent package's `__path__` attribute is passed in as a second argument. The method returns a module spec, or `None` if the module cannot be found.

The abstract base class defining the interface of finder objects on `meta_path`.

The concrete class which `find_spec()` should return instances of.

sys.modules

This is a dictionary that maps module names to modules which have already been loaded. This can be manipulated to force reloading of modules and other tricks. However, replacing the dictionary will not necessarily work as expected and deleting essential items from the dictionary may cause Python to fail. If you want to iterate over this global dictionary always use `sys.modules.copy()` or `tuple(sys.modules)` to avoid exceptions as its size may change during iteration as a side effect of code or activity.

sys.orig_argv

The list of the original command line arguments passed to the Python executable. The elements of `sys.orig_argv` are the arguments to the Python interpreter, while the elements of `sys.argv` are the arguments to the user's program. Arguments consumed by the interpreter itself will be present in `sys.orig_argv` and missing from `sys.argv`. Added in version 3.10.

sys.path

A list of strings that specifies the search path for modules. Initialized from the environment variable `PYTHONPATH`, plus an installation-dependent default. By default, as initialized upon program startup, a potentially unsafe path is prepended to `sys.path` (before the entries inserted as a result of `PYTHONPATH`): `python-mmodule` command line: prepend the current working directory. `python-script.py` command line: prepend the script's directory. If it's a symbolic link, resolve symbolic links. `python-ccode` and `python`.

sys.path_hooks

A list of callables that take a path argument to try to create a finder for the path. If a finder can be created, it is to be returned by the callable, else raise `ImportError`. Originally specified in PEP 302.

sys.path_importer_cache

A dictionary acting as a cache for finder objects. The keys are paths that have been passed to `sys.path_hooks` and the values are the finders that are found. If a path is a valid file system path but no finder is found on `sys.path_hooks` then `None` is stored. Originally specified in PEP 302.

sys.platform

A string containing a platform identifier. Known values are: `SystemPlatformValue` `AIX` `aix` `Android` `android` `Emscripten` `emscripten` `FreeBSD` `freebsd` `iOS` `ios` `Linux` `linux` `macOS` `darwin` `Windows` `win32` `Windows/Cygwin` `cygwin` `WASI` `wasi` On Unix systems not listed in the table, the value is the lowercased OS name as returned by `uname -s`, with the first part of the version as returned by `uname -r` appended, e.g. `'sunos5'`, at the time when Python was built. Unless you want to test for a specific system version, it is there.

sys.platlibdir

Name of the platform-specific library directory. It is used to build the path of standard library and the paths of installed extension modules. It is equal to `"lib"` on most platforms. On Fedora and SuSE, it is equal to `"lib64"` on 64-bit platforms which gives the following `sys.path` paths (where `X.Y` is the Python major.minor version): `/usr/lib64/pythonX.Y/`: Standard library (like `os.py` of the `os` module) `/usr/lib64/pythonX.Y/lib-dynload/`: C extension modules of the standard library (like `theerrnomodule`, the exact f

sys.prefix

A string giving the site-specific directory prefix where the platform independent Python files are installed; on Unix, the default is `/usr/local`. This can be set at build time with the `--prefix` argument to `theconfigurescript`. See `Installation paths for derived paths`. Note if a virtual environment is in effect, this prefix will point to the virtual environment. The value for the Python installation will still be available, via `base_prefix`. Refer to `Virtual Environments` for more information. Changed in version 3.3

sys.ps1

sys.ps2

Strings specifying the primary and secondary prompt of the interpreter. These are only defined if the interpreter is in interactive mode. Their initial values in this case are `'>>>'` and `'...'`. If a non-string object is assigned to either variable, its `str()` is re-evaluated each time the interpreter prepares to read a new interactive command; this can be used to implement a dynamic prompt.

sys.setdlopenflags(n)

Set the flags used by the interpreter for `dlopen()` calls, such as when the interpreter loads extension modules. Among other things, this will enable a lazy resolving of symbols when importing a module, if called as `sys.setdlopenflags(0)`. To share symbols across extension modules, call `sys.setdlopenflags(os.RTLD_GLOBAL)`. Symbolic names for the flag values can be found in the `os` module (`RTLD_XXX` constants, e.g. `os.RTLD_LAZY`). Availability: Unix.

sys.set_int_max_str_digits(maxdigits)

Set the integer string conversion length limitation used by this interpreter. See `sys.get_int_max_str_digits()`. Added in version 3.11.

sys.setprofile(profilefunc)

'call'

'return'

'c_call'

'c_return'

'c_exception'

Set the system's profile function, which allows you to implement a Python source code profiler in Python. See chapter `The Python Profilers` for more information on the Python profiler. The system's profile function is called similarly to the system's trace function (see `settrace()`), but it is called with different events, for example it isn't called for each executed line of code (only on call and return, but the return event is reported even when an exception has been set). The function is thread

A function is called (or some other code block entered). The profile function is called; `arg` is `None`.

A function (or other code block) is about to return. The profile function is called; `arg` is the value that will be returned, or `None` if the event is caused by an exception being raised.

A C function is about to be called. This may be an extension function or a built-in. `arg` is the C function object.

A C function has returned. `arg` is the C function object.

A C function has raised an exception. `arg` is the C function object.

sys.setrecursionlimit(limit)

Set the maximum depth of the Python interpreter stack to `limit`. This limit prevents infinite recursion from causing an overflow of the C stack and crashing Python. The highest possible limit is platform-dependent. A user

may need to set the limit higher when they have a program that requires deep recursion and a platform that supports a higher limit. This should be done with care, because a too-high limit can lead to a crash. If the new limit is too low at the current recursion depth, a `RecursionError`

`sys.setswitchinterval(interval)`

Set the interpreter's thread switch interval (in seconds). This floating-point value determines the ideal duration of the "timeslices" allocated to concurrently running Python threads. Please note that the actual value can be higher, especially if long-running internal functions or methods are used. Also, which thread becomes scheduled at the end of the interval is the operating system's decision. The interpreter doesn't have its own scheduler. Added in version 3.2.

`sys.settrace(tracefunc)`

'call'

'line'

'return'

'exception'

'opcode'

Set the system's trace function, which allows you to implement a Python source code debugger in Python. The function is thread-specific; for a debugger to support multiple threads, it must register a trace function using `sys.settrace()` for each thread being debugged or `threading.settrace()`. Trace functions should have three arguments: `frame`, `event`, and `arg`. `frame` is the current stack frame. `event` is a string: 'call', 'line', 'return', 'exception' or 'opcode'. `arg` depends on the event type. The trace function is invoked

A function is called (or some other code block entered). The global trace function is called; `arg` is `None`; the return value specifies the local trace function.

The interpreter is about to execute a new line of code or re-execute the condition of a loop. The local trace function is called; `arg` is `None`; the return value specifies the new local trace function. See `Objects/notab_notes.txt` for a detailed explanation of how this works. Per-line events may be disabled for a frame by setting `frame.f_trace_line` to `False` on that frame.

A function (or other code block) is about to return. The local trace function is called; `arg` is the value that will be returned, or `None` if the event is caused by an exception being raised. The trace function's return value is ignored.

An exception has occurred. The local trace function is called; `arg` is a tuple (`exception`, `value`, `traceback`); the return value specifies the new local trace function.

The interpreter is about to execute a new opcode (see `dis` for opcode details). The local trace function is called; `arg` is `None`; the return value specifies the new local trace function. Per-opcode events are not emitted by default: they must be explicitly requested by setting `frame.f_trace_opcode` to `True` on the frame.

`sys.set_asyncgen_hooks([firstiter][,finalizer])`

Accepts two optional keyword arguments which are callables that accept an asynchronous generator iterator as an argument. The `firstiter` callable will be called when an asynchronous generator is iterated for the first time. The `finalizer` will be called when an asynchronous generator is about to be garbage collected. Raises an `auditing` event `sys.set_asyncgen_hooks_firstiter` with no arguments. Raises an `auditing` event `sys.set_asyncgen_hooks_finalizer` with no arguments. Two auditing events are raised because the u

`sys.set_coroutine_origin_tracking_depth(depth)`

Allows enabling or disabling coroutine origin tracking. When enabled, the `cr_origin` attribute on coroutine objects will contain a tuple of (filename, line number, function name) tuples describing the traceback where the coroutine object was created, with the most recent call first. When disabled, `cr_origin` will be `None`. To enable,

pass a depth value greater than zero; this sets the number of frames whose information will be captured. To disable, set depth to zero. This setting is thread-specific. Added in v

sys.activate_stack_trampoline(backend, /)

Activate the stack profiler trampoline backend. The only supported backend is "perf". Stack trampolines cannot be activated if the JIT is active. Availability: Linux. Added in version 3.12. See also Python support for the Linux perf profiler <https://perf.wiki.kernel.org>

sys.deactivate_stack_trampoline()

Deactivate the current stack profiler trampoline backend. If no stack profiler is activated, this function has no effect. Availability: Linux. Added in version 3.12.

sys.is_stack_trampoline_active()

Return True if a stack profiler trampoline is active. Availability: Linux. Added in version 3.12.

sys.remote_exec(pid, script)

Execute script, a file containing Python code in the remote process with the given pid. This function returns immediately, and the code will be executed by the target process's main thread at the next available opportunity, similarly to how signals are handled. There is no interface to determine when the code has been executed. The caller is responsible for making sure that the file still exists whenever the remote process tries to read it and that it hasn't been overwritten. The remote process must

sys._enablelegacywindowsfsencoding()

Changes the filesystem encoding and error handler to 'mbcs' and 'replace' respectively, for consistency with versions of Python prior to 3.6. This is equivalent to defining the PYTHONLEGACYWINDOWSFSENCODING environment variable before launching Python. See also `sys.getfilesystemencoding()` and `sys.getfilesystemencodeerrors()`. Availability: Windows. Note Changing the filesystem encoding after Python startup is risky because the old fsencoding or paths encoded by the old fsencoding may be cached somewhere. Use

sys.stdin

sys.stdout

sys.stderr

File objects used by the interpreter for standard input, output and errors: `stdin` is used for all interactive input (including calls to `input()`); `stdout` is used for the output of `print()` and expression statements and for the prompts of `input()`; The interpreter's own prompts and its error messages go to `stderr`. These streams are regular text files like those returned by the `open()` function. Their parameters are chosen as follows: The encoding and error handling are initialized from `PyConfig.stdio_encoding` and `PyCo`

sys.__stdin__

sys.__stdout__

sys.__stderr__

These objects contain the original values of `stdin`, `stderr` and `stdout` at the start of the program. They are used during finalization, and could be useful to print to the actual standard stream no matter if the `sys.std*object` has been redirected. It can also be used to restore the actual files to known working file objects in case they have been overwritten with a broken object. However, the preferred way to do this is to explicitly save the previous stream before replacing it, and restore the saved o

sys.stdlib_module_names

A frozenset of strings containing the names of standard library modules. It is the same on all platforms. Modules which are not available on some platforms and modules disabled at Python build are also listed. All module kinds are listed: pure Python, built-in, frozen and extension modules. Test modules are excluded. For packages, only the main package is listed: sub-packages and sub-modules are not listed. For example, the email package is listed, but email.mime sub-package and email.message module

sys.thread_info

thread_info.name

thread_info.lock

thread_info.version

A named tuple holding information about the thread implementation.

thread_info.name The name of the thread implementation: "nt": Windows threads "pthread": POSIX threads "pthread-stubs": stub POSIX threads (on WebAssembly platforms without threading support) "solaris": Solaris threads

thread_info.lock The name of the lock implementation: "semaphore": a lock uses a semaphore "mutex+cond": a lock uses a mutex and a condition variable None if this information is unknown

thread_info.version The name and version of

The name of the thread implementation: "nt": Windows threads "pthread": POSIX threads "pthread-stubs": stub POSIX threads (on WebAssembly platforms without threading support) "solaris": Solaris threads

The name of the lock implementation: "semaphore": a lock uses a semaphore "mutex+cond": a lock uses a mutex and a condition variable None if this information is unknown

The name and version of the thread library. It is a string, or None if this information is unknown.

sys.tracebacklimit

When this variable is set to an integer value, it determines the maximum number of levels of traceback information printed when an unhandled exception occurs. The default is 1000. When set to 0 or less, all traceback information is suppressed and only the exception type and value are printed.

sys.unraisablehook(unraisable, /)

Handle an unraisable exception. Called when an exception has occurred but there is no way for Python to handle it. For example, when a destructor raises an exception or during garbage collection (gc.collect()).

The unraisable argument has the following attributes:

- exc_type**: Exception type
- exc_value**: Exception value, can be None
- exc_traceback**: Exception traceback, can be None
- err_msg**: Error message, can be None
- object**: Object causing the exception, can be None

The default hook format is: `serr_msg and object as: f`

sys.version

A string containing the version number of the Python interpreter plus additional information on the build number and compiler used. This string is displayed when the interactive interpreter is started. Do not extract version information out of it, rather, use `version_info` and the functions provided by the `platform` module.

sys.api_version

The C API version, equivalent to the C macro `PYTHON_API_VERSION`. Defined for backwards compatibility. Currently, this constant is not updated in new Python versions, and is not useful for versioning. This may change in the future.

sys.version_info

A tuple containing the five components of the version number: major, minor, micro, release level, and serial. All values except release level are integers; the release level is 'alpha', 'beta', 'candidate', or 'final'.

The `version_info` value corresponding to the Python version 2.0 is (2, 0, 0, 'final', 0). The components can also be accessed by name, so `sys.version_info[0]` is equivalent to `sys.version_info.major` and so on. Changed in version 3.1: Added named component attributes.

sys.warnoptions

This is an implementation detail of the warnings framework; do not modify this value. Refer to thewarningsmodule for more information on the warnings framework.

sys.winver

The version number used to form registry keys on Windows platforms. This is stored as string resource 1000 in the Python DLL. The value is normally the major and minor versions of the running Python interpreter. It is provided in thesysmodule for informational purposes; modifying this value has no effect on the registry keys used by Python.Availability: Windows.

sys.monitoring

Namespace containing functions and constants for register callbacks and controlling monitoring events. Seesys.monitoringfor details.

sys._xoptions

A dictionary of the various implementation-specific flags passed through the-Xcommand-line option. Option names are either mapped to their values, if given explicitly, or toTrue. Example:./python-Xa=b-XcPython 3.2a3+ (py3k, Oct 16 2010, 20:14:50)[GCC 4.4.3] on linux2Type "help", "copyright", "credits" or "license" for more information.>>> import sys>>> sys._xoptions{'a': 'b', 'c': True}CPython implementation detail:This is a CPython-specific way of accessing options passed through-X. Other i

Citations

ISO/IEC 9899:1999. "Programming languages – C." A public draft of this standard is available at<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1256.pdf>.

sys.monitoring— Execution event monitoring

Added in version 3.12.

■ Nota: `sys.monitoring` is a namespace within the `sys` module, not an independent module, so there is no need to `import sys.monitoring`, simply `import sys` and then use `sys.monitoring`.

This namespace provides access to the functions and constants necessary to activate and control event monitoring.

As programs execute, events occur that might be of interest to tools that monitor execution. The `sys.monitoring` namespace provides means to receive callbacks when events of interest occur.

The monitoring API consists of three components:

- Tool identifiers
- Events
- Callbacks

Tool identifiers

A tool identifier is an integer and the associated name. Tool identifiers are used to discourage tools from interfering with each other and to allow multiple tools to operate at the same time. Currently tools are completely independent and cannot be used to monitor each other. This restriction may be lifted in the future.

Before registering or activating events, a tool should choose an identifier. Identifiers are integers in the range 0 to 5 inclusive.

Registering and using tools

`sys.monitoring.use_tool_id(tool_id:int,name:str,f)→None`

Must be called before `tool_id` can be used. `tool_id` must be in the range 0 to 5 inclusive. Raises a `ValueError` if `tool_id` is in use.

`sys.monitoring.clear_tool_id(tool_id:int,f)→None`

Unregister all events and callback functions associated with `tool_id`.

`sys.monitoring.free_tool_id(tool_id:int,f)→None`

Should be called once a tool no longer requires `tool_id`. Will call `clear_tool_id()` before releasing `tool_id`.

`sys.monitoring.get_tool(tool_id:int,f)→str|None`

Returns the name of the tool if `tool_id` is in use, otherwise it returns `None`. `tool_id` must be in the range 0 to 5 inclusive.

All IDs are treated the same by the VM with regard to events, but the following IDs are pre-defined to make co-operation of tools easier:

```
sys.monitoring.DEBUGGER_ID = 0
sys.monitoring.COVERAGE_ID = 1
sys.monitoring.PROFILER_ID = 2
sys.monitoring.OPTIMIZER_ID = 5
```

Events

The following events are supported:

sys.monitoring.events.BRANCH_LEFT

A conditional branch goes left. It is up to the tool to determine how to present “left” and “right” branches. There is no guarantee which branch is “left” and which is “right”, except that it will be consistent for the duration of the program.

sys.monitoring.events.BRANCH_RIGHT

A conditional branch goes right.

sys.monitoring.events.CALL

A call in Python code (event occurs before the call).

sys.monitoring.events.C_RAISE

An exception raised from any callable, except for Python functions (event occurs after the exit).

sys.monitoring.events.C_RETURN

Return from any callable, except for Python functions (event occurs after the return).

sys.monitoring.events.EXCEPTION_HANDLED

An exception is handled.

sys.monitoring.events.INSTRUCTION

A VM instruction is about to be executed.

sys.monitoring.events.JUMP

An unconditional jump in the control flow graph is made.

sys.monitoring.events.LINE

An instruction is about to be executed that has a different line number from the preceding instruction.

sys.monitoring.events.PY_RESUME

Resumption of a Python function (for generator and coroutine functions), except for throw() calls.

sys.monitoring.events.PY_RETURN

Return from a Python function (occurs immediately before the return, the callee’s frame will be on the stack).

sys.monitoring.events.PY_START

Start of a Python function (occurs immediately after the call, the callee’s frame will be on the stack)

sys.monitoring.events.PY_THROW

A Python function is resumed by throw() call.

sys.monitoring.events.PY_UNWIND

Exit from a Python function during exception unwinding. This includes exceptions raised directly within the function and that are allowed to continue to propagate.

sys.monitoring.events.PY_YIELD

Yield from a Python function (occurs immediately before the yield, the callee’s frame will be on the stack).

sys.monitoring.events.RAISE

An exception is raised, except those that cause a STOP_ITERATION event.

sys.monitoring.events.RERAISE

An exception is re-raised, for example at the end of a finally block.

sys.monitoring.events.STOP_ITERATION

An artificial StopIteration is raised; see the STOP_ITERATION event.

More events may be added in the future.

These events are attributes of the sys.monitoring.events namespace. Each event is represented as a power-of-2 integer constant. To define a set of events, simply bitwise OR the individual events together. For example, to specify both PY_RETURN and PY_START events, use the expression PY_RETURN|PY_START.

sys.monitoring.events.NO_EVENTS

An alias for 0 so users can do explicit comparisons like: if get_events(DEBUGGER_ID) == NO_EVENTS: ... Setting this event deactivates all events.

Local events

Local events are associated with normal execution of the program and happen at clearly defined locations. All local events can be disabled. The local events are:

- PY_START
- PY_RESUME
- PY_RETURN
- PY_YIELD
- CALL
- LINE
- INSTRUCTION
- JUMP
- BRANCH_LEFT
- BRANCH_RIGHT
- STOP_ITERATION

Deprecated event

- BRANCH

The BRANCH event is deprecated in 3.14. Using BRANCH_LEFT and BRANCH_RIGHT events will give much better performance as they can be disabled independently.

Ancillary events

Ancillary events can be monitored like other events, but are controlled by another event:

- C_RAISE
- C_RETURN

The C_RETURN and C_RAISE events are controlled by the CALL event. C_RETURN and C_RAISE events will only be seen if the corresponding CALL event is being monitored.

Other events

Other events are not necessarily tied to a specific location in the program and cannot be individually disabled via DISABLE.

The other events that can be monitored are:

- PY_THROW
- PY_UNWIND
- RAISE
- EXCEPTION_HANDLED

The STOP_ITERATION event

PEP 380 specifies that a `StopIteration` exception is raised when returning a value from a generator or coroutine. However, this is a very inefficient way to return a value, so some Python implementations, notably CPython 3.12+, do not raise an exception unless it would be visible to other code.

To allow tools to monitor for real exceptions without slowing down generators and coroutines, the `STOP_ITERATION` event is provided. `STOP_ITERATION` can be locally disabled, unlike `RAISE`.

Note that the `STOP_ITERATION` event and the `RAISE` event for a `StopIteration` exception are equivalent, and are treated as interchangeable when generating events. Implementations will favor `STOP_ITERATION` for performance reasons, but may generate a `RAISE` event with a `StopIteration`.

Turning events on and off

In order to monitor an event, it must be turned on and a corresponding callback must be registered. Events can be turned on or off by setting the events either globally and/or for a particular code object. An event will trigger only once, even if it is turned on both globally and locally.

Setting events globally

Events can be controlled globally by modifying the set of events being monitored.

`sys.monitoring.get_events(tool_id:int,/)→int`

Returns the int representing all the active events.

`sys.monitoring.set_events(tool_id:int,event_set:int,/)→None`

Activates all events which are set in `event_set`. Raises a `ValueError` if `tool_id` is not in use.

No events are active by default.

Per code object events

Events can also be controlled on a per code object basis. The functions defined below which accept `types.CodeType` should be prepared to accept a look-alike object from functions which are not defined in Python (see `Monitoring C API`).

`sys.monitoring.get_local_events(tool_id:int,code:CodeType,/)→int`

Returns all the local events for `code`

`sys.monitoring.set_local_events(tool_id:int,code:CodeType,event_set:int,/)→None`

Activates all the local events for `code` which are set in `event_set`. Raises a `ValueError` if `tool_id` is not in use.

Disabling events

`sys.monitoring.DISABLE`

A special value that can be returned from a callback function to disable events for the current code location.

Local events can be disabled for a specific code location by returning `sys.monitoring.DISABLE` from a callback function. This does not change which events are set, or any other code locations for the same event.

Disabling events for specific locations is very important for high performance monitoring. For example, a program can be run under a debugger with no overhead if the debugger disables all monitoring except for a few breakpoints.

If `DISABLE` is returned by a callback for a global event, `ValueError` will be raised by the interpreter in a non-specific location (that is, no traceback will be provided).

`sys.monitoring.restart_events()` → `None`

Enable all the events that were disabled by `sys.monitoring.DISABLE` for all tools.

Registering callback functions

`sys.monitoring.register_callback(tool_id:int,event:int,func:Callable|None,/) → Callable|None`

Registers the callable `func` for the event with the given `tool_id`. If another callback was registered for the given `tool_id` and event, it is unregistered and returned. Otherwise `register_callback()` returns `None`. Raises an auditing event `sys.monitoring.register_callback` with argument `func`.

Functions can be unregistered by calling `sys.monitoring.register_callback(tool_id,event,None)`.

Callback functions can be registered and unregistered at any time.

Callbacks are called only once regardless if the event is turned on both globally and locally. As such, if an event could be turned on for both global and local events by your code then the callback needs to be written to handle either trigger.

Callback function arguments

`sys.monitoring.MISSING`

A special value that is passed to a callback function to indicate that there are no arguments to the call.

When an active event occurs, the registered callback function is called. Callback functions returning an object other than `DISABLE` will have no effect. Different events will provide the callback function with different arguments, as follows:

- `PY_START` and `PY_RESUME`: `func(code:CodeType,instruction_offset:int)→object`
- `PY_RETURN` and `PY_YIELD`: `func(code:CodeType,instruction_offset:int,retval:object)→object`
- `CALL`, `C_RAISE` and `C_RETURN` (`arg0` can be `MISSING` specifically): `func(code:CodeType,instruction_offset:int,callable:object,arg0:object)→object` `code` represents the code object where the call is being made, while `callable` is the object that is about to be called (and thus triggered the event). If there are no arguments, `arg0` is set to `sys.monitoring.MISSING`. For instance methods, `callable` will be the function object `a`
- `RAISE`, `RERAISE`, `EXCEPTION_HANDLED`, `PY_UNWIND`, `PY_THROW` and `STOP_ITERATION`: `func(code:CodeType,instruction_offset:int,exception:BaseException)→object`
- `LINE`: `func(code:CodeType,line_number:int)→object`
- `BRANCH_LEFT`, `BRANCH_RIGHT` and `JUMP`: `func(code:CodeType,instruction_offset:int,destination_offset:int)→object` Note that the `destination_offset` is where the code will next execute.
- `INSTRUCTION`: `func(code:CodeType,instruction_offset:int)→object`

sysconfig— Provide access to Python's configuration information

Added in version 3.2.

Source code: `Lib/sysconfig`

The `sysconfig` module provides access to Python's configuration information like the list of installation paths and the configuration variables relevant for the current platform.

Configuration variables

A Python distribution contains a `Makefile` and `pyconfig.h` header file that are necessary to build both the Python binary itself and third-party C extensions compiled using `setuptools`.

`sysconfig` puts all variables found in these files in a dictionary that can be accessed using `get_config_vars()` or `get_config_var()`.

Notice that on Windows, it's a much smaller set.

`sysconfig.get_config_vars(*args)`

With no arguments, return a dictionary of all configuration variables relevant for the current platform. With arguments, return a list of values that result from looking up each argument in the configuration variable dictionary. For each argument, if the value is not found, return `None`.

`sysconfig.get_config_var(name)`

Return the value of a single variable `name`. Equivalent to `get_config_vars().get(name)`. If `name` is not found, return `None`.

Example of usage:

```
>>> import sysconfig
>>> sysconfig.get_config_var('Py_ENABLE_SHARED')
0
>>> sysconfig.get_config_var('LIBDIR')
'/usr/local/lib'
>>> sysconfig.get_config_vars('AR', 'CXX')
['ar', 'g++']
```

Installation paths

Python uses an installation scheme that differs depending on the platform and on the installation options. These schemes are stored in `sysconfig` under unique identifiers based on the value returned by `os.name`. The schemes are used by package installers to determine where to copy files to.

Python currently supports nine schemes:

- `posix_prefix`: scheme for POSIX platforms like Linux or macOS. This is the default scheme used when Python or a component is installed.
- `posix_home`: scheme for POSIX platforms, when the `home` option is used. This scheme defines paths located under a specific home prefix.
- `posix_user`: scheme for POSIX platforms, when the `user` option is used. This scheme defines paths located under the user's home directory (`site.USER_BASE`).

- `posix_venv`: scheme for Python virtual environments on POSIX platforms; by default it is the same as `posix_prefix`.
- `nt`: scheme for Windows. This is the default scheme used when Python or a component is installed.
- `nt_user`: scheme for Windows, when the `user` option is used.
- `nt_venv`: scheme for Python virtual environments on Windows; by default it is the same as `nt`.
- `venv`: a scheme with values from either `posix_venv` or `nt_venv` depending on the platform Python runs on.
- `osx_framework_user`: scheme for macOS, when the `user` option is used.

Each scheme is itself composed of a series of paths and each path has a unique identifier. Python currently uses eight paths:

- `stdlib`: directory containing the standard Python library files that are not platform-specific.
- `platstdlib`: directory containing the standard Python library files that are platform-specific.
- `platlib`: directory for site-specific, platform-specific files.
- `purelib`: directory for site-specific, non-platform-specific files ('pure' Python).
- `include`: directory for non-platform-specific header files for the Python C-API.
- `platinclude`: directory for platform-specific header files for the Python C-API.
- `scripts`: directory for script files.
- `data`: directory for data files.

User scheme

This scheme is designed to be the most convenient solution for users that don't have write permission to the global site-packages directory or don't want to install into it.

Files will be installed into subdirectories of `site.USER_BASE` (written as `userbase` hereafter). This scheme installs pure Python modules and extension modules in the same location (also known as `site.USER_SITE`).

posix_user

```
Path | Installation directory | stdlib | userbase | /lib/python | X.Y | platstdlib | userbase |
/lib/python | X.Y | platlib | userbase | /lib/python | X.Y | /site-packages | purelib | userbase |
/lib/python | X.Y | /site-packages | include | userbase | /include/python | X.Y | scripts |
userbase | /bin | data | userbase
```

nt_user

```
Path | Installation directory | stdlib | userbase | \Python | XY | platstdlib | userbase |
\Python | XY | platlib | userbase | \Python | XY | \site-packages | purelib | userbase | \Python
| XY | \site-packages | include | userbase | \Python | XY | \Include | scripts | userbase |
\Python | XY | \Scripts | data | userbase
```

osx_framework_user

```
Path | Installation directory | stdlib | userbase | /lib/python | platstdlib | userbase |
/lib/python | platlib | userbase | /lib/python/site-packages | purelib | userbase |
/lib/python/site-packages | include | userbase | /include/python | X.Y | scripts | userbase |
/bin | data | userbase
```

Home scheme

The idea behind the “home scheme” is that you build and maintain a personal stash of Python modules. This scheme’s name is derived from the idea of a “home” directory on Unix, since it’s not unusual for a Unix user to make their home directory have a layout similar to `/usr/or/usr/local/`. This scheme can be used by anyone, regardless of the operating system they are installing for.

posix_home

```
Path | Installation directory | stdlib | home | /lib/python | platstdlib | home | /lib/python |
platlib | home | /lib/python | purelib | home | /lib/python | include | home | /include/python |
platinclude | home | /include/python | scripts | home | /bin | data | home
```

Prefix scheme

The “prefix scheme” is useful when you wish to use one Python installation to perform the build/install (i.e., to run the setup script), but install modules into the third-party module directory of a different Python installation (or something that looks like a different Python installation). If this sounds a trifle unusual, it is—that’s why the user and home schemes come before. However, there are at least two known cases where the prefix scheme will be useful.

First, consider that many Linux distributions put Python in `/usr`, rather than the more traditional `/usr/local`. This is entirely appropriate, since in those cases Python is part of “the system” rather than a local add-on. However, if you are installing Python modules from source, you probably want them to go in `/usr/local/lib/python2.X` rather than `/usr/lib/python2.X`.

Another possibility is a network filesystem where the name used to write to a remote directory is different from the name used to read it: for example, the Python interpreter accessed as `/usr/local/bin/python` might search for modules in `/usr/local/lib/python2.X`, but those modules would have to be installed to, say, `/mnt/@server/export/lib/python2.X`.

posix_prefix

```
Path | Installation directory | stdlib | prefix | /lib/python | X.Y | platstdlib | prefix |
/lib/python | X.Y | platlib | prefix | /lib/python | X.Y | /site-packages | purelib | prefix |
/lib/python | X.Y | /site-packages | include | prefix | /include/python | X.Y | platinclude |
prefix | /include/python | X.Y | scripts | prefix | /bin | data | prefix
```

nt

```
Path | Installation directory | stdlib | prefix | \Lib | platstdlib | prefix | \Lib | platlib |
prefix | \Lib\site-packages | purelib | prefix | \Lib\site-packages | include | prefix |
\Include | platinclude | prefix | \Include | scripts | prefix | \Scripts | data | prefix
```

Installation path functions

`sysconfig` provides some functions to determine these installation paths.

sysconfig.get_scheme_names()

Return a tuple containing all schemes currently supported in `sysconfig`.

sysconfig.get_default_scheme()

Return the default scheme name for the current platform. Added in version 3.10: This function was previously named `_get_default_scheme()` and considered an implementation detail. Changed in version 3.11: When Python runs from a virtual environment, `thevenvscheme` is returned.

sysconfig.get_preferred_scheme(key)

Return a preferred scheme name for an installation layout specified by `key`. `key` must be either "prefix", "home", or "user". The return value is a scheme name listed in `get_scheme_names()`. It can be passed to `sysconfig` functions that take a scheme argument, such as `get_paths()`. Added in version 3.10. Changed in version 3.11: When Python runs from a virtual environment and `key="prefix"`, the `venv` scheme is returned.

`sysconfig._get_preferred_schemes()`

Return a dict containing preferred scheme names on the current platform. Python implementers and redistributors may add their preferred schemes to the `_INSTALL_SCHEMES` module-level global value, and modify this function to return those scheme names, to e.g. provide different schemes for system and language package managers to use, so packages installed by either do not mix with those by the other. End users should not use this function, but `get_default_scheme()` and `get_preferred_scheme()` instead. Added

`sysconfig.get_path_names()`

Return a tuple containing all path names currently supported in `sysconfig`.

`sysconfig.get_path(name[,scheme[,vars[,expand]]])`

Return an installation path corresponding to the `pathname`, from the install scheme named `scheme`. `name` has to be a value from the list returned by `get_path_names()`. `sysconfig` stores installation paths corresponding to each path name, for each platform, with variables to be expanded. For instance the `stdlibpath` for the `ts` scheme is: `{base}/Lib.get_path()` will use the variables returned by `get_config_vars()` to expand the path. All variables have default values for each platform so one may call this function a

`sysconfig.get_paths([scheme[,vars[,expand]]])`

Return a dictionary containing all installation paths corresponding to an installation scheme. See `get_path()` for more information. If `scheme` is not provided, will use the default scheme for the current platform. If `vars` is provided, it must be a dictionary of variables that will update the dictionary used to expand the paths. If `expand` is set to false, the paths will not be expanded. If `scheme` is not an existing scheme, `get_paths()` will raise a `KeyError`.

Other functions

`sysconfig.get_python_version()`

Return the MAJOR.MINOR Python version number as a string. Similar to `'%d.%d'%sys.version_info[:2]`.

`sysconfig.get_platform()`

Return a string that identifies the current platform. This is used mainly to distinguish platform-specific build directories and platform-specific built distributions. Typically includes the OS name and version and the architecture (as supplied by `os.uname()`), although the exact information included depends on the OS; e.g., on Linux, the kernel version isn't particularly important. Examples of returned values: `Windows:win-amd64` (64-bit Windows on AMD64, aka x86_64, Intel64, and EM64T) `win-arm64` (64-

`sysconfig.is_python_build()`

Return `True` if the running Python interpreter was built from source and is being run from its built location, and not from a location resulting from e.g. running `make install` or installing via a binary installer.

`sysconfig.parse_config_h(fp[,vars])`

Parse a `config.h`-style file. `fp` is a file-like object pointing to the `config.h`-like file. A dictionary containing name/value pairs is returned. If an optional dictionary is passed in as the second argument, it is used instead of a new dictionary, and updated with the values read in the file.

`sysconfig.get_config_h_filename()`

Return the path of `pyconfig.h`.

sysconfig.get_makefile_filename()

Return the path of Makefile.

Command-line usage

You can use `sysconfig` as a script with Python's `-m` option:

```
$ python -m sysconfig
Platform: "macosx-10.4-i386"
Python version: "3.2"
Current installation scheme: "posix_prefix"
```

Paths:

```
data = "/usr/local"
include = "/Users/tarek/Dev/svn.python.org/py3k/Include"
platinclude = "."
platlib = "/usr/local/lib/python3.2/site-packages"
platstdlib = "/usr/local/lib/python3.2"
purelib = "/usr/local/lib/python3.2/site-packages"
scripts = "/usr/local/bin"
stdlib = "/usr/local/lib/python3.2"
```

Variables:

```
AC_APPLE_UNIVERSAL_BUILD = "0"
AIX_GENUINE_CPLUSPLUS = "0"
AR = "ar"
ARFLAGS = "rc"
...
```

This call will print in the standard output the information returned by `get_platform()`, `get_python_version()`, `get_path()` and `get_config_vars()`.

builtins— Built-in objects

This module provides direct access to all ‘built-in’ identifiers of Python; for example, `builtins.open` is the full name for the built-in function `open()`.

This module is not normally accessed explicitly by most applications, but can be useful in modules that provide objects with the same name as a built-in value, but in which the built-in of that name is also needed. For example, in a module that wants to implement an `open()` function that wraps the built-in `open()`, this module can be used directly:

```
import builtins

def open(path):
    f = builtins.open(path, 'r')
    return UpperCaser(f)

class UpperCaser:
    '''Wrapper around a file that converts output to uppercase.'''

    def __init__(self, f):
        self._f = f

    def read(self, count=-1):
        return self._f.read(count).upper()

    # ...
```

As an implementation detail, most modules have the name `__builtins__` made available as part of their globals. The value of `__builtins__` is normally either this module or the value of this module’s `__dict__` attribute. Since this is an implementation detail, it may not be used by alternate implementations of Python.

See also

- Built-in Constants
- Built-in Exceptions
- Built-in Functions
- Built-in Types

`__main__` — Top-level code environment

In Python, the special name `__main__` is used for two important constructs:

- the name of the top-level environment of the program, which can be checked using the `__name__ == '__main__'` expression; and
- the `__main__.py` file in Python packages.

Both of these mechanisms are related to Python modules; how users interact with them and how they interact with each other. They are explained in detail below. If you're new to Python modules, see the tutorial section [Modules](#) for an introduction.

`__name__ == '__main__'`

When a Python module or package is imported, `__name__` is set to the module's name. Usually, this is the name of the Python file itself without the `.py` extension:

```
>>> import configparser
>>> configparser.__name__
'configparser'
```

If the file is part of a package, `__name__` will also include the parent package's path:

```
>>> from concurrent.futures import process
>>> process.__name__
'concurrent.futures.process'
```

However, if the module is executed in the top-level code environment, its `__name__` is set to the string `'__main__'`.

What is the “top-level code environment”?

`__main__` is the name of the environment where top-level code is run. “Top-level code” is the first user-specified Python module that starts running. It's “top-level” because it imports all other modules that the program needs. Sometimes “top-level code” is called an entry point to the application.

The top-level code environment can be:

- the scope of an interactive prompt: `>>> __name__` `'__main__'`
- the Python module passed to the Python interpreter as a file argument: `$python helloworld.py` Hello, world!
- the Python module or package passed to the Python interpreter with the `-m` argument: `$python -m tarfile usage: tarfile.py [-h] [-v] (...)`
- Python code read by the Python interpreter from standard input: `$echo "import this" | python` The Zen of Python, by Tim Peters Beautiful is better than ugly. Explicit is better than implicit....
- Python code passed to the Python interpreter with the `-c` argument: `$python -c "import this"` The Zen of Python, by Tim Peters Beautiful is better than ugly. Explicit is better than implicit....

In each of these situations, the top-level module's `__name__` is set to `'__main__'`.

As a result, a module can discover whether or not it is running in the top-level environment by checking its own `__name__`, which allows a common idiom for conditionally executing code when the module is not initialized from an import statement:

```
if __name__ == '__main__':
    # Execute when the module is not initialized from an import statement.
    ...
```

See also

For a more detailed look at how `__name__` is set in all situations, see the tutorial section `Modules`.

Idiomatic Usage

Some modules contain code that is intended for script use only, like parsing command-line arguments or fetching data from standard input. If a module like this was imported from a different module, for example to unit test it, the script code would unintentionally execute as well.

This is where using the `if __name__ == '__main__':` code block comes in handy. Code within this block won't run unless the module is executed in the top-level environment.

Putting as few statements as possible in the block below `if __name__ == '__main__':` can improve code clarity and correctness. Most often, a function named `main` encapsulates the program's primary behavior:

```
# echo.py

import shlex
import sys

def echo(phrase: str) -> None:
    """A dummy wrapper around print."""
    # for demonstration purposes, you can imagine that there is some
    # valuable and reusable logic inside this function
    print(phrase)

def main() -> int:
    """Echo the input arguments to standard output"""
    phrase = shlex.join(sys.argv)
    echo(phrase)
    return 0

if __name__ == '__main__':
    sys.exit(main()) # next section explains the use of sys.exit
```

Note that if the module didn't encapsulate code inside the `main` function but instead put it directly within the `if __name__ == '__main__':` block, the `phrase` variable would be global to the entire module. This is error-prone as other functions within the module could be unintentionally using the global variable instead of a local name. A `main` function solves this problem.

Using a `main` function has the added benefit of the `echo` function itself being isolated and importable elsewhere. When `echo.py` is imported, the `echo` and `main` functions will be defined, but neither of them will be called, because `__name__ != '__main__'`.

Packaging Considerations

`main` functions are often used to create command-line tools by specifying them as entry points for console scripts. When this is done, `pip` inserts the function call into a template script, where the return value of `main` is passed into `sys.exit()`. For example:

```
sys.exit(main())
```

Since the call to `main` is wrapped in `sys.exit()`, the expectation is that your function will return some value acceptable as an input to `sys.exit()`; typically, an integer or `None` (which is implicitly returned if your function does not have a return statement).

By proactively following this convention ourselves, our module will have the same behavior when run directly (i.e. `python echo.py`) as it will have if we later package it as a console script entry-point in a `pip`-installable package.

In particular, be careful about returning strings from your `main` function. `sys.exit()` will interpret a string argument as a failure message, so your program will have an exit code of 1, indicating failure, and the string will be written to `sys.stderr`. The `echo.py` example from earlier exemplifies using the `sys.exit(main())` convention.

See also

Python Packaging User Guide contains a collection of tutorials and references on how to distribute and install Python packages with modern tools.

`__main__.py` in Python Packages

If you are not familiar with Python packages, see section [Packages](#) of the tutorial. Most commonly, the `__main__.py` file is used to provide a command-line interface for a package. Consider the following hypothetical package, “bandclass”:

```
bandclass
  ■■■ __init__.py
  ■■■ __main__.py
  ■■■ student.py
```

`__main__.py` will be executed when the package itself is invoked directly from the command line using the `-m` flag. For example:

```
$ python -m bandclass
```

This command will cause `__main__.py` to run. How you utilize this mechanism will depend on the nature of the package you are writing, but in this hypothetical case, it might make sense to allow the teacher to search for students:

```
# bandclass/__main__.py

import sys
from .student import search_students

student_name = sys.argv[1] if len(sys.argv) >= 2 else ''
print(f'Found student: {search_students(student_name)}')
```

Note that `from .student import search_students` is an example of a relative import. This import style can be used when referencing modules within a package. For more details, see [Intra-package References](#) in the [Modules](#) section of the tutorial.

Idiomatic Usage

The content of `__main__.py` typically isn't fenced with an `if __name__ == '__main__':` block. Instead, those files are kept short and import functions to execute from other modules. Those other modules can then be easily unit-tested and are properly reusable.

If used, an `if __name__ == '__main__':` block will still work as expected for a `__main__.py` file within a package, because its `__name__` attribute will include the package's path if imported:

```
>>> import asyncio.__main__
>>> asyncio.__main__.__name__
'asyncio.__main__'
```

This won't work for `__main__.py` files in the root directory of a .zip file though. Hence, for consistency, a minimal `__main__.py` without a `__name__` check is preferred.

See also

See `env` for an example of a package with a minimal `__main__.py` in the standard library. It doesn't contain an `if __name__ == '__main__':` block. You can invoke it with `python -m env [directory]`.

See [runpy](#) for more details on the `-m` flag to the interpreter executable.

See [zipapp](#) for how to run applications packaged as .zipfiles. In this case Python looks for a `__main__.pyfile` in the root directory of the archive.

import __main__

Regardless of which module a Python program was started with, other modules running within that same program can import the top-level environment's scope (namespace) by importing the `__main__` module. This doesn't import a `__main__.pyfile` but rather whichever module that received the special name `'__main__'`.

Here is an example module that consumes the `__main__` namespace:

```
# namely.py

import __main__

def did_user_define_their_name():
    return 'my_name' in dir(__main__)

def print_user_name():
    if not did_user_define_their_name():
        raise ValueError('Define the variable `my_name`!')

    print(__main__.my_name)
```

Example usage of this module could be as follows:

```
# start.py

import sys

from namely import print_user_name

# my_name = "Dinsdale"

def main():
    try:
        print_user_name()
    except ValueError as ve:
        return str(ve)

if __name__ == "__main__":
    sys.exit(main())
```

Now, if we started our program, the result would look like this:

```
$ python start.py
Define the variable `my_name`!
```

The exit code of the program would be 1, indicating an error. Uncommenting the line `my_name="Dinsdale"` fixes the program and now it exits with status code 0, indicating success:

```
$ python start.py
Dinsdale
```

Note that importing `__main__` doesn't cause any issues with unintentionally running top-level code meant for script use which is put in their `if __name__ == "__main__":` block of the start module. Why does this work?

Python inserts an empty `__main__` module into `sys.modules` at interpreter startup, and populates it by running top-level code. In our example this is the start module which runs line by line and imports `namely`. In turn, `namely` imports `__main__` (which is really `start`). That's an import cycle! Fortunately, since the partially populated `__main__` module is present in `sys.modules`, Python passes that to `namely`. See [Special considerations](#)

for `__main__` in the import system's reference for details on how this works.

The Python REPL is another example of a “top-level environment”, so anything defined in the REPL becomes part of the `__main__` scope:

```
>>> import namely
>>> namely.did_user_define_their_name()
False
>>> namely.print_user_name()
Traceback (most recent call last):
...
ValueError: Define the variable `my_name`!
>>> my_name = 'Jabberwocky'
>>> namely.did_user_define_their_name()
True
>>> namely.print_user_name()
Jabberwocky
```

The `__main__` scope is used in the implementation of `pdb` and `rlcompleter`.

warnings— Warning control

Source code: `Lib/warnings.py`

Warning messages are typically issued in situations where it is useful to alert the user of some condition in a program, where that condition (normally) doesn't warrant raising an exception and terminating the program. For example, one might want to issue a warning when a program uses an obsolete module.

Python programmers issue warnings by calling the `warn()` function defined in this module. (C programmers use `PyErr_WarnEx()`; see [Exception Handling](#) for details).

Warning messages are normally written to `sys.stderr`, but their disposition can be changed flexibly, from ignoring all warnings to turning them into exceptions. The disposition of warnings can vary based on the warning category, the text of the warning message, and the source location where it is issued. Repetitions of a particular warning for the same source location are typically suppressed.

There are two stages in warning control: first, each time a warning is issued, a determination is made whether a message should be issued or not; next, if a message is to be issued, it is formatted and printed using a user-settable hook.

The determination whether to issue a warning message is controlled by the warning filter, which is a sequence of matching rules and actions. Rules can be added to the filter by calling `filterwarnings()` and reset to its default state by calling `resetwarnings()`.

The printing of warning messages is done by calling `showwarning()`, which may be overridden; the default implementation of this function formats the message by calling `formatwarning()`, which is also available for use by custom implementations.

See also

`logging.captureWarnings()` allows you to handle all warnings with the standard logging infrastructure.

Warning Categories

There are a number of built-in exceptions that represent warning categories. This categorization is useful to be able to filter out groups of warnings.

While these are technically built-in exceptions, they are documented here, because conceptually they belong to the warnings mechanism.

User code can define additional warning categories by subclassing one of the standard warning categories. A warning category must always be a subclass of the `Warning` class.

The following warnings category classes are currently defined:

```
Class | Description | Warning | This is the base class of all warning category classes. It is a
subclass of | Exception | . | UserWarning | The default category for | warn() | . |
DeprecationWarning | Base category for warnings about deprecated features when those warnings
are intended for other Python developers (ignored by default, unless triggered by code in |
__main__ | ). | SyntaxWarning | Base category for warnings about dubious syntactic features
(typically emitted when compiling Python source code, and hence may not be suppressed by runtime
filters) | RuntimeWarning | Base category for warnings about dubious runtime features. |
FutureWarning | Base category for warnings about deprecated features when those warnings are
intended for end users of applications that are written in Pyt
```

Changed in version 3.7: Previously `DeprecationWarning` and `FutureWarning` were distinguished based on whether a feature was being removed entirely or changing its behaviour. They are now distinguished based on their intended audience and the way they're handled by the default warnings filters.

The Warnings Filter

The warnings filter controls whether warnings are ignored, displayed, or turned into errors (raising an exception).

Conceptually, the warnings filter maintains an ordered list of filter specifications; any specific warning is matched against each filter specification in the list in turn until a match is found; the filter determines the disposition of the match. Each entry is a tuple of the form (action,message,category,module,lineno), where:

- **action** is one of the following strings: "default" print the first occurrence of matching warnings for each location (module + line number) where the warning is issued; "error" turn matching warnings into exceptions; "ignore" never print matching warnings; "always" always print matching warnings; "all" alias to "always"; "module" print the first occurrence of matching warnings for each module where t
- **message** is a string containing a regular expression that the start of the warning message must match, case-insensitively. In-WandPYTHONWARNINGS, **message** is a literal string that the start of the warning message must contain (case-insensitively), ignoring any whitespace at the start or end of **message**.
- **category** is a class (a subclass of `Warning`) of which the warning category must be a subclass in order to match.
- **module** is a string containing a regular expression that the start of the fully qualified module name must match, case-sensitively. In-WandPYTHONWARNINGS, **module** is a literal string that the fully qualified module name must be equal to (case-sensitively), ignoring any whitespace at the start or end of **module**.
- **lineno** is an integer that the line number where the warning occurred must match, or 0 to match all line numbers.

Since the `Warning` class is derived from the built-in `Exception` class, to turn a warning into an error we simply `raise category(message)`.

If a warning is reported and doesn't match any registered filter then the "default" action is applied (hence its name).

Repeated Warning Suppression Criteria

The filters that suppress repeated warnings apply the following criteria to determine if a warning is considered a repeat:

- "default": A warning is considered a repeat only if the (message,category,module,lineno) are all the same.
- "module": A warning is considered a repeat if the (message,category,module) are the same, ignoring the line number.
- "once": A warning is considered a repeat if the (message,category) are the same, ignoring the module and line number.

Describing Warning Filters

The warnings filter is initialized by -Woptions passed to the Python interpreter command line and the PYTHONWARNINGSEnvironment variable. The interpreter saves the arguments for all supplied entries without interpretation in `sys.warnoptions`; the `warnings` module parses these when it is first imported (invalid options are ignored, after printing a message to `sys.stderr`).

Individual warnings filters are specified as a sequence of fields separated by colons:

```
action:message:category:module:line
```

The meaning of each of these fields is as described in The Warnings Filter. When listing multiple filters on a single line (as for PYTHONWARNINGS), the individual filters are separated by commas and the filters listed later take precedence over those listed before them (as they're applied left-to-right, and the most recently applied filters take precedence over earlier ones).

Commonly used warning filters apply to either all warnings, warnings in a particular category, or warnings raised by particular modules or packages. Some examples:

```
default                # Show all warnings (even those ignored by default)
ignore                 # Ignore all warnings
error                  # Convert all warnings to errors
error::ResourceWarning # Treat ResourceWarning messages as errors
default::DeprecationWarning # Show DeprecationWarning messages
ignore,default::mymodule # Only report warnings triggered by "mymodule"
error::mymodule         # Convert warnings to errors in "mymodule"
```

Default Warning Filter

By default, Python installs several warning filters, which can be overridden by the `-W` command-line option, the `PYTHONWARNINGS` environment variable and calls to `filterwarnings()`.

In regular release builds, the default warning filter has the following entries (in order of precedence):

```
default::DeprecationWarning: __main__
ignore::DeprecationWarning
ignore::PendingDeprecationWarning
ignore::ImportWarning
ignore::ResourceWarning
```

In a debug build, the list of default warning filters is empty.

Changed in version 3.2: `DeprecationWarning` is now ignored by default in addition to `PendingDeprecationWarning`.

Changed in version 3.7: `DeprecationWarning` is once again shown by default when triggered directly by code in `__main__`.

Changed in version 3.7: `BytesWarning` no longer appears in the default filter list and is instead configured via `sys.warnoptions` when `-bb` is specified twice.

Overriding the default filter

Developers of applications written in Python may wish to hide all Python level warnings from their users by default, and only display them when running tests or otherwise working on the application.

The `sys.warnoptions` attribute used to pass filter configurations to the interpreter can be used as a marker to indicate whether or not warnings should be disabled:

```
import sys

if not sys.warnoptions:
    import warnings
    warnings.simplefilter("ignore")
```

Developers of test runners for Python code are advised to instead ensure that all warnings are displayed by default for the code under test, using code like:

```
import sys

if not sys.warnoptions:
    import os, warnings
    warnings.simplefilter("default") # Change the filter in this process
    os.environ["PYTHONWARNINGS"] = "default" # Also affect subprocesses
```

Finally, developers of interactive shells that run user code in a namespace other than `__main__` are advised to ensure that `DeprecationWarning` messages are made visible by default, using code like the following (where `user_ns` is the module used to execute code entered interactively):

```
import warnings
warnings.filterwarnings("default", category=DeprecationWarning,
```

```
module=user_ns.get("__name__"))
```

Temporarily Suppressing Warnings

If you are using code that you know will raise a warning, such as a deprecated function, but do not want to see the warning (even when warnings have been explicitly configured via the command line), then it is possible to suppress the warning using the `catch_warnings` context manager:

```
import warnings

def fxn():
    warnings.warn("deprecated", DeprecationWarning)

with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    fxn()
```

While within the context manager all warnings will simply be ignored. This allows you to use known-deprecated code without having to see the warning while not suppressing the warning for other code that might not be aware of its use of deprecated code.

■ **Nota:** NoteSeeConcurrent safety of Context Managersfor details on the concurrency-safety of the `catch_warnings` context manager when used in programs using multiple threads or async functions.

Testing Warnings

To test warnings raised by code, use the `catch_warnings` context manager. With it you can temporarily mutate the warnings filter to facilitate your testing. For instance, do the following to capture all raised warnings to check:

```
import warnings

def fxn():
    warnings.warn("deprecated", DeprecationWarning)

with warnings.catch_warnings(record=True) as w:
    # Cause all warnings to always be triggered.
    warnings.simplefilter("always")
    # Trigger a warning.
    fxn()
    # Verify some things
    assert len(w) == 1
    assert isinstance(w[-1].category, DeprecationWarning)
    assert "deprecated" in str(w[-1].message)
```

One can also cause all warnings to be exceptions by using `error` instead of `always`. One thing to be aware of is that if a warning has already been raised because of a `once/default` rule, then no matter what filters are set the warning will not be seen again unless the warnings registry related to the warning has been cleared.

Once the context manager exits, the warnings filter is restored to its state when the context was entered. This prevents tests from changing the warnings filter in unexpected ways between tests and leading to indeterminate test results.

■ **Nota:** NoteSeeConcurrent safety of Context Managersfor details on the concurrency-safety of the `catch_warnings` context manager when used in programs using multiple threads or async functions.

When testing multiple operations that raise the same kind of warning, it is important to test them in a manner that confirms each operation is raising a new warning (e.g. set warnings to be raised as exceptions and check the operations raise exceptions, check that the length of the warning list continues to increase after each operation, or else delete the previous entries from the warnings list before each new operation).

Updating Code For New Versions of Dependencies

Warning categories that are primarily of interest to Python developers (rather than end users of applications written in Python) are ignored by default.

Notably, this “ignored by default” list includes `DeprecationWarning` (for every module except `__main__`), which means developers should make sure to test their code with typically ignored warnings made visible in order to receive timely notifications of future breaking API changes (whether in the standard library or third party packages).

In the ideal case, the code will have a suitable test suite, and the test runner will take care of implicitly enabling all warnings when running tests (the test runner provided by the `unittest` module does this).

In less ideal cases, applications can be checked for use of deprecated interfaces by passing `-W` to the Python interpreter (this is shorthand for `-Wdefault`) or setting `PYTHONWARNINGS=default` in the environment. This enables default handling for all warnings, including those that are ignored by default. To change what action is taken for encountered warnings you can change what argument is passed to `-W` (e.g. `-Werror`). See the `-W` flag for more details on what is possible.

Available Functions

`warnings.warn(message,category=None,stacklevel=1,source=None,*,skip_file_prefixes=())`

Issue a warning, or maybe ignore it or raise an exception. The `category` argument, if given, must be a warning category class; it defaults to `UserWarning`. Alternatively, `message` can be a `Warning` instance, in which case `category` will be ignored and `message.__class__` will be used. In this case, the message text will be `str(message)`. This function raises an exception if the particular warning issued is changed into an error by the warnings filter. The `stacklevel` argument can be used by wrapper functions written

`warnings.warn_explicit(message,category,filename,lineno,module=None,registry=None,module_globals=None,source=None)`

This is a low-level interface to the functionality of `warn()`, passing in explicitly the message, category, filename and line number, and optionally other arguments. `message` must be a string and `category` a subclass of `Warning` or `message` may be a `Warning` instance, in which case `category` will be ignored. `module`, if supplied, should be the module name. If no module is passed, the filename with `.py` stripped is used. `registry`, if supplied, should be the `__warningregistry__` dictionary of the module. If no registry is pas

`warnings.showwarning(message,category,filename,lineno,file=None,line=None)`

Write a warning to a file. The default implementation calls `formatwarning(message,category,filename,lineno,line)` and writes the resulting string to `file`, which defaults to `sys.stderr`. You may replace this function with any callable by assigning to `warnings.showwarning`. `line` is a line of source code to be included in the warning message; if `line` is not supplied, `showwarning()` will try to read the line specified by `filename` and `lineno`.

`warnings.formatwarning(message,category,filename,lineno,line=None)`

Format a warning the standard way. This returns a string which may contain embedded newlines and ends in a newline. `line` is a line of source code to be included in the warning message; if `line` is not supplied, `formatwarning()` will try to read the line specified by `filename` and `lineno`.

`warnings.filterwarnings(action,message="",category=Warning,module="",lineno=0,append=False)`

Insert an entry into the list of warnings filter specifications. The entry is inserted at the front by default; if `append` is true, it is inserted at the end. This checks the types of the arguments, compiles

the message and module regular expressions, and inserts them as a tuple in the list of warnings filters. Entries closer to the front of the list override entries later in the list, if both match a particular warning. Omitted arguments default to a value that matches everything.

warnings.simplefilter(action,category=Warning,lineno=0,append=False)

Insert a simple entry into the list of warnings filter specifications. The meaning of the function parameters is as for `filterwarnings()`, but regular expressions are not needed as the filter inserted always matches any message in any module as long as the category and line number match.

warnings.resetwarnings()

Reset the warnings filter. This discards the effect of all previous calls to `filterwarnings()`, including that of the `-W` command line options and calls to `simplefilter()`.

@warnings.deprecated(msg,*,category=DeprecationWarning,stacklevel=1)

Decorator to indicate that a class, function or overload is deprecated. When this decorator is applied to an object, deprecation warnings may be emitted at runtime when the object is used. Static type checkers will also generate a diagnostic on usage of the deprecated object. Usage: `from warnings import deprecated from typing import overload @deprecated("Use B instead") class A: pass @deprecated("Use g instead") def f(): pass @overload @deprecated("int support is deprecated") def g(x:int)->int: ... @overload def g(x:str)->`

Available Context Managers

class warnings.catch_warnings(*,record=False,module=None,action=None,category=Warning,lineno=0,append=False)

A context manager that copies and, upon exit, restores the warnings filter and the `showwarning()` function. If the `record` argument is `False` (the default) the context manager returns `None` on entry. If `record` is `True`, a list is returned that is progressively populated with objects as seen by a custom `showwarning()` function (which also suppresses output to `sys.stdout`). Each object in the list has attributes with the same names as the arguments to `showwarning()`. The `module` argument takes a module that will be used in

Concurrent safety of Context Managers

The behavior of `catch_warnings` context manager depends on the `sys.flags.context_aware_warnings` flag. If the flag is true, the context manager behaves in a concurrent-safe fashion and otherwise not. Concurrent-safe means that it is both thread-safe and safe to use within an `asyncio` coroutines and tasks. Being thread-safe means that behavior is predictable in a multi-threaded program. The flag defaults to true for free-threaded builds and false otherwise.

If the `context_aware_warnings` flag is false, then `catch_warnings` will modify the global attributes of the `warnings` module. This is not safe if used within a concurrent program (using multiple threads or using `asyncio` coroutines). For example, if two or more threads use the `catch_warnings` class at the same time, the behavior is undefined.

If the flag is true, `catch_warnings` will not modify global attributes and will instead use a `ContextVar` to store the newly established warning filtering state. A context variable provides thread-local storage and it makes the use of `catch_warnings` thread-safe.

The `record` parameter of the context handler also behaves differently depending on the value of the flag. When `record` is true and the flag is false, the context manager works by replacing and then later restoring the module's `showwarning()` function. That is not concurrent-safe.

When `recordis` is true and the flag is true, the `showwarning()` function is not replaced. Instead, the recording status is indicated by an internal property in the context variable. In this case, the `showwarning()` function will not be restored when exiting the context handler.

The `context_aware_warnings` flag can be set the `-Xcontext_aware_warnings` command-line option or by the `PYTHON_CONTEXT_AWARE_WARNINGS` environment variable.

■ Nota: Note it is likely that most programs that desire thread-safe behaviour of the warnings module will also want to set the `thread_inherit_context` flag to true. That flag causes threads created by `threading.Thread` to start with a copy of the context variables from the thread starting it. When true, the context established by `catch_warnings` in one thread will also apply to new threads started by it. If false, new threads will start with an empty warnings context variable, meaning that any filtering that was established by `acatch_warnings` context manager will no longer be active.

Changed in version 3.14: Added the `sys.flags.context_aware_warnings` flag and the use of a context variable for `catch_warnings` if the flag is true. Previous versions of Python acted as if the flag was always set to false.

dataclasses— Data Classes

Source code: `Lib/dataclasses.py`

This module provides a decorator and functions for automatically adding generated special methods such as `__init__()` and `__repr__()` to user-defined classes. It was originally described in PEP 557.

The member variables to use in these generated methods are defined using PEP 526 type annotations. For example, this code:

```
from dataclasses import dataclass

@dataclass
class InventoryItem:
    """Class for keeping track of an item in inventory."""
    name: str
    unit_price: float
    quantity_on_hand: int = 0

    def total_cost(self) -> float:
        return self.unit_price * self.quantity_on_hand
```

will add, among other things, a `__init__()` that looks like:

```
def __init__(self, name: str, unit_price: float, quantity_on_hand: int = 0):
    self.name = name
    self.unit_price = unit_price
    self.quantity_on_hand = quantity_on_hand
```

Note that this method is automatically added to the class: it is not directly specified in the `InventoryItem` definition shown above.

Added in version 3.7.

Module contents

`@dataclasses.dataclass(*,init=True,repr=True,eq=True,order=False,unsafe_hash=False,frozen=False,match_args=True,kw_only=False,slots=False,weakref_slot=False)`

This function is a decorator that is used to add generated special methods to classes, as described below. The `@dataclass` decorator examines the class to find fields. A field is defined as a class variable that has a type annotation. With two exceptions described below, nothing in `@dataclass` examines the type specified in the variable annotation. The order of the fields in all of the generated methods is the order in which they appear in the class definition. The `@dataclass` decorator will add various “dunder”

`dataclasses.field(*,default=MISSING,default_factory=MISSING,init=True,repr=True,hash=None,compare=True,metadata=None,kw_only=MISSING,doc=None)`

For common and simple use cases, no other functionality is required. There are, however, some dataclass features that require additional per-field information. To satisfy this need for additional information, you can replace the default field value with a call to the provided `field()` function. For example: `@dataclass class C: mylist: list[int] = field(default_factory=list) c = C() c.mylist += [1, 2, 3]` As shown above, the `MISSING` value is a sentinel object used to detect if some parameters are provided by the user.

`class dataclasses.Field`

Field objects describe each defined field. These objects are created internally, and are returned by the `fields()` module-level method (see below). Users should never instantiate a `Field` object directly. Its documented attributes are: `name`: The name of the field. `type`: The type of the

field.default, default_factory, init, repr, hash, compare, metadata, and kw_only have the identical meaning and values as they do in the field() function. Other attributes may exist, but they are private and must not be inspected or

dataclasses.InitVar

InitVar[T] type annotations describe variables that are init-only. Fields annotated with InitVar are considered pseudo-fields, and thus are neither returned by the fields() function nor used in any way except adding them as parameters to __init__() and an optional __post_init__().

dataclasses.fields(class_or_instance)

Returns a tuple of Field objects that define the fields for this dataclass. Accepts either a dataclass, or an instance of a dataclass. Raises TypeError if not passed a dataclass or instance of one. Does not return pseudo-fields which are ClassVar or InitVar.

dataclasses.asdict(obj, *, dict_factory=dict)

Converts the dataclass obj to a dict (by using the factory function dict_factory). Each dataclass is converted to a dict of its fields, as name: value pairs. dataclasses, dicts, lists, and tuples are recursed into. Other objects are copied with copy.deepcopy(). Example of using asdict() on nested dataclasses: @dataclass class Point: x: int; y: int @dataclass class C: mylist: list[Point] p = Point(10, 20) assert asdict(p) == {'x': 10, 'y': 20} c = C([Point(0, 0), Point(10, 4)]) assert asdict(c) == {'mylist': [{'x': 0, 'y': 0}, {'x': 10, 'y': 4}]

dataclasses.astuple(obj, *, tuple_factory=tuple)

Converts the dataclass obj to a tuple (by using the factory function tuple_factory). Each dataclass is converted to a tuple of its field values. dataclasses, dicts, lists, and tuples are recursed into. Other objects are copied with copy.deepcopy(). Continuing from the previous

example: assert astuple(p) == (10, 20) assert astuple(c) == ((0, 0), (10, 4),) To create a shallow copy, the following workaround may be

used: tuple((getattr(obj, field.name) for field in dataclasses.fields(obj))) astuple() raises TypeError if obj is

dataclasses.make_dataclass(cls_name, fields, *, bases=(), namespace=None, init=True, repr=True, eq=True, order=False, unsafe_hash=False, frozen=False, match_args=True, kw_only=False, slots=False, weakref_slot=False, module=None, decorator=dataclass)

Creates a new dataclass with name cls_name, fields as defined in fields, base classes as given in bases, and initialized with a namespace as given in namespace. fields is an iterable whose elements are each either name, (name, type), or (name, type, Field). If just name is supplied, typing.Any is used for type. The values of init, repr, eq, order, unsafe_hash, frozen, match_args, kw_only, slots, and weakref_slot have the same meaning as they do in @dataclass. If module is defined, the __module__ attribute of the dataclass is s

dataclasses.replace(obj, /, **changes)

Creates a new object of the same type as obj, replacing fields with values from changes. If obj is not a Data Class, raises TypeError. If keys in changes are not field names of the given dataclass, raises TypeError. The newly returned object is created by calling the __init__() method of the dataclass. This ensures that __post_init__(), if present, is also called. Init-only variables without default values, if any exist, must be specified on the call to replace() so that they can be passed to __init__() and __

dataclasses.is_dataclass(obj)

Return True if its parameter is a dataclass (including subclasses of a dataclass) or an instance of one, otherwise return False. If you need to know if a class is an instance of a dataclass (and not a dataclass itself), then add a further check

for not isinstance(obj, type): def is_dataclass_instance(obj): return is_dataclass(obj) and not isinstance(obj, type)

dataclasses.MISSING

A sentinel value signifying a missing default or default_factory.

dataclasses.KW_ONLY

A sentinel value used as a type annotation. Any fields after a pseudo-field with the type of `KW_ONLY` are marked as keyword-only fields. Note that a pseudo-field of type `KW_ONLY` is otherwise completely ignored. This includes the name of such a field. By convention, a name of `_` is used for a `KW_ONLY` field. Keyword-only fields signify `__init__()` parameters that must be specified as keywords when the class is instantiated. In this example, the fields `y` and `z` will be marked as keyword-only fields: `@dataclass class C:`

exception dataclasses.FrozenInstanceError

Raised when an implicitly defined `__setattr__()` or `__delattr__()` is called on a dataclass which was defined with `frozen=True`. It is a subclass of `AttributeError`.

Post-init processing

dataclasses.__post_init__()

When defined on the class, it will be called by the generated `__init__()`, normally as `self.__post_init__()`. However, if any `InitVar` fields are defined, they will also be passed to `__post_init__()` in the order they were defined in the class. If no `__init__()` method is generated, then `__post_init__()` will not automatically be called. Among other uses, this allows for initializing field values that depend on one or more other fields. For example: `@dataclass class C: a: float b: float c: float = field(init=False) def __p`

The `__init__()` method generated by `@dataclass` does not call base class `__init__()` methods. If the base class has an `__init__()` method that has to be called, it is common to call this method in a `__post_init__()` method:

```
class Rectangle:
    def __init__(self, height, width):
        self.height = height
        self.width = width

@dataclass
class Square(Rectangle):
    side: float

    def __post_init__(self):
        super().__init__(self.side, self.side)
```

Note, however, that in general the dataclass-generated `__init__()` methods don't need to be called, since the derived dataclass will take care of initializing all fields of any base class that is a dataclass itself.

See the section below on init-only variables for ways to pass parameters to `__post_init__()`. Also see the warning about how `replace()` handles `init=False` fields.

Class variables

One of the few places where `@dataclass` actually inspects the type of a field is to determine if a field is a class variable as defined in PEP 526. It does this by checking if the type of the field is `typing.ClassVar`. If a field is a `ClassVar`, it is excluded from consideration as a field and is ignored by the dataclass mechanisms. Such `ClassVar` pseudo-fields are not returned by the module-level `fields()` function.

Init-only variables

Another place where `@dataclass` inspects a type annotation is to determine if a field is an init-only variable. It does this by seeing if the type of a field is of type `InitVar`. If a field is an `InitVar`, it is considered a pseudo-field called an init-only field. As it is not a true field, it is not returned by the module-level `fields()` function. Init-only

fields are added as parameters to the generated `__init__()` method, and are passed to the optional `__post_init__()` method. They are not otherwise used by dataclasses.

For example, suppose a field will be initialized from a database, if a value is not provided when creating the class:

```
@dataclass
class C:
    i: int
    j: int | None = None
    database: InitVar[DatabaseType | None] = None

    def __post_init__(self, database):
        if self.j is None and database is not None:
            self.j = database.lookup('j')

c = C(10, database=my_database)
```

In this case, `fields()` will return `Field` objects for `i` and `j`, but not for `database`.

Frozen instances

It is not possible to create truly immutable Python objects. However, by passing `frozen=True` to the `@dataclass` decorator you can emulate immutability. In that case, dataclasses will add `__setattr__()` and `__delattr__()` methods to the class. These methods will raise a `FrozenInstanceError` when invoked.

There is a tiny performance penalty when using `frozen=True`: `__init__()` cannot use simple assignment to initialize fields, and must use `object.__setattr__()`.

Inheritance

When the dataclass is being created by the `@dataclass` decorator, it looks through all of the class's base classes in reverse MRO (that is, starting at `object`) and, for each dataclass that it finds, adds the fields from that base class to an ordered mapping of fields. After all of the base class fields are added, it adds its own fields to the ordered mapping. All of the generated methods will use this combined, calculated ordered mapping of fields. Because the fields are in insertion order, derived classes override base classes. An example:

```
@dataclass
class Base:
    x: Any = 15.0
    y: int = 0

@dataclass
class C(Base):
    z: int = 10
    x: int = 15
```

The final list of fields is, in order, `x`, `y`, `z`. The final type of `x` is `int`, as specified in class `C`.

The generated `__init__()` method for `C` will look like:

```
def __init__(self, x: int = 15, y: int = 0, z: int = 10):
```

Re-ordering of keyword-only parameters in `__init__()`

After the parameters needed for `__init__()` are computed, any keyword-only parameters are moved to come after all regular (non-keyword-only) parameters. This is a requirement of how keyword-only parameters are implemented in Python: they must come after non-keyword-only parameters.

In this example, `Base.y`, `Base.w`, and `D.t` are keyword-only fields, and `Base.x` and `D.z` are regular fields:

```
@dataclass
class Base:
    x: Any = 15.0
    _: KW_ONLY
    y: int = 0
    w: int = 1

@dataclass
class D(Base):
    z: int = 10
    t: int = field(kw_only=True, default=0)
```

The generated `__init__()` method for `D` will look like:

```
def __init__(self, x: Any = 15.0, z: int = 10, *, y: int = 0, w: int = 1, t: int = 0):
```

Note that the parameters have been re-ordered from how they appear in the list of fields: parameters derived from regular fields are followed by parameters derived from keyword-only fields.

The relative ordering of keyword-only parameters is maintained in the re-ordered `__init__()` parameter list.

Default factory functions

If `field()` specifies a `default_factory`, it is called with zero arguments when a default value for the field is needed. For example, to create a new instance of a list, use:

```
mylist: list = field(default_factory=list)
```

If a field is excluded from `__init__()` (using `init=False`) and the field also specifies `default_factory`, then the default factory function will always be called from the generated `__init__()` function. This happens because there is no other way to give the field an initial value.

Mutable default values

Python stores default member variable values in class attributes. Consider this example, not using dataclasses:

```
class C:
    x = []
    def add(self, element):
        self.x.append(element)

o1 = C()
o2 = C()
o1.add(1)
o2.add(2)
assert o1.x == [1, 2]
assert o1.x is o2.x
```

Note that the two instances of class `C` share the same class variable `x`, as expected.

Using dataclasses, if this code was valid:

```
@dataclass
class D:
    x: list = [] # This code raises ValueError
    def add(self, element):
```

```
self.x.append(element)
```

it would generate code similar to:

```
class D:
    x = []
    def __init__(self, x=x):
        self.x = x
    def add(self, element):
        self.x.append(element)
```

```
assert D().x is D().x
```

This has the same issue as the original example using classC. That is, two instances of classD that do not specify a value for x when creating a class instance will share the same copy of x. Because dataclasses just use normal Python class creation they also share this behavior. There is no general way for Data Classes to detect this condition. Instead, the @dataclass decorator will raise a ValueError if it detects an unhashable default parameter. The assumption is that if a value is unhashable, it is mutable. This is a partial solution, but it does protect against many common errors.

Using default factory functions is a way to create new instances of mutable types as default values for fields:

```
@dataclass
class D:
    x: list = field(default_factory=list)
```

```
assert D().x is not D().x
```

Changed in version 3.11: Instead of looking for and disallowing objects of type list, dict, or set, unhashable objects are now not allowed as default values. Unhashability is used to approximate mutability.

Descriptor-typed fields

Fields that are assigned descriptor objects as their default value have the following special behaviors:

- The value for the field passed to the dataclass's __init__() method is passed to the descriptor's __set__() method rather than overwriting the descriptor object.
- Similarly, when getting or setting the field, the descriptor's __get__() or __set__() method is called rather than returning or overwriting the descriptor object.
- To determine whether a field contains a default value, @dataclass will call the descriptor's __get__() method using its class access form: descriptor.__get__(obj=None, type=cls). If the descriptor returns a value in this case, it will be used as the field's default. On the other hand, if the descriptor raises AttributeError in this situation, no default value will be provided for the field.

```
class IntConversionDescriptor:
    def __init__(self, *, default):
        self._default = default

    def __set_name__(self, owner, name):
        self._name = "_" + name

    def __get__(self, obj, type):
        if obj is None:
            return self._default

        return getattr(obj, self._name, self._default)

    def __set__(self, obj, value):
        setattr(obj, self._name, int(value))
```

```
@dataclass
class InventoryItem:
    quantity_on_hand: IntConversionDescriptor = IntConversionDescriptor(default=100)

i = InventoryItem()
print(i.quantity_on_hand)    # 100
i.quantity_on_hand = 2.5     # calls __set__ with 2.5
print(i.quantity_on_hand)    # 2
```

Note that if a field is annotated with a descriptor type, but is not assigned a descriptor object as its default value, the field will act like a normal field.

contextlib— Utilities for with-statement contexts

Source code: `Lib/contextlib.py`

This module provides utilities for common tasks involving the `with` statement. For more information see also `Context Manager Types` and `With Statement Context Managers`.

Utilities

Functions and classes provided:

class contextlib.AbstractContextManager

An abstract base class for classes that implement `__enter__()` and `__exit__()`. A default implementation for `__enter__()` is provided which returns `self` while `__exit__()` is an abstract method which by default returns `None`. See also the definition of `Context Manager Types`. Added in version 3.6.

class contextlib.AbstractAsyncContextManager

An abstract base class for classes that implement `__aenter__()` and `__aexit__()`. A default implementation for `__aenter__()` is provided which returns `self` while `__aexit__()` is an abstract method which by default returns `None`. See also the definition of `Asynchronous Context Managers`. Added in version 3.7.

@contextlib.contextmanager

This function is a decorator that can be used to define a factory function for `with` statement context managers, without needing to create a class or separate `__enter__()` and `__exit__()` methods. While many objects natively support use in `with` statements, sometimes a resource needs to be managed that isn't a context manager in its own right, and doesn't implement a `close()` method for use with `contextlib.closing`. An abstract example would be the following to ensure correct resource management: `from contextlib import`

@contextlib.asynccontextmanager

Similar to `contextmanager()`, but creates an asynchronous context manager. This function is a decorator that can be used to define a factory function for `async with` statement asynchronous context managers, without needing to create a class or separate `__aenter__()` and `__aexit__()` methods. It must be applied to an asynchronous generator function. A simple example: `from contextlib import asynccontextmanager`
`@asynccontextmanager`
`async def get_connection():`
 `conn = await acquire_db_connection()`
 `try:`
 `yield conn`
 `finally:`
 `await release_`

contextlib.closing(thing)

Return a context manager that closes `thing` upon completion of the block. This is basically equivalent to: `from contextlib import contextmanager`
`@contextmanager`
`def closing(thing):`
 `try:`
 `yield thing`
 `finally:`
 `thing.close()`
And lets you write code like this: `from contextlib import closing`
`from urllib.request import urlopen`
`with closing(urlopen('https://www.python.org')) as page:`
 `for line in page:`
 `print(line)`
without needing to explicitly close `page`. Even if an error occurs, `page.close()` will be called when the `with` block is exited. Not

contextlib.aclosing(thing)

Return an async context manager that calls the `aclose()` method of `thing` upon completion of the block. This is basically equivalent to: `from contextlib import asynccontextmanager`
`@asynccontextmanager`
`async def aclosing(thing):`
 `try:`
 `yield thing`
 `finally:`
 `await thing.aclose()`
Significantly, `aclosing()` supports deterministic cleanup of async generators when they happen to exit early by `break` or an exception. For example: `from contextlib import aclosing`
`async with aclosing(my_generator()) as values:`
 `async for value in values:`
 `if value == 4:`

contextlib.nullcontext(enter_result=None)

Return a context manager that returns `enter_result` from `__enter__()`, but otherwise does nothing. It is intended to be used as a stand-in for an optional context manager, for

```
example: def myfunction(arg, ignore_exceptions=False):
    if ignore_exceptions: # Use suppress to ignore all exceptions.
        cm = contextlib.suppress(Exception)
    else: # Do not ignore any exceptions, cm has no effect.
        cm = contextlib.nullcontext()
    with cm: # Do something
        ...
An example using enter_result:
def process_file(file_or_path):
    if isinstance(file_or_path, str):
        # ...
```

contextlib.suppress(*exceptions)

Return a context manager that suppresses any of the specified exceptions if they occur in the body of `with` statement and then resumes execution with the first statement following the end of the `with` statement. As with any other mechanism that completely suppresses exceptions, this context manager should be used only to cover very specific errors where silently continuing with program execution is known to be the right thing to do. For example:

```
from contextlib import suppress
with suppress(FileNotFoundError):
    ...
```

contextlib.redirect_stdout(new_target)

Context manager for temporarily redirecting `sys.stdout` to another file or file-like object. This tool adds flexibility to existing functions or classes whose output is hardwired to `stdout`. For example, the output of `help()` normally is sent to `sys.stdout`. You can capture that output in a string by redirecting the output to `io.StringIO` object. The replacement stream is returned from the `__enter__()` method and so is available as the target of the `with` statement:

```
with redirect_stdout(io.StringIO()) as f:
    help(pow)
```

contextlib.redirect_stderr(new_target)

Similar to `redirect_stdout()` but redirecting `sys.stderr` to another file or file-like object. This context manager is reentrant. Added in version 3.5.

contextlib.chdir(path)

Non parallel-safe context manager to change the current working directory. As this changes a global state, the working directory, it is not suitable for use in most threaded or async contexts. It is also not suitable for most non-linear code execution, like generators, where the program execution is temporarily relinquished – unless explicitly desired, you should not yield when this context manager is active. This is a simple wrapper around `chdir()`, it changes the current working directory upon `enter` and `exit`.

class contextlib.ContextDecorator

A base class that enables a context manager to also be used as a decorator. Context managers inheriting from `ContextDecorator` have to implement `__enter__()` and `__exit__()` as normal. `__exit__` retains its optional exception handling even when used as a decorator. `ContextDecorator` is used by `contextmanager()`, so you get this functionality automatically. Example of `ContextDecorator`:

```
from contextlib import ContextDecorator
class mycontext(ContextDecorator):
    def __enter__(self):
        print('Starting')
        return self
    def __exit__(self, *exc_info):
        print('Finishing')
        return False
```

class contextlib.AsyncContextDecorator

Similar to `ContextDecorator` but only for asynchronous functions. Example of `AsyncContextDecorator`:

```
from contextlib import AsyncContextDecorator
class mycontext(AsyncContextDecorator):
    async def __aenter__(self):
        print('Starting')
        return self
    async def __aexit__(self, *exc_info):
        print('Finishing')
        return False
```

The class can then be used like this:

```
>>> @mycontext()
... async def function():
...     print('The bit in the middle')
... >>> run(function())
Starting
The bit in the middle
Finishing
```

class contextlib.ExitStack

enter_context(cm)

push(exit)

callback(callback, /, *args, **kwargs)

pop_all()

close()

A context manager that is designed to make it easy to programmatically combine other context managers and cleanup functions, especially those that are optional or otherwise driven by input data. For example, a set of files may easily be handled in a single with statement as

follows: `with ExitStack() as stack: files = [stack.enter_context(open(fname)) for fname in filenames]` # All opened files will automatically be closed at the end of the with statement, even if attempts to open files later in the list fail

Enters a new context manager and adds its `__exit__()` method to the callback stack. The return value is the result of the context manager's own `__enter__()` method. These context managers may suppress exceptions just as they normally would if used directly as part of a with statement. Changed in version

3.11: Raises `TypeError` instead of `AttributeError` if `cm` is not a context manager.

Adds a context manager's `__exit__()` method to the callback stack. As `__enter__` is not invoked, this method can be used to cover part of an `__enter__()` implementation with a context manager's own `__exit__()` method. If passed an object that is not a context manager, this method assumes it is a callback with the same signature as a context manager's `__exit__()` method and adds it directly to the callback stack. By returning true values, these callbacks can suppress exceptions the same way context manager `__exit__()`

Accepts an arbitrary callback function and arguments and adds it to the callback stack. Unlike the other methods, callbacks added this way cannot suppress exceptions (as they are never passed the exception details). The passed in callback is returned from the function, allowing this method to be used as a function decorator.

Transfers the callback stack to a fresh `ExitStack` instance and returns it. No callbacks are invoked by this operation - instead, they will now be invoked when the new stack is closed (either explicitly or implicitly at the end of a with statement). For example, a group of files can be opened as an "all or nothing" operation as follows: `with ExitStack() as stack: files = [stack.enter_context(open(fname)) for fname in filenames]` # Hold onto the close method, but don't call it yet. `close_files = stack.pop_all().close` #

Immediately unwinds the callback stack, invoking callbacks in the reverse order of registration. For any context managers and exit callbacks registered, the arguments passed in will indicate that no exception occurred.

class contextlib.AsyncExitStack

async def enter_async_context(cm)

push_async_exit(exit)

push_async_callback(callback, /, *args, **kwargs)

async def aclose()

An asynchronous context manager, similar to `ExitStack`, that supports combining both synchronous and asynchronous context managers, as well as having coroutines for cleanup logic. The `close()` method is not implemented; `aclose()` must be used instead. `async def enter_async_context(cm)` Similar to `ExitStack.enter_context()` but expects an asynchronous context manager. Changed in version 3.11: Raises `TypeError` instead of `AttributeError` if `cm` is not an asynchronous context manager. `push_async_exit(exit)` Similar to `ExitStack.push()`

Similar to `ExitStack.enter_context()` but expects an asynchronous context manager. Changed in version 3.11: Raises `TypeError` instead of `AttributeError` if `cm` is not an asynchronous context manager.

Similar to `ExitStack.push()` but expects either an asynchronous context manager or a coroutine function.

Similar to `ExitStack.callback()` but expects a coroutine function.

Similar to `ExitStack.close()` but properly handles awaitables.

Examples and Recipes

This section describes some examples and recipes for making effective use of the tools provided by `contextlib`.

Supporting a variable number of context managers

The primary use case for `ExitStack` is the one given in the class documentation: supporting a variable number of context managers and other cleanup operations in a single `with` statement. The variability may come from the number of context managers needed being driven by user input (such as opening a user specified collection of files), or from some of the context managers being optional:

```
with ExitStack() as stack:
    for resource in resources:
        stack.enter_context(resource)
    if need_special_resource():
        special = acquire_special_resource()
        stack.callback(release_special_resource, special)
    # Perform operations that use the acquired resources
```

As shown, `ExitStack` also makes it quite easy to use `with` statements to manage arbitrary resources that don't natively support the context management protocol.

Catching exceptions from `__enter__` methods

It is occasionally desirable to catch exceptions from an `__enter__()` method implementation, without inadvertently catching exceptions from the `with` statement body or the context manager's `__exit__()` method. By using `ExitStack` the steps in the context management protocol can be separated slightly in order to allow this:

```
stack = ExitStack()
try:
    x = stack.enter_context(cm)
except Exception:
    # handle __enter__ exception
else:
    with stack:
        # Handle normal case
```

Actually needing to do this is likely to indicate that the underlying API should be providing a direct resource management interface for use with `try/except/finally` statements, but not all APIs are well designed in that regard. When a context manager is the only resource management API provided, then `ExitStack` can make it easier to handle various situations that can't be handled directly in a `with` statement.

Cleaning up in an `__enter__` implementation

As noted in the documentation of `ExitStack.push()`, this method can be useful in cleaning up an already allocated resource if later steps in the `__enter__()` implementation fail.

Here's an example of doing this for a context manager that accepts resource acquisition and release functions, along with an optional validation function, and maps them to the context management protocol:

```
from contextlib import contextmanager, AbstractContextManager, ExitStack
```

```
class ResourceManager(AbstractContextManager):

    def __init__(self, acquire_resource, release_resource, check_resource_ok=None):
        self.acquire_resource = acquire_resource
        self.release_resource = release_resource
        if check_resource_ok is None:
            def check_resource_ok(resource):
```

```

        return True
    self.check_resource_ok = check_resource_ok

    @contextmanager
    def _cleanup_on_error(self):
        with ExitStack() as stack:
            stack.push(self)
            yield
            # The validation check passed and didn't raise an exception
            # Accordingly, we want to keep the resource, and pass it
            # back to our caller
            stack.pop_all()

    def __enter__(self):
        resource = self.acquire_resource()
        with self._cleanup_on_error():
            if not self.check_resource_ok(resource):
                msg = "Failed validation for {!r}"
                raise RuntimeError(msg.format(resource))
        return resource

    def __exit__(self, *exc_details):
        # We don't need to duplicate any of our resource release logic
        self.release_resource()

```

Replacing any use of try-finally and flag variables

A pattern you will sometimes see is a try-finally statement with a flag variable to indicate whether or not the body of the finally clause should be executed. In its simplest form (that can't already be handled just by using an except clause instead), it looks something like this:

```

cleanup_needed = True
try:
    result = perform_operation()
    if result:
        cleanup_needed = False
finally:
    if cleanup_needed:
        cleanup_resources()

```

As with any try statement based code, this can cause problems for development and review, because the setup code and the cleanup code can end up being separated by arbitrarily long sections of code.

ExitStack makes it possible to instead register a callback for execution at the end of a with statement, and then later decide to skip executing that callback:

```

from contextlib import ExitStack

with ExitStack() as stack:
    stack.callback(cleanup_resources)
    result = perform_operation()
    if result:
        stack.pop_all()

```

This allows the intended cleanup behaviour to be made explicit up front, rather than requiring a separate flag variable.

If a particular application uses this pattern a lot, it can be simplified even further by means of a small helper class:

```

from contextlib import ExitStack

class Callback(ExitStack):
    def __init__(self, callback, /, *args, **kwargs):
        super().__init__()

```

```

        self.callback(callback, *args, **kwargs)

    def cancel(self):
        self.pop_all()

with Callback(cleanup_resources) as cb:
    result = perform_operation()
    if result:
        cb.cancel()

```

If the resource cleanup isn't already neatly bundled into a standalone function, then it is still possible to use the decorator form of `ExitStack.callback()` to declare the resource cleanup in advance:

```

from contextlib import ExitStack

with ExitStack() as stack:
    @stack.callback
    def cleanup_resources():
        ...
    result = perform_operation()
    if result:
        stack.pop_all()

```

Due to the way the decorator protocol works, a callback function declared this way cannot take any parameters. Instead, any resources to be released must be accessed as closure variables.

Using a context manager as a function decorator

`ContextDecorator` makes it possible to use a context manager in both an ordinary `with` statement and also as a function decorator.

For example, it is sometimes useful to wrap functions or groups of statements with a logger that can track the time of entry and time of exit. Rather than writing both a function decorator and a context manager for the task, inheriting from `ContextDecorator` provides both capabilities in a single definition:

```

from contextlib import ContextDecorator
import logging

logging.basicConfig(level=logging.INFO)

class track_entry_and_exit(ContextDecorator):
    def __init__(self, name):
        self.name = name

    def __enter__(self):
        logging.info('Entering: %s', self.name)

    def __exit__(self, exc_type, exc, exc_tb):
        logging.info('Exiting: %s', self.name)

```

Instances of this class can be used as both a context manager:

```

with track_entry_and_exit('widget loader'):
    print('Some time consuming activity goes here')
    load_widget()

```

And also as a function decorator:

```

@track_entry_and_exit('widget loader')
def activity():
    print('Some time consuming activity goes here')
    load_widget()

```

Note that there is one additional limitation when using context managers as function decorators: there's no way to access the return value of `__enter__()`. If that value is needed, then it is still necessary to use an

explicitwithstatement.

See also

PEP 343- The “with” statement

The specification, background, and examples for the Pythonwithstatement.

Single use, reusable and reentrant context managers

Most context managers are written in a way that means they can only be used effectively in awithstatement once. These single use context managers must be created afresh each time they’re used - attempting to use them a second time will trigger an exception or otherwise not work correctly.

This common limitation means that it is generally advisable to create context managers directly in the header of thewithstatement where they are used (as shown in all of the usage examples above).

Files are an example of effectively single use context managers, since the firstwithstatement will close the file, preventing any further IO operations using that file object.

Context managers created usingcontextmanager()are also single use context managers, and will complain about the underlying generator failing to yield if an attempt is made to use them a second time:

```
>>> from contextlib import contextmanager
>>> @contextmanager
... def singleuse():
...     print("Before")
...     yield
...     print("After")
...
>>> cm = singleuse()
>>> with cm:
...     pass
...
Before
After
>>> with cm:
...     pass
...
Traceback (most recent call last):
...
RuntimeError: generator didn't yield
```

Reentrant context managers

More sophisticated context managers may be “reentrant”. These context managers can not only be used in multiplewithstatements, but may also be usedinsideawithstatement that is already using the same context manager.

threading.RLockis an example of a reentrant context manager, as aresuppress(),redirect_stdout(), andchdir(). Here's a very simple example of reentrant use:

```
>>> from contextlib import redirect_stdout
>>> from io import StringIO
>>> stream = StringIO()
>>> write_to_stream = redirect_stdout(stream)
>>> with write_to_stream:
...     print("This is written to the stream rather than stdout")
...     with write_to_stream:
...         print("This is also written to the stream")
...
>>> print("This is written directly to stdout")
This is written directly to stdout
```

```
>>> print(stream.getvalue())
This is written to the stream rather than stdout
This is also written to the stream
```

Real world examples of reentrancy are more likely to involve multiple functions calling each other and hence be far more complicated than this example.

Note also that being reentrant is not the same thing as being thread safe. `redirect_stdout()`, for example, is definitely not thread safe, as it makes a global modification to the system state by binding `sys.stdout` to a different stream.

Reusable context managers

Distinct from both single use and reentrant context managers are “reusable” context managers (or, to be completely explicit, “reusable, but not reentrant” context managers, since reentrant context managers are also reusable). These context managers support being used multiple times, but will fail (or otherwise not work correctly) if the specific context manager instance has already been used in a containing with statement.

`threading.Lock` is an example of a reusable, but not reentrant, context manager (for a reentrant lock, it is necessary to use `threading.RLock` instead).

Another example of a reusable, but not reentrant, context manager is `ExitStack`, as it invokes all currently registered callbacks when leaving any with statement, regardless of where those callbacks were added:

```
>>> from contextlib import ExitStack
>>> stack = ExitStack()
>>> with stack:
...     stack.callback(print, "Callback: from first context")
...     print("Leaving first context")
...
Leaving first context
Callback: from first context
>>> with stack:
...     stack.callback(print, "Callback: from second context")
...     print("Leaving second context")
...
Leaving second context
Callback: from second context
>>> with stack:
...     stack.callback(print, "Callback: from outer context")
...     with stack:
...         stack.callback(print, "Callback: from inner context")
...         print("Leaving inner context")
...     print("Leaving outer context")
...
Leaving inner context
Callback: from inner context
Callback: from outer context
Leaving outer context
```

As the output from the example shows, reusing a single stack object across multiple with statements works correctly, but attempting to nest them will cause the stack to be cleared at the end of the innermost with statement, which is unlikely to be desirable behaviour.

Using separate `ExitStack` instances instead of reusing a single instance avoids that problem:

```
>>> from contextlib import ExitStack
>>> with ExitStack() as outer_stack:
...     outer_stack.callback(print, "Callback: from outer context")
...     with ExitStack() as inner_stack:
...         inner_stack.callback(print, "Callback: from inner context")
...         print("Leaving inner context")
...     print("Leaving outer context")
...
Leaving inner context
Leaving outer context
```

Leaving inner context
Callback: from inner context
Leaving outer context
Callback: from outer context

abc— Abstract Base Classes

Source code: `Lib/abc.py`

This module provides the infrastructure for defining abstract base classes (ABCs) in Python, as outlined in PEP 3119; see the PEP for why this was added to Python. (See also PEP 3141 and the `numbers` module regarding a type hierarchy for numbers based on ABCs.)

The `collections` module has some concrete classes that derive from ABCs; these can, of course, be further derived. In addition, the `collections.abc` submodule has some ABCs that can be used to test whether a class or instance provides a particular interface, for example, if it is hashable or if it is mapping.

This module provides the metaclass `ABCMeta` for defining ABCs and a helper class `ABC` to alternatively define ABCs through inheritance:

class `abc.ABC`

A helper class that has `ABCMeta` as its metaclass. With this class, an abstract base class can be created by simply deriving from `ABC` avoiding sometimes confusing metaclass usage, for example: `from abc import ABC; class MyABC(ABC): pass`. Note that the type of `ABC` is still `ABCMeta`, therefore inheriting from `ABC` requires the usual precautions regarding metaclass usage, as multiple inheritance may lead to metaclass conflicts. One may also define an abstract base class by passing the metaclass keyword and using `ABCMeta`.

class `abc.ABCMeta`

`register(subclass)`

`__subclasshook__(subclass)`

Metaclass for defining Abstract Base Classes (ABCs). Use this metaclass to create an ABC. An ABC can be subclassed directly, and then acts as a mix-in class. You can also register unrelated concrete classes (even built-in classes) and unrelated ABCs as “virtual subclasses” – these and their descendants will be considered subclasses of the registering ABC by the built-in `issubclass()` function, but the registering ABC won’t show up in their MRO (Method Resolution Order) nor will method implementation.

Registers `subclass` as a “virtual subclass” of this ABC. For example: `from abc import ABC; class MyABC(ABC): pass; MyABC.register(tuple); assert issubclass(tuple, MyABC); assert isinstance((), MyABC)`. Changed in version 3.3: Returns the registered subclass, to allow usage as a class decorator. Changed in version 3.4: To detect calls to `register()`, you can use the `get_cache_token()` function.

(Must be defined as a class method.) Check whether `subclass` is considered a subclass of this ABC. This means that you can customize the behavior of `issubclass()` further without the need to call `register()` on every class you want to consider a subclass of the ABC. (This class method is called from the `__subclasscheck__()` method of the ABC.) This method should return `True`, `False` or `NotImplemented`. If it returns `True`, the `subclass` is considered a subclass of this ABC. If it returns `False`, the `subclass` is not considered.

The `abc` module also provides the following decorator:

@`abc.abstractmethod`

A decorator indicating abstract methods. Using this decorator requires that the class’s metaclass is `ABCMeta` or is derived from it. A class that has a metaclass derived from `ABCMeta` cannot be instantiated unless all of its abstract methods and properties are overridden. The abstract methods can be called using any of the normal ‘super’ call mechanisms. `abstractmethod()` may be used to declare abstract methods for properties and descriptors. Dynamically adding abstract methods to a class, or attempting

The `abc` module also supports the following legacy decorators:

@abc.abstractclassmethod

Added in version 3.2. Deprecated since version 3.3: It is now possible to use `classmethod` with `abstractmethod()`, making this decorator redundant. A subclass of the built-in `classmethod()`, indicating an abstract classmethod. Otherwise it is similar to `abstractmethod()`. This special case is deprecated, as the `classmethod()` decorator is now correctly identified as abstract when applied to an abstract

method: `class C(ABC): @classmethod @abstractmethod def my_abstract_classmethod(cls, arg): ...`

@abc.abstractstaticmethod

Added in version 3.2. Deprecated since version 3.3: It is now possible to use `staticmethod` with `abstractmethod()`, making this decorator redundant. A subclass of the built-in `staticmethod()`, indicating an abstract staticmethod. Otherwise it is similar to `abstractmethod()`. This special case is deprecated, as the `staticmethod()` decorator is now correctly identified as abstract when applied to an abstract

method: `class C(ABC): @staticmethod @abstractmethod def my_abstract_staticmethod(arg): ...`

@abc.abstractproperty

Deprecated since version 3.3: It is now possible to use `property`, `property.getter()`, `property.setter()` and `property.deleter()` with `abstractmethod()`, making this decorator redundant. A subclass of the built-in `property()`, indicating an abstract property. This special case is deprecated, as the `property()` decorator is now correctly identified as abstract when applied to an abstract

method: `class C(ABC): @property @abstractmethod def my_abstract_property(self): ...` The above example defines a read-only property; you can

The `abc` module also provides the following functions:

abc.get_cache_token()

Returns the current abstract base class cache token. The token is an opaque object (that supports equality testing) identifying the current version of the abstract base class cache for virtual subclasses. The token changes with every call to `ABCMeta.register()` on any ABC. Added in version 3.4.

abc.update_abstractmethods(cls)

A function to recalculate an abstract class's abstraction status. This function should be called if a class's abstract methods have been implemented or changed after it was created. Usually, this function should be called from within a class decorator. Returns `cls`, to allow usage as a class decorator. If `cls` is not an instance of `ABCMeta`, does nothing. Note This function assumes that `cls`'s superclasses are already updated. It does not update any subclasses. Added in version 3.10.

Footnotes

C++ programmers should note that Python's virtual base class concept is not the same as C++'s.

atexit—Exit handlers

The `atexit` module defines functions to register and unregister cleanup functions. Functions thus registered are automatically executed upon normal interpreter termination. `atexit` runs these functions in the reverse order in which they were registered; if you register `A`, `B`, and `C`, at interpreter termination time they will be run in the order `C`, `B`, `A`.

Note: The functions registered via this module are not called when the program is killed by a signal not handled by Python, when a Python fatal internal error is detected, or when `os._exit()` is called.

Note: The effect of registering or unregistering functions from within a cleanup function is undefined.

Changed in version 3.7: When used with C-API subinterpreters, registered functions are local to the interpreter they were registered in.

atexit.register(func,*args,kwargs)**

Register `func` as a function to be executed at termination. Any optional arguments that are to be passed to `func` must be passed as arguments to `register()`. It is possible to register the same function and arguments more than once. At normal program termination (for instance, if `sys.exit()` is called or the main module's execution completes), all functions registered are called in last in, first out order. The assumption is that lower level modules will normally be imported before higher level modules.

atexit.unregister(func)

Remove `func` from the list of functions to be run at interpreter shutdown. `unregister()` silently does nothing if `func` was not previously registered. If `func` has been registered more than once, every occurrence of that function in the `atexit` call stack will be removed. Equality comparisons (`==`) are used internally during unregistration, so function references do not need to have matching identities.

See also

Module readline

Useful example of `atexit` to read and write `readline` history files.

atexit Example

The following simple example demonstrates how a module can initialize a counter from a file when it is imported and save the counter's updated value automatically when the program terminates without relying on the application making an explicit call into this module at termination.

```
try:
    with open('counterfile') as infile:
        _count = int(infile.read())
except FileNotFoundError:
    _count = 0

def incrcounter(n):
    global _count
    _count = _count + n

def savecounter():
    with open('counterfile', 'w') as outfile:
        outfile.write('%d' % _count)

import atexit

atexit.register(savecounter)
```

Positional and keyword arguments may also be passed to `register()` to be passed along to the registered function when it is called:

```
def goodbye(name, adjective):
    print('Goodbye %s, it was %s to meet you.' % (name, adjective))

import atexit

atexit.register(goodbye, 'Donny', 'nice')
# or:
atexit.register(goodbye, adjective='nice', name='Donny')
```

Usage as a decorator:

```
import atexit

@atexit.register
def goodbye():
    print('You are now leaving the Python sector.')
```

This only works with functions that can be called without arguments.

traceback— Print or retrieve a stack traceback

Source code: `Lib/traceback.py`

This module provides a standard interface to extract, format and print stack traces of Python programs. It is more flexible than the interpreter's default traceback display, and therefore makes it possible to configure certain aspects of the output. Finally, it contains a utility for capturing enough information about an exception to print it later, without the need to save a reference to the actual exception. Since exceptions can be the roots of large objects graph, this utility can significantly improve memory management.

The module uses traceback objects— these are objects of type `TracebackType`, which are assigned to the `__traceback__` field of `BaseException` instances.

See also

Module `faulthandler`

Module `pdb`

Used to dump Python tracebacks explicitly, on a fault, after a timeout, or on a user signal.

Interactive source code debugger for Python programs.

The module's API can be divided into two parts:

- Module-level functions offering basic functionality, which are useful for interactive inspection of exceptions and tracebacks.
- `TracebackException` class and its helper classes `StackSummary` and `FrameSummary`. These offer both more flexibility in the output generated and the ability to store the information necessary for later formatting without holding references to actual exception and traceback objects.

Added in version 3.13: Output is colorized by default and can be controlled using environment variables.

Module-Level Functions

`traceback.print_tb(tb, limit=None, file=None)`

Print up to `limit` stack trace entries from traceback object `tb` (starting from the caller's frame) if `limit` is positive. Otherwise, print the last `abs(limit)` entries. If `limit` is omitted or `None`, all entries are printed. If `file` is omitted or `None`, the output goes to `sys.stderr`; otherwise it should be an open file or file-like object to receive the output. Note the meaning of the `limit` parameter is different than the meaning of `sys.tracebacklimit`. A negative `limit` value corresponds to a positive value of `sys.tracebacklimit`.

`traceback.print_exception(exc, /, [value, tb,], limit=None, file=None, chain=True)`

Print exception information and stack trace entries from traceback object `tb` to `file`. This differs from `print_tb()` in the following ways: if `tb` is not `None`, it prints a header `Traceback (most recent call last):`; it prints the exception type and `value` after the stack trace if `type(value) is SyntaxError` and `value` has the appropriate format, it prints the line where the syntax error occurred with a caret indicating the approximate position of the error. Since Python 3.10, instead of passing `value` and `tb`, an exception object can

`traceback.print_exc(limit=None, file=None, chain=True)`

This is a shorthand for `print_exception(sys.exception(), limit=limit, file=file, chain=chain)`.

`traceback.print_last(limit=None, file=None, chain=True)`

This is a shorthand for `print_exception(sys.last_exc, limit=limit, file=file, chain=chain)`. In general it will work only after an exception has reached an interactive prompt (see `sys.last_exc`).

`traceback.print_stack(f=None, limit=None, file=None)`

Print up to `limit` stack trace entries (starting from the invocation point) if `limit` is positive. Otherwise, print the last `limit` entries. If `limit` is omitted or `None`, all entries are printed. The optional `file` argument can be used to specify an alternate stack frame to start. The optional `file` argument has the same meaning as for `print_tb()`. Changed in version 3.5: Added negative `limit` support.

`traceback.extract_tb(tb, limit=None)`

Return a `StackSummary` object representing a list of “pre-processed” stack trace entries extracted from the `traceback` object `tb`. It is useful for alternate formatting of stack traces. The optional `limit` argument has the same meaning as for `print_tb()`. A “pre-processed” stack trace entry is a `FrameSummary` object containing attributes `filename`, `lineno`, `name`, and `linerepr` representing the information that is usually printed for a stack trace.

`traceback.extract_stack(f=None, limit=None)`

Extract the raw traceback from the current stack frame. The return value has the same format as for `extract_tb()`. The optional `file` and `limit` arguments have the same meaning as for `print_stack()`.

`traceback.print_list(extracted_list, file=None)`

Print the list of tuples as returned by `extract_tb()` or `extract_stack()` as a formatted stack trace to the given file. If `file` is `None`, the output is written to `sys.stderr`.

`traceback.format_list(extracted_list)`

Given a list of tuples or `FrameSummary` objects as returned by `extract_tb()` or `extract_stack()`, return a list of strings ready for printing. Each string in the resulting list corresponds to the item with the same index in the argument list. Each string ends in a newline; the strings may contain internal newlines as well, for those items whose source text line is not `None`.

`traceback.format_exception_only(exc, /, [value,]*, show_group=False)`

Format the exception part of a traceback using an exception value such as given by `sys.last_value`. The return value is a list of strings, each ending in a newline. The list contains the exception's message, which is normally a single string; however, for `SyntaxError` exceptions, it contains several lines that (when printed) display detailed information about where the syntax error occurred. Following the message, the list contains the exception's notes. Since Python 3.10, instead of passing `value`, an

`traceback.format_exception(exc, /, [value, tb,] limit=None, chain=True)`

Format a stack trace and the exception information. The arguments have the same meaning as the corresponding arguments to `print_exception()`. The return value is a list of strings, each ending in a newline and some containing internal newlines. When these lines are concatenated and printed, exactly the same text is printed as does `print_exception()`. Changed in version 3.5: The `type` argument is ignored and inferred from the type of `value`. Changed in version 3.10: This function's behavior and signature

`traceback.format_exc(limit=None, chain=True)`

This is like `print_exc(limit)` but returns a string instead of printing to a file.

`traceback.format_tb(tb, limit=None)`

A shorthand for `format_list(extract_tb(tb, limit))`.

`traceback.format_stack(f=None, limit=None)`

A shorthand for `format_list(extract_stack(f, limit))`.

`traceback.clear_frames(tb)`

Clears the local variables of all the stack frames in a traceback by calling the `clear()` method of each frame object. Added in version 3.4.

`traceback.walk_stack(f)`

Walk a stack following `f.back` from the given frame, yielding the frame and line number for each frame. If `f` is `None`, the current stack is used. This helper is used with `StackSummary.extract()`. Added in version 3.5. Changed in version 3.14: This function previously returned a generator that would walk the stack when first iterated over. The generator returned now is the state of the stack when `walk_stack` is called.

`traceback.walk_tb(tb)`

Walk a traceback following `tb.next` yielding the frame and line number for each frame. This helper is used with `StackSummary.extract()`. Added in version 3.5.

TracebackException Objects

Added in version 3.5.

`TracebackException` objects are created from actual exceptions to capture data for later printing. They offer a more lightweight method of storing this information by avoiding holding references to traceback and frame objects. In addition, they expose more options to configure the output compared to the module-level functions described above.

`class traceback.TracebackException(exc_type, exc_value, exc_traceback, *, limit=None, lookup_lines=True, capture_locals=False, compact=False, max_group_width=15, max_group_depth=10)`

`__cause__`

`__context__`

`exceptions`

`__suppress_context__`

`__notes__`

`stack`

`exc_type`

`exc_type_str`

`filename`

`lineno`

`end_lineno`

`text`

`offset`

`end_offset`

`msg`

`classmethod from_exception(exc, *, limit=None, lookup_lines=True, capture_locals=False)`

`print(*, file=None, chain=True)`

`format(*, chain=True)`

`format_exception_only(*, show_group=False)`

Capture an exception for later rendering. The meaning of `limit`, `lookup_lines` and `capture_locals` are as for the `StackSummary` class. If `compact` is true, only data that is required by `TracebackException`'s `format()` method is saved in the class attributes. In particular, the `__context__` field is calculated only if `__cause__` is `None` and `__suppress_context__` is false. Note that when locals are captured, they are also shown in the traceback. `max_group_width` and `max_group_depth` control the formatting of exception groups (see `BaseException`).

A `TracebackException` of the original `__cause__`.

A `TracebackException` of the original `__context__`.

If self represents an `ExceptionGroup`, this field holds a list of `TracebackException` instances representing the nested exceptions. Otherwise it is `None`. Added in version 3.11.

The `__suppress_context__` value from the original exception.

The `__notes__` value from the original exception, or `None` if the exception does not have any notes. If it is not `None` it is formatted in the traceback after the exception string. Added in version 3.11.

A `StackSummary` representing the traceback.

The class of the original traceback. Deprecated since version 3.13.

String display of the class of the original exception. Added in version 3.13.

For syntax errors - the file name where the error occurred.

For syntax errors - the line number where the error occurred.

For syntax errors - the end line number where the error occurred. Can be `None` if not present. Added in version 3.10.

For syntax errors - the text where the error occurred.

For syntax errors - the offset into the text where the error occurred.

For syntax errors - the end offset into the text where the error occurred. Can be `None` if not present. Added in version 3.10.

For syntax errors - the compiler error message.

Capture an exception for later rendering. `limit`, `lookup_lines` and `capture_locals` are as for the `StackSummary` class. Note that when locals are captured, they are also shown in the traceback.

Print to file (default `sys.stderr`) the exception information returned by `format()`. Added in version 3.11.

Format the exception. If `chain` is not `True`, `__cause__` and `__context__` will not be formatted. The return value is a generator of strings, each ending in a newline and some containing internal newlines. `print_exception()` is a wrapper around this method which just prints the lines to a file.

Format the exception part of the traceback. The return value is a generator of strings, each ending in a newline. When `show_group` is `False`, the generator emits the exception's message followed by its notes (if it has any). The exception message is normally a single string; however, for `SyntaxError` exceptions, it consists of several lines that (when printed) display detailed information about where the syntax error occurred. When `show_group` is `True`, and the exception is an instance of `BaseExceptionGroup`, the

StackSummary Objects

Added in version 3.5.

`StackSummary` objects represent a call stack ready for formatting.

class `traceback.StackSummary`

classmethodextract(frame_gen,*,limit=None,lookup_lines=True,capture_locals=False)

classmethodfrom_list(a_list)

format()

format_frame_summary(frame_summary)

classmethodextract(frame_gen,*,limit=None,lookup_lines=True,capture_locals=False) Construct a `StackSummary` object from a frame generator (such as is returned by `walk_stack()` or `walk_tb()`). If `limit` is supplied, only this many frames are taken from `frame_gen`. If `lookup_lines` is `False`, the returned `FrameSummary` objects will not have read their lines in yet, making the cost of creating the `StackSummary` cheaper (which may be valuable if it may not actually get formatted). If `capture_locals` is `True` the local variables in

Construct a `StackSummary` object from a frame generator (such as is returned by `walk_stack()` or `walk_tb()`). If `limit` is supplied, only this many frames are taken from `frame_gen`. If `lookup_lines` is `False`, the returned `FrameSummary` objects will not have read their lines in yet, making the cost of creating the `StackSummary` cheaper (which may be valuable if it may not actually get formatted). If `capture_locals` is `True` the local variables in each `FrameSummary` are captured as object representations. Changed in version 3.12: E

Construct a `StackSummary` object from a supplied list of `FrameSummary` objects or old-style list of tuples. Each tuple should be a 4-tuple with `filename`, `lineno`, `name`, `line` as the elements.

Returns a list of strings ready for printing. Each string in the resulting list corresponds to a single frame from the stack. Each string ends in a newline; the strings may contain internal newlines as well, for those items with source text lines. For long sequences of the same frame and line, the first few repetitions are shown, followed by a summary line stating the exact number of further repetitions. Changed in version 3.6: Long sequences of repeated frames are now abbreviated.

Returns a string for printing one of the frames involved in the stack. This method is called for each `FrameSummary` object to be printed by `StackSummary.format()`. If it returns `None`, the frame is omitted from the output. Added in version 3.11.

FrameSummary Objects

Added in version 3.5.

A `FrameSummary` object represents a single frame in a `traceback`.

class traceback.FrameSummary(filename,lineno,name,*,lookup_line=True,locals=None,line=None,end_lineno=None,colno=None,end_colno=None)

filename

lineno

name

line

end_lineno

colno

end_colno

Represents a single frame in the `traceback` or stack that is being formatted or printed. It may optionally have a stringified version of the frame's locals included in it. If `lookup_line` is `False`, the source code is not looked up until the `FrameSummary` has the `line` attribute accessed (which also happens when casting it to a tuple). `line` may be

directly provided, and will prevent line lookups happening at all. `locals` is an optional local variable mapping, and if supplied the variable representations are stored in

The filename of the source code for this frame. Equivalent to accessing `f_code.co_filename` on a frame object `f`.

The line number of the source code for this frame.

Equivalent to accessing `f_code.co_name` on a frame object `f`.

A string representing the source code for this frame, with leading and trailing whitespace stripped. If the source is not available, it is `None`.

The last line number of the source code for this frame. By default, it is set to `lineno` and indexation starts from 1. Changed in version 3.13: The default value changed from `None` to `lineno`.

The column number of the source code for this frame. By default, it is `None` and indexation starts from 0.

The last column number of the source code for this frame. By default, it is `None` and indexation starts from 0.

Examples of Using the Module-Level Functions

This simple example implements a basic read-eval-print loop, similar to (but less useful than) the standard Python interactive interpreter loop. For a more complete implementation of the interpreter loop, refer to `thecodemodule`.

```
import sys, traceback

def run_user_code(envdir):
    source = input(">>> ")
    try:
        exec(source, envdir)
    except Exception:
        print("Exception in user code:")
        print("-"*60)
        traceback.print_exc(file=sys.stdout)
        print("-"*60)

envdir = {}
while True:
    run_user_code(envdir)
```

The following example demonstrates the different ways to print and format the exception and traceback:

```
import sys, traceback

def lumberjack():
    bright_side_of_life()

def bright_side_of_life():
    return tuple()[0]

try:
    lumberjack()
except IndexError as exc:
    print("*** print_tb:")
    traceback.print_tb(exc.__traceback__, limit=1, file=sys.stdout)
    print("*** print_exception:")
    traceback.print_exception(exc, limit=2, file=sys.stdout)
    print("*** print_exc:")
    traceback.print_exc(limit=2, file=sys.stdout)
    print("*** format_exc, first and last line:")
    formatted_lines = traceback.format_exc().splitlines()
    print(formatted_lines[0])
```

```

print(formatted_lines[-1])
print("*** format_exception:")
print(repr(traceback.format_exception(exc)))
print("*** extract_tb:")
print(repr(traceback.extract_tb(exc.__traceback__)))
print("*** format_tb:")
print(repr(traceback.format_tb(exc.__traceback__)))
print("*** tb_lineno:", exc.__traceback__.tb_lineno)

```

The output for the example would look similar to this:

```

*** print_tb:
File "<doctest...>", line 10, in <module>
    lumberjack()
    ~~~~~^^
*** print_exception:
Traceback (most recent call last):
  File "<doctest...>", line 10, in <module>
    lumberjack()
    ~~~~~^^

  File "<doctest...>", line 4, in lumberjack
    bright_side_of_life()
    ~~~~~^^

IndexError: tuple index out of range
*** print_exc:
Traceback (most recent call last):
  File "<doctest...>", line 10, in <module>
    lumberjack()
    ~~~~~^^

  File "<doctest...>", line 4, in lumberjack
    bright_side_of_life()
    ~~~~~^^

IndexError: tuple index out of range
*** format_exc, first and last line:
Traceback (most recent call last):
IndexError: tuple index out of range
*** format_exception:
['Traceback (most recent call last):\n',
 '  File "<doctest default[0]>", line 10, in <module>\n    lumberjack()\n    ~~~~~^^\n',
 '  File "<doctest default[0]>", line 4, in lumberjack\n    bright_side_of_life()\n    ~~~~~^^\n',
 '  File "<doctest default[0]>", line 7, in bright_side_of_life\n    return tuple()[0]\n    ~~~~~^^\n',
 'IndexError: tuple index out of range\n']
*** extract_tb:
[<FrameSummary file <doctest...>, line 10 in <module>>,
 <FrameSummary file <doctest...>, line 4 in lumberjack>,
 <FrameSummary file <doctest...>, line 7 in bright_side_of_life>]
*** format_tb:
['  File "<doctest default[0]>", line 10, in <module>\n    lumberjack()\n    ~~~~~^^\n',
 '  File "<doctest default[0]>", line 4, in lumberjack\n    bright_side_of_life()\n    ~~~~~^^\n',
 '  File "<doctest default[0]>", line 7, in bright_side_of_life\n    return tuple()[0]\n    ~~~~~^^\n']
*** tb_lineno: 10

```

The following example shows the different ways to print and format the stack:

```

>>> import traceback
>>> def another_function():
...     lumberstack()
...
>>> def lumberstack():
...     traceback.print_stack()
...     print(repr(traceback.extract_stack()))
...     print(repr(traceback.format_stack()))
...
>>> another_function()
File "<doctest>", line 10, in <module>
    another_function()

```

```

File "<doctest>", line 3, in another_function
    lumberstack()
File "<doctest>", line 6, in lumberstack
    traceback.print_stack()
[('<doctest>', 10, '<module>', 'another_function()'),
 ('<doctest>', 3, 'another_function', 'lumberstack()'),
 ('<doctest>', 7, 'lumberstack', 'print(repr(traceback.extract_stack()))')]
[' File "<doctest>", line 10, in <module>\n    another_function()\n',
 ' File "<doctest>", line 3, in another_function\n    lumberstack()\n',
 ' File "<doctest>", line 8, in lumberstack\n    print(repr(traceback.format_stack()))\n']

```

This last example demonstrates the final few formatting functions:

```

>>> import traceback
>>> traceback.format_list([('spam.py', 3, '<module>', 'spam.eggs()'),
...                        ('eggs.py', 42, 'eggs', 'return "bacon"')])
[' File "spam.py", line 3, in <module>\n    spam.eggs()\n',
 ' File "eggs.py", line 42, in eggs\n    return "bacon"\n']
>>> an_error = IndexError('tuple index out of range')
>>> traceback.format_exception_only(an_error)
['IndexError: tuple index out of range\n']

```

Examples of Using TracebackException

With the helper class, we have more options:

```

>>> import sys
>>> from traceback import TracebackException
>>>
>>> def lumberjack():
...     bright_side_of_life()
...
>>> def bright_side_of_life():
...     t = "bright", "side", "of", "life"
...     return t[5]
...
>>> try:
...     lumberjack()
... except IndexError as e:
...     exc = e
...
>>> try:
...     try:
...         lumberjack()
...     except:
...         1/0
... except Exception as e:
...     chained_exc = e
...
>>> # limit works as with the module-level functions
>>> TracebackException.from_exception(exc, limit=-2).print()
Traceback (most recent call last):
  File "<python-input-1>", line 6, in lumberjack
    bright_side_of_life()
    ~~~~~^
  File "<python-input-1>", line 10, in bright_side_of_life
    return t[5]
           ~^^
IndexError: tuple index out of range

```

```

>>> # capture_locals adds local variables in frames
>>> TracebackException.from_exception(exc, limit=-2, capture_locals=True).print()
Traceback (most recent call last):
  File "<python-input-1>", line 6, in lumberjack

```

```

    bright_side_of_life()
    ~~~~~^^
File "<python-input-1>", line 10, in bright_side_of_life
    return t[5]
    ~^^^
    t = ("bright", "side", "of", "life")
IndexError: tuple index out of range

```

```

>>> # The *chain* kwarg to print() controls whether chained
>>> # exceptions are displayed
>>> TracebackException.from_exception(chained_exc).print()
Traceback (most recent call last):
  File "<python-input-19>", line 4, in <module>
    lumberjack()
    ~~~~~^^
  File "<python-input-8>", line 7, in lumberjack
    bright_side_of_life()
    ~~~~~^^
  File "<python-input-8>", line 11, in bright_side_of_life
    return t[5]
    ~^^^
IndexError: tuple index out of range

```

During handling of the above exception, another exception occurred:

```

Traceback (most recent call last):
  File "<python-input-19>", line 6, in <module>
    1/0
    ~^~
ZeroDivisionError: division by zero

```

```

>>> TracebackException.from_exception(chained_exc).print(chain=False)
Traceback (most recent call last):
  File "<python-input-19>", line 6, in <module>
    1/0
    ~^~
ZeroDivisionError: division by zero

```

`__future__` — Future statement definitions

Source code: `Lib/__future__.py`

Imports of the form `from __future__ import feature` are called future statements. These are special-cased by the Python compiler to allow the use of new Python features in modules containing the future statement before the release in which the feature becomes standard.

While these future statements are given additional special meaning by the Python compiler, they are still executed like any other import statement and the `__future__` exists and is handled by the import system the same way any other Python module would be. This design serves three purposes:

- To avoid confusing existing tools that analyze import statements and expect to find the modules they're importing.
- To document when incompatible changes were introduced, and when they will be — or were — made mandatory. This is a form of executable documentation, and can be inspected programmatically via `importing __future__` and examining its contents.
- To ensure that future statements run under releases prior to Python 2.1 at least yield runtime exceptions (the import of `__future__` will fail, because there was no module of that name prior to 2.1).

Module Contents

No feature description will ever be deleted from `__future__`. Since its introduction in Python 2.1 the following features have found their way into the language using this mechanism:

```
feature | optional in | mandatory in | effect | __future__. | nested_scopes | 2.1.0b1 | 2.2 |
PEP 227 | : | Statically Nested Scopes | __future__. | generators | 2.2.0a1 | 2.3 | PEP 255 | :
| Simple Generators | __future__. | division | 2.2.0a2 | 3.0 | PEP 238 | : | Changing the
Division Operator | __future__. | absolute_import | 2.5.0a1 | 3.0 | PEP 328 | : | Imports:
Multi-Line and Absolute/Relative | __future__. | with_statement | 2.5.0a1 | 2.6 | PEP 343 | : |
The "with" Statement | __future__. | print_function | 2.6.0a2 | 3.0 | PEP 3105 | : | Make print
a function | __future__. | unicode_literals | 2.6.0a2 | 3.0 | PEP 3112 | : | Bytes literals in
Python 3000 | __future__. | generator_stop | 3.5.0b1 | 3.7 | PEP 479 | : | StopIteration
handling inside generators | __future__. | annotation
```

`class __future__.Feature`

Each statement in `__future__.py` is of the form: `FeatureName = _Feature(OptionalRelease, MandatoryRelease, CompilerFlag)` where, normally, `OptionalRelease` is less than `MandatoryRelease`, and both are 5-tuples of the same form `sys.version_info: (PY_MAJOR_VERSION, # the 2 in 2.1.0a3; an int PY_MINOR_VERSION, # the 1; an int PY_MICRO_VERSION, # the 0; an int PY_RELEASE_LEVEL, # "alpha", "beta", "candidate" or "final"; string PY_RELEASE_SERIAL # the 3; an int)`

`__Feature.getOptionalRelease()`

`OptionalRelease` records the first release in which the feature was accepted.

`__Feature.getMandatoryRelease()`

In the case of a `MandatoryRelease` that has not yet occurred, `MandatoryRelease` predicts the release in which the feature will become part of the language. Else `MandatoryRelease` records when the feature became part of the language; in releases at or after that, modules no longer need a future statement to use the feature in question, but may continue to use such imports. `MandatoryRelease` may also be `None`, meaning that a planned feature got dropped or that it is not yet decided.

`_Feature.compiler_flag`

`CompilerFlag` is the (bitfield) flag that should be passed in the fourth argument to the built-in function `compile()` to enable the feature in dynamically compiled code. This flag is stored in the `_Feature.compiler_flag` attribute on `_Feature` instances.

`from __future__ import annotations` was previously scheduled to become mandatory in Python 3.10, but the change was delayed and ultimately canceled. This feature will eventually be deprecated and removed. See PEP 649 and PEP 749.

See also

Future statements

PEP 236- Back to the `__future__`

How the compiler treats future imports.

The original proposal for the `__future__` mechanism.

gc— Garbage Collector interface

This module provides an interface to the optional garbage collector. It provides the ability to disable the collector, tune the collection frequency, and set debugging options. It also provides access to unreachable objects that the collector found but cannot free. Since the collector supplements the reference counting already used in Python, you can disable the collector if you are sure your program does not create reference cycles. Automatic collection can be disabled by calling `gc.disable()`. To debug a leaking program call `gc.set_debug(gc.DEBUG_LEAK)`. Notice that this includes `gc.DEBUG_SAVEALL`, causing garbage-collected objects to be saved in `gc.garbage` for inspection.

The `gc` module provides the following functions:

`gc.enable()`

Enable automatic garbage collection.

`gc.disable()`

Disable automatic garbage collection.

`gc.isenabled()`

Return `True` if automatic collection is enabled.

`gc.collect(generation=2)`

Perform a collection. The optional argument `generation` may be an integer specifying which generation to collect (from 0 to 2). A `ValueError` is raised if the generation number is invalid. The sum of collected objects and uncollectable objects is returned. Calling `gc.collect(0)` will perform a GC collection on the young generation. Calling `gc.collect(1)` will perform a GC collection on the young generation and an increment of the old generation. Calling `gc.collect(2)` or `gc.collect()` performs a full collection. The

`gc.set_debug(flags)`

Set the garbage collection debugging flags. Debugging information will be written to `sys.stderr`. See below for a list of debugging flags which can be combined using bit operations to control debugging.

`gc.get_debug()`

Return the debugging flags currently set.

`gc.get_objects(generation=None)`

Returns a list of all objects tracked by the collector, excluding the list returned. If `generation` is not `None`, return only the objects as follows:
0: All objects in the young generation
1: No objects, as there is no generation 1 (as of Python 3.14)
2: All objects in the old generation
Changed in version 3.8: New `generation` parameter.
Changed in version 3.14: Generation 1 is removed.
Raises an `auditing` event `gc.get_objects` with argument `generation`.

`gc.get_stats()`

Return a list of three per-generation dictionaries containing collection statistics since interpreter start. The number of keys may change in the future, but currently each dictionary will contain the following items:
`collections` is the number of times this generation was collected;
`collected` is the total number of objects collected inside this generation;
`uncollectable` is the total number of objects which were found to be uncollectable (and were therefore moved to the `gc.garbage` list) inside this generation.

`gc.set_threshold(threshold0[,threshold1[,threshold2]])`

Set the garbage collection thresholds (the collection frequency). Setting `threshold0` to zero disables collection. The GC classifies objects into two generations depending on whether they have survived a collection. New objects are placed in the young generation. If an object survives a collection it is moved into the old

generation. In order to decide when to run, the collector keeps track of the number of object allocations and deallocations since the last collection. When the number of allocation

gc.get_count()

Return the current collection counts as a tuple of (count0, count1, count2).

gc.get_threshold()

Return the current collection thresholds as a tuple of (threshold0, threshold1, threshold2).

gc.get_referrers(*objs)

Return the list of objects that directly refer to any of objs. This function will only locate those containers which support garbage collection; extension types which do refer to other objects but do not support garbage collection will not be found. Note that objects which have already been dereferenced, but which live in cycles and have not yet been collected by the garbage collector can be listed among the resulting referrers. To get only currently live objects, call `collect()` before calling `get_`

gc.get_referents(*objs)

Return a list of objects directly referred to by any of the arguments. The referents returned are those objects visited by the arguments' C-level `tp_traverse` methods (if any), and may not be all objects actually directly reachable. `tp_traverse` methods are supported only by objects that support garbage collection, and are only required to visit objects that may be involved in a cycle. So, for example, if an integer is directly reachable from an argument, that integer object may or may not appear in

gc.is_tracked(obj)

Returns `True` if the object is currently tracked by the garbage collector, `False` otherwise. As a general rule, instances of atomic types aren't tracked and instances of non-atomic types (containers, user-defined objects...) are. However, some type-specific optimizations can be present in order to suppress the garbage collector footprint of simple instances (e.g. dicts containing only atomic keys and values):
`>>>gc.is_tracked(0)False>>>gc.is_tracked("a")False>>>gc.is_tracked([])True>>>gc.is_tracked({})`

gc.is_finalized(obj)

Returns `True` if the given object has been finalized by the garbage collector, `False` otherwise.
`>>>x=None>>>class Lazarus:...def __del__(self):...global x...x=self...>>>lazarus=Lazarus()>>>gc.is_finalized(lazarus)False>>>dellazarus>>>gc.is_finalized(x)True` Added in version 3.9.

gc.freeze()

Freeze all the objects tracked by the garbage collector; move them to a permanent generation and ignore them in all the future collections. If a process will `fork()` without `exec()`, avoiding unnecessary copy-on-write in child processes will maximize memory sharing and reduce overall memory usage. This requires both avoiding creation of freed "holes" in memory pages in the parent process and ensuring that GC collections in child processes won't touch the `gc_refscouter` of long-lived objects originating

gc.unfreeze()

Unfreeze the objects in the permanent generation, put them back into the oldest generation. Added in version 3.7.

gc.get_freeze_count()

Return the number of objects in the permanent generation. Added in version 3.7.

The following variables are provided for read-only access (you can mutate the values but should not rebind them):

gc.garbage

A list of objects which the collector found to be unreachable but could not be freed (uncollectable objects). Starting with Python 3.4, this list should be empty most of the time, except when using instances of C extension types with a non-NULL `tp_dealloc`. If `DEBUG_SAVEALL` is set, then all unreachable objects will be added to this list rather than freed. Changed in version 3.2: If this list is non-empty at interpreter shutdown, a `ResourceWarning` is emitted, which is silent by default. If `DEBUG_UNCOLLECT`

gc.callbacks

A list of callbacks that will be invoked by the garbage collector before and after collection. The callbacks will be called with two arguments, `phase` and `info`. `phase` can be one of two values: "start": The garbage collection is about to start. "stop": The garbage collection has finished. `info` is a dict providing more information for the callback. The following keys are currently defined: "generation": The oldest generation being collected. "collected": When `phase` is "stop", the number of objects successful

The following constants are provided for use with `set_debug()`:

gc.DEBUG_STATS

Print statistics during collection. This information can be useful when tuning the collection frequency.

gc.DEBUG_COLLECTABLE

Print information on collectable objects found.

gc.DEBUG_UNCOLLECTABLE

Print information of uncollectable objects found (objects which are not reachable but cannot be freed by the collector). These objects will be added to `the_gc.garbage_list`. Changed in version 3.2: Also print the contents of `the_gc.garbage_list` at interpreter shutdown, if it isn't empty.

gc.DEBUG_SAVEALL

When set, all unreachable objects found will be appended to `the_gc.garbage_list` rather than being freed. This can be useful for debugging a leaking program.

gc.DEBUG_LEAK

The debugging flags necessary for the collector to print information about a leaking program (equal to `DEBUG_COLLECTABLE|DEBUG_UNCOLLECTABLE|DEBUG_SAVEALL`).

inspect— Inspect live objects

Source code: `Lib/inspect.py`

The `inspect` module provides several useful functions to help get information about live objects such as modules, classes, methods, functions, tracebacks, frame objects, and code objects. For example, it can help you examine the contents of a class, retrieve the source code of a method, extract and format the argument list for a function, or get all the information you need to display a detailed traceback.

There are four main kinds of services provided by this module: type checking, getting source code, inspecting classes and functions, and examining the interpreter stack.

Types and members

The `getmembers()` function retrieves the members of an object such as a class or module. The functions whose names begin with “is” are mainly provided as convenient choices for the second argument to `getmembers()`. They also help you determine when you can expect to find the following special attributes (see `Import-related attributes on module objects` for module attributes):

Type	Attribute	Description
class	<code>__doc__</code>	documentation string
	<code>__name__</code>	name with which this class was defined
	<code>__qualname__</code>	qualified name
	<code>__module__</code>	name of module in which this class was defined
	<code>__type_params__</code>	A tuple containing the type parameters of a generic class
method	<code>__doc__</code>	documentation string
	<code>__name__</code>	name with which this method was defined
	<code>__qualname__</code>	qualified name
	<code>__func__</code>	function object containing implementation of method
	<code>__self__</code>	instance to which this method is bound, or <code>None</code>
	<code>__module__</code>	name of module in which this method was defined
function	<code>__doc__</code>	documentation string
	<code>__name__</code>	name with which this function was defined
	<code>__qualname__</code>	qualified name
	<code>__code__</code>	code object containing compiled function

Changed in version 3.5: Add `__qualname__` and `__yieldfrom__` attributes to generators.

The `__name__` attribute of generators is now set from the function name, instead of the code name, and it can now be modified.

Changed in version 3.7: Add `__origin__` attribute to coroutines.

Changed in version 3.10: Add `__builtins__` attribute to functions.

Changed in version 3.11: Add `__suspended__` attribute to generators.

Changed in version 3.11: Add `__suspended__` attribute to coroutines.

Changed in version 3.12: Add `__suspended__` attribute to async generators.

Changed in version 3.14: Add `__generator__` attribute to frames.

`inspect.getmembers(object[, predicate])`

Return all the members of an object in a list of (name, value) pairs sorted by name. If the optional `predicate` argument—which will be called with the value object of each member—is supplied, only members for which the predicate returns a true value are included. Note that `getmembers()` will only return class attributes defined in the metaclass when the argument is a class and those attributes have been listed in the metaclass' custom `__dir__()`.

`inspect.getmembers_static(object[, predicate])`

Return all the members of an object in a list of (name, value) pairs sorted by name without triggering dynamic lookup via the descriptor protocol, `__getattr__` or `__getattribute__`. Optionally, only return members that satisfy a given predicate. Note that `getmembers_static()` may not be able to retrieve all members that `getmembers` can fetch.

(like dynamically created attributes) and may find members that getmembers can't (like descriptors that raise `AttributeError`). It can also return descriptor objects instead.

inspect.getmodule(path)

Return the name of the module named by the filepath, without including the names of enclosing packages. The file extension is checked against all of the entries in `importlib.machinery.all_suffixes()`. If it matches, the final path component is returned with the extension removed. Otherwise, `None` is returned. Note that this function only returns a meaningful name for actual Python modules - paths that potentially refer to Python packages will still return `None`. Changed in version 3.3: The function is based

inspect.ismodule(object)

Return `True` if the object is a module.

inspect.isclass(object)

Return `True` if the object is a class, whether built-in or created in Python code.

inspect.ismethod(object)

Return `True` if the object is a bound method written in Python.

inspect.ispackage(object)

Return `True` if the object is a package. Added in version 3.14.

inspect.isfunction(object)

Return `True` if the object is a Python function, which includes functions created by lambda expression.

inspect.isgeneratorfunction(object)

Return `True` if the object is a Python generator function. Changed in version 3.8: Functions wrapped in `functools.partial()` now return `True` if the wrapped function is a Python generator function. Changed in version 3.13: Functions wrapped in `functools.partialmethod()` now return `True` if the wrapped function is a Python generator function.

inspect.isgenerator(object)

Return `True` if the object is a generator.

inspect.iscoroutinefunction(object)

Return `True` if the object is a coroutine function (a function defined with an `async def` syntax), a `functools.partial()` wrapping a coroutine function, or a sync function marked with `markcoroutinefunction()`. Added in version 3.5. Changed in version 3.8: Functions wrapped in `functools.partial()` now return `True` if the wrapped function is a coroutine function. Changed in version 3.12: Sync functions marked with `markcoroutinefunction()` now return `True`. Changed in version 3.13: Functions wrapped in `functools.partialmethod()` now r

inspect.markcoroutinefunction(func)

Decorator to mark a callable as a coroutine function if it would not otherwise be detected by `iscoroutinefunction()`. This may be of use for sync functions that return a coroutine, if the function is passed to an API that requires `iscoroutinefunction()`. When possible, using `async def` function is preferred. Also acceptable is calling the function and testing the return with `iscoroutine()`. Added in version 3.12.

inspect.iscoroutine(object)

Return `True` if the object is a coroutine created by an `async def` function. Added in version 3.5.

inspect.isawaitable(object)

Return True if the object can be used in a wait expression. Can also be used to distinguish generator-based coroutines from regular generators: `import types; def gen(): yield 1`...`>>> inspect.iscoroutine(gen_coro())` yields `True` if `gen_coro()` is a coroutine, `False` if it is a generator. Added in version 3.5.

inspect.isasyncgenfunction(object)

Return True if the object is an asynchronous generator function, for example: `>>> async def agen(): ... yield 1`...`>>> inspect.isasyncgenfunction(agen)` yields `True`. Added in version 3.6. Changed in version 3.8: Functions wrapped in `functools.partial()` now return `True` if the wrapped function is an asynchronous generator function. Changed in version 3.13: Functions wrapped in `functools.partialmethod()` now return `True` if the wrapped function is an asynchronous generator function.

inspect.isasyncgen(object)

Return True if the object is an asynchronous generator iterator created by an asynchronous generator function. Added in version 3.6.

inspect.istraceback(object)

Return True if the object is a traceback.

inspect.isframe(object)

Return True if the object is a frame.

inspect.iscode(object)

Return True if the object is a code.

inspect.isbuiltin(object)

Return True if the object is a built-in function or a bound built-in method.

inspect.ismethodwrapper(object)

Return True if the type of object is a `MethodWrapperType`. These are instances of `MethodWrapperType`, such as `__str__()`, `__eq__()` and `__repr__()`. Added in version 3.11.

inspect.isroutine(object)

Return True if the object is a user-defined or built-in function or method.

inspect.isabstract(object)

Return True if the object is an abstract base class.

inspect.ismethoddescriptor(object)

Return True if the object is a method descriptor, but not if `ismethod()`, `isclass()`, `isfunction()` or `isbuiltin()` are true. This, for example, is true of `int.__add__`. An object passing this test has a `__get__()` method, but not a `__set__()` method or a `__delete__()` method. Beyond that, the set of attributes varies. A `__name__` attribute is usually sensible, and `__doc__` often is. Methods implemented via descriptors that also pass one of the other tests return `False` from the `ismethoddescriptor()` test, simply because the other

inspect.isdatadescriptor(object)

Return True if the object is a data descriptor. Data descriptors have a `__set__` or a `__delete__` method. Examples are properties (defined in Python), `getsets`, and `members`. The latter two are defined in C and there are more specific tests available for those types, which is robust across Python implementations. Typically, data descriptors will also have `__name__` and `__doc__` attributes (properties, `getsets`, and `members` have both of these attributes), but this is not guaranteed.

inspect.isgetsetdescriptor(object)

Return `True` if the object is a `getset` descriptor. CPython implementation detail: `getsets` are attributes defined in extension modules via `PyGetSetDef` structures. For Python implementations without such types, this method will always return `False`.

`inspect.ismemberdescriptor(object)`

Return `True` if the object is a member descriptor. CPython implementation detail: Member descriptors are attributes defined in extension modules via `PyMemberDef` structures. For Python implementations without such types, this method will always return `False`.

Retrieving source code

`inspect.getdoc(object)`

Get the documentation string for an object, cleaned up with `cleandoc()`. If the documentation string for an object is not provided and the object is a class, a method, a property or a descriptor, retrieve the documentation string from the inheritance hierarchy. Return `None` if the documentation string is invalid or missing. Changed in version 3.5: Documentation strings are now inherited if not overridden.

`inspect.getcomments(object)`

Return in a single string any lines of comments immediately preceding the object's source code (for a class, function, or method), or at the top of the Python source file (if the object is a module). If the object's source code is unavailable, return `None`. This could happen if the object has been defined in C or the interactive shell.

`inspect.getfile(object)`

Return the name of the (text or binary) file in which an object was defined. This will fail with a `TypeError` if the object is a built-in module, class, or function.

`inspect.getmodule(object)`

Try to guess which module an object was defined in. Return `None` if the module cannot be determined.

`inspect.getsourcefile(object)`

Return the name of the Python source file in which an object was defined or `None` if no way can be identified to get the source. This will fail with a `TypeError` if the object is a built-in module, class, or function.

`inspect.getsourcelines(object)`

Return a list of source lines and starting line number for an object. The argument may be a module, class, method, function, traceback, frame, or code object. The source code is returned as a list of the lines corresponding to the object and the line number indicates where in the original source file the first line of code was found. An `OSError` is raised if the source code cannot be retrieved. A `TypeError` is raised if the object is a built-in module, class, or function. Changed in version 3.3: `OSError`

`inspect.getsource(object)`

Return the text of the source code for an object. The argument may be a module, class, method, function, traceback, frame, or code object. The source code is returned as a single string. An `OSError` is raised if the source code cannot be retrieved. A `TypeError` is raised if the object is a built-in module, class, or function. Changed in version 3.3: `OSError` is raised instead of `IOError`, now an alias of the former.

`inspect.cleandoc(doc)`

Clean up indentation from docstrings that are indented to line up with blocks of code. All leading whitespace is removed from the first line. Any leading whitespace that can be uniformly removed from the second line onwards is removed. Empty lines at the beginning and end are subsequently removed. Also, all tabs are expanded to spaces.

Introspecting callables with the Signature object

Added in version 3.3.

The `Signature` object represents the call signature of a callable object and its return annotation. To retrieve a `Signature` object, use the `signature()` function.

`inspect.signature(callable, *, follow_wrapped=True, globals=None, locals=None, eval_str=False, annotation_format=Format.VALUE)`

Return a `Signature` object for the given callable:

```
>>> from inspect import signature
>>> def foo(a, *, b: int, **kwargs): ...
>>> sig = signature(foo)
>>> str(sig)
'(a, *, b: int, **kwargs)'
>>> str(sig.parameters['b'])
'b: int'
>>> sig.parameters['b'].annotation
<class 'int'>
```

Accepts a wide range of Python callables, from plain functions and classes to `functools.partial()` objects. If some of the annotations are strings (e.g., because from `__future__` import `annotations` was used), `signature()` will attempt to automatically un-stringi

`class inspect.Signature(parameters=None, *, return_annotation=Signature.empty)`

`empty`

`parameters`

`return_annotation`

`bind(*args, **kwargs)`

`bind_partial(*args, **kwargs)`

`replace(*[, parameters][, return_annotation])`

`format(*, max_width=None, quote_annotation_strings=True)`

`classmethod from_callable(obj, *, follow_wrapped=True, globals=None, locals=None, eval_str=False)`

A `Signature` object represents the call signature of a function and its return annotation. For each parameter accepted by the function it stores a `Parameter` object in its `parameters` collection. The optional `parameters` argument is a sequence of `Parameter` objects, which is validated to check that there are no parameters with duplicate names, and that the parameters are in the right order, i.e. positional-only first, then positional-or-keyword, and that parameters with defaults follow parameters without default.

A special class-level marker to specify absence of a return annotation.

An ordered mapping of parameters' names to the corresponding `Parameter` objects. Parameters appear in strict definition order, including keyword-only parameters. Changed in version 3.7: Python only explicitly guaranteed that it preserved the declaration order of keyword-only parameters as of version 3.7, although in practice this order had always been preserved in Python 3.

The "return" annotation for the callable. If the callable has no "return" annotation, this attribute is set to `Signature.empty`.

Create a mapping from positional and keyword arguments to parameters.

Returns `BoundArguments` if `*args` and `**kwargs` match the signature, or raises a `TypeError`.

Works the same way as `Signature.bind()`, but allows the omission of some required arguments (mimics `functools.partial()` behavior.) Returns `BoundArguments`, or raises a `TypeError` if the passed arguments do not match the signature.

Create a new `Signature` instance based on the instance `replace()` was invoked on. It is possible to pass different `parameters` and/or `return_annotation` to override the corresponding properties of the base signature. To remove `return_annotation` from the copied `Signature`, pass in `Signature.empty`.

```
>>> def test(a, b): ...
>>> sig = sig
```

```
nature(test)>>>new_sig=sig.replace(return_annotation="new return anno")>>>str(new_sig)"(a, b) -> 'new return anno'"Signature objects are also supported by the generic function copy.repl
```

Create a string representation of the `Signature` object. If `max_width` is passed, the method will attempt to fit the signature into lines of at most `max_width` characters. If the signature is longer than `max_width`, all parameters will be on separate lines. If `quote_annotation_strings` is `False`, annotations in the signature are displayed without opening and closing quotation marks if they are strings. This is useful if the signature was created with the `STRING` format or if from `__future__` import `annotations` was used. Add

Return a `Signature` (or its subclass) object for a given callable `obj`. This method simplifies subclassing of `Signature`:
`class MySignature(Signature): pass`
`sig = MySignature.from_callable(sum)`
`assert isinstance(sig, MySignature)`
Its behavior is otherwise identical to that of `signature()`. Added in version 3.5. Changed in version 3.10: The `globals`, `locals`, and `eval_str` parameters were added.

`inspect.Parameter(name, kind, *, default=Parameter.empty, annotation=Parameter.empty)`

`empty`

`name`

`default`

`annotation`

`kind`

`kind.description`

`replace(*[, name][, kind][, default][, annotation])`

`Parameter` objects are immutable. Instead of modifying a `Parameter` object, you can use `Parameter.replace()` or `copy.replace()` to create a modified copy. Changed in version 3.5: `Parameter` objects are now picklable and hashable. `empty` A special class-level marker to specify absence of default values and annotations. `name` The name of the parameter as a string. The name must be a valid Python identifier. CPython implementation detail: CPython generates implicit parameter names of the form `0` on the code objects used to

A special class-level marker to specify absence of default values and annotations.

The name of the parameter as a string. The name must be a valid Python identifier. CPython implementation detail: CPython generates implicit parameter names of the form `0` on the code objects used to implement comprehensions and generator expressions. Changed in version 3.6: These parameter names are now exposed by this module as names like `implicit0`.

The default value for the parameter. If the parameter has no default value, this attribute is set to `Parameter.empty`.

The annotation for the parameter. If the parameter has no annotation, this attribute is set to `Parameter.empty`.

Describes how argument values are bound to the parameter. The possible values are accessible via `Parameter` (like `Parameter.KEYWORD_ONLY`), and support comparison and ordering, in the following order: `Name` `Meaning` `POSITIONAL_ONLY` `Value` must be supplied as a positional argument. Positional only parameters are those which appear before a `/` entry (if present) in a Python function definition. `POSITIONAL_OR_KEYWORD` `Value` may be supplied as either a keyword or positional argument (this is the standard binding behavior)

Describes an enum value of `Parameter.kind`. Added in version 3.8. Example: print all descriptions of arguments:

```
>>> def foo(a, b, *, c, d=10): ... pass
>>> sig = signature(foo)
>>> for param in sig.parameters.values(): ... print(param.kind,
description)
positional or keyword
positional or keyword
keyword-only
keyword-only
```

Create a new `Parameter` instance based on the instance `replaced` was invoked on. To override a `Parameter` attribute, pass the corresponding argument. To remove a default value or/and an annotation from

```
aParameter, passParameter.empty.>>>frominspectimportParameter>>>param=Parameter('foo',Parameter.KEYWORD_ONLY,default=42)>>>str(param)'foo=42'>>>str(param.replace())# Will create a shallow copy of
'param"foo=42">>>str(param.replace(default=Parameter.empty,annotation='spam'))"foo:
'spam'"Parameterobject
```

classinspect.BoundArguments

arguments

args

kwargs

signature

apply_defaults()

Result of a `Signature.bind()` or `Signature.bind_partial()` call. Holds the mapping of arguments to the function's parameters. `arguments` A mutable mapping of parameters' names to arguments' values. Contains only explicitly bound arguments. Changes in `arguments` will reflect in `args` and `kwargs`. Should be used in conjunction with `Signature.parameters` for any argument processing purposes. Note `Arguments` for which `Signature.bind()` or `Signature.bind_partial()` relied on a default value are skipped. However, if needed, use `BoundArguments.apply_defaults()` to add them. Changed in version 3.9: `arguments` is now of type `dict`. Formerly, it was of type `collections`.

A mutable mapping of parameters' names to arguments' values. Contains only explicitly bound arguments. Changes in `arguments` will reflect in `args` and `kwargs`. Should be used in conjunction with `Signature.parameters` for any argument processing purposes. Note `Arguments` for which `Signature.bind()` or `Signature.bind_partial()` relied on a default value are skipped. However, if needed, use `BoundArguments.apply_defaults()` to add them. Changed in version 3.9: `arguments` is now of type `dict`. Formerly, it was of type `collections`.

A tuple of positional arguments values. Dynamically computed from the `arguments` attribute.

A dict of keyword arguments values. Dynamically computed from the `arguments` attribute. Arguments that can be passed positionally are included in `args` instead.

A reference to the parent `Signature` object.

Set default values for missing arguments. For variable-positional arguments (`*args`) the default is an empty tuple. For variable-keyword arguments (`**kwargs`) the default is an empty dict.

```
>>>def foo(a,b='ham',*args):pass>>>ba=inspect.signature(foo).bind('spam')>>>ba.apply_defaults()>>>ba.arguments{'a': 'spam', 'b': 'ham', 'args': ()}
```

 Added in version 3.5.

See also

PEP 362- Function Signature Object.

The detailed specification, implementation details and examples.

Classes and functions

inspect.getclasstree(classes,unique=False)

Arrange the given list of classes into a hierarchy of nested lists. Where a nested list appears, it contains classes derived from the class whose entry immediately precedes the list. Each entry is a 2-tuple containing a class and a tuple of its base classes. If the `unique` argument is true, exactly one entry appears in the returned structure for each class in the given list. Otherwise, classes using multiple inheritance and their descendants will appear multiple times.

inspect.getfullargspec(func)

Get the names and default values of a Python function's parameters. A named tuple is returned: `FullArgSpec(args, varargs, varkw, defaults, kwonlyargs, kwonlydefaults, annotations)` `args` is a list of the positional parameter names. `varargs` is the name of the *parameter or `None` if arbitrary positional arguments are not accepted. `varkw` is the name of the **parameter or `None` if arbitrary keyword arguments are not accepted. `defaults` is an n-tuple of default argument values corresponding to the last `n` positional parameters, or

`inspect.getargvalues(frame)`

Get information about arguments passed into a particular frame. A named tuple `ArgInfo(args, varargs, keywords, locals)` is returned. `args` is a list of the argument names. `varargs` and `keywords` are the names of the * and ** arguments or `None`. `locals` is the locals dictionary of the given frame. Note This function was inadvertently marked as deprecated in Python 3.5.

`inspect.formatargvalues(args[, varargs, varkw, locals, formatarg, formatvarargs, formatvarkw, formatvalue])`

Format a pretty argument spec from the four values returned by `getargvalues()`. The `format*` arguments are the corresponding optional formatting functions that are called to turn names and values into strings. Note This function was inadvertently marked as deprecated in Python 3.5.

`inspect.getmro(cls)`

Return a tuple of class `cls`'s base classes, including `cls`, in method resolution order. No class appears more than once in this tuple. Note that the method resolution order depends on `cls`'s type. Unless a very peculiar user-defined metatype is in use, `cls` will be the first element of the tuple.

`inspect.getcallargs(func, /, *args, **kwds)`

Bind the `args` and `kwds` to the argument names of the Python function or method `func`, as if it was called with them. For bound methods, bind also the first argument (typically named `self`) to the associated instance. A dict is returned, mapping the argument names (including the names of the * and ** arguments, if any) to their values from `args` and `kwds`. In case of invoking `func` incorrectly, i.e. whenever `func(*args, **kwds)` would raise an exception because of incompatible signature, an exception of the same type and

`inspect.getclosurevars(func)`

Get the mapping of external name references in a Python function or method `func` to their current values. A named tuple `ClosureVars(nonlocals, globals, builtins, unbound)` is returned. `nonlocals` maps referenced names to lexical closure variables, `globals` to the function's module globals and `builtins` to the builtins visible from the function body. `unbound` is the set of names referenced in the function that could not be resolved at all given the current module globals and builtins. `TypeError` is raised if `func` is not a

`inspect.unwrap(func, *, stop=None)`

Get the object wrapped by `func`. It follows the chain of `__wrapped__` attributes returning the last object in the chain. `stop` is an optional callback accepting an object in the wrapper chain as its sole argument that allows the unwrapping to be terminated early if the callback returns a true value. If the callback never returns a true value, the last object in the chain is returned as usual. For example, `signature()` uses this to stop unwrapping if any object in the chain has a `__signature__` attribute defined.

`inspect.get_annotations(obj, *, globals=None, locals=None, eval_str=False, format=annotationlib.Format.VALUE)`

Compute the annotations dict for an object. This is an alias for `annotationlib.get_annotations()`; see the documentation of that function for more information. Caution This function may execute arbitrary code contained in annotations. See Security implications of introspecting annotations for more information. Added in version 3.10. Changed in version 3.14: This function is now an alias for `annotationlib.get_annotations()`. Calling it as `inspect.get_annotations` will continue to work.

The interpreter stack

Some of the following functions return `FrameInfo` objects. For backwards compatibility these objects allow tuple-like operations on all attributes except `positions`. This behavior is considered deprecated and may be removed in the future.

`classinspect.FrameInfo`

frame

filename

lineno

function

code_context

index

positions

frame The frame object that the record corresponds to.
filename The file name associated with the code being executed by the frame this record corresponds to.
lineno The line number of the current line associated with the code being executed by the frame this record corresponds to.
function The function name that is being executed by the frame this record corresponds to.
code_context A list of lines of context from the source code that's being executed by the frame this record corresponds to.
index The index

The frame object that the record corresponds to.

The file name associated with the code being executed by the frame this record corresponds to.

The line number of the current line associated with the code being executed by the frame this record corresponds to.

The function name that is being executed by the frame this record corresponds to.

A list of lines of context from the source code that's being executed by the frame this record corresponds to.

The index of the current line being executed in the `code_context` list.

Adis.Positions object containing the start line number, end line number, start column offset, and end column offset associated with the instruction being executed by the frame this record corresponds to.

`classinspect.Traceback`

filename

lineno

function

code_context

index

positions

filename The file name associated with the code being executed by the frame this traceback corresponds to.
lineno The line number of the current line associated with the code being executed by the frame this traceback corresponds to.
function The function name that is being executed by the frame this traceback corresponds to.
code_context A list of lines of context from the source code that's being executed by the frame this traceback corresponds to.
index The index of the current line being executed in

The file name associated with the code being executed by the frame this traceback corresponds to.

The line number of the current line associated with the code being executed by the frame this traceback corresponds to.

The function name that is being executed by the frame this traceback corresponds to.

A list of lines of context from the source code that's being executed by the frame this traceback corresponds to.

The index of the current line being executed in the `code_contextlist`.

An `Index` object containing the start line number, end line number, start column offset, and end column offset associated with the instruction being executed by the frame this traceback corresponds to.

■ **Nota:** Note that keeping references to frame objects, as found in the first element of the frame records these functions return, can cause your program to create reference cycles. Once a reference cycle has been created, the lifespan of all objects which can be accessed from the objects which form the cycle can become much longer even if Python's optional cycle detector is enabled. If such cycles must be created, it is important to ensure they are explicitly broken to avoid the delayed destruction of objects and increased memory consumption which occurs. Though the cycle detector will catch these, destruction

The optional `context` argument supported by most of these functions specifies the number of lines of context to return, which are centered around the current line.

`inspect.getframeinfo(frame, context=1)`

Get information about a frame or traceback object. A `TracebackObject` is returned. Changed in version 3.11: A `TracebackObject` is returned instead of a named tuple.

`inspect.getouterframes(frame, context=1)`

Get a list of `FrameInfo` objects for a frame and all outer frames. These frames represent the calls that lead to the creation of frame. The first entry in the returned list represents frame; the last entry represents the outermost call on frame's stack. Changed in version 3.5: A list of named tuples `FrameInfo(frame, filename, lineno, function, code_context, index)` is returned. Changed in version 3.11: A list of `FrameInfo` objects is returned.

`inspect.getinnerframes(traceback, context=1)`

Get a list of `FrameInfo` objects for a traceback's frame and all inner frames. These frames represent calls made as a consequence of frame. The first entry in the list represents traceback; the last entry represents where the exception was raised. Changed in version 3.5: A list of named tuples `FrameInfo(frame, filename, lineno, function, code_context, index)` is returned. Changed in version 3.11: A list of `FrameInfo` objects is returned.

`inspect.currentframe()`

Return the frame object for the caller's stack frame. CPython implementation detail: This function relies on Python stack frame support in the interpreter, which isn't guaranteed to exist in all implementations of Python. If running in an implementation without Python stack frame support this function returns `None`.

`inspect.stack(context=1)`

Return a list of `FrameInfo` objects for the caller's stack. The first entry in the returned list represents the caller; the last entry represents the outermost call on the stack. Changed in version 3.5: A list of named tuples `FrameInfo(frame, filename, lineno, function, code_context, index)` is returned. Changed in version 3.11: A list of `FrameInfo` objects is returned.

`inspect.trace(context=1)`

Return a list of `FrameInfo` objects for the stack between the current frame and the frame in which an exception currently being handled was raised in. The first entry in the list represents the caller; the last entry represents where the exception was raised. Changed in version 3.5: A list of named

`tuplesFrameInfo(frame,filename,lineno,function,code_context,index)` is returned.Changed in version 3.11:A list of `FrameInfo` objects is returned.

Fetching attributes statically

Both `getattr()` and `hasattr()` can trigger code execution when fetching or checking for the existence of attributes. Descriptors, like properties, will be invoked and `__getattr__()` and `__getattribute__()` may be called.

For cases where you want passive introspection, like documentation tools, this can be inconvenient. `getattr_static()` has the same signature as `getattr()` but avoids executing code when it fetches attributes.

`inspect.getattr_static(obj,attr,default=None)`

Retrieve attributes without triggering dynamic lookup via the descriptor protocol, `__getattr__()` or `__getattribute__()`. Note: this function may not be able to retrieve all attributes that `getattr` can fetch (like dynamically created attributes) and may find attributes that `getattr` can't (like descriptors that raise `AttributeError`). It can also return descriptors objects instead of instance members. If the instance `__dict__` is shadowed by another member (for example a property) then this function will be

`getattr_static()` does not resolve descriptors, for example slot descriptors or getset descriptors on objects implemented in C. The descriptor object is returned instead of the underlying attribute.

You can handle these with code like the following. Note that for arbitrary getset descriptors invoking these may trigger code execution:

```
# example code for resolving the builtin descriptor types
class _foo:
    __slots__ = ['foo']

slot_descriptor = type(_foo.foo)
getset_descriptor = type(type(open(__file__)).name)
wrapper_descriptor = type(str.__dict__['__add__'])
descriptor_types = (slot_descriptor, getset_descriptor, wrapper_descriptor)

result = getattr_static(some_object, 'foo')
if type(result) in descriptor_types:
    try:
        result = result.__get__()
    except AttributeError:
        # descriptors can raise AttributeError to
        # indicate there is no underlying value
        # in which case the descriptor itself will
        # have to do
        pass
```

Current State of Generators, Coroutines, and Asynchronous Generators

When implementing coroutine schedulers and for other advanced uses of generators, it is useful to determine whether a generator is currently executing, is waiting to start or resume or execution, or has already terminated. `getgeneratorstate()` allows the current state of a generator to be determined easily.

`inspect.getgeneratorstate(generator)`

Get current state of a generator-iterator. Possible states are: `GEN_CREATED`: Waiting to start execution. `GEN_RUNNING`: Currently being executed by the interpreter. `GEN_SUSPENDED`: Currently suspended at a yield expression. `GEN_CLOSED`: Execution has completed. Added in version 3.2.

`inspect.getcoroutinestate(coroutine)`

Get current state of a coroutine object. The function is intended to be used with coroutine objects created by `async def` functions, but will accept any coroutine-like object that has `cr_running` and `cr_frame` attributes. Possible states are: `CORO_CREATED`: Waiting to start execution. `CORO_RUNNING`: Currently being executed by the interpreter. `CORO_SUSPENDED`: Currently suspended at an `await` expression. `CORO_CLOSED`: Execution has completed. Added in version 3.5.

`inspect.getasyncgenstate(agen)`

Get current state of an asynchronous generator object. The function is intended to be used with asynchronous iterator objects created by `async def` functions which use the `yield` statement, but will accept any asynchronous generator-like object that has `ag_running` and `ag_frame` attributes. Possible states are: `AGEN_CREATED`: Waiting to start execution. `AGEN_RUNNING`: Currently being executed by the interpreter. `AGEN_SUSPENDED`: Currently suspended at a `yield` expression. `AGEN_CLOSED`: Execution has completed. Added in version 3.5.

The current internal state of the generator can also be queried. This is mostly useful for testing purposes, to ensure that internal state is being updated as expected:

`inspect.getgeneratorlocals(generator)`

Get the mapping of live local variables in generator to their current values. A dictionary is returned that maps from variable names to values. This is the equivalent of calling `locals()` in the body of the generator, and all the same caveats apply. If generator is a generator with no currently associated frame, then an empty dictionary is returned. `TypeError` is raised if generator is not a Python generator object. CPython implementation detail: This function relies on the generator exposing a Python stack frame.

`inspect.getcoroutinelocals(coroutine)`

This function is analogous to `getgeneratorlocals()`, but works for coroutine objects created by `async def` functions. Added in version 3.5.

`inspect.getasyncgenlocals(agen)`

This function is analogous to `getgeneratorlocals()`, but works for asynchronous generator objects created by `async def` functions which use the `yield` statement. Added in version 3.12.

Code Objects Bit Flags

Python code objects have a `co_flags` attribute, which is a bitmap of the following flags:

`inspect.CO_OPTIMIZED`

The code object is optimized, using fast locals.

`inspect.CO_NEWLOCALS`

If set, a new dict will be created for the frame's `f_locals` when the code object is executed.

`inspect.CO_VARARGS`

The code object has a variable positional parameter (*args-like).

`inspect.CO_VARKEYWORDS`

The code object has a variable keyword parameter (**kwargs-like).

`inspect.CO_NESTED`

The flag is set when the code object is a nested function.

`inspect.CO_GENERATOR`

The flag is set when the code object is a generator function, i.e. a generator object is returned when the code object is executed.

inspect.CO_COROUTINE

The flag is set when the code object is a coroutine function. When the code object is executed it returns a coroutine object. SeePEP 492for more details.Added in version 3.5.

inspect.CO_ITERABLE_COROUTINE

The flag is used to transform generators into generator-based coroutines. Generator objects with this flag can be used inawaitexpression, and canyieldfromcoroutine objects. SeePEP 492for more details.Added in version 3.5.

inspect.CO_ASYNC_GENERATOR

The flag is set when the code object is an asynchronous generator function. When the code object is executed it returns an asynchronous generator object. SeePEP 525for more details.Added in version 3.6.

inspect.CO_HAS_DOCSTRING

The flag is set when there is a docstring for the code object in the source code. If set, it will be the first item inco_consts.Added in version 3.14.

inspect.CO_METHOD

The flag is set when the code object is a function defined in class scope.Added in version 3.14.

■ Nota: NoteThe flags are specific to CPython, and may not be defined in other Python implementations. Furthermore, the flags are an implementation detail, and can be removed or deprecated in future Python releases. It's recommended to use public APIs from theinspectmodule for any introspection needs.

Buffer flags

classinspect.BufferFlags

SIMPLE

WRITABLE

FORMAT

ND

STRIDES

C_CONTIGUOUS

F_CONTIGUOUS

ANY_CONTIGUOUS

INDIRECT

CONTIG

CONTIG_RO

STRIDED

STRIDED_RO

RECORDS

RECORDS_RO

FULL

FULL_RO

READ

WRITE

This is an `enum.IntFlag` that represents the flags that can be passed to the `__buffer__()` method of objects implementing the `buffer` protocol. The meaning of the flags is explained at `Buffer request types`. `SIMPLEWRITABLE` `FORMATNDSTRIDESC` `_CONTIGUOUSF` `_CONTIGUOUSANY` `_CONTIGUOUSINDIRECT` `CONTIG` `CONTIG_RO` `STRIDED` `STRIDED_RO` `RECORDS` `RECORDS_RO` `FULL` `FULL_RO` `READWRITE` Added in version 3.12.

Command-line interface

The `inspect` module also provides a basic introspection capability from the command line.

By default, accepts the name of a module and prints the source of that module. A class or function within the module can be printed instead by appending a colon and the qualified name of the target object.

--details

Print information about the specified object rather than the source code

annotationlib— Functionality for introspecting annotations

Added in version 3.14.

Source code: `Lib/annotationlib.py`

The `annotationlib` module provides tools for introspecting annotations on modules, classes, and functions.

Annotations are lazily evaluated and often contain forward references to objects that are not yet defined when the annotation is created. This module provides a set of low-level tools that can be used to retrieve annotations in a reliable way, even in the presence of forward references and other edge cases.

This module supports retrieving annotations in three main formats (see `Format`), each of which works best for different use cases:

- `VALUE` evaluates the annotations and returns their value. This is most straightforward to work with, but it may raise errors, for example if the annotations contain references to undefined names.
- `FORWARDREF` returns `ForwardRef` objects for annotations that cannot be resolved, allowing you to inspect the annotations without evaluating them. This is useful when you need to work with annotations that may contain unresolved forward references.
- `STRING` returns the annotations as a string, similar to how it would appear in the source file. This is useful for documentation generators that want to display annotations in a readable way.

The `get_annotations()` function is the main entry point for retrieving annotations. Given a function, class, or module, it returns an annotations dictionary in the requested format. This module also provides functionality for working directly with the `annotate` function that is used to evaluate annotations, such as `get_annotate_from_class_namespace()` and `call_annotate_function()`, as well as the `call_evaluate_function()` function for working with evaluate functions.

Caution

Most functionality in this module can execute arbitrary code; see the security section for more information.

See also

PEP 649 proposed the current model for how annotations work in Python.

PEP 749 expanded on various aspects of PEP 649 and introduced the `annotationlib` module.

Annotations Best Practices provides best practices for working with annotations.

`typing-extensions` provides a backport of `get_annotations()` that works on earlier versions of Python.

Annotation semantics

The way annotations are evaluated has changed over the history of Python 3, and currently still depends on a future import. There have been execution models for annotations:

- Stock semantics (default in Python 3.0 through 3.13; see PEP 3107 and PEP 526): Annotations are evaluated eagerly, as they are encountered in the source code.
- Stringified annotations (used with `from __future__ import annotations` in Python 3.7 and newer; see PEP 563): Annotations are stored as strings only.
- Deferred evaluation (default in Python 3.14 and newer; see PEP 649 and PEP 749): Annotations are evaluated lazily, only when they are accessed.

As an example, consider the following program:

```
def func(a: Cls) -> None:
    print(a)

class Cls: pass

print(func.__annotations__)
```

This will behave as follows:

- Under stock semantics (Python 3.13 and earlier), it will throw a `NameError` at the line where `func` is defined, because `Cls` is an undefined name at that point.
- Under stringified annotations (if `from __future__ import annotations` is used), it will print `{'a': 'Cls', 'return': 'None'}`.
- Under deferred evaluation (Python 3.14 and later), it will print `{'a': <class 'Cls'>, 'return': None}`.

Stock semantics were used when function annotations were first introduced in Python 3.0 (by PEP 3107) because this was the simplest, most obvious way to implement annotations. The same execution model was used when variable annotations were introduced in Python 3.6 (by PEP 526). However, stock semantics caused problems when using annotations as type hints, such as a need to refer to names that are not yet defined when the annotation is encountered. In addition, there were performance problems with executing annotations at module import time. Therefore, in Python 3.7, PEP 563 introduced the ability to store annotations as strings using `from __future__ import annotation` syntax. The plan at the time was to eventually make this behavior the default, but a problem appeared: stringified annotations are more difficult to process for those who introspect annotations at runtime. An alternative proposal, PEP 649, introduced the third execution model, deferred evaluation, and was implemented in Python 3.14. Stringified annotations are still used if `from __future__ import annotations` is present, but this behavior will eventually be removed.

Classes

class annotationlib.Format

VALUE=1

VALUE_WITH_FAKE_GLOBALS=2

FORWARDREF=3

STRING=4

An `IntEnum` describing the formats in which annotations can be returned. Members of the enum, or their equivalent integer values, can be passed to `get_annotations()` and other functions in this module, as well as to `__annotate__` functions. **VALUE=1** Values are the result of evaluating the annotation expressions. **VALUE_WITH_FAKE_GLOBALS=2** Special value used to signal that an `annotate` function is being evaluated in a special environment with fake globals. When passed this value, `annotate` functions should either

Values are the result of evaluating the annotation expressions.

Special value used to signal that an `annotate` function is being evaluated in a special environment with fake globals. When passed this value, `annotate` functions should either return the same value as for the `Format.VALUE` format, or raise `NotImplementedError` to signal that they do not support execution in this environment. This format is only used internally and should not be passed to the functions in this module.

Values are real annotation values (as per `Format.VALUE` format) for defined values, and `ForwardRef` proxies for undefined values. Real objects may contain references to `ForwardRef` proxy objects.

Values are the text string of the annotation as it appears in the source code, up to modifications including, but not restricted to, whitespace normalizations and constant values optimizations. The exact values of these strings

may change in future versions of Python.

class `annotationlib.ForwardRef`

`__forward_arg__`

`evaluate(*,owner=None,globals=None,locals=None,type_params=None,format=Format.VALUE)`

A proxy object for forward references in annotations. Instances of this class are returned when the `FORWARDREF` format is used and annotations contain a name that cannot be resolved. This can happen when a forward reference is used in an annotation, such as when a class is referenced before it is defined. `__forward_arg__` A string containing the code that was evaluated to produce the `ForwardRef`. The string may not be exactly equivalent to the original source. `evaluate(*,owner=None,globals=None,locals=None`

A string containing the code that was evaluated to produce the `ForwardRef`. The string may not be exactly equivalent to the original source.

Evaluate the forward reference, returning its value. If the `format` argument is `VALUE` (the default), this method may throw an exception, such as `NameError`, if the forward reference refers to a name that cannot be resolved. The arguments to this method can be used to provide bindings for names that would otherwise be undefined. If the `format` argument is `FORWARDREF`, the method will never throw an exception, but may return a `ForwardRef` instance. For example, if the forward reference object contains the code `is`

Functions

`annotationlib.annotations_to_string(annotations)`

Convert an annotations dict containing runtime values to a dict containing only strings. If the values are not already strings, they are converted using `type_repr()`. This is meant as a helper for user-provided annotate functions that support the `STRING` format but do not have access to the code creating the annotations. For example, this is used to implement the `STRING` for `typing.TypedDict` classes created through the functional syntax: `>>> from typing import TypedDict >>> Movie = TypedDict("movie", {"name": str, "ye`

`annotationlib.call_annotate_function(annotate,format,*,owner=None)`

Call the `annotate` function `annotate` with the given `format`, a member of the `Format` enum, and return the annotations dictionary produced by the function. This helper function is required because `annotate` functions generated by the compiler for functions, classes, and modules only support the `VALUE` format when called directly. To support other formats, this function calls the `annotate` function in a special environment that allows it to produce annotations in the other formats. This is a useful building bloc

`annotationlib.call_evaluate_function(evaluate,format,*,owner=None)`

Call the `evaluate` function `evaluate` with the given `format`, a member of the `Format` enum, and return the value produced by the function. This is similar to `call_annotate_function()`, but the latter always returns a dictionary mapping strings to annotations, while this function returns a single value. This is intended for use with the `evaluate` functions generated for lazily evaluated elements related to type aliases and type parameters: `typing.TypeAliasType.evaluate_value()`, the value of type alias `typing.T`

`annotationlib.get_annotate_from_class_namespace(namespace)`

Retrieve the `annotate` function from a class namespace dictionary `namespace`. Return `None` if the namespace does not contain an `annotate` function. This is primarily useful before the class has been fully created (e.g., in a metaclass); after the class exists, the `annotate` function can be retrieved with `cls.__annotate__`. See below for an example using this function in a metaclass. Added in version 3.14.

`annotationlib.get_annotations(obj,*,globals=None,locals=None,eval_str=False,format=Format.VALUE)`

Compute the annotations dict for an object. obj may be a callable, class, module, or other object with `__annotate__` or `__annotations__` attributes. Passing any other object raises `TypeError`. The `format` parameter controls the format in which annotations are returned, and must be a member of the `FormatEnum` or its integer equivalent. The different formats work as follows: `VALUE:object.__annotations__` is tried first; if that does not exist, the `object.__annotate__` function is called if it exists. `FORWARDREF`: If objec

annotationlib.type_repr(value)

Convert an arbitrary Python value to a format suitable for use by the `STRING` format. This calls `repr()` for most objects, but has special handling for some objects, such as type objects. This is meant as a helper for user-provided `annotate` functions that support the `STRING` format but do not have access to the code creating the annotations. It can also be used to provide a user-friendly string representation for other objects that contain values that are commonly encountered in annotations. Added in versi

Recipes

Using annotations in a metaclass

A metaclass may want to inspect or even modify the annotations in a class body during class creation. Doing so requires retrieving annotations from the class namespace dictionary. For classes created with `from __future__ import annotations`, the annotations will be in the `__annotations__` key of the dictionary. For other classes with annotations, `get_annotate_from_class_namespace()` can be used to get the `annotate` function, and `call_annotate_function()` can be used to call it and retrieve the annotations. Using the `FORWARDREF` format will usually be best, because this allows the annotations to refer to names that cannot yet be resolved when the class is created.

To modify the annotations, it is best to create a wrapper `annotate` function that calls the original `annotate` function, makes any necessary adjustments, and returns the result.

Below is an example of a metaclass that filters out all `typing.ClassVar` annotations from the class and puts them in a separate attribute:

```
import annotationlib
import typing

class ClassVarSeparator(type):
    def __new__(mcls, name, bases, ns):
        if "__annotations__" in ns: # from __future__ import annotations
            annotations = ns["__annotations__"]
            classvar_keys = {
                key for key, value in annotations.items()
                # Use string comparison for simplicity; a more robust solution
                # could use annotationlib.ForwardRef.evaluate
                if value.startswith("ClassVar")}
            classvars = {key: annotations[key] for key in classvar_keys}
            ns["__annotations__"] = {
                key: value for key, value in annotations.items()
                if key not in classvar_keys}
            wrapped_annotate = None
        elif annotate := annotationlib.get_annotate_from_class_namespace(ns):
            annotations = annotationlib.call_annotate_function(
                annotate, format=annotationlib.Format.FORWARDREF)
            classvar_keys = {
                key for key, value in annotations.items()
                if typing.get_origin(value) is typing.ClassVar}
        }
```

```

classvars = {key: annotations[key] for key in classvar_keys}

def wrapped_annotate(format):
    annos = annotationlib.call_annotate_function(annotate, format, owner=typ)
    return {key: value for key, value in annos.items() if key not in classvar_keys}

else: # no annotations
    classvars = {}
    wrapped_annotate = None
typ = super().__new__(mcls, name, bases, ns)

if wrapped_annotate is not None:
    # Wrap the original __annotate__ with a wrapper that removes ClassVars
    typ.__annotate__ = wrapped_annotate
typ.classvars = classvars # Store the ClassVars in a separate attribute
return typ

```

Limitations of theSTRINGformat

TheSTRINGformat is meant to approximate the source code of the annotation, but the implementation strategy used means that it is not always possible to recover the exact source code.

First, the stringifier of course cannot recover any information that is not present in the compiled code, including comments, whitespace, parenthesization, and operations that get simplified by the compiler.

Second, the stringifier can intercept almost all operations that involve names looked up in some scope, but it cannot intercept operations that operate fully on constants. As a corollary, this also means it is not safe to request theSTRINGformat on untrusted code: Python is powerful enough that it is possible to achieve arbitrary code execution even with no access to any globals or builtins. For example:

```

>>> def f(x: (1).__class__.__base__.__subclasses__)[-1].__init__.__builtins__["print"]("Hello world")): pass
...
>>> annotationlib.get_annotations(f, format=annotationlib.Format.STRING)
Hello world
{'x': 'None'}

```

■ **Nota:** NoteThis particular example works as of the time of writing, but it relies on implementation details and is not guaranteed to work in the future.

Among the different kinds of expressions that exist in Python, as represented by theastmodule, some expressions are supported, meaning that theSTRINGformat can generally recover the original source code; others are unsupported, meaning that they may result in incorrect output or an error.

The following are supported (sometimes with caveats):

- ast.BinOp
- ast.UnaryOpast.Invert(~),ast.UAdd(+), andast.USub(-) are supportedast.Not(not) is not supported
- ast.Dict(except when using**unpacking)
- ast.Set
- ast.Compareast.Eqandast.NotEqare supportedast.Lt,ast.LtE,ast.Gt, andast.GtEare supported, but the operand may be flippedast.Is,ast.IsNot,ast.In, andast.NotInare not supported
- ast.Call(except when using**unpacking)
- ast.Constant(though not the exact representation of the constant; for example, escape sequences in strings are lost; hexadecimal numbers are converted to decimal)
- ast.Attribute(assuming the value is not a constant)

- `ast.Subscript`(assuming the value is not a constant)
- `ast.Starred`(*unpacking)
- `ast.Name`
- `ast.List`
- `ast.Tuple`
- `ast.Slice`

The following are unsupported, but throw an informative error when encountered by the stringifier:

- `ast.FormattedValue`(f-strings; error is not detected if conversion specifiers like `!rare` are used)
- `ast.JoinedStr`(f-strings)

The following are unsupported and result in incorrect output:

- `ast.BoolOp`(`and`/`or`)
- `ast.IfExp`
- `ast.Lambda`
- `ast.ListComp`
- `ast.SetComp`
- `ast.DictComp`
- `ast.GeneratorExp`

The following are disallowed in annotation scopes and therefore not relevant:

- `ast.NamedExpr`(`:=`)
- `ast.Await`
- `ast.Yield`
- `ast.YieldFrom`

Limitations of the `FORWARDREF` format

The `FORWARDREF` format aims to produce real values as much as possible, with anything that cannot be resolved replaced with `ForwardRef` objects. It is affected by broadly the same Limitations as the `STRING` format: annotations that perform operations on literals or that use unsupported expression types may raise exceptions when evaluated using the `FORWARDREF` format.

Below are a few examples of the behavior with unsupported expressions:

```
>>> from annotationlib import get_annotations, Format
>>> def zerodiv(x: 1 / 0): ...
>>> get_annotations(zerodiv, format=Format.STRING)
Traceback (most recent call last):
...
ZeroDivisionError: division by zero
>>> get_annotations(zerodiv, format=Format.FORWARDREF)
Traceback (most recent call last):
...
ZeroDivisionError: division by zero
>>> def ifexp(x: 1 if y else 0): ...
>>> get_annotations(ifexp, format=Format.STRING)
{'x': '1'}
```

Security implications of introspecting annotations

Much of the functionality in this module involves executing code related to annotations, which can then do arbitrary things. For example, `get_annotations()` may call an arbitrary `annotate` function, and `ForwardRef.evaluate()` may call `eval()` on an arbitrary string. Code contained in an annotation might make arbitrary system calls, enter an infinite loop, or perform any other operation. This is also true for any access of the `__annotations__` attribute, and for various functions in the `typing` module that work with annotations, such as `astyping.get_type_hints()`.

Any security issue arising from this also applies immediately after importing code that may contain untrusted annotations: importing code can always cause arbitrary operations to be performed. However, it is unsafe to accept strings or other input from an untrusted source and pass them to any of the APIs for introspecting annotations, for example by editing an `__annotations__` dictionary or directly creating a `ForwardRef` object.

site— Site-specific configuration hook

Source code: `Lib/site.py`

This module is automatically imported during initialization. The automatic import can be suppressed using the interpreter's `-S` option.

Importing this module normally appends site-specific paths to the module search path and adds callables, including `help()` to the built-in namespace. However, Python startup option `-S` blocks this and this module can be safely imported with no automatic modifications to the module search path or additions to the builtins. To explicitly trigger the usual site-specific additions, call `main()` function.

Changed in version 3.3: Importing the module used to trigger paths manipulation even when using `-S`.

It starts by constructing up to four directories from a head and a tail part. For the head part, it uses `sys.prefix` and `sys.exec_prefix`; empty heads are skipped. For the tail part, it uses the empty string and then `lib/site-packages` (on Windows) or `lib/pythonX.Y/site-packages` (on Unix and macOS). (The optional suffix “t” indicates the free-threaded build, and is appended if “t” is present in `sys.abiflags` constant.) For each of the distinct head-tail combinations, it sees if it refers to an existing directory, and if so, adds it to `sys.path` and also inspects the newly added path for configuration files.

Changed in version 3.5: Support for the “site-python” directory has been removed.

Changed in version 3.13: On Unix, Free threading Python installations are identified by the “t” suffix in the version-specific directory name, such as `lib/python3.13t/`.

Changed in version 3.14: `site` is no longer responsible for updating `sys.prefix` and `sys.exec_prefix` on Virtual Environments. This is now done during the path initialization. As a result, under Virtual Environments, `sys.prefix` and `sys.exec_prefix` no longer depend on the site initialization, and are therefore unaffected by `-S`.

When running under a virtual environment, the `pyvenv.cfg` file in `sys.prefix` is checked for site-specific configurations. If the `include-system-site-packages` key exists and is set to `true` (case-insensitive), the system-level prefixes will be searched for site-packages, otherwise they won't.

A path configuration file is a file whose name has the form `name.pth` and exists in one of the four directories mentioned above; its contents are additional items (one per line) to be added to `sys.path`. Non-existing items are never added to `sys.path`, and no check is made that the item refers to a directory rather than a file. No item is added to `sys.path` more than once. Blank lines and lines beginning with `#` are skipped. Lines starting with `import` (followed by space or tab) are executed.

■ **Note:** An executable line in a `.pth` file is run at every Python startup, regardless of whether a particular module is actually going to be used. Its impact should thus be kept to a minimum. The primary intended purpose of executable lines is to make the corresponding module(s) importable (load 3rd-party import hooks, adjust `PATHEXT` etc). Any other initialization is supposed to be done upon a module's actual import, if and when it happens. Limiting a code chunk to a single line is a deliberate measure to discourage putting anything more complex here.

Changed in version 3.13: The `.pth` files are now decoded by UTF-8 at first and then by the locale encoding if it fails.

For example, suppose `sys.prefix` and `sys.exec_prefix` are set to `/usr/local`. The Python X.Y library is then installed in `/usr/local/lib/pythonX.Y`. Suppose this has a subdirectory `/usr/local/lib/pythonX.Y/site-packages` with three subdirectories, `foo`, `bar` and `spam`, and two path configuration files, `foo.pth` and `bar.pth`. Assume `foo.pth` contains the following:

```
# foo package configuration
```

```
foo
bar
```

bletch

and bar.pth contains:

```
# bar package configuration
```

bar

Then the following version-specific directories are added to `sys.path`, in this order:

```
/usr/local/lib/pythonX.Y/site-packages/bar  
/usr/local/lib/pythonX.Y/site-packages/foo
```

Note that `bletch` is omitted because it doesn't exist; `thebar` directory precedes `thefoo` directory because `bar.pth` comes alphabetically before `foo.pth`; and `spam` is omitted because it is not mentioned in either path configuration file.

sitecustomize

After these path manipulations, an attempt is made to import a module named `sitecustomize`, which can perform arbitrary site-specific customizations. It is typically created by a system administrator in the `site-packages` directory. If this import fails with an `ImportError` or its subclass exception, and the exception's `name` attribute equals `'sitecustomize'`, it is silently ignored. If Python is started without output streams available, as with `pythonw.exe` on Windows (which is used by default to start IDLE), attempted output from `sitecustomize` is ignored. Any other exception causes a silent and perhaps mysterious failure of the process.

usercustomize

After this, an attempt is made to import a module named `usercustomize`, which can perform arbitrary user-specific customizations, if `ENABLE_USER_SITE` is true. This file is intended to be created in the user `site-packages` directory (see below), which is part of `sys.path` unless disabled by `-s`. If this import fails with an `ImportError` or its subclass exception, and the exception's `name` attribute equals `'usercustomize'`, it is silently ignored.

Note that for some non-Unix systems, `sys.prefix` and `sys.exec_prefix` are empty, and the path manipulations are skipped; however the import of `sitecustomize` and `usercustomize` is still attempted.

Readline configuration

On systems that support readline, this module will also import and configure `rlcompleter` module, if Python is started in interactive mode and without the `-S` option. The default behavior is to enable tab completion and to use `~/.python_history` as the history save file. To disable it, delete (or override) the `sys.__interactivehook__` attribute in your `sitecustomize` or `usercustomize` module or your `PYTHONSTARTUP` file.

Changed in version 3.4: Activation of `rlcompleter` and history was made automatic.

Module contents

`site.PREFIXES`

A list of prefixes for site-packages directories.

`site.ENABLE_USER_SITE`

Flag showing the status of the user site-packages directory. True means that it is enabled and was added to `sys.path`. False means that it was disabled by user request (with `-s` or `PYTHONNOUSERSITE`). None means it was disabled for security reasons (mismatch between user or group id and effective id) or by an administrator.

site.USER_SITE

Path to the user site-packages for the running Python. Can be None if `getusersitepackages()` hasn't been called yet. Default value is `~/.local/lib/pythonX.Y[t]/site-packages` for UNIX and non-framework macOS builds, `~/Library/Python/X.Y/lib/python/site-packages` for macOS framework builds, and `%APPDATA%\Python\PythonXY\site-packages` on Windows. The optional "t" indicates the free-threaded build. This directory is a site directory, which means that `.pth` files in it will be processed.

site.USER_BASE

Path to the base directory for the user site-packages. Can be None if `getuserbase()` hasn't been called yet. Default value is `~/.local` for UNIX and macOS non-framework builds, `~/Library/Python/X.Y` for macOS framework builds, and `%APPDATA%\Python` for Windows. This value is used to compute the installation directories for scripts, data files, Python modules, etc. for the user installation scheme. See also `PYTHONUSERBASE`.

site.main()

Adds all the standard site-specific directories to the module search path. This function is called automatically when this module is imported, unless the Python interpreter was started with the `-S` flag. Changed in version 3.3: This function used to be called unconditionally.

site.addsitedir(sitedir, known_paths=None)

Add a directory to `sys.path` and process its `.pth` files. Typically used in `sitecustomize` or `usercustomize` (see above).

site.getsitepackages()

Return a list containing all global site-packages directories. Added in version 3.2.

site.getuserbase()

Return the path of the user base directory, `USER_BASE`. If it is not initialized yet, this function will also set it, respecting `PYTHONUSERBASE`. Added in version 3.2.

site.getusersitepackages()

Return the path of the user-specific site-packages directory, `USER_SITE`. If it is not initialized yet, this function will also set it, respecting `USER_BASE`. To determine if the user-specific site-packages was added to `sys.path`, `ENABLE_USER_SITE` should be used. Added in version 3.2.

Command-line interface

The `site` module also provides a way to get the user directories from the command line:

```
$ python -m site --user-site
/home/user/.local/lib/python3.11/site-packages
```

If it is called without arguments, it will print the contents of `sys.path` on the standard output, followed by the value of `USER_BASE` and whether the directory exists, then the same thing for `USER_SITE`, and finally the value of `ENABLE_USER_SITE`.

--user-base

Print the path to the user base directory.

--user-site

Print the path to the user site-packages directory.

If both options are given, user base and user site will be printed (always in this order), separated by `os.pathsep`.

If any option is given, the script will exit with one of these values: 0 if the user site-packages directory is enabled, 1 if it was disabled by the user, 2 if it is disabled for security reasons or by an administrator, and a value greater than 2 if there is an error.

See also

- PEP 370— Per user site-packages directory
- The initialization of the `sys.path` module search path— The initialization of `sys.path`.

Custom Python Interpreters

The modules described in this chapter allow writing interfaces similar to Python's interactive interpreter. If you want a Python interpreter that supports some special feature in addition to the Python language, you should look at the `code` module. (The `codeop` module is lower-level, used to support compiling a possibly incomplete chunk of Python code.)

The full list of modules described in this chapter is:

- `code`— Interpreter base classes `InteractiveInterpreter` Objects `InteractiveConsole` Objects
- `codeop`— Compile Python code

code— Interpreter base classes

Source code: `Lib/code.py`

The `code` module provides facilities to implement read-eval-print loops in Python. Two classes and convenience functions are included which can be used to build applications which provide an interactive interpreter prompt.

`class code.InteractiveInterpreter(locals=None)`

This class deals with parsing and interpreter state (the user's namespace); it does not deal with input buffering or prompting or input file naming (the filename is always passed in explicitly). The optional `locals` argument specifies a mapping to use as the namespace in which code will be executed; it defaults to a newly created dictionary with key `'__name__'` set to `'__console__'` and key `'__doc__'` set to `None`. Note that functions and classes objects created under an `InteractiveInterpreter` instance will belong

`class code.InteractiveConsole(locals=None, filename='<console>', local_exit=False)`

Closely emulate the behavior of the interactive Python interpreter. This class builds on `InteractiveInterpreter` and adds prompting using the familiar `sys.ps1` and `sys.ps2`, and input buffering. If `local_exit` is true, `exit()` and `quit()` in the console will not raise `SystemExit`, but instead return to the calling code. Changed in version 3.13: Added `local_exit` parameter.

`code.interact(banner=None, readfunc=None, local=None, exitmsg=None, local_exit=False)`

Convenience function to run a read-eval-print loop. This creates a new instance of `InteractiveConsole` and sets `readfunc` to be used as the `InteractiveConsole.raw_input()` method, if provided. If `locals` is provided, it is passed to the `InteractiveConsole` constructor for use as the default namespace for the interpreter loop. If `local_exit` is provided, it is passed to the `InteractiveConsole` constructor. The `interact()` method of the instance is then run with `banner` and `exitmsg` passed as the banner and exit message to

`code.compile_command(source, filename='<input>', symbol='single')`

This function is useful for programs that want to emulate Python's interpreter main loop (a.k.a. the read-eval-print loop). The tricky part is to determine when the user has entered an incomplete command that can be completed by entering more text (as opposed to a complete command or a syntax error). This function almost always makes the same decision as the real interpreter main loop. `source` is the source string; `filename` is the optional filename from which source was read, defaulting to `'<input>'`;

Interactive Interpreter Objects

`InteractiveInterpreter.runsource(source, filename='<input>', symbol='single')`

Compile and run some source in the interpreter. Arguments are the same as for `compile_command()`; the default for `filename` is `'<input>'`, and for `symbol` is `'single'`. One of several things can happen: The input is incorrect; `compile_command()` raised an exception (`SyntaxError` or `OverflowError`). A syntax traceback will be printed by calling the `showsyntaxerror()` method. `runsource()` returns `False`. The input is incomplete, and more input is required; `compile_command()` returned `None`. `runsource()` returns `True`. The input is com

`InteractiveInterpreter.runcode(code)`

Execute a code object. When an exception occurs, `showtraceback()` is called to display a traceback. All exceptions are caught except `SystemExit`, which is allowed to propagate. A note about `KeyboardInterrupt`: this exception may occur elsewhere in this code, and may not always be caught. The caller should be prepared to deal with it.

`InteractiveInterpreter.showsyntaxerror(filename=None)`

Display the syntax error that just occurred. This does not display a stack trace because there isn't one for syntax errors. If `filename` is given, it is stuffed into the exception instead of the default filename provided by Python's parser, because it always uses '<string>' when reading from a string. The output is written by the `write()` method.

`InteractiveInterpreter.showtraceback()`

Display the exception that just occurred. We remove the first stack item because it is within the interpreter object implementation. The output is written by the `write()` method. Changed in version 3.5: The full chained traceback is displayed instead of just the primary traceback.

`InteractiveInterpreter.write(data)`

Write a string to the standard error stream (`sys.stderr`). Derived classes should override this to provide the appropriate output handling as needed.

Interactive Console Objects

The `InteractiveConsole` class is a subclass of `InteractiveInterpreter`, and so offers all the methods of the interpreter objects as well as the following additions.

`InteractiveConsole.interact(banner=None, exitmsg=None)`

Closely emulate the interactive Python console. The optional `banner` argument specifies the banner to print before the first interaction; by default it prints a banner similar to the one printed by the standard Python interpreter, followed by the class name of the console object in parentheses (so as not to confuse this with the real interpreter – since it's so close!). The optional `exitmsg` argument specifies an exit message printed when exiting. Pass the empty string to suppress the exit message. If `exit`

`InteractiveConsole.push(line)`

Push a line of source text to the interpreter. The line should not have a trailing newline; it may have internal newlines. The line is appended to a buffer and the interpreter's `runsource()` method is called with the concatenated contents of the buffer as source. If this indicates that the command was executed or invalid, the buffer is reset; otherwise, the command is incomplete, and the buffer is left as it was after the line was appended. The return value is `True` if more input is required, `False` if

`InteractiveConsole.resetbuffer()`

Remove any unhandled source text from the input buffer.

`InteractiveConsole.raw_input(prompt="")`

Write a prompt and read a line. The returned line does not include the trailing newline. When the user enters the EOF key sequence, `EOFError` is raised. The base implementation reads from `sys.stdin`; a subclass may replace this with a different implementation.